FREECAD

A MANUAL

BY YORIK VAN HAVRE
AND THE FREECAD COMMUNITY

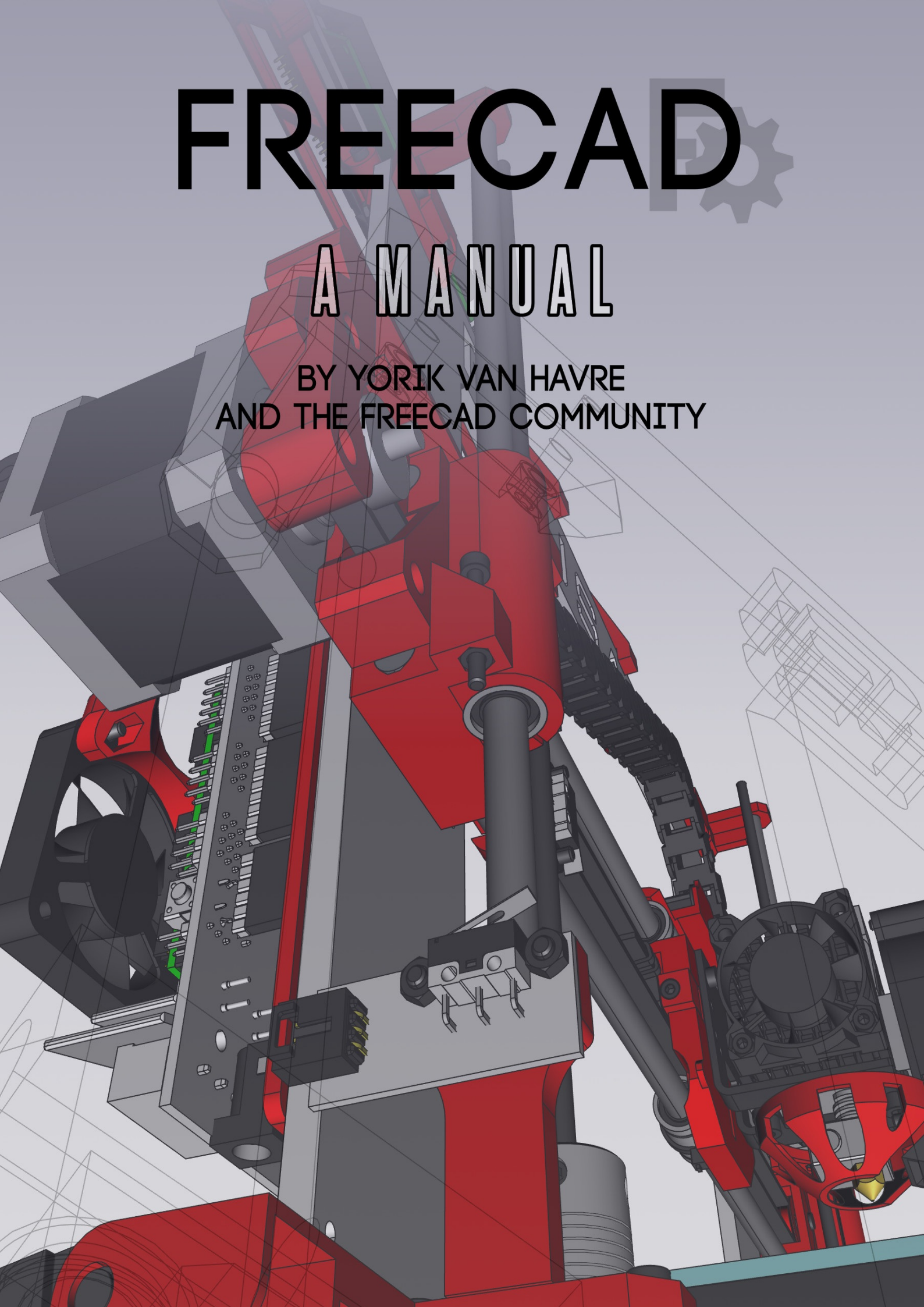


Table of Contents

- Introduction 1.1
- Discovering FreeCAD 1.2
  - What is FreeCAD? 1.2.1
  - Installing 1.2.2
    - Installing on Windows 1.2.2.1
    - Installing on Linux 1.2.2.2
    - Installing on Mac OS 1.2.2.3
    - Uninstalling 1.2.2.4
    - Setting basic preferences 1.2.2.5
    - Installing additional content 1.2.2.6
  - The FreeCAD interface 1.2.3
    - Workbenches 1.2.3.1
    - The interface 1.2.3.2
    - Customizing the interface 1.2.3.3
  - Navigating in the 3D view 1.2.4
    - A word about the 3D space 1.2.4.1
    - The FreeCAD 3D view 1.2.4.2
    - Selecting objects 1.2.4.3
  - The FreeCAD document 1.2.5
  - Parametric objects 1.2.6
  - Import and export to other filetypes 1.2.7
- Working with FreeCAD 1.3
  - All workbenches at a glance 1.3.1
  - Traditional modeling, the CSG way 1.3.2
  - Traditional 2D drafting 1.3.3
  - Modeling for product design 1.3.4
  - Preparing models for 3D printing 1.3.5
    - Exporting to slicers 1.3.5.1
    - Converting objects to meshes 1.3.5.2
    - Using Slic3r 1.3.5.3

Using the Cura addon 1.3.5.4
Generating G-code 1.3.5.5
Generating 2D drawings 1.3.6
BIM modeling 1.3.7
Using spreadsheets 1.3.8
Reading properties 1.3.8.1
Writing properties 1.3.8.2
Creating FEM analyses 1.3.9
Creating renderings 1.3.10
Python scripting 1.4
A gentle introduction 1.4.1
Writing Python code 1.4.1.1
Manipulating FreeCAD objects 1.4.1.2
Vectors and Placements 1.4.1.3
Creating and manipulating geometry 1.4.2
Creating parametric objects 1.4.3
Creating interface tools 1.4.4
The community 1.5

Посібник із FreeCAD

Примітка: Посібник перенесено до офіційної вікі FreeCAD — тепер це його нове місце розташування. Якщо ви бажаєте запропонувати зміни, будь ласка, вносьте їх саме там, оскільки цей репозиторій надалі використовуватиметься лише для створення версій у форматі електронних книг і безпосередньо редагуватися більше не буде.

Вступ

FreeCAD — це безкоштовна програма для параметричного 3D-моделювання з відкритим вихідним кодом. Вона створена насамперед для моделювання реальних об’єктів: від дрібних електронних компонентів до будівель та інженерних споруд, причому особлива увага приділяється об’єктам, призначеним для 3D-друку. FreeCAD можна вільно завантажувати, використовувати, поширювати та змінювати; його вихідний код є відкритим і публікується на умовах дуже ліберальної ліцензії LGPL. Дані, створені вами у FreeCAD, належать виключно вам, і їх можна відновити навіть без використання цієї програми.

FreeCAD — це, по суті, також соціальний проєкт, оскільки його розробляє та підтримує спільнота розробників і користувачів, об’єднаних пристрастю до FreeCAD.

Цей посібник — спроба піти шляхом, протилежним тому, що пропонує офіційна вікі-документація FreeCAD. Вікі створюється спільними зусиллями десятків учасників спільноти; як і більшість вікі-ресурсів, вона містить величезний обсяг інформації, проте новачкам буває досить складно в ній орієнтуватися. Завдяки цьому вона є цінним довідковим ресурсом, але не надто зручним інструментом для вивчення FreeCAD. У цьому посібнику розглядається той самий матеріал, що й у вікі. Однак ми сподіваємося, що завдяки покроковому викладу, опорі на конкретні приклади та єдиному стилю (оскільки авторів тут менше), цей посібник краще підійде для першого знайомства з FreeCAD і стане чудовим доповненням до вікі.

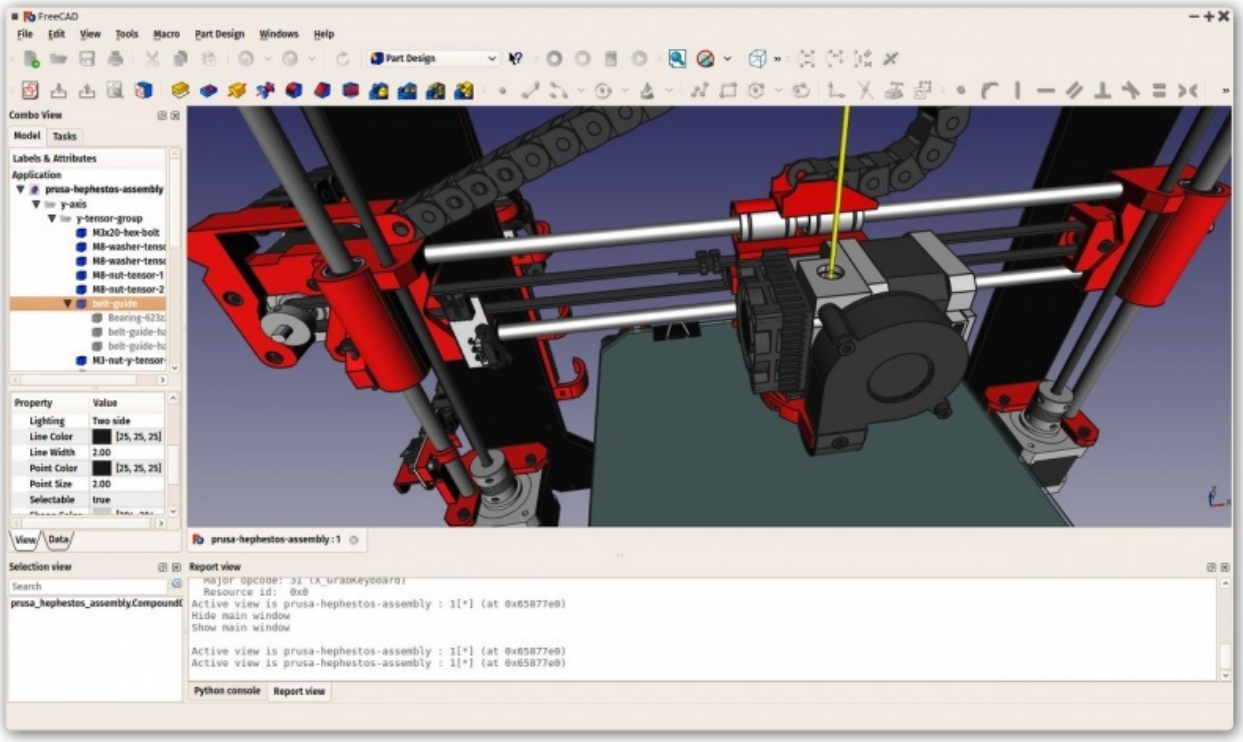
Цей посібник написаний для поточної стабільної версії FreeCAD — версії **0.16**.

Увесь вміст цього посібника опубліковано на умовах ліцензії Creative Commons 4.0; його можна вільно використовувати, завантажувати, копіювати та змінювати. Вихідні файли посібника розміщено на GitHub.

Цю книгу написав переважно Yorik, проте в ній використано багато інформації, зібраної користувачами FreeCAD — здебільшого з вікі FreeCAD. Тож справжнім автором цієї книги є вся спільнота FreeCAD!

Знайомство з FreeCAD

Що таке FreeCAD?



FreeCAD — це програма параметричного 3D-моделювання з відкритим вихідним кодом, створена переважно для проектування реальних об’єктів. Параметричне моделювання описує певний тип моделювання, де форма тривимірних об’єктів, які ви проектуєте, контролюється параметрами. Наприклад, форма цегли може регулюватися трьома параметрами: висотою, шириною та довжиною. У FreeCAD, як і в інших параметричних моделях, ці параметри є частиною об’єкта, і їх можна змінювати в будь-який час після створення об’єкта. Деякі об’єкти можуть мати інші об’єкти як параметри, наприклад, ви можете мати об’єкт, який приймає нашу цеглинку як вхідні дані та створює з неї стовпець. Ви можете думати про параметричний об’єкт як про невелику програму, яка створює геометрію з параметрів.

FreeCAD не призначений для якогось одного конкретного виду робіт чи створення певного типу об'єктів. Натомість він пропонує широкі можливості застосування, дозволяючи користувачам створювати моделі будь-яких розмірів і призначень — від дрібних електронних компонентів і деталей для 3D-друку аж до будівель. Для кожного з таких завдань передбачено відповідний набір інструментів і робочих процесів.

FreeCAD також є кросплатформним (програма працює однаково в операційних системах Windows, Mac OS та Linux) і має відкритий вихідний код. Завдяки відкритому коду FreeCAD розвивається завдяки внеску та зусиллям великої спільноти програмістів, ентузіастів і користувачів з усього світу. По суті, FreeCAD — це програма, створена самими користувачами, а не комерційною компанією, яка намагається продати вам свій продукт. І, звісно, це означає, що FreeCAD є безкоштовним: його можна не лише використовувати, а й поширювати, копіювати, змінювати чи навіть продавати.

FreeCAD також спирається на величезний досвід, накопичений у сфері програмного забезпечення з відкритим вихідним кодом. У своїй основі програма містить низку інших компонентів із відкритим кодом; водночас і сам FreeCAD може використовуватися як компонент в інших застосунках. Він також має широкий набір функцій, що стали стандартом у світі відкритого ПЗ: підтримку великої кількості форматів файлів, широкі можливості для написання скриптів, а також гнучкі засоби налаштування та модифікації. Усе це стало можливим завдяки активній спільноті користувачів-ентузіастів.

Офіційний вебсайт FreeCAD: http://www.freecadweb.org

Читати далі:

- Про FreeCAD: http://www.freecadweb.org/wiki/index.php?title=About_FreeCAD
- Список функцій: http://www.freecadweb.org/wiki/index.php?title=Feature_list
- Знімки екрана та приклади: http://forum.freecadweb.org/viewforum.php?f=24

Встановлення

FreeCAD розповсюджується на умовах ліцензії LGPL. Це означає, що ви можете вільно завантажувати, встановлювати, поширювати та використовувати FreeCAD на власний розсуд — незалежно від того, чи йдеться про комерційну, чи про некомерційну діяльність. Ви не обмежені жодними додатковими умовами, а файли, створені за допомогою цієї програми, належать виключно вам. Єдине, що фактично забороняє ця ліцензія, — це стверджувати, що ви самі розробили FreeCAD!

FreeCAD працює однаково в операційних системах Windows, Mac OS та Linux. Проте способи його встановлення дещо відрізняються залежно від платформи. Для Windows і Mac спільнота FreeCAD надає готові до завантаження скомпільовані пакети (інсталятори), тоді як для Linux доступним є вихідний код, який отримують супроводжувачі дистрибутивів; саме вони відповідають за підготовку пакета FreeCAD для свого конкретного дистрибутива. Як наслідок, у Linux встановити FreeCAD зазвичай можна безпосередньо через менеджер програм.

Офіційна сторінка завантаження FreeCAD для Windows та Mac OS є:
https://github.com/FreeCAD/FreeCAD/releases

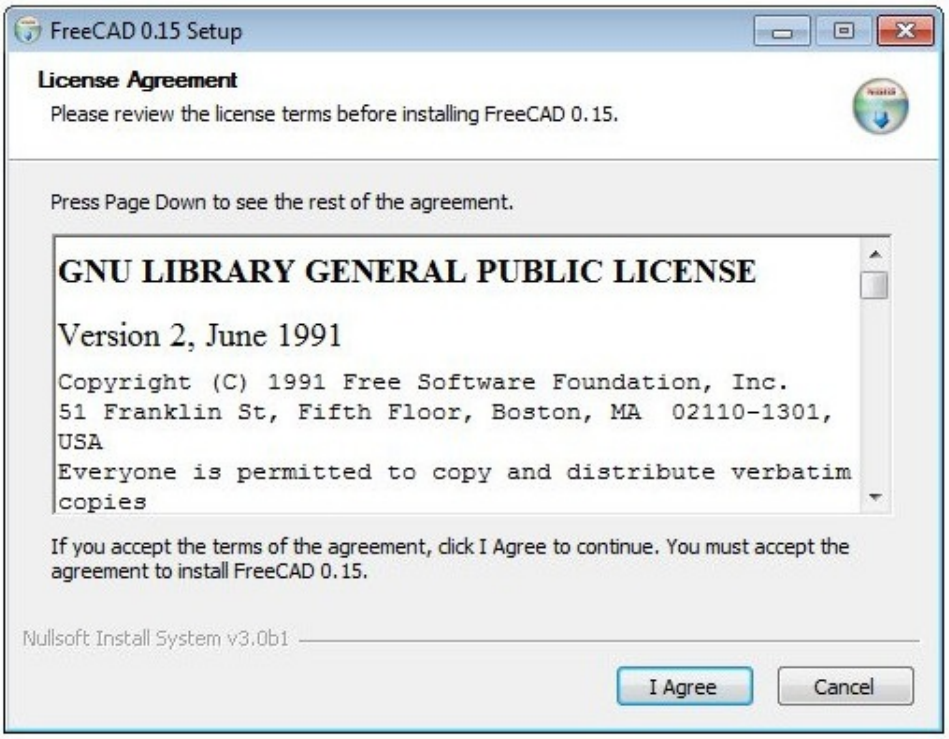
Версії FreeCAD

Офіційні випуски FreeCAD, доступні на згаданій вище сторінці або в менеджері програм вашого дистрибутива, є стабільними версіями. Проте розробка FreeCAD просувається дуже швидко! Майже щодня з'являються нові функції та виправляються помилки. Оскільки між стабільними випусками іноді минає чимало часу, вам може бути цікаво спробувати новішу, експериментальну версію FreeCAD. Такі версії для розробників (або попередні випуски) час від часу додаються на згадану вище сторінку завантаження. Крім того, якщо ви використовуєте Ubuntu, спільнота FreeCAD підтримує репозиторії PPA (Personal Package Archives) — так звані «щоденні збірки» (daily builds), які регулярно оновлюються й містять найсвіжіші зміни.

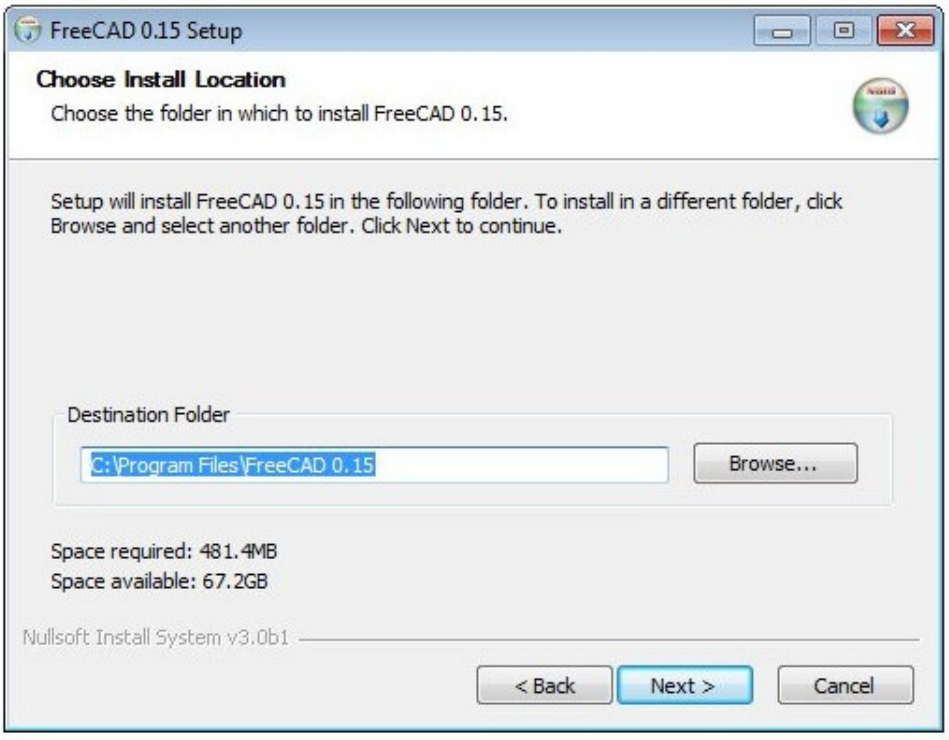
Якщо ви встановлюєте FreeCAD у віртуальній машині, майте на увазі, що продуктивність може бути низькою — або ж програма може взагалі виявитися непридатною для використання — через обмеження підтримки OpenGL у більшості віртуальних машин.

Встановлення в Windows

1. Завантажте інсталяційний пакет (.exe), що відповідає вашій версії Windows (32-розрядній або 64-розрядній), зі сторінки завантаження. Інсталятори FreeCAD мають працювати в будь-якій версії Windows, починаючи з Windows 7.
2. Двічі клацніть завантажений інсталяційний файл.
3. Прийміть умови ліцензії LGPL (це один із небагатьох випадків, коли можна сміливо натискати кнопку «Прийняти», не читаючи тексту — жодних прихованих пунктів тут немає):

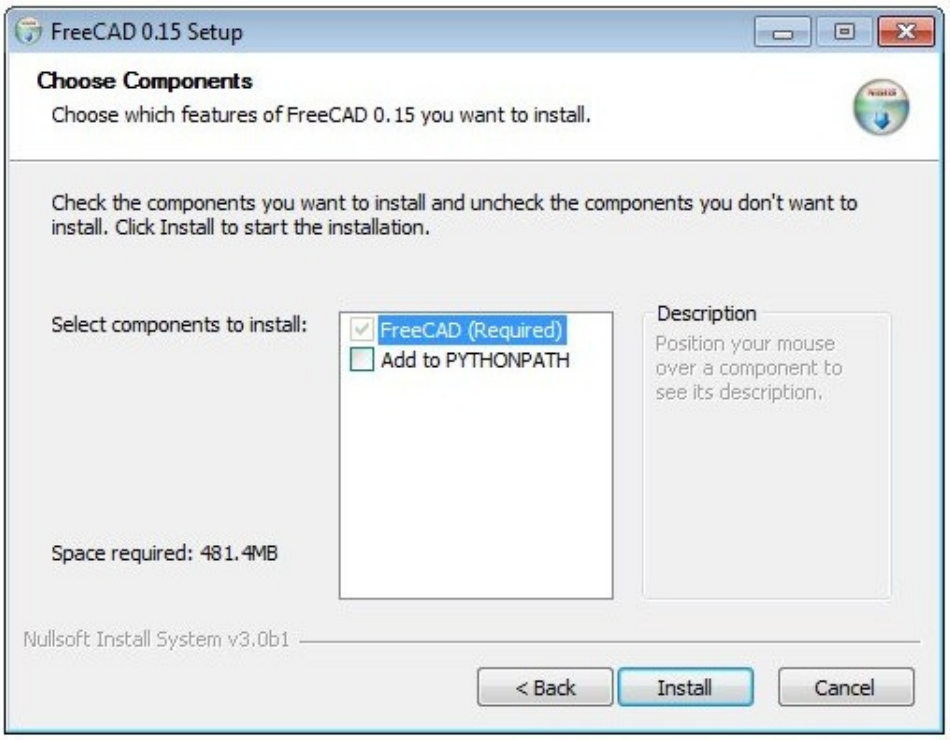


4. Тут можна залишити шлях за замовчуванням або, за бажанням, змінити його:

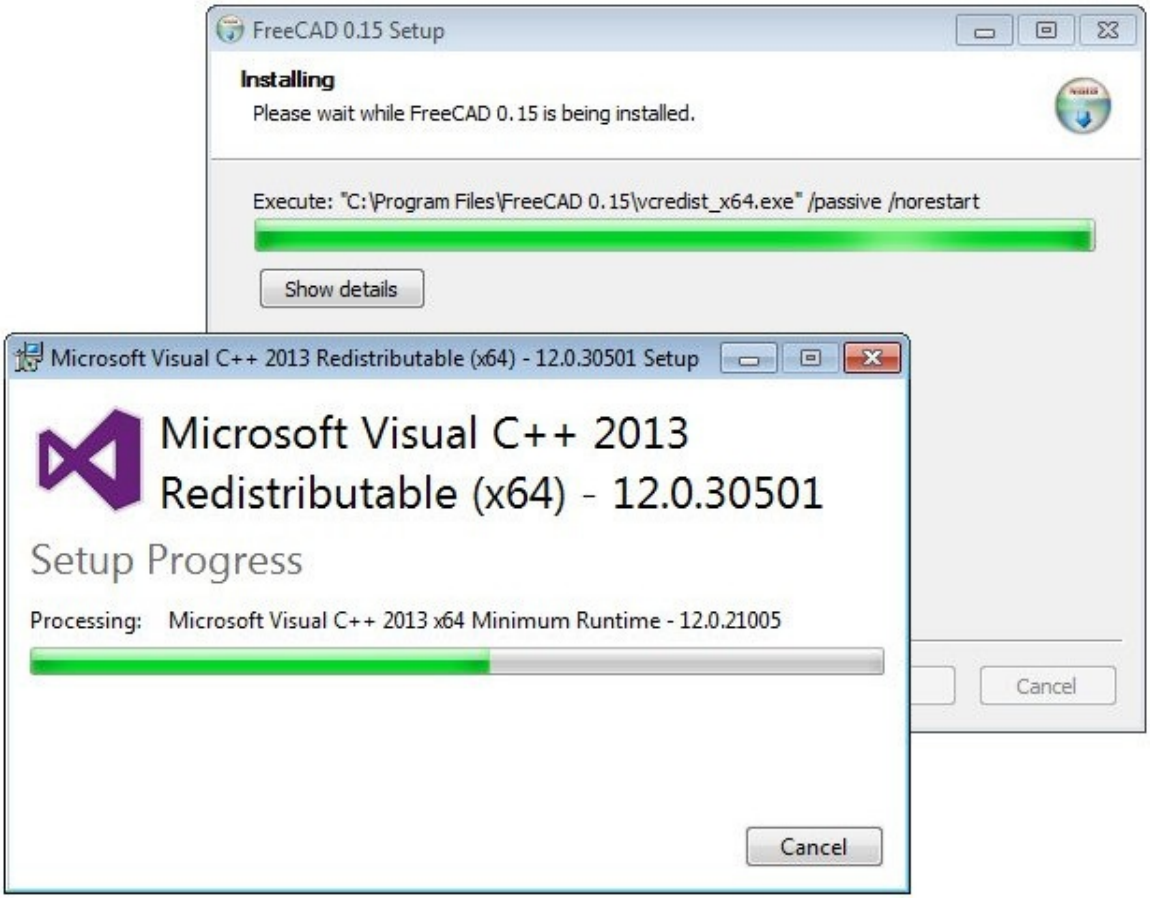


5. Немає потреби встановлювати змінну PYTHONPATH, хіба що ви плануєте займатися складнішим програмуванням на Python — у такому разі ви, ймовірно, вже знаєте,

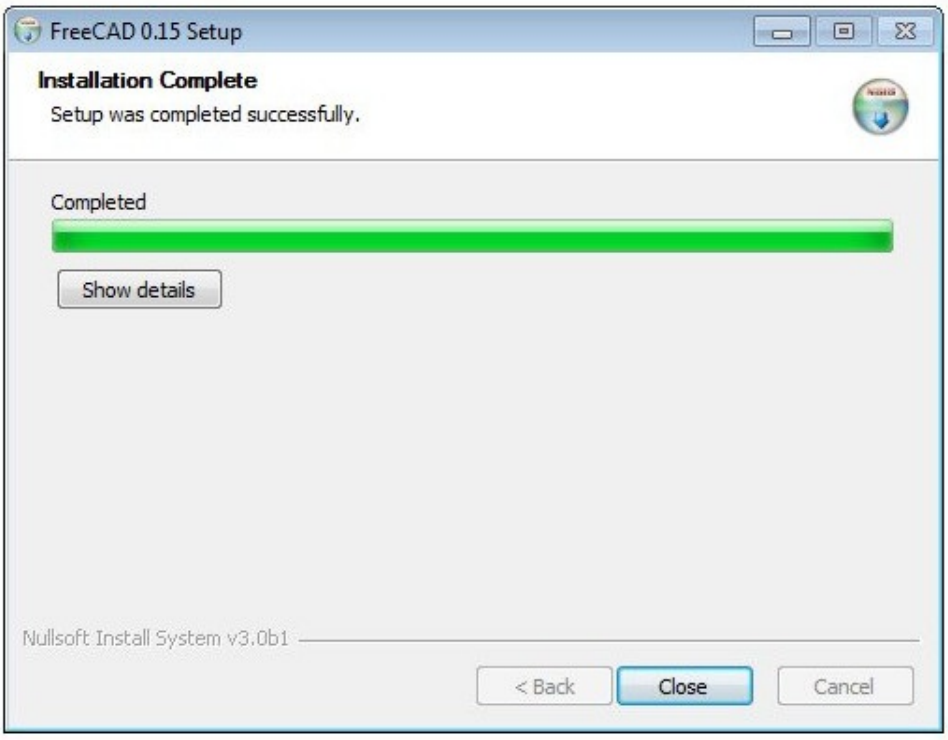
для чого вона потрібна:



6. Під час встановлення також буде встановлено кілька додаткових компонентів, що входять до складу інсталятора:



7. Ось і все — FreeCAD встановлено. Ви знайдете його в меню «Пуск».



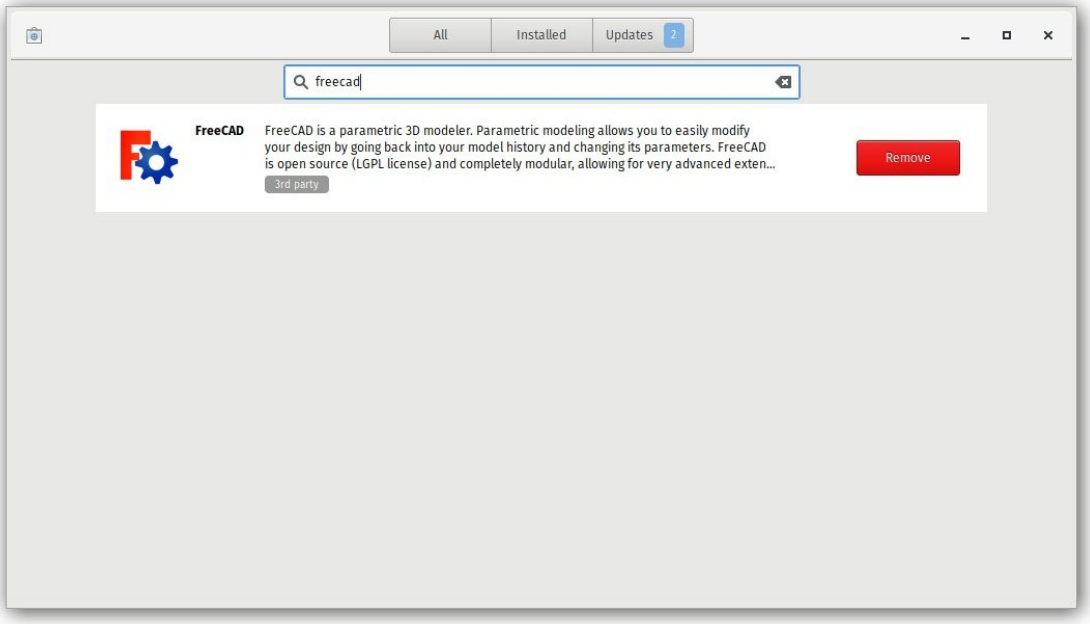
Встановлення версії для розробки

Підготовка FreeCAD до розповсюдження та створення інсталятора потребують певного часу й зусиль, тому версії для розробників (їх також називають попередніми версіями) зазвичай надаються у вигляді архівів .zip (або .7z). Їх не потрібно встановлювати: достатньо розпакувати архів і запустити FreeCAD, двічі клацнувши файл FreeCAD.exe, який міститься всередині. Це також дозволяє використовувати стабільну та «нестабільну» версії одночасно на одному комп'ютері.

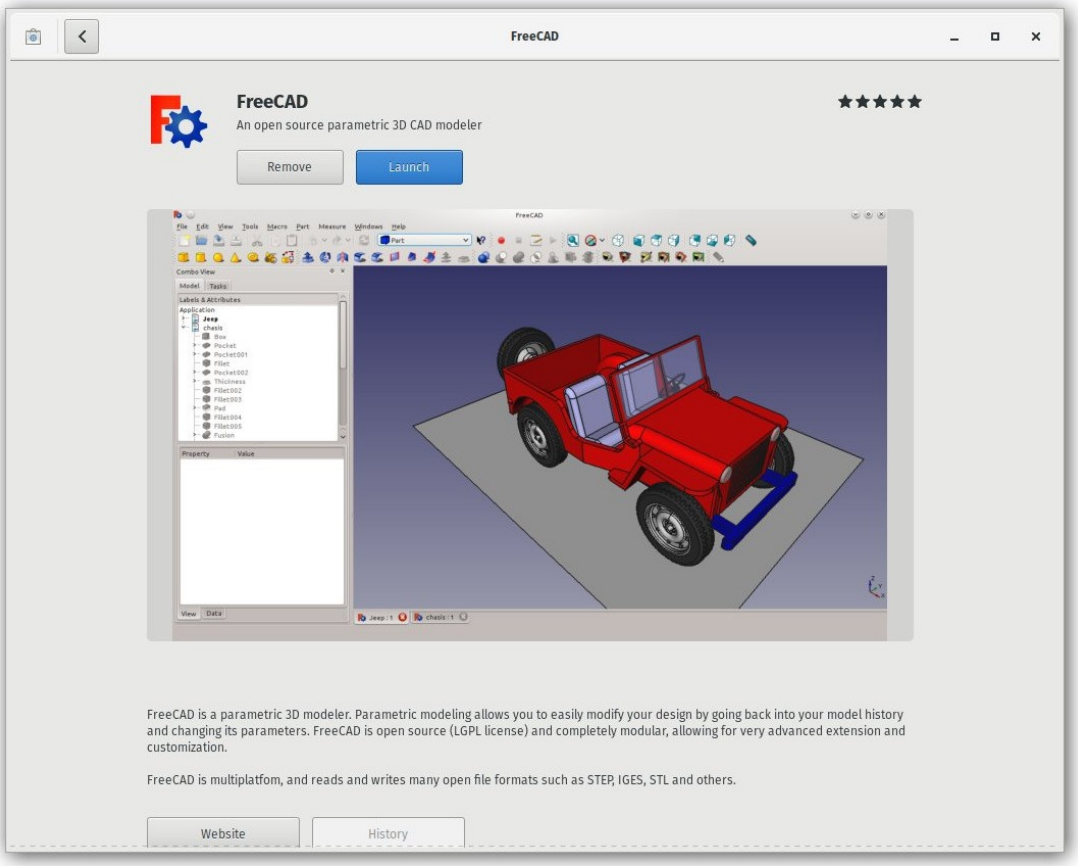
Встановлення в Linux

У більшості сучасних дистрибутивів Linux (Ubuntu, Fedora, OpenSUSE, Debian, Mint, Elementary тощо) FreeCAD можна встановити одним натисканням кнопки — безпосередньо через програму для керування програмним забезпеченням, що входить до складу вашого дистрибутива (зовнішній вигляд цієї програми може відрізнятися від наведеного на зображеннях нижче, оскільки кожен дистрибутив використовує власний інструмент).

1. Відкрийте менеджер програмного забезпечення та знайдіть «FreeCAD»:



2. Натисніть кнопку «Встановити» — і все, FreeCAD буде встановлено. Не забудьте потім оцінити програму!



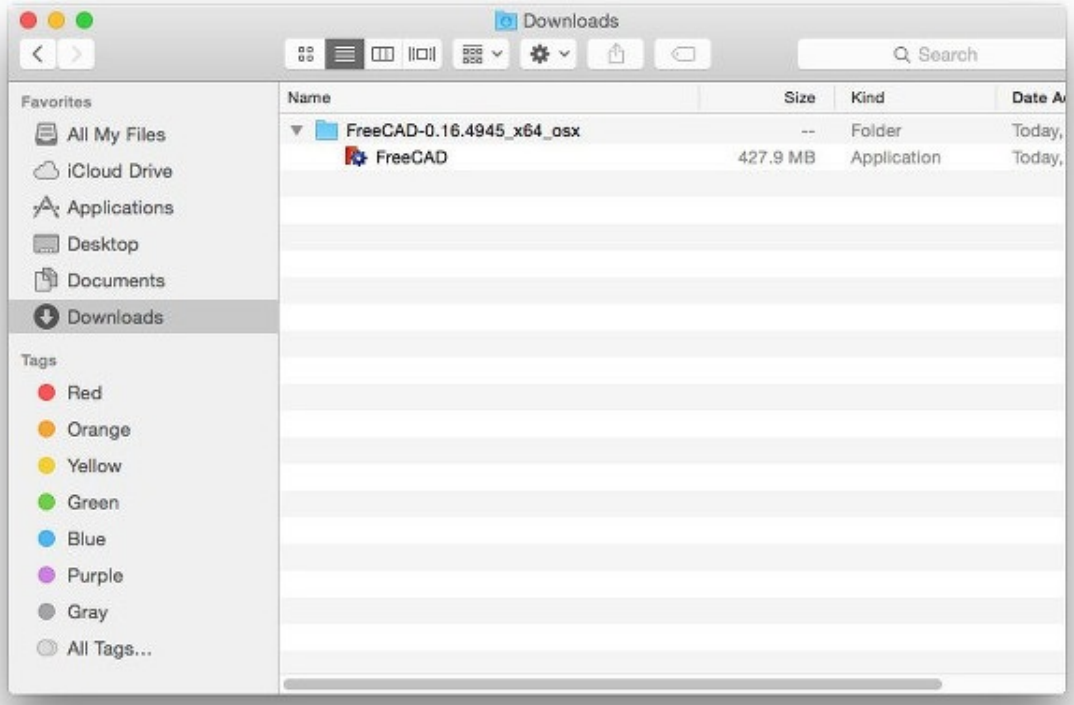
Альтернативні шляхи

Однією з великих переваг використання Linux є безліч можливостей для налаштування програмного забезпечення на власний розсуд, тож не обмежуйте себе. В Ubuntu та її похідних дистрибутивах FreeCAD також можна встановити з PPA-репозиторію, який підтримується спільнотою FreeCAD (він містить як стабільні версії, так і версії для розробки). А оскільки це програмне забезпечення з відкритим вихідним кодом, ви також можете легко зібрати FreeCAD самостійно.

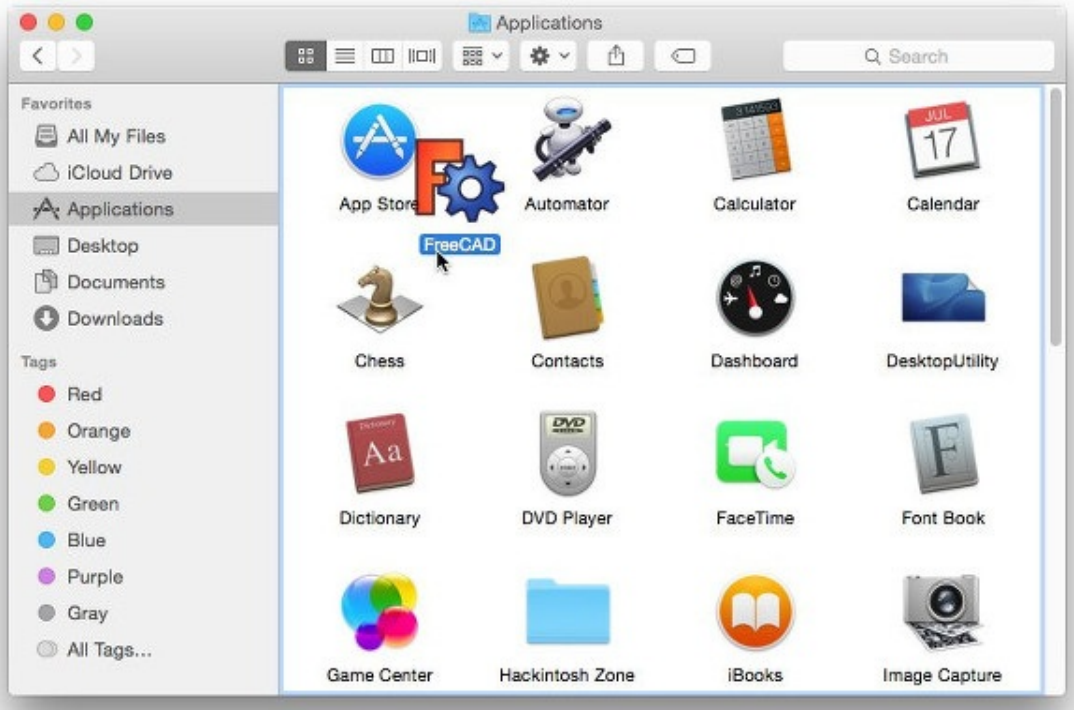
Встановлення в Mac OS

Встановлення FreeCAD у Mac OS сьогодні таке ж просте, як і на інших платформах. Однак, оскільки в спільноті менше користувачів, які мають комп'ютери Mac, доступні пакети часто відстають на кілька версій від тих, що пропонуються для інших платформ.

1. Завантажте зі сторінки завантажень архів, що відповідає вашій версії.
2. Відкрийте папку «Завантаження» та розпакуйте завантажений ZIP-файл:



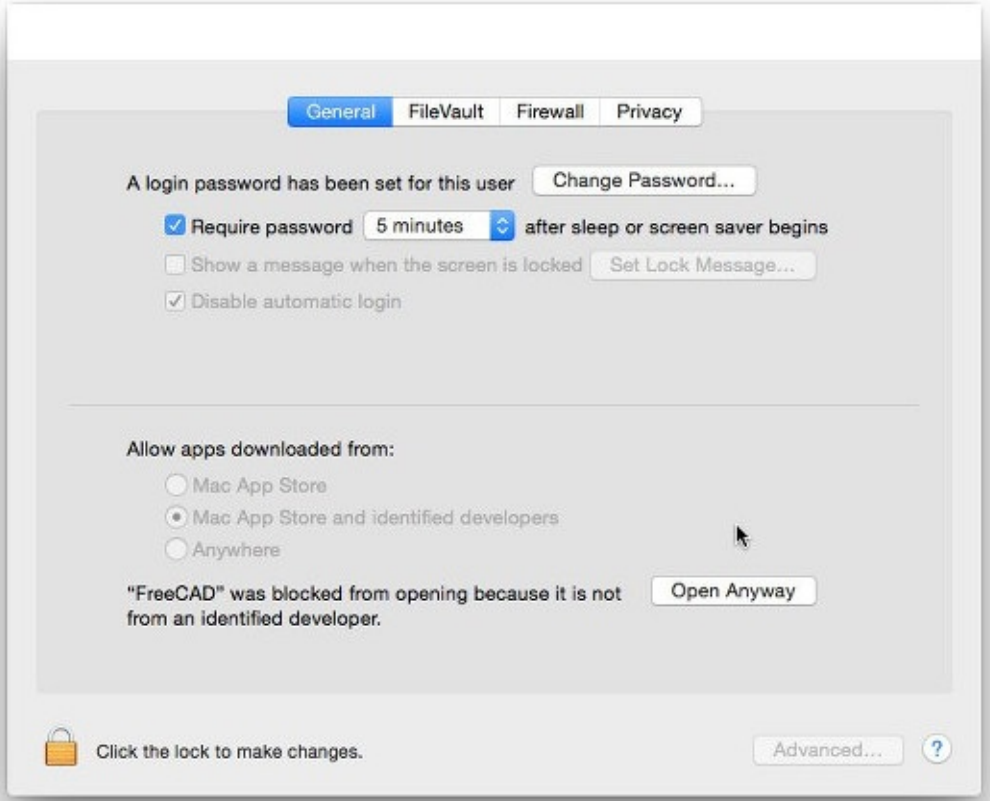
3. Перетягніть програму FreeCAD із ZIP-архіву до папки «Програми»:



4. Ось і все, FreeCAD встановлено!



5. Якщо система перешкоджає запуску FreeCAD через обмеження прав доступу для програм, завантажених не з App Store, вам потрібно дозволити його запуск:



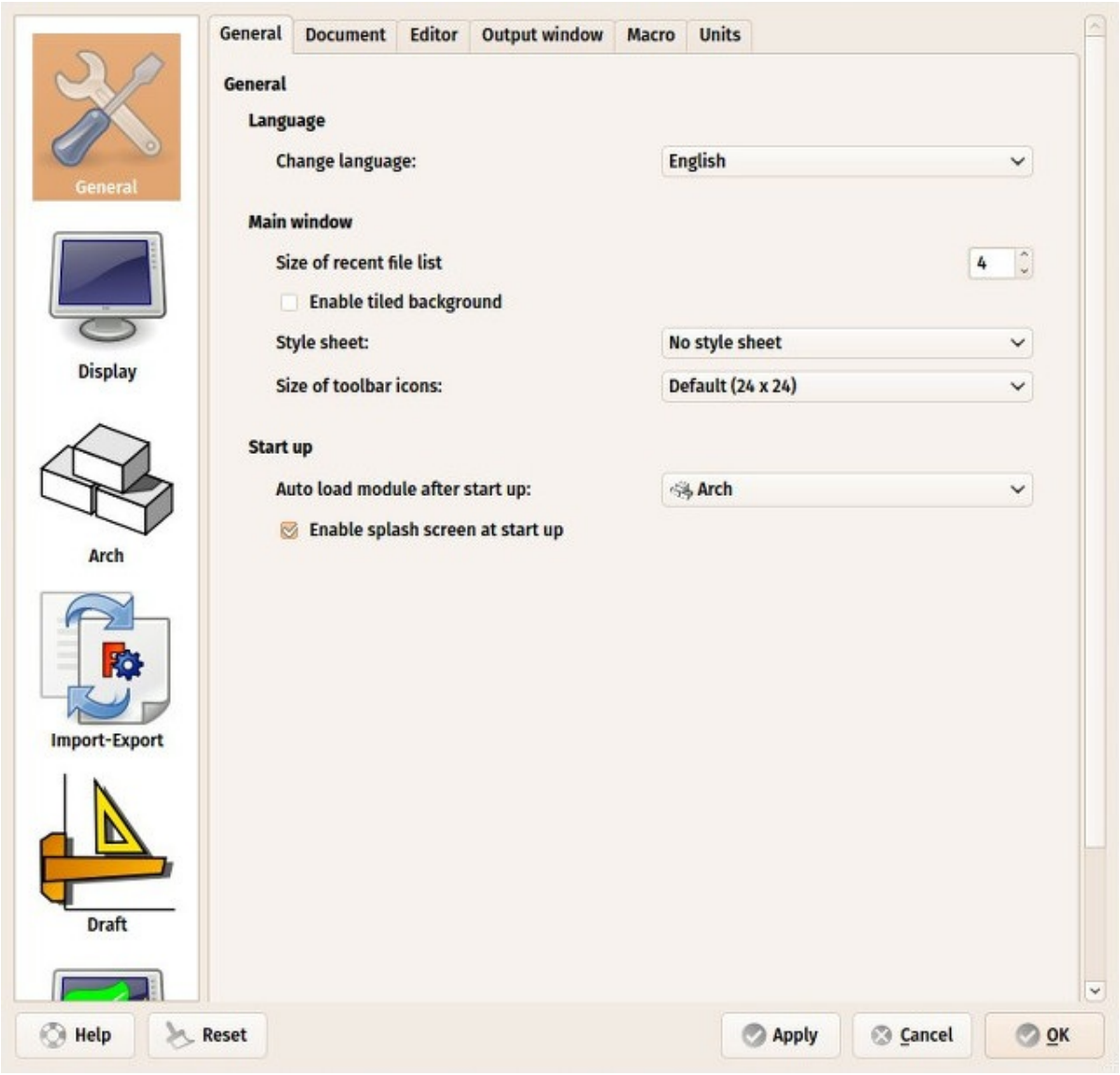
Видалення

Сподіваємося, вам це не знадобиться, але знати про таку можливість усе ж варто. У Windows і Linux видалити FreeCAD дуже просто. У Windows скористайтеся стандартною функцією «Видалення програм» у Панелі керування, а в Linux — тим самим менеджером програм, за допомогою якого ви встановлювали FreeCAD. На комп'ютерах Mac достатньо просто видалити програму з папки «Програми» (Applications).

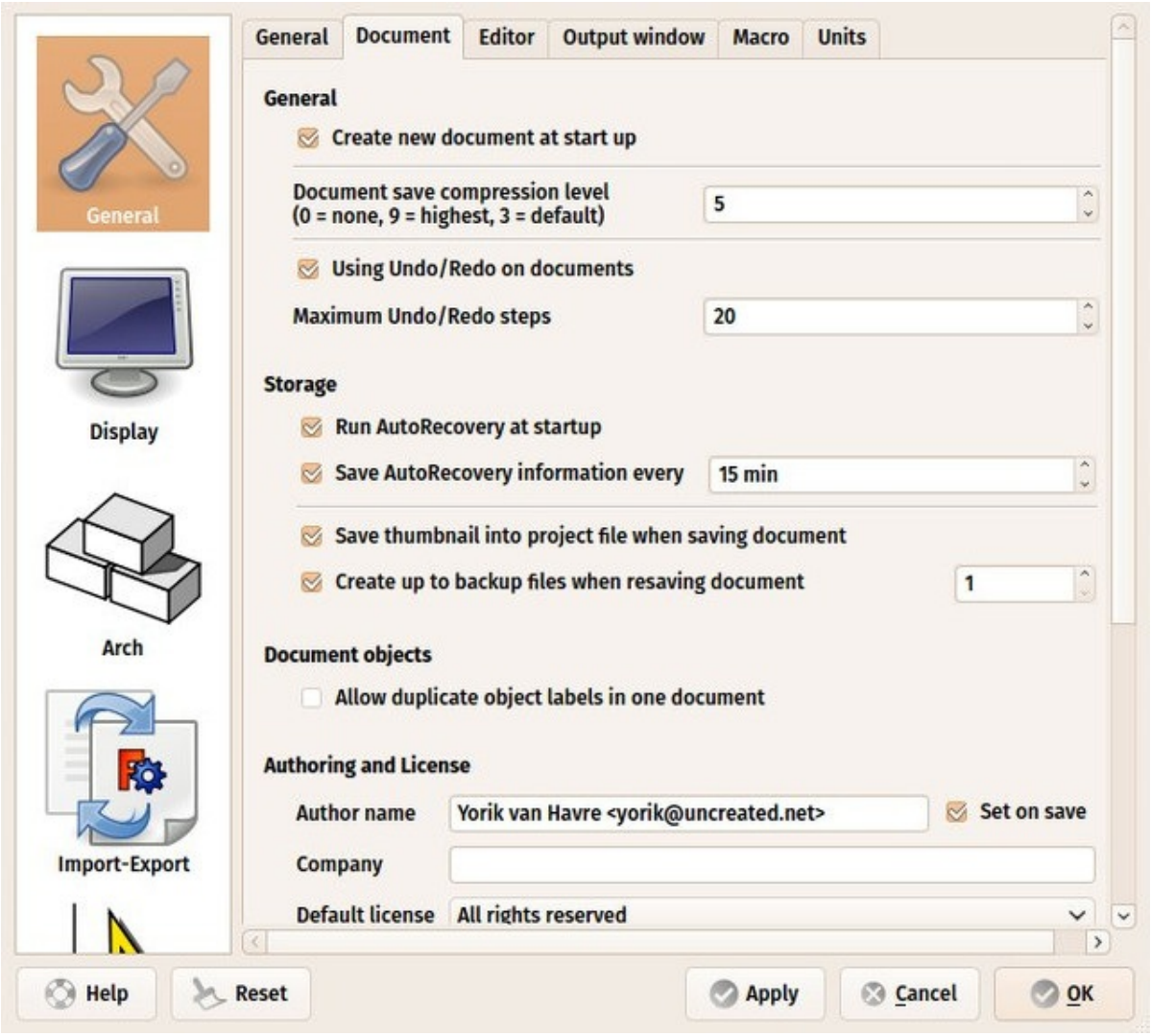
Налаштування основних параметрів

Після встановлення FreeCAD варто відкрити програму та задати кілька налаштувань. Параметри у FreeCAD знаходяться в меню «Редагування» (Edit) -> «Налаштування» (Preferences). Ви можете переглянути різні сторінки налаштувань, щоб дізнатися, чи є ще щось, що ви хотіли б змінити, але ось кілька основних параметрів:

1. **Мова:** FreeCAD автоматично обере мову вашої операційної системи, проте ви можете змінити її. FreeCAD майже повністю перекладено 5–6 мовами; крім того, існує багато інших мов, переклад якими наразі виконано лише частково. Ви можете легко долучитися до перекладу FreeCAD.

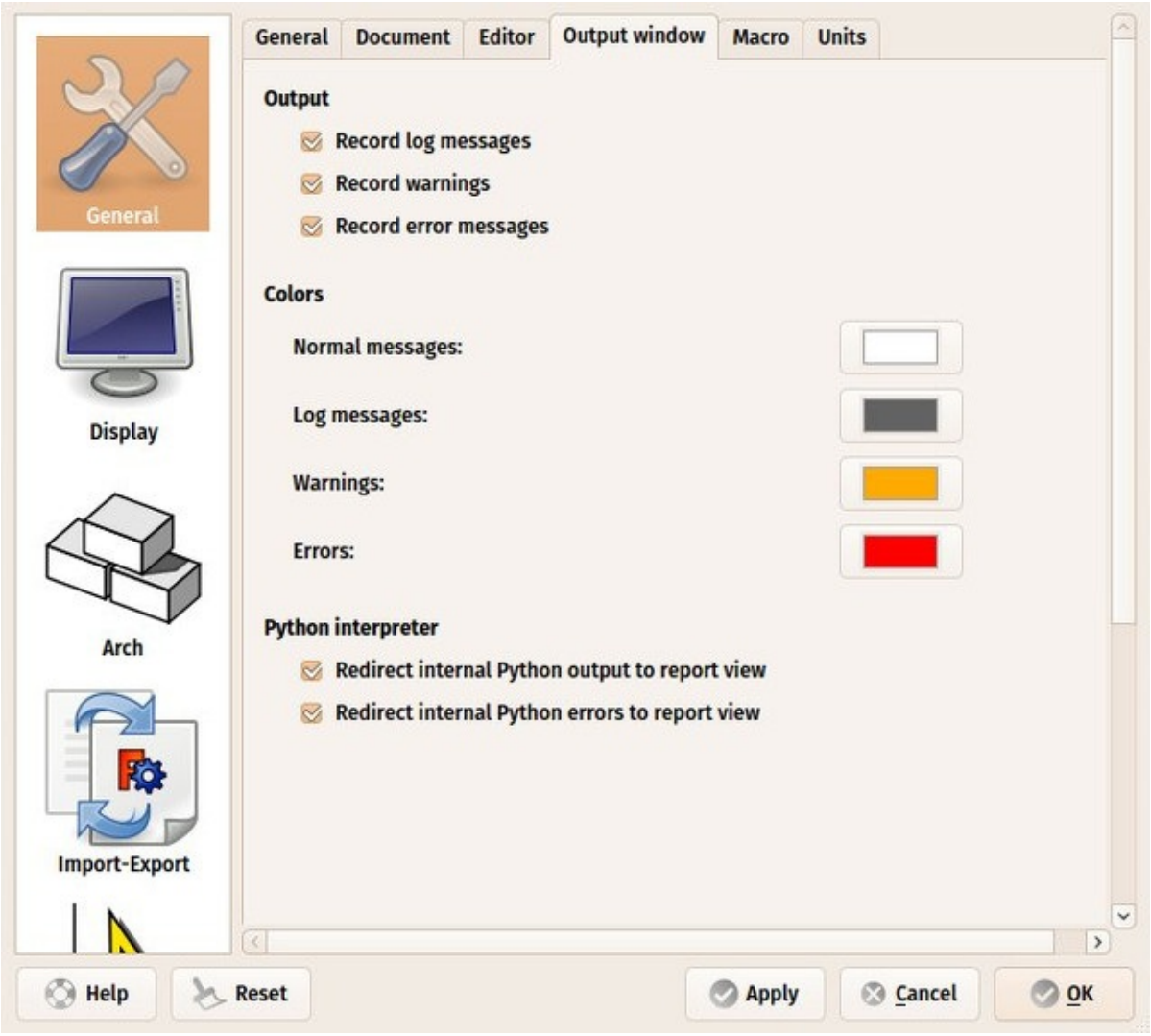


2. **Автоматичне завантаження модуля:** зазвичай після запуску FreeCAD відкривається «Центр запуску» (Start Center). Ви можете пропустити цей етап і розпочати роботу у FreeCAD одразу в обраному вами робочому середовищі (workbench). Детальніше про робочі середовища йтиметься в наступному розділі.
3. **Створення документа під час запуску:** у поєднанні з попередньою опцією це дозволяє запустити FreeCAD у стані, готовому до роботи.

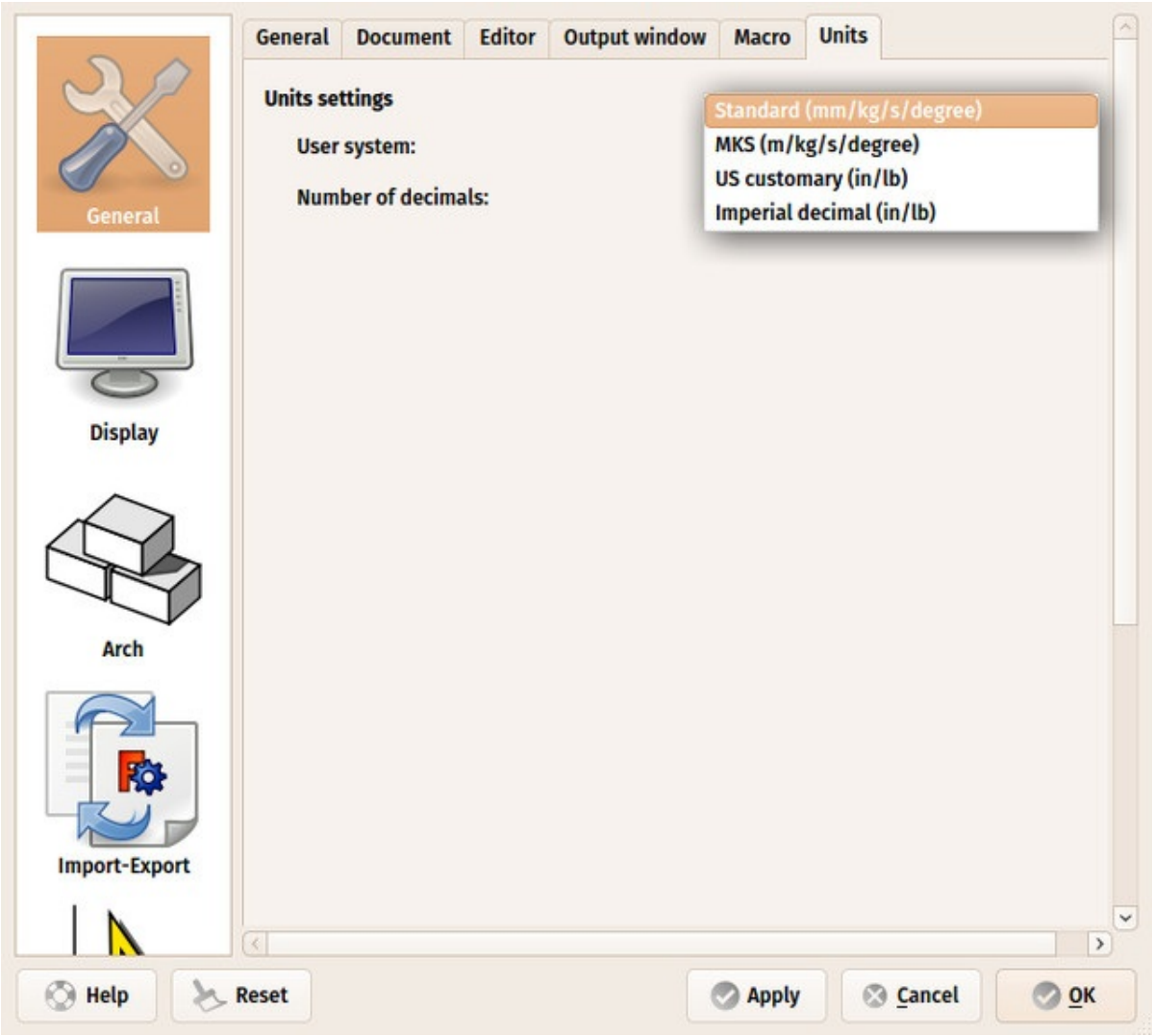


4. **Параметри збереження:** як і будь-яка складна програма, FreeCAD може час від часу аварійно завершувати роботу. Тут можна налаштувати кілька параметрів, які допоможуть відновити результати вашої праці в разі збою.

5. **Авторство та ліцензія:** Ви можете задати параметри за замовчуванням, які застосовуватимуться до нових файлів. Варто подбати про те, щоб ваші файли були доступні для поширення від самого початку — для цього можна обрати ліцензію типу «copyleft» (наприклад, Creative Commons).
6. **Перенаправлення повідомлень Python у вікно виводу:** ці два параметри варто активувати, оскільки вони дозволять бачити в панелі звітів (Report View) інформацію про помилки, що виникають під час виконання певних скриптів Python.



7. **Одиниці вимірювання:** тут можна встановити одиниці вимірювання, які використовуватимуться за замовчуванням. Надалі, під час роботи з прикладами, буде зручніше дотримуватися системи CI.



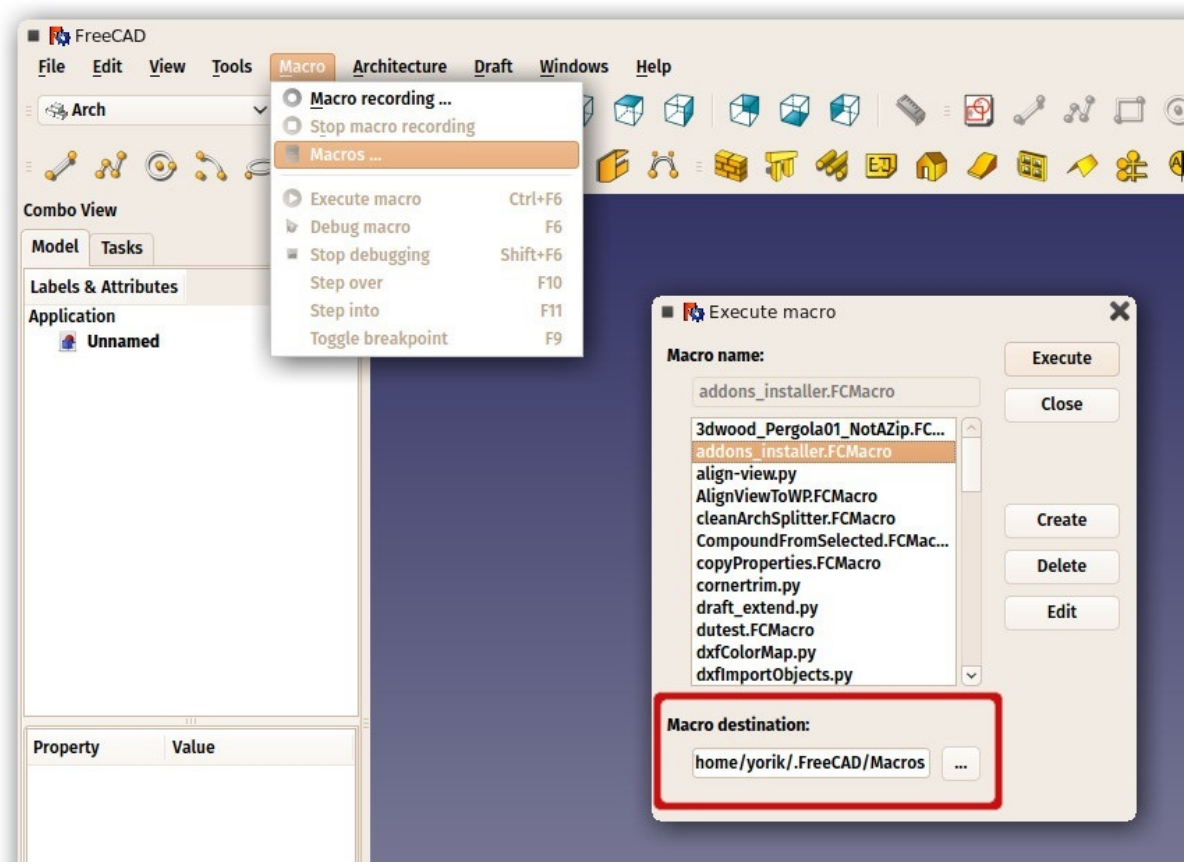
Встановлення додаткового вмісту

Оскільки проєкт FreeCAD та його спільнота стрімко розвиваються, а сама програма вирізняється легкістю розширення, в інтернеті з'являється дедалі більше сторонніх розробок і додаткових проєктів, створених учасниками спільноти та іншими ентузіастами. Наразі триває робота зі збору всіх цих цікавих доповнень в одному місці — на сторінці FreeCAD у GitHub. Там, серед іншого, ви знайдете:

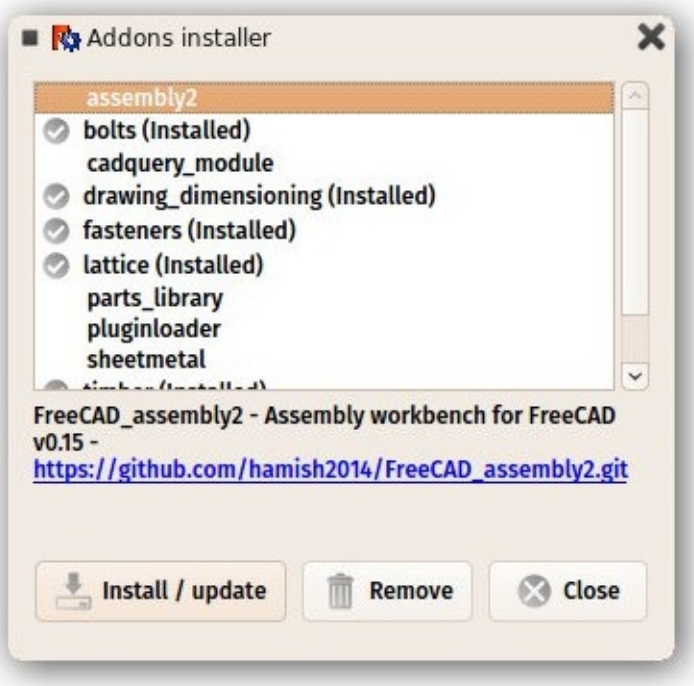
1. Бібліотеку деталей, що містить різноманітні корисні моделі (або їхні фрагменти), створені користувачами FreeCAD; їх можна вільно використовувати у власних проєктах. Доступ до бібліотеки здійснюється безпосередньо з інтерфейсу встановленої програми FreeCAD.
2. Колекцію доповнень — переважно додаткових робочих середовищ (workbenches), — які розширюють функціональні можливості FreeCAD для виконання певних завдань. Інструкції зі встановлення наведено на сторінці кожного окремого доповнення.
3. Колекцію макросів, котра доступна також у вікі FreeCAD разом із документацією щодо їх використання. У вікі можна знайти значно більше макросів.

Якщо ви використовуєте Ubuntu або будь-який з її похідних дистрибутивів, у репозиторії PPA «FreeCAD-extras» міститься більшість із цих доповнень. На інших платформах будь-яке з доповнень — зокрема й бібліотеку деталей (Parts library) — можна легко встановити за допомогою макросу «addon-installer», доступного в репозиторії доповнень. Нижче описано процедуру встановлення «addon-installer» (у такий самий спосіб можна встановлювати й інші макроси).

1. Завантажте файл `addons-installer.FCMacro` з https://github.com/FreeCAD/FreeCAD-addons: клацніть на нього, потім натисніть правою кнопкою миші на кнопку «RAW» і виберіть «Зберегти як».
2. Розмістіть цей макрос у папці для макросів FreeCAD. Шлях до цієї папки вказано в нижній частині діалогового вікна **«Виконати макрос» (Execute macro)** у програмі FreeCAD:



3. Закрийте та знову відкрийте діалогове вікно **«Виконати макрос» (Execute macro)** і запустіть **`addons_installer.FCMacro`**. Відкриється інсталятор, за допомогою якого ви зможете встановлювати, оновлювати та видаляти будь-які доповнення:

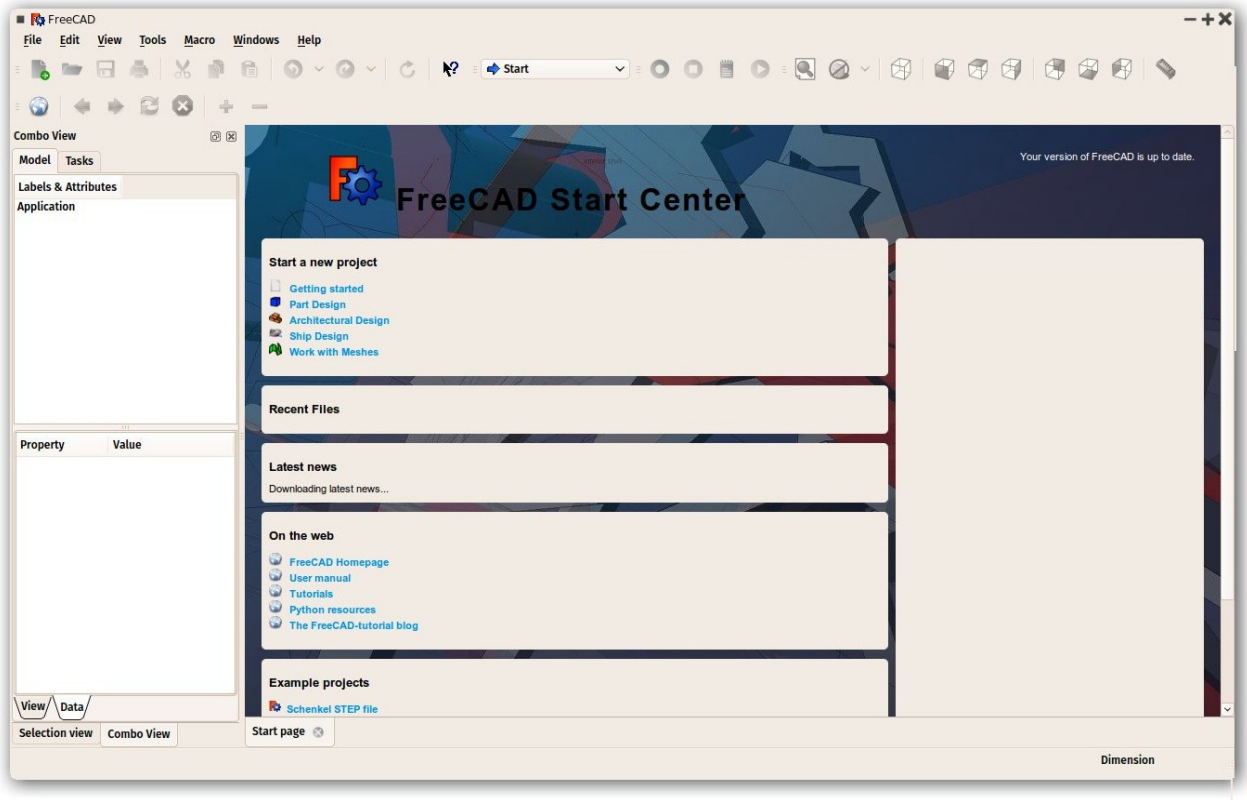


Читати далі

- Додаткові варіанти завантаження: http://www.freecadweb.org/wiki/index.php?title=Download
- Детальні інструкції зі встановлення: http://www.freecadweb.org/wiki/index.php?title=Installing
- PPA-репозиторій FreeCAD для Ubuntu: https://launchpad.net/~freecad-maintainers
- PPA-репозиторій додатків FreeCAD для Ubuntu: https://launchpad.net/freecad-extras
- Самостійна компіляція FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Compiling
- Переклади FreeCAD: https://crowdin.com/project/freecad
- Сторінка FreeCAD на GitHub: https://github.com/FreeCAD

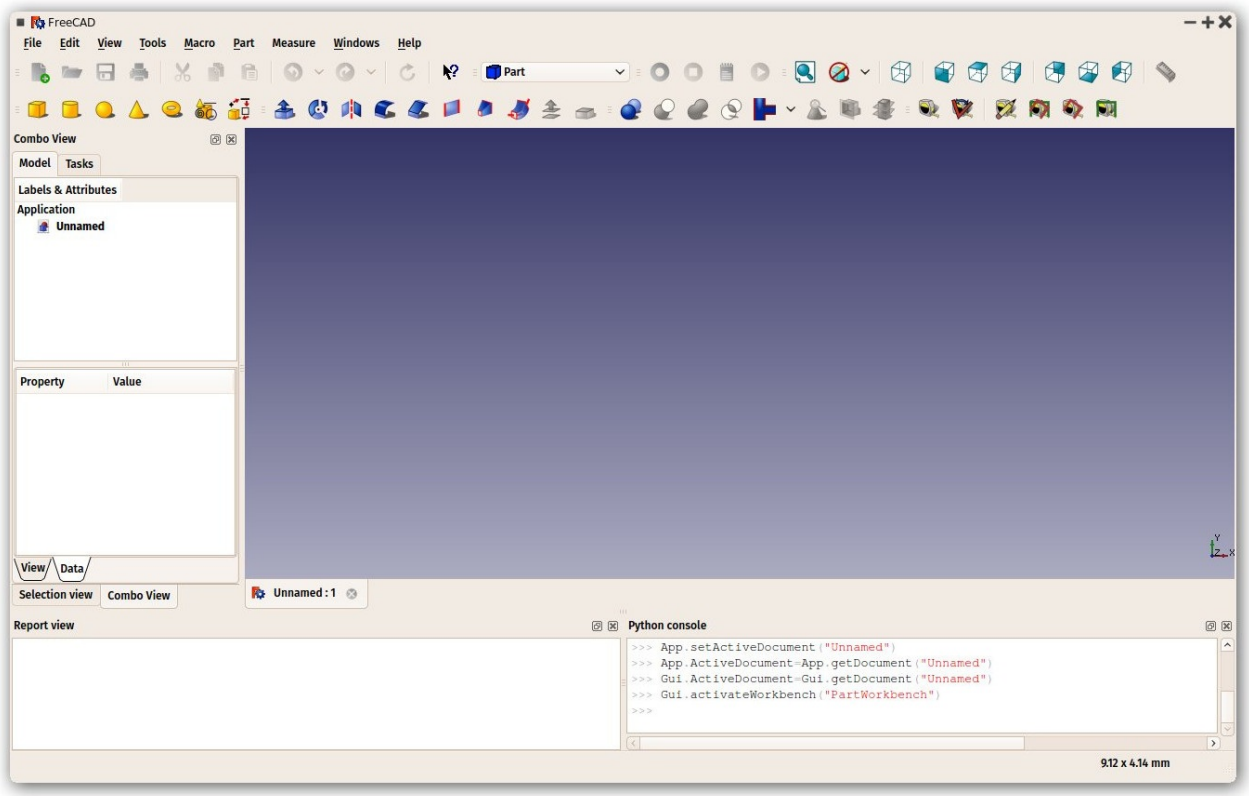
Інтерфейс FreeCAD

Для створення та керування інтерфейсом FreeCAD використовується фреймворк Qt. Цей фреймворк застосовується в багатьох програмах, тому інтерфейс FreeCAD є цілком класичним і не викликає жодних труднощів у розумінні. Більшість кнопок — стандартні, і ви знайдете їх саме там, де й очікуєте (Файл -> Відкрити, Редагування -> Вставити тощо). Ось як виглядає FreeCAD під час першого запуску відразу після встановлення — на екрані відображається «Центр запуску» (Start Center):



Центр запуску — це зручний «екран привітання», де відображається корисна для новачків інформація: наприклад, останні файли, з якими ви працювали, новини зі світу FreeCAD або стислі відомості про найпопулярніші робочі середовища (Workbenches). Також він сповістить вас, якщо з’явиться нова стабільна версія FreeCAD.

Закрийте вкладку «Початкова сторінка» (натисніть на «x» на вкладці, ближче до нижньої частини) і створіть новий документ (Ctrl-N):

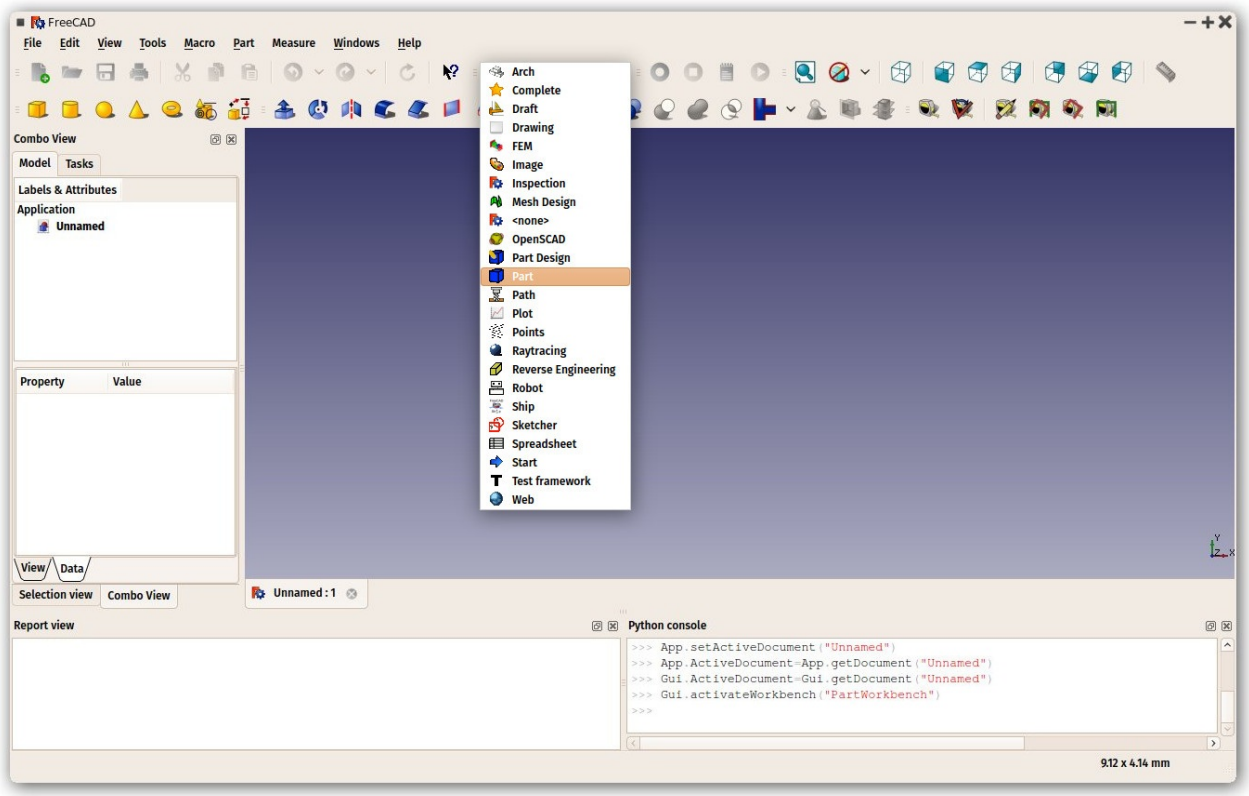


Робочі середовища

Зверніть увагу, що деякі піктограми на двох наведених вище знімках екрана відрізняються. Тут у гру вступає найважливіша концепція інтерфейсу FreeCAD — «Робочі середовища» (Workbenches).

Робочі середовища — це набори інструментів (кнопки на панелях інструментів, пункти меню та інші елементи керування інтерфейсом), згруповані за функціональним призначенням. Уявіть собі майстерню, де працюють різні фахівці: один займається металом, інший — деревом. У кожного з них є окремий робочий стіл зі спеціальними інструментами, необхідними для виконання його завдань. При цьому всі вони можуть працювати з одними й тими самими об'єктами. Те саме відбувається і у FreeCAD.

Найважливішим елементом керування в інтерфейсі FreeCAD є перемикач робочих середовищ, за допомогою якого можна переходити від одного робочого середовища до іншого:

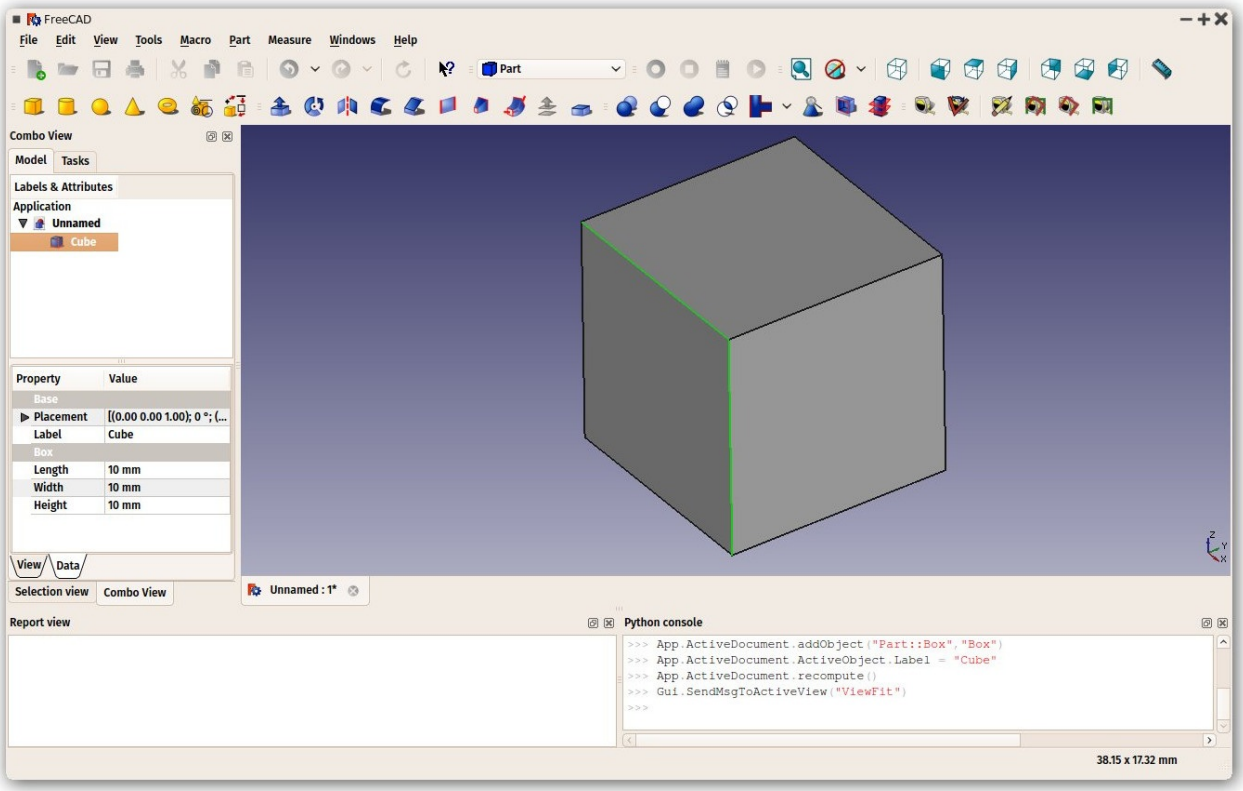


Робочі середовища (Workbenches) часто спантеличують нових користувачів, оскільки не завжди легко зрозуміти, у якому саме середовищі шукати потрібний інструмент. Проте опанувати їх можна досить швидко, і вже незабаром робота з ними стане цілком природною. Нові користувачі швидко усвідомлюють, що робочі середовища — це зручний спосіб упорядкувати величезну кількість інструментів, які пропонує FreeCAD. Крім того, робочі середовища можна повністю налаштувати під власні потреби (див. нижче).

Далі в цьому посібнику ви знайдете таблицю, у якій показано типовий вміст робочих середовищ.

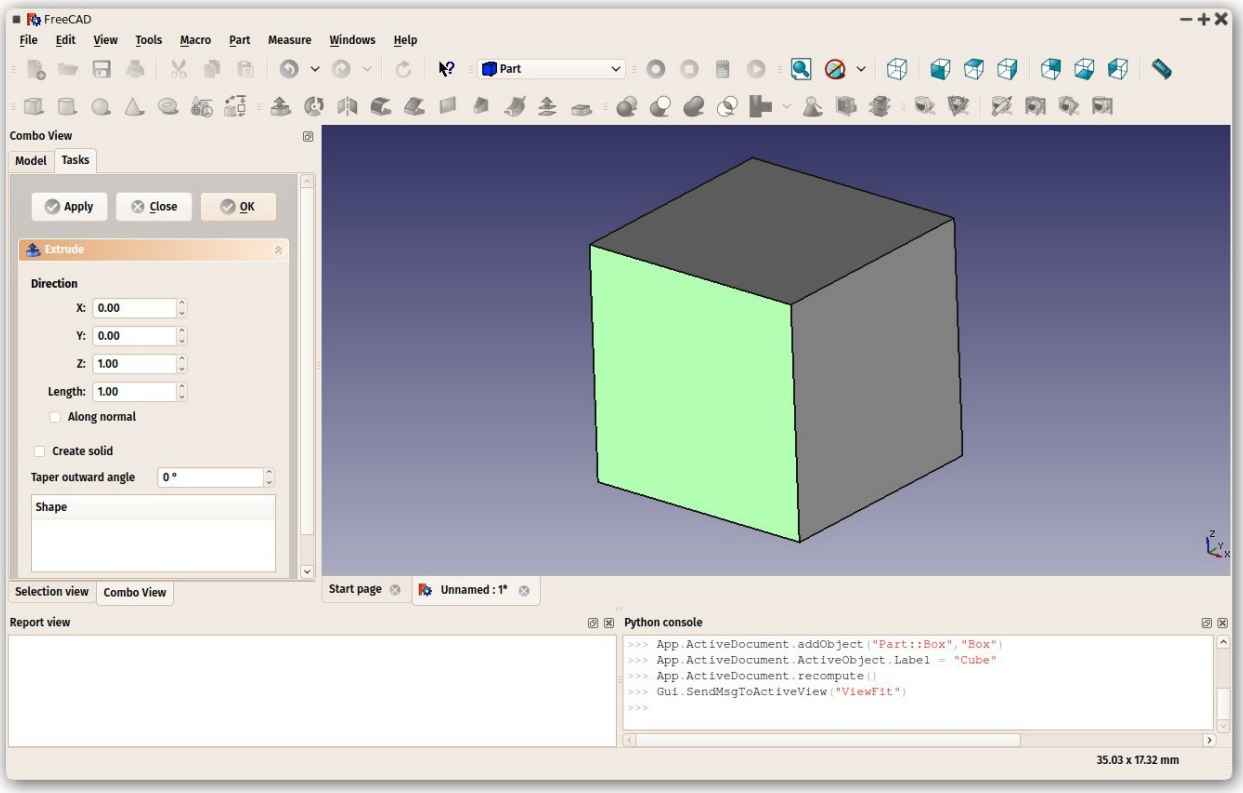
Інтерфейс

Розгляньмо детальніше різні елементи інтерфейсу:



- **3D-вид** є основним компонентом інтерфейсу. Його можна відокремити від головного вікна; також можна відкрити кілька видів одного й того ж документа (або одних і тих самих об'єктів) або ж працювати з кількома документами одночасно. Ви можете вибирати об'єкти або їхні частини, клацаючи по них, а також переміщувати, масштабувати та обертати вид за допомогою кнопок миші. Детальніше про це йтиметься в наступному розділі.
- **Комбінована панель (Combo View)** у лівій частині вікна містить дві вкладки:
  - Вкладка «Модель» (Model) відображає вміст і структуру вашого документа (див. вище), а також властивості (або параметри) вибраного об'єкта (або об'єктів) (див. нижче). Властивості на цій вкладці поділяються на дві категорії:
    - Дані (Data) — властивості, що стосуються безпосередньо геометрії
    - Вид (View) — властивості, що визначають зовнішній вигляд геометрії на екрані.
  - На вкладці «Завдання» (Tasks) FreeCAD запитує значення, необхідні для інструмента, який ви використовуєте в даний момент — наприклад, введення значення «довжини» під час роботи з інструментом «Лінія». Ця панель автоматично закривається після натискання кнопки «OK» (або «Скасувати»). Крім того, для більшості інструментів передбачена можливість знову відкрити панель завдань — для цього достатньо двічі клацнути відповідний об'єкт у комбінованій панелі, що дозволить змінити налаштування.
- **Панель звітів (Report View)** зазвичай прихована, проте її варто тримати відкритою: у ній відображаються повідомлення, попередження або помилки, які допоможуть вам зрозуміти (або діагностувати), що саме пішло не так. (Меню «Вид» -> «Панелі» -> позначте пункт «Панель звітів»).
- **Консоль Python** також за замовчуванням прихована. Тут ви можете взаємодіяти з вмістом документа за допомогою мови програмування Python. Оскільки кожна дія в інтерфейсі FreeCAD фактично виконує певний фрагмент коду Python, відкриття цієї консолі дозволяє спостерігати за виконанням коду в режимі реального часу. Це чудовий і простий спосіб попутно вивчити основи Python, майже не помічаючи самого процесу навчання.

(Меню «Вигляд» -> «Панелі» -> позначте пункт «Консоль Python»).

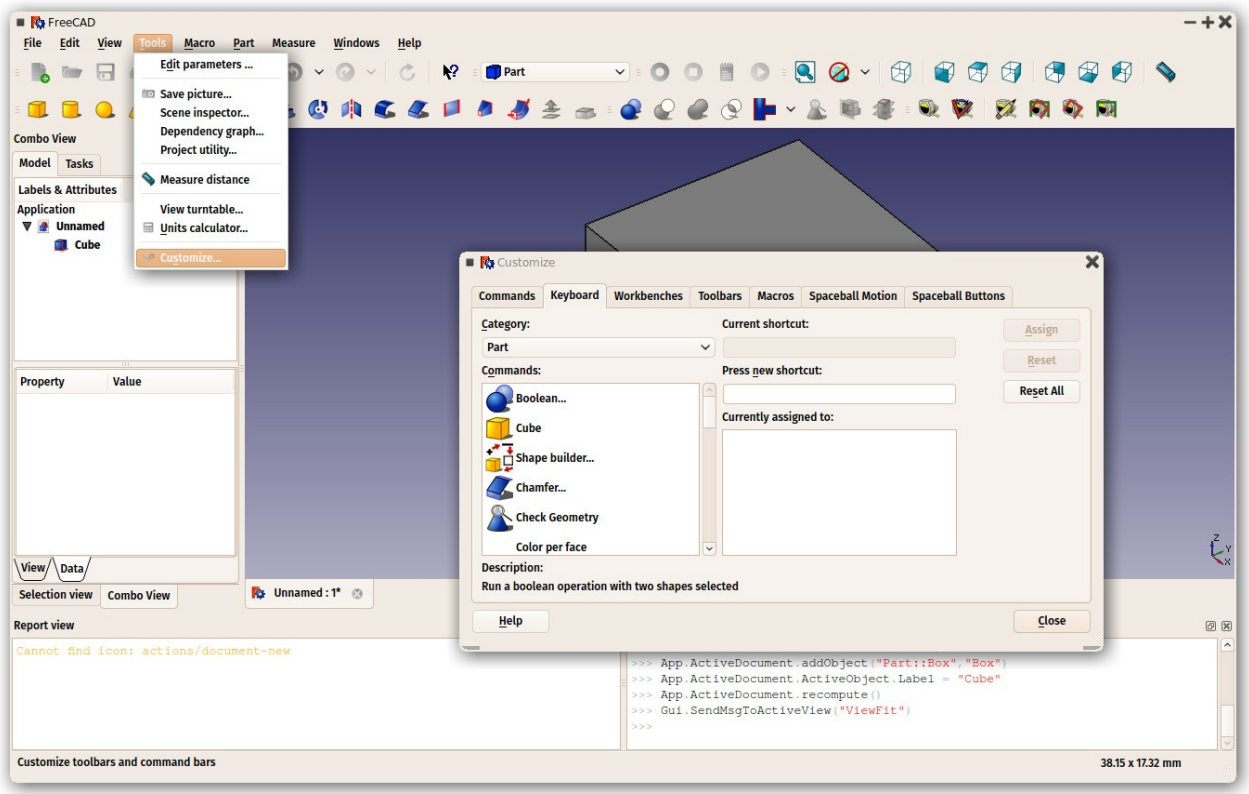


Будь-яку з наведених вище панелей можна ввімкнути або вимкнути в меню «Вигляд» -> «Панелі».

Налаштування інтерфейсу

Інтерфейс FreeCAD можна гнучко налаштовувати. Усі панелі та панелі інструментів можна переміщувати в різні місця або розміщувати одну над одною. Їх також можна закривати, а за потреби — знову відкривати через меню «Вигляд» (View) або клацнувши правою кнопкою миші на порожній ділянці інтерфейсу. Крім того, доступно ще багато інших можливостей: наприклад, створення власних панелей інструментів із набором інструментів із будь-якого робочого середовища (Workbench), а також призначення чи зміна комбінацій клавіш.

Ці розширені параметри налаштування доступні в меню «Інструменти» -> «Налаштувати» (Tools -> Customize):



Читати далі

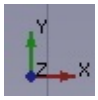
- Початок роботи з FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Getting_started
- Налаштування інтерфейсу: http://www.freecadweb.org/wiki/index.php?title=Interface_Customization
- Робочі середовища: http://www.freecadweb.org/wiki/index.php?title=Workbenches
- Більше про Python: https://www.python.org

Навігація в 3D-просторі

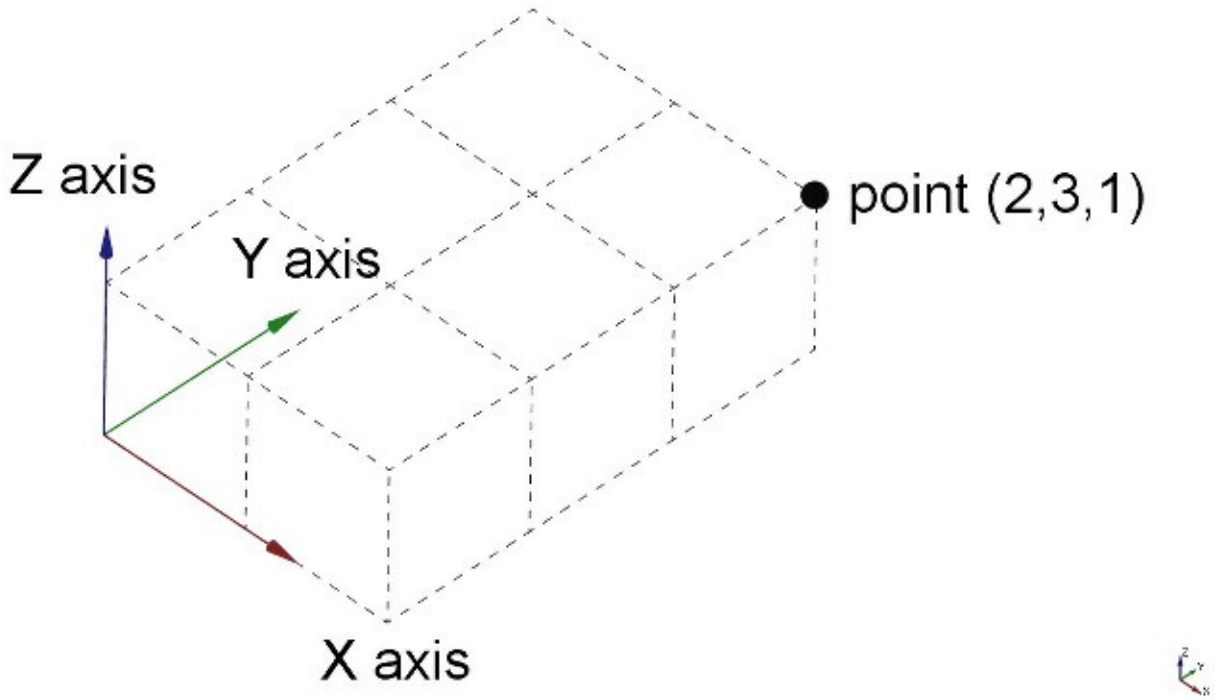
Кілька слів про 3D-простір

Якщо ви вперше працюєте з 3D-програмою, вам спочатку варто ознайомитися з деякими базовими поняттями. Якщо ж ні — можете сміливо пропустити цей розділ.

3D-простір у FreeCAD — це евклідів простір. Він має точку відліку (початок координат) і три осі: X, Y та Z. Якщо поглянути на сцену зверху, то, за загальноприйнятою домовленістю, вісь X спрямована праворуч, вісь Y — углиб (назад), а вісь Z — угору. У правому нижньому куті вікна перегляду FreeCAD завжди можна побачити, з якого ракурсу ви розглядаєте сцену:



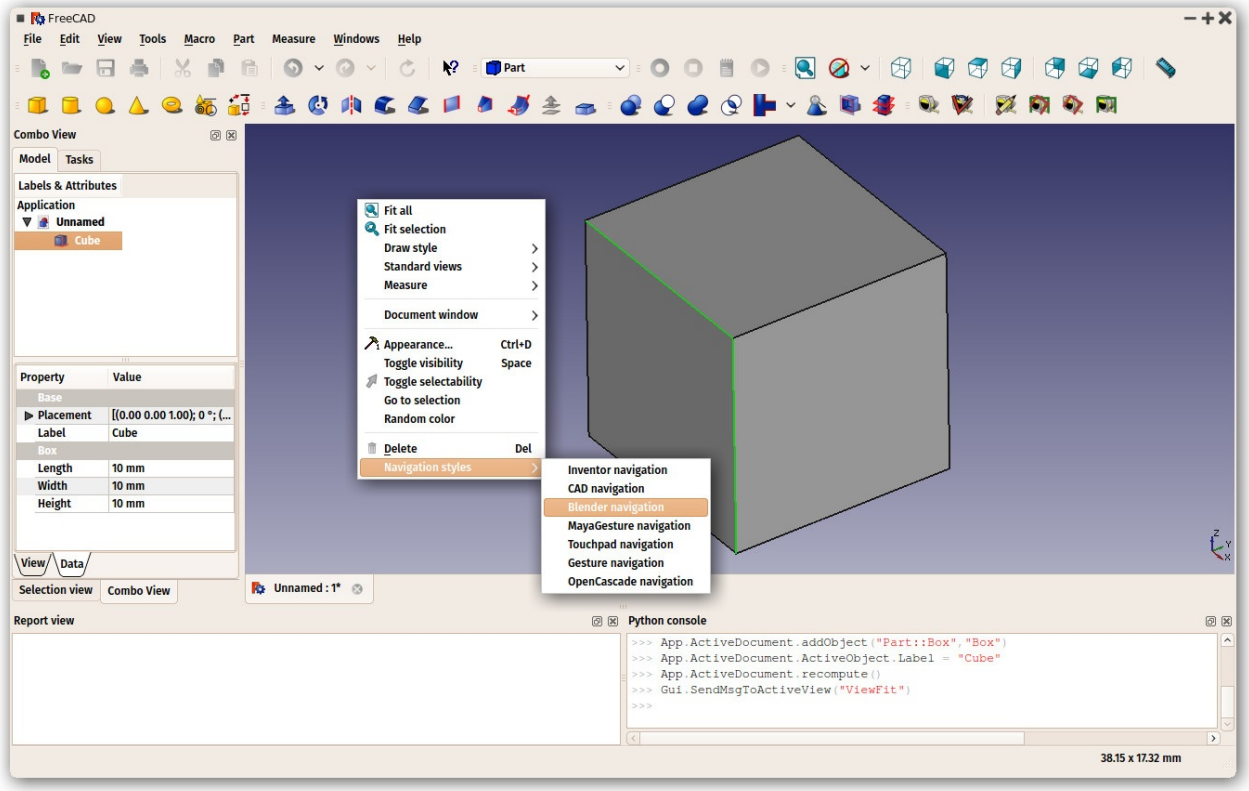
Будь-яку точку будь-якого об'єкта в цьому просторі можна визначити за допомогою координат (x, y, z). Наприклад, точка з координатами (2, 3, 1) розташована на відстані 2 одиниць уздовж осі X, 3 одиниць уздовж осі Y та 1 одиниці вздовж осі Z:



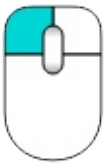
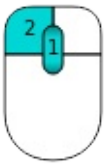
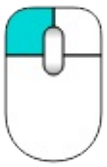
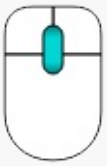
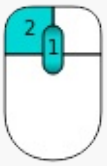
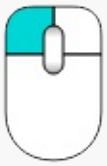
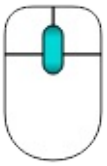
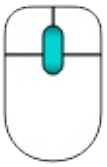
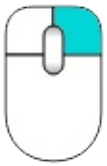
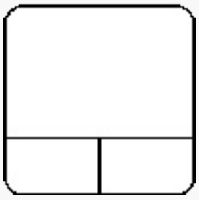
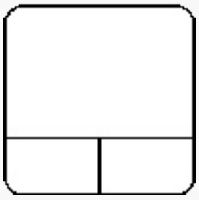
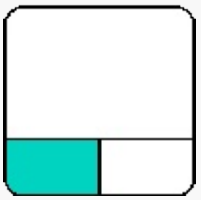
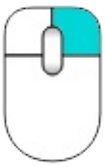
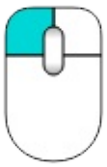
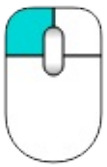
Ви можете розглядати сцену з будь-якого ракурсу — так, ніби тримаєте в руках камеру. Цю камеру можна переміщувати вліво, вправо, вгору та вниз (панорамування), обертати навколо об'єкта, на який вона спрямована (обертання), а також наближати до сцени або віддаляти від неї (масштабування).

3D-простір у FreeCAD

Навігацію в 3D-вигляді FreeCAD можна здійснювати за допомогою миші, маніпулятора Space Navigator, клавіатури, сенсорної панелі або їх комбінації. У FreeCAD доступно кілька режимів навігації, які визначають спосіб виконання трьох основних операцій із зображенням (переміщення, обертання та масштабування), а також порядок вибору об'єктів на екрані. Вибрати режим навігації можна в меню налаштувань (Preferences) або безпосередньо в 3D-вигляді, клацнувши правою кнопкою миші в будь-якому місці:



У кожному з цих режимів із чотирма зазначеними операціями пов’язані різні кнопки миші, комбінації миші й клавіатури або жести миші. У наведеній нижче таблиці представлено основні доступні режими:

Mode	Pan	Rotate	Zoom	Select
Inventor				CTRL + 
CAD (default)				
Blender	SHIFT + 			
Touchpad	SHIFT + 	ALT + 	PGUP / PGDOWN	
Gesture	 + DRAG	 + DRAG		

Крім того, деякі елементи керування за допомогою клавіатури доступні завжди, незалежно від режиму навігації:

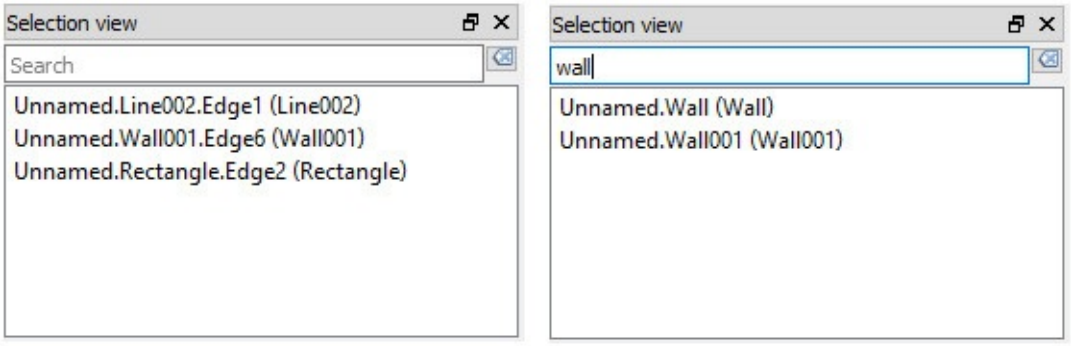
- **CTRL +** та **CTRL –** — для збільшення й зменшення масштабу;
- **клавіші зі стрілками** — для переміщення (панорамування) вигляду вліво/вправо та вгору/вниз;
- цифрові клавіші **від 1 до 6** — для переходу до шести стандартних видів: зверху, спереду, справа, знизу, ззаду та зліва;
- клавіша **O** перемикає камеру в ортографічний режим;
- клавіша **P** перемикає її в перспективний режим;
- клавіша **CTRL** дозволяє вибирати кілька об’єктів або елементів одночасно.

Ці функції також доступні в меню «Вид» (View), а деякі з них — на панелі інструментів «Вид».

Вибір об'єктів

Об'єкти в 3D-вигляді можна вибрати, клацнувши на них відповідною кнопкою миші, залежно від режиму навігації. (Для решти посібника ми вважатимемо, що використовується навігація CAD за замовчуванням.) Одне клацання миші вибере об'єкт та один з його підкомпонентів (ребро, грань, вершина). Подвійне клацання миші вибере об'єкт та всі його підкомпоненти. Ви можете вибрати більше одного підкомпонента або навіть різні підкомпоненти з різних об'єктів, натиснувши клавішу CTRL. Якщо вибрано кілька елементів, утримування клавіші CTRL натиснутою та натискання вибраного елемента видаляє його з вибору. Клацання кнопкою вибору на порожній частині 3D-вигляду зніме вибір з усього.

Також можна ввімкнути панель під назвою "Вид вибору", доступну в меню "Вигляд", яка показує, що наразі вибрано:



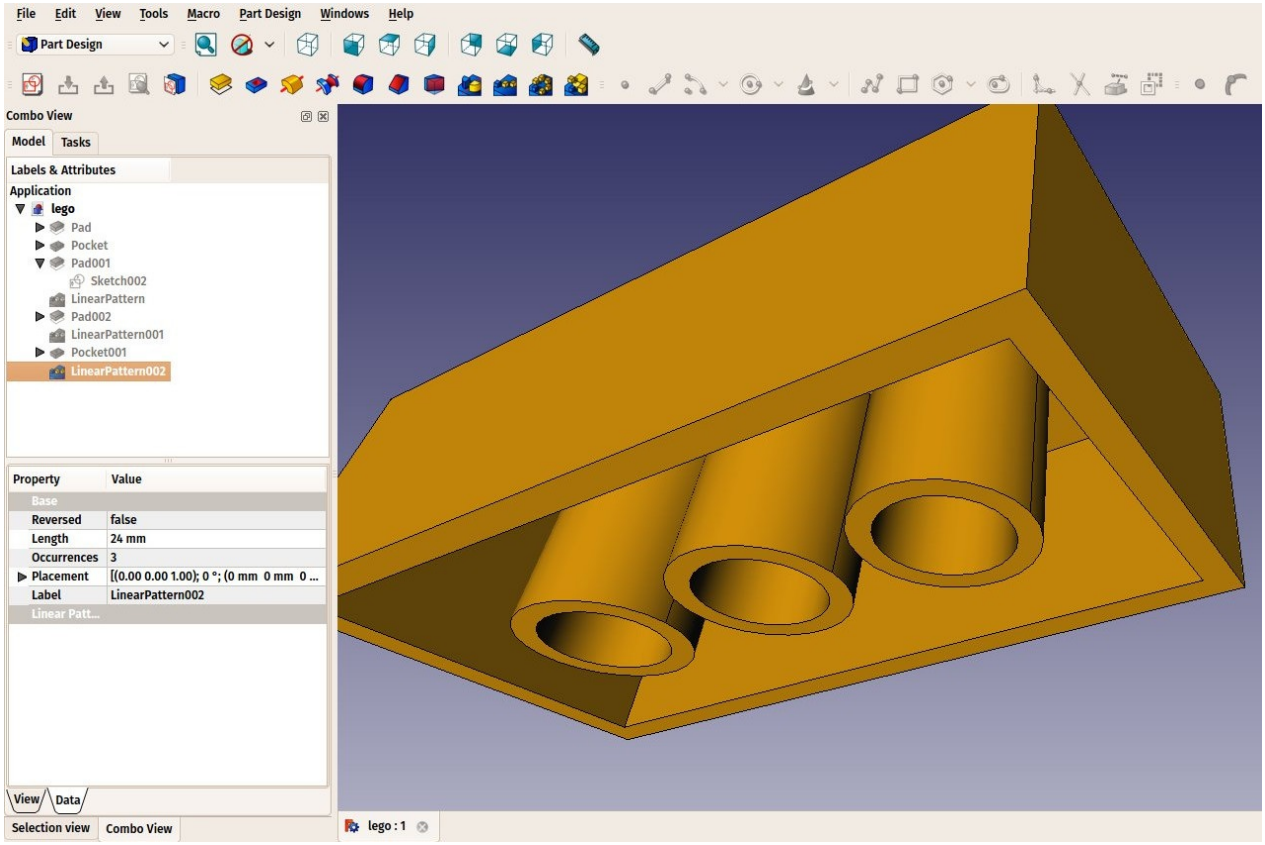
Ви також можете використовувати Вигляд вибору для вибору об'єктів, шукаючи певний об'єкт.

Читати далі

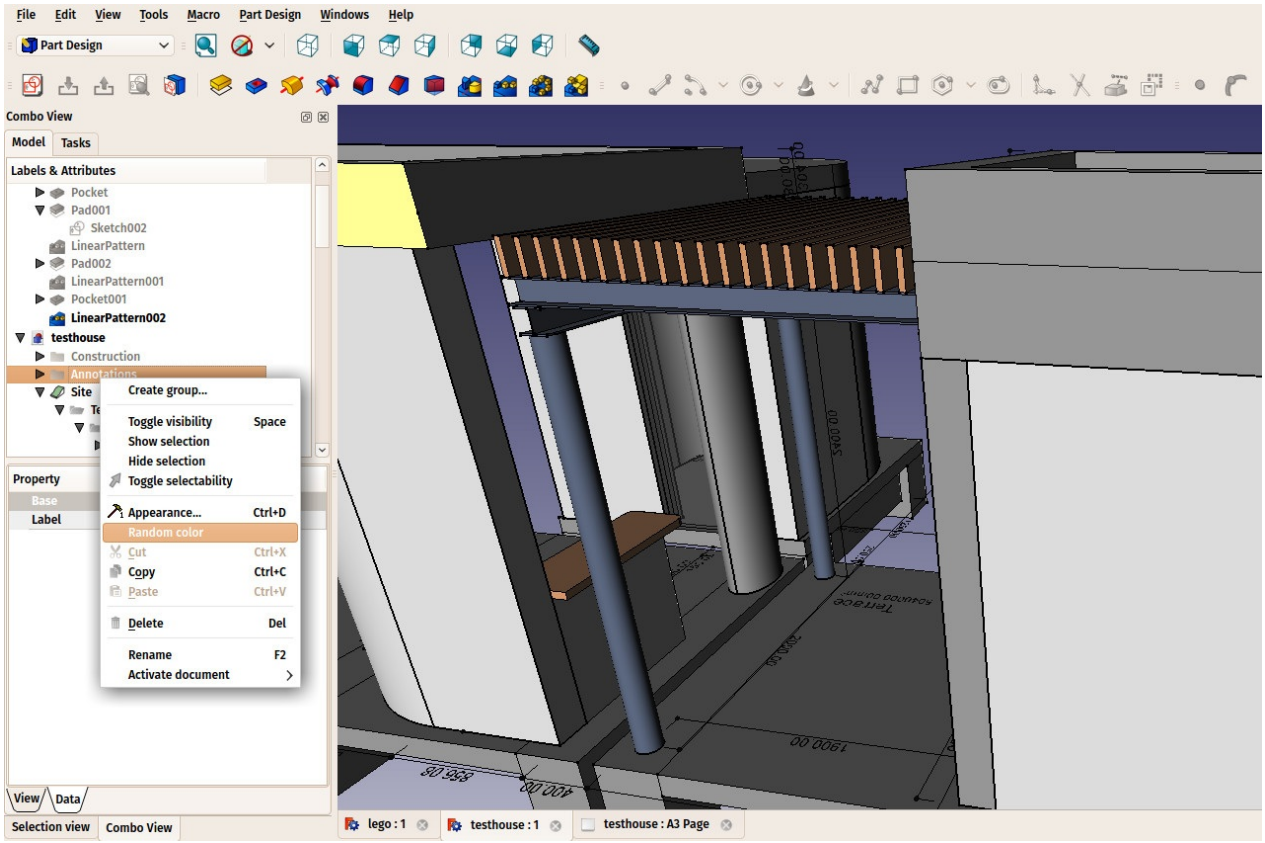
- Режими навігації FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Mouse_Model

Документ FreeCAD

Документ FreeCAD містить усі об'єкти вашої сцени. Він може включати групи та об'єкти, створені за допомогою будь-якого робочого середовища (верстака). Таким чином, ви можете перемикатися між різними робочими середовищами, продовжуючи працювати з тим самим документом та/або об'єктами в ньому. Саме документ зберігається на диску, коли ви зберігаєте свою роботу. У FreeCAD також можна відкривати кілька документів одночасно, а також створювати кілька вікон перегляду для одного й того ж документа.



Усередині документа об'єкти можна об'єднувати в групи; кожен об'єкт має унікальну назву. Керування групами, об'єктами та їхніми назвами здійснюється переважно через дерево проєкту (Tree view). Там ви можете створювати групи, переміщувати в них об'єкти, а також видаляти об'єкти чи групи. Клацнувши правою кнопкою миші в дереві проєкту або на конкретному об'єкті, ви можете перейменувати об'єкт, змінити його колір, приховати або відобразити його, а також виконати інші дії — залежно від того, яке робоче середовище активоване в цей момент.



Об'єкти в документі FreeCAD можуть належати до різних типів. Кожен робочий верстат (workbench) може додавати власні типи об'єктів: наприклад, верстат «Сітка» (Mesh) додає об'єкти типу «сітка», верстат «Деталь» (Part) — об'єкти типу «деталь» тощо.

У програмі завжди є один — і тільки один — активний документ. Це той документ, який відображається в поточному 3D-вікні; саме з ним ви працюєте в даний момент. Якщо ви перемкнетеся на вкладку з іншим документом, він стане активним. Більшість операцій виконуються саме з активним документом.

Документи FreeCAD зберігаються з розширенням `.FcStd`. Це складений формат на основі архіву ZIP — подібний до того, що використовується в LibreOffice. Якщо виникне серйозна помилка, часто можна розпакувати такий файл, щоб усунути проблему або відновити дані.

Докладніше

- Документ FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Document_structure
- Формат файлу .FcStd: http://www.freecadweb.org/wiki/index.php?title=File_Format_FCStd

Параметричні об'єкти

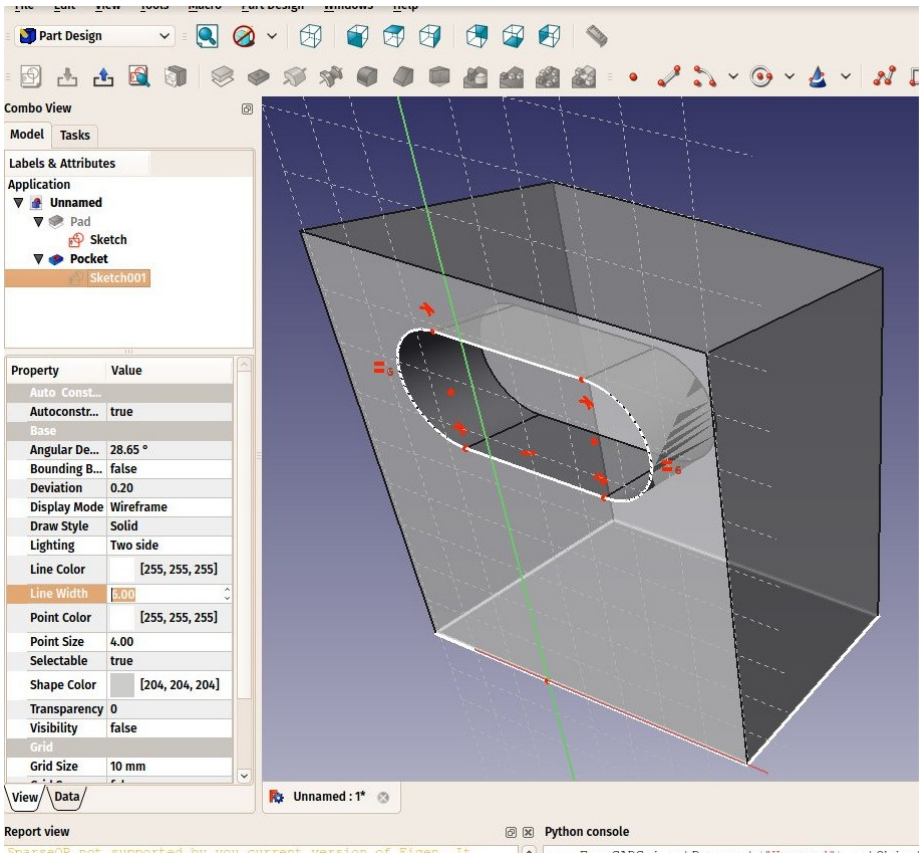
FreeCAD призначений для параметричного моделювання. Це означає, що створювана вами геометрія не формується шляхом вільного «ліплення», а будується на основі певних правил і параметрів. Наприклад, циліндр може бути побудований на основі радіуса та висоти. Маючи ці два параметри, програма отримує достатньо інформації для побудови циліндра.

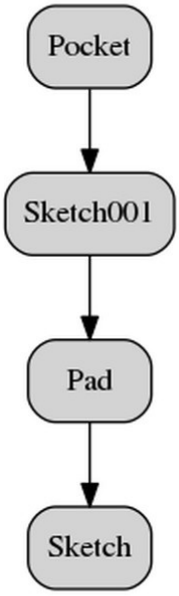
У FreeCAD параметричні об'єкти — це, по суті, невеликі фрагменти програмного коду, які виконуються щоразу, коли змінюється будь-який із параметрів. Об'єкти можуть мати найрізноманітніші типи параметрів: числа (цілі, як-от 1, 2, 3, або числа з плаваючою комою, наприклад 3,1416), фізичні розміри (1 мм, 2,4 м, 4,5 фута), координати (x, y, z), текстові рядки («hello!») або навіть інші об'єкти.

Останній тип дозволяє швидко створювати складні ланцюжки операцій, де кожен новий об'єкт базується на попередньому, доповнюючи його новими властивостями.

У наведеному нижче прикладі суцільний кубічний об'єкт (Pad — «видавлювання») базується на прямокутній 2D-фігурі (Sketch — «ескіз») і має задану відстань видавлювання. Завдяки цим двом властивостям програма створює об'ємну фігуру, «видавлюючи» базовий контур на вказану відстань. Надалі цей об'єкт можна використовувати як основу для подальших операцій: наприклад, побудувати нову 2D-фігуру на одній із його граней (Sketch001), а потім виконати віднімання (Pocket — «виріз»), доки не буде отримано кінцевий об'єкт.

Усі проміжні операції (2D-фігури, видавлювання, вирізи тощо) зберігаються в системі, і ви можете будь-коли змінити будь-який із їхніх параметрів. Увесь ланцюжок буде автоматично перебудовано (перераховано) за потреби.





Важливо знати дві речі:

1. Переобчислення не завжди відбувається автоматично. «Важкі» операції, які можуть змінити значну частину вашого документа і, відповідно, потребують певного часу, не виконуються автоматично. Натомість об'єкт (а також усі об'єкти, що від нього залежать) позначається для переобчислення (у дереві об'єктів на ньому з'являється невелика синя піктограма). Щоб виконати переобчислення всіх позначених об'єктів, потрібно натиснути кнопку переобчислення (клавіша F5 або піктограма з двома синіми стрілками, розташованими одна за одною).
2. Дерево залежностей завжди має спрямовуватися в один бік. Циклічні залежності заборонені. Об'єкт A може залежати від об'єкта B, який, своєю чергою, залежить від об'єкта C. Але неможлива ситуація, коли об'єкт A залежить від об'єкта B, а той — від об'єкта A; це називається циклічною залежністю. Водночас багато об'єктів можуть залежати від одного й того самого об'єкта (наприклад, об'єкти B і C можуть обидва залежати від об'єкта A). У меню «Інструменти» (Tools) -> «Граф залежностей» (Dependency graph) можна переглянути схему залежностей, подібну до наведеної вище. Це може бути корисним для виявлення проблем.

У FreeCAD не всі об'єкти є параметричними. Часто геометрія, імпортована з інших файлів, не містить жодних параметрів і являє собою прості, непараметричні об'єкти. Проте їх часто можна використовувати як основу або відправну точку для створення нових параметричних об'єктів — звісно, залежно від вимог самого параметричного об'єкта та якості імпортованої геометрії.

Усі об’єкти — незалежно від того, чи є вони параметричними — мають кілька базових параметрів: «Назва» (Name) — унікальний ідентифікатор у межах документа, який не можна редагувати; «Мітка» (Label) — назва, що задається користувачем і яку можна змінювати; а також «Розміщення» (Placement), що визначає положення об’єкта у тривимірному просторі.

Насамкінець варто зазначити, що власні параметричні об’єкти легко створювати за допомогою мови програмування Python.

Детальніше

- Редактор властивостей: http://www.freecadweb.org/wiki/index.php?title=Property_editor
- Програмування параметричних об’єктів: http://www.freecadweb.org/wiki/index.php?title=Scripted_objects
- Розміщення об’єктів у FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Placement
- Увімкнення графа залежностей: http://www.freecadweb.org/wiki/index.php?title=Std_DependencyGraph

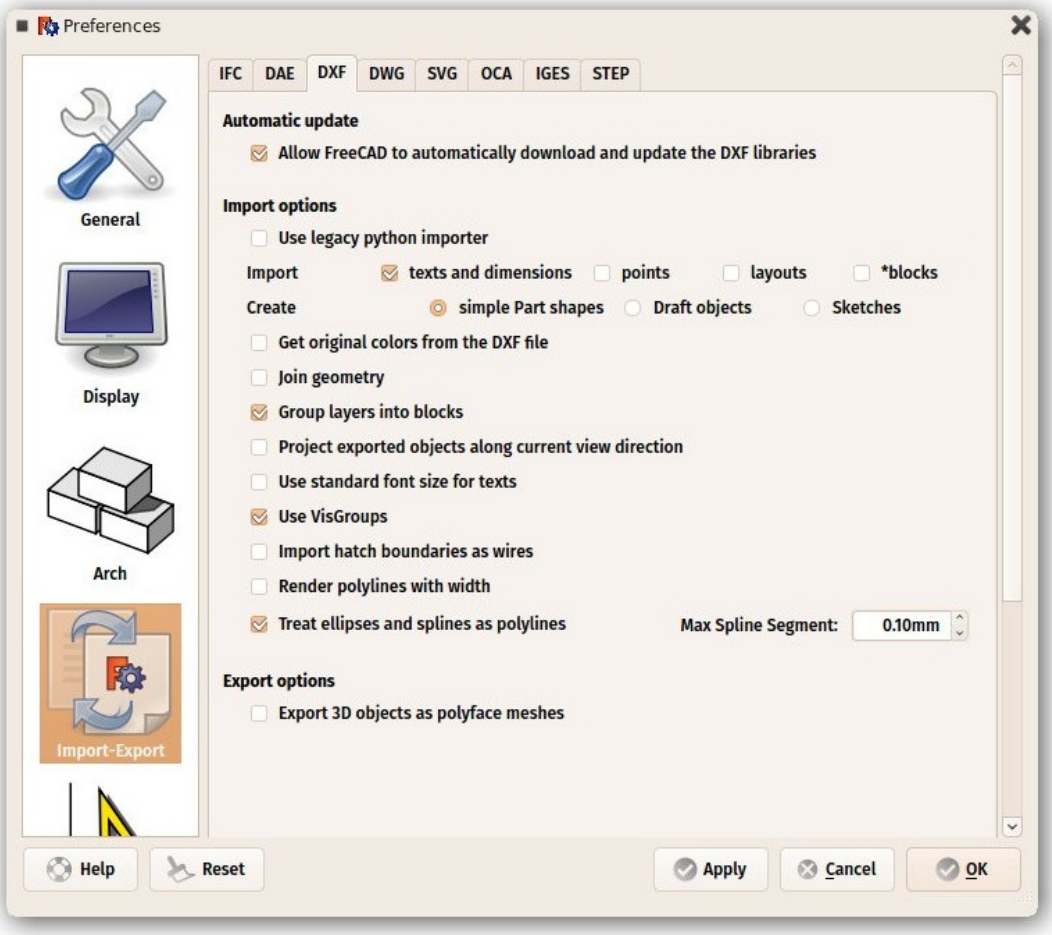
Імпорт та експорт у файли інших форматів

FreeCAD дозволяє імпортувати та експортувати файли багатьох форматів. Нижче наведено перелік найважливіших із них із коротким описом доступних можливостей:

Формат	Імпорт	Експорт	Примітки
STEP	Так	Так	Це найбільш точний формат для імпорту та експорту, оскільки він підтримує твердотільну геометрію та NURBS. Використовуйте його, коли це можливо.
IGES	Так	Так	Старіший формат для твердотільних моделей, який також має дуже хорошу підтримку. Деякі застарілі програми не підтримують STEP, але працюють з IGES.
BREP	Так	Так	Рідний формат OpenCasCade — геометричного ядра FreeCAD.
DXF	Так	Так	Відкритий формат, що підтримується компанією Autodesk. Оскільки 3D-дані у файлах DXF закодовані у пропрієтарному (закритому) форматі, FreeCAD може імпортувати та експортувати лише 2D-дані.
DWG	Так	Так	Пропрієтарний формат файлів. Потребує встановлення утиліти Teigha File Converter. Цей формат має ті самі обмеження, зумовлені його закритою природою, що й DXF.
OBJ	Так	Так	Формат на основі сіток (mesh). Може містити лише тріангульовані сітки. Під час експорту всі твердотільні об'єкти та об'єкти на основі NURBS у FreeCAD перетворюються на сітки. Робоче середовище (верстат) Arch пропонує альтернативний інструмент експорту, який краще підходить для архітектурних моделей.
DAE	Так	Так	Основний формат імпорту та експорту для SketchUp. Може містити лише тріангульовані сітки. Під час експорту всі твердотільні об'єкти та об'єкти на основі NURBS у FreeCAD перетворюються на сітки.
STL	Так	Так	Формат на основі сіток, що часто використовується для 3D-друку. Може містити лише тріангульовані сітки. Під час експорту всі твердотільні об'єкти та об'єкти на основі NURBS у FreeCAD перетворюються на сітки.
PLY	Так	Так	Застарілий формат на основі сіток. Може містити лише тріангульовані сітки. Під час експорту всі твердотільні об'єкти та об'єкти на основі NURBS у FreeCAD перетворюються на сітки.
IFC	Так	Так	Industry Foundation Classes (галузевий стандарт для будівельної інформації). Потребує встановлення IfcOpenShell-python. Формат IFC та питання його сумісності з іншими програмами — досить складна тема, тому використовуйте його з обережністю.

SVG	Так	Так	Чудовий і широко поширений формат 2D-графіки.
VRML	Так	Так	Досить старий веб-формат на основі сіток.
GCODE	Так	Так	FreeCAD підтримує імпорт та експорт кількох різновидів G-коду (також відомого як RS-274), проте наразі підтримується лише обмежена кількість верстатів.
CSG	Так	Ні	CSG (конструктивна твердотільна геометрія) в OpenSCAD

Деякі з цих форматів файлів мають додаткові параметри. Їх можна налаштувати в меню: **«Редагування» (Edit) -> «Параметри» (Preferences) -> «Імпорт/експорт» (Import/export)**:



Детальніше

- Усі формати файлів, що підтримуються FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Import_Export
- Робота з файлами DXF у FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Draft_DXF
- Увімкнення підтримки DXF та DWG: http://www.freecadweb.org/wiki/index.php?title=Dxf_Importer_Install
- Робота з файлами SVG у FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Draft_SVG

- Імпорт та експорт у форматі IFC: http://www.freecadweb.org/wiki/index.php?title=Arch_IFC
- OpenCasCade: http://www.opencascade.com
- Teigha File Converter: https://www.opendesign.com/guestfiles
- Формат IFC: http://www.buildingsmart-tech.org/ifc/IFC4/final/html/index.htm
- IfcOpenShell: http://ifcopenshell.org/

Робота з FreeCAD

Огляд робочих середовищ

Однією з найбільших проблем для нових користувачів FreeCAD є розуміння того, в якому робочому середовищі (workbench) шукати той чи інший інструмент. У наведеній нижче таблиці представлено огляд найважливіших робочих середовищ та інструментів, що в них містяться. Повний перелік можна знайти на сторінці кожного робочого середовища в документації FreeCAD.

Чотири робочі середовища розроблені для роботи в парах, причому одне з них повністю входить до складу іншого: середовище Arch містить усі інструменти з Draft, а PartDesign — усі інструменти зі Sketcher. Однак для більшої наочності нижче вони розглядаються окремо.

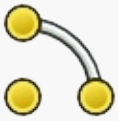
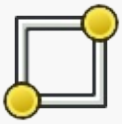
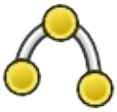
Part

Робоче середовище Part надає базові інструменти для роботи з твердотільними об'єктами: примітивами (наприклад, кубом чи сферою), а також для виконання простих геометричних і булевих операцій. Будучи основним інтерфейсом для взаємодії з ядром OpenCasCade, середовище Part слугує фундаментом геометричної системи FreeCAD; фактично, майже всі інші робочі середовища створюють геометрію на основі об'єктів Part.

Інструмент	Опис	Інструмент	Опис
 Box	Draws a box	 Cone	Draws a cone
 Cylinder	Draws a cylinder	 Sphere	Draws a sphere
 Torus	Draws a torus (ring)	 Create Primitives	Creates various other parametric geometric primitives
 Shape Builder	Create more complex shapes from primitives	 Fuse	Fuses (unions) two objects
 Common	Extracts the common (intersection) part of two objects	 Cut	Cuts (subtracts) one object from another
 Join Connect	Connects interiors of walled objects	 Join Embed	Embeds a walled object into another walled object
 Join Cutout	Creates a cutout in a wall of an object for another walled object	 Extrude	Extrudes planar faces of an object
 Fillet	Fillets (rounds) edges of an object	 Revolve	Creates a solid by revolving another object (not solid) around an axis
 Section	Creates a section by intersecting an object with a section plane	 Section Cross	Creates multiple cross sections along an object
 Chamfer	Chamfers edges of an object	 Mirror	Mirrors the selected object on a given mirror plane
 Ruled Surface	Create a ruled surface between selected curves	 Sweep	Sweeps one or more profiles along a path
 Loft	Lofts from one profile to another	 Offset	Creates a scaled copy of the original object
 Thickness	Assign a thickness to the faces of a shape		

Draft

Робочий стіл Draft надає інструменти для виконання базових завдань 2D-креслення (створення ліній, кіл тощо), а також низку універсальних корисних інструментів, таких як переміщення, обертання або масштабування. Він також пропонує допоміжні засоби для креслення, зокрема сітку та прив'язки. Хоча основне призначення цього модуля — створення допоміжних ліній для об'єктів Arch, він також слугує своєрідним «швейцарським ножем» у FreeCAD.

Tool	Description	Tool	Description
 Line	Draws a line segment between 2 points	 Wire	Draws a line made of multiple line segments (polyline)
 Circle	Draws a circle from center and radius	 Arc	Draws an arc segment from center, radius, start angle and end angle
 Ellipse	Draws an ellipse from two corner points	 Polygon	Draws a regular polygon from a center and a radius
 Rectangle	Draws a rectangle from 2 opposite points	 Text	Draws a multi-line text annotation
 Dimension	Draws a dimension annotation	 BSpline	Draws a B-Spline from a series of points
 Point	Inserts a single point	 Shape String	The ShapeString tool inserts a compound shape representing a text string at a given point in the current document
 Facebinder	Creates a new object from selected faces on existing objects	 Bezier Curve	Draws a Bezier curve from a series of points
 Move	Moves or copies objects from one location to another	 Rotate	Rotates objects by a certain angle around a point

 Offset	Offsets an object to a certain distance	 Trimex	Trims, extends or extrudes an object
 Upgrade	Turns or joins objects into a higher-level object	 Downgrade	Turns or separtes objects into lower-level objects
 Scale	Scales objects in relation to a point	 Shape2D View	Creates a 2D object which is a flattened view of another object
 Draft2Sketch	Converts a Draft object to a Sketch and vice-versa	 Array	Creates a polar or rectangular array from an object
 PathArray	Creates an array from an object by placing copies along a path	 Clone	Creates linked copies of objects
 Mirror	Mirrors objects across a line		

Sketcher

Робоче середовище Sketcher містить інструменти для створення та редагування складних 2D-об'єктів, які називаються ескізами (sketches). Геометричні елементи в таких ескізах можна точно розміщувати та пов'язувати між собою за допомогою обмежень (constraints). Хоча вони насамперед призначені для використання як базові елементи при побудові геометрії в середовищі PartDesign, вони можуть бути корисними в будь-якій частині FreeCAD.

Tool	Description	Tool	Description
 Point	Draws a point	 Line by 2 points	Draws a line segment from 2 points
 Arc	Draws an arc segment from center, radius, start angle and end angle	 Arc by 3 points	Draws an arc segment from two endpoints and another point on the circumference
 Circle	Draws a circle from center and radius	 Circle by 3 points	Draws a circle from three points on the circumference
	Draws an ellipse by center point, major radius point and	 Ellipse	Draws an ellipse by major diameter (2

center	minor radius point	by 3 points	points) and minor radius point
 Arc of ellipse	Draws an arc of ellipse by center point, major radius point, starting point and ending point	 Polyline	Draws a line made of multiple line segments. Several drawing modes available
 Rectangle	Draws a rectangle from 2 opposite points	 Triangle	Draws a regular triangle inscribed in a construction geometry circle
 Square	Draws a regular square inscribed in a construction geometry circle	 Pentagon	Draws a regular pentagon inscribed in a construction geometry circle
 Hexagon	Draws a regular hexagon inscribed in a construction geometry circle	 Heptagon	Draws a regular heptagon inscribed in a construction geometry circle
 Octagon	Draws a regular octagon inscribed in a construction geometry circle	 Slot	Draws an oval by selecting the center of one semicircle and an endpoint of the other semicircle
 Fillet	Makes a fillet between two lines joined at one point	 Trim	Trims a line, circle or arc with respect to a clicked point
 External Geometry	Creates an edge linked to external geometry	 Construction Mode	Toggles an element to/from construction mode. A construction object will not be used in a 3D geometry operation and is only visible while editing the Sketch that contains it
 Coincident constraint	Affixes a point onto (coincident with) one or more other points.	 Point On Object constraint	Affixes a point onto another object such as a line, arc, or axis.
 Vertical constraint	Constrains the selected lines or polyline elements to a true vertical orientation. More than one object can be selected before applying this constraint.	Horizontal constraint	Constrains the selected lines or polyline elements to a true horizontal orientation. More than one object can be selected before applying this constraint.

			constraint.
 Parallel constraint	Constrains two or more lines parallel to one another.	 Perpendicular constraint	Constrains two lines perpendicular to one another, or constrains a line perpendicular to an arc endpoint.
 Tangent constraint	Creates a tangent constraint between two selected entities, or a co-linear constraint between two line segments.	 Equal Length constraint	Constrains two selected entities equal to one another. If used on circles or arcs their radii will be set equal.
 Symmetric constraint	Constrains two points symmetrically about a line, or constrains the first two selected points symmetrically about a third selected point.	 Lock constraint	Constrains the selected item by setting vertical and horizontal distances relative to the origin, thereby locking the location of that item
 Horizontal Distance constraint	Fixes the horizontal distance between two points or line endpoints. If only one item is selected, the distance is set to the origin.	 Vertical Distance constraint	Fixes the vertical distance between 2 points or line endpoints. If only one item is selected, the distance is set to the origin.
 Length constraint	Defines the distance of a selected line by constraining its length, or defines the distance between two points by constraining the distance between them.	 Radius constraint	Defines the radius of a selected arc or circle by constraining the radius.
 Internal Angle constraint	Defines the internal angle between two selected lines.	 Snell's Law constraint	Constrains two lines to obey a refraction law to simulate the light going through an interface
 Internal Alignment constraint	Aligns selected elements to selected shape (e.g. a line to become major axis of an ellipse)	 Map sketch to face	Maps a sketch to the previously selected face of a solid
 Merge	Merge two or more sketches	 Mirror	Mirrors selected elements of a sketch

Part Design

Робоче середовище Part Design містить розширені інструменти для створення твердотільних деталей. До нього також входять усі інструменти з модуля ескізування (Sketcher). Оскільки в цьому середовищі можна створювати лише твердотільні об'єкти (це головне правило Part Design), воно є основним інструментом для проєктування деталей, призначених для виробництва або 3D-друку — адже в результаті ви завжди отримаєте об'єкт, придатний для друку.

Tool	Description	Tool	Description
 Pad	Extrudes a solid object from a selected sketch	 Pocket	Creates a pocket from a selected sketch. The sketch must be mapped to an existing solid object's face
 Revolution	Creates a solid by revolving a sketch around an axis	 Groove	Creates a groove by revolving a sketch around an axis
 Fillet	Fillets (rounds) edges of an object	 Chamfer	Chamfers edges of an object
 Draft	Applies angular draft to faces of an object	 Mirrored	Mirrors features on a plane or face
 Linear Pattern	Creates a linear pattern of features	 Polar Pattern	Creates a polar pattern of features
 Scaled	Scales features to a different size	 MultiTransform	Allows creating a pattern with any combination of the other transformations
 Shaft wizard	Generates a shaft from a table of values and allows to analyze forces and moments	 Involute Gear wizard	Allows you to create several types of gears

Arch

Робоче середовище Arch містить інструменти для роботи з BIM-проєктами (у сферах цивільного будівництва та архітектури). Він також включає всі інструменти з робочого верстака Draft. Основне призначення верстака Arch — створення BIM-об'єктів або надання BIM-атрибутів об'єктам, створеним за допомогою інших верстаків, для подальшого експорту у формат IFC.

Tool	Description	Tool	Description
 Wall	Creates a wall from scratch or using a selected object as a base	 Structure	Creates a structural element from scratch or using a selected object as a base
 Reinforcement Bar	Creates a reinforcement bar in a selected structural element	 Floor	Creates a floor including selected objects
 Building	Creates a building including selected objects	 Site	Creates a site including selected objects
 Window	Creates a window using a selected object as a base	 Section Plane	Adds a section plane object to the document
 Axes	Adds an axes system to the document	 Roof	Creates a sloped roof from a selected face
 Space	Creates a space object in the document	 Stairs	Creates a stairs object in the document
 Panel	Creates a panel object from a selected 2D object	 Frame	Creates a frame object from a selected layout
 Equipment	Creates an equipment or furniture object	 Set Material	Attributes a material to selected objects
 Schedule	Creates different types of schedules	 Cut Plane	Cut an object according to a plan.
 Add Component	Adds objects to a component	 Remove Component	Subtracts or removes objects from a component
 Survey Mode	Enters or leaves surveying mode		

Drawing

Робоче середовище «Креслення» (Drawing) призначене для створення та редагування двовимірних креслярських аркушів, які використовуються для відображення проекцій вашої 3D-моделі у 2D-форматі. Ці аркуші можна експортувати для використання в інших 2D-програмах (у форматах SVG або DXF), зберегти у вигляді PDF-файлу або роздрукувати.

Tool	Description	Tool	Description
New sheet	Creates a new drawing sheet	Insert view	Inserts a view of the selected object in the active drawing sheet
Annotation	Adds an annotation to the current drawing sheet	Clip	Adds a clip group to the current drawing sheet
Browser preview	Opens a preview of the current sheet in the browser	Ortho Views	Automatically creates orthographic views of an object on the current drawing sheet
Symbol	Adds the contents of a SVG file as a symbol on the current drawing sheet	Draft View	Inserts a special Draft view of the selected object in the current drawing sheet
Export	Saves the current sheet as a SVG file		

Інші вбудовані робочі середовища

Хоча вище наведено огляд найважливіших інструментів FreeCAD, існує ще багато інших робочих середовищ, зокрема:

- Робоче середовище «Сітки» (Mesh) дозволяє працювати з полігональними сітками. Хоча сітки не є бажаним типом геометрії для роботи у FreeCAD (через недостатню точність і відсутність підтримки кривих), вони все ж мають широке застосування, і FreeCAD повністю їх підтримує. Це середовище також містить низку інструментів для перетворення об'єктів у сітки та навпаки.
- Робоче середовище «Трасування променів» (Raytracing) надає інструменти для взаємодії із зовнішніми системами візуалізації (рендерерами), такими як POV-Ray або LuxRender. Воно дозволяє створювати високоякісні візуалізації ваших моделей безпосередньо у FreeCAD.
- Робоче середовище «Електронні таблиці» (Spreadsheet) дозволяє створювати та редагувати табличні дані, які можна отримувати з моделей FreeCAD. На комірки таблиць також можна посилатися в різних частинах FreeCAD, що дозволяє використовувати їх як основні структури даних.
- Робоче середовище «МЕК» (FEM — метод скінченних елементів) призначене для аналізу методом скінченних елементів; воно дозволяє виконувати попередню та подальшу обробку розрахунків, а також графічно відображати отримані результати.

Зовнішні робочі середовища

Існує також низка інших дуже корисних робочих середовищ (верстаків), створених учасниками спільноти FreeCAD. Хоча вони не входять до стандартного комплекту встановлення FreeCAD, їх легко додати як додаткові модулі (плагіни). Усі вони представлені в репозиторії FreeCAD-addons. Серед найбільш розвинених — такі:

- Робоче середовище «Drawing Dimensioning» (Проставляння розмірів на кресленнях) пропонує багато нових інструментів для безпосередньої роботи з аркушами креслень; воно дозволяє додавати розміри, анотації та інші технічні позначення, надаючи широкі можливості для керування їхнім виглядом.
- Робоче середовище «Fasteners» (Кріпильні вироби) містить великий набір готових до використання кріпильних елементів, таких як гвинти, болти, стрижні, шайби та гайки. Доступно багато варіантів і налаштувань.
- Робоче середовище «Assembly2» пропонує набір інструментів для створення вузлів (складок) та роботи з ними.

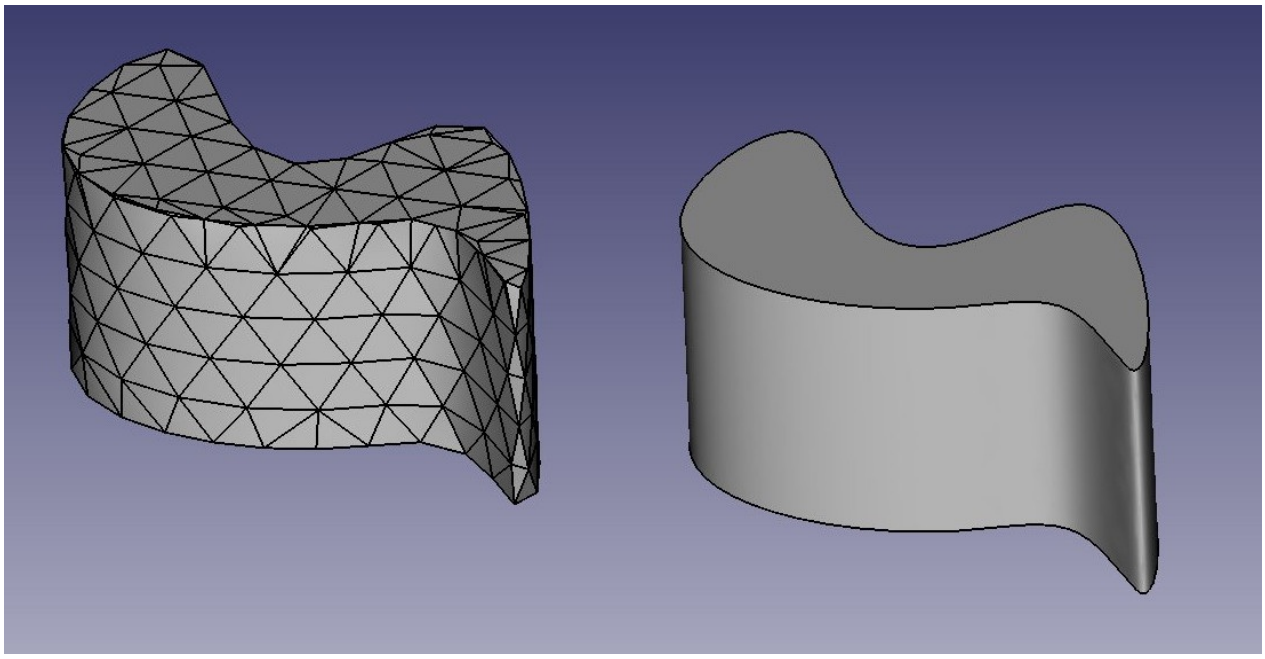
Детальніше

- Повний перелік робочих середовищ: http://www.freecadweb.org/wiki/index.php?title=Workbenches
- Робоче середовище «Part»: http://www.freecadweb.org/wiki/index.php?title=Part_Module
- Робоче середовище «Draft»: http://www.freecadweb.org/wiki/index.php?title=Draft_Module
- Робочі середовища «Sketcher» (Ескіз) та «Part Design» (Проектування деталей): http://www.freecadweb.org/wiki/index.php?title=PartDesign_Workbench
- Робоче середовище «Arch» (Архітектура): http://www.freecadweb.org/wiki/index.php?title=Arch_Module
- Робоче середовище «Drawing» (Креслення): http://www.freecadweb.org/wiki/index.php?title=Drawing_Module
- Робоче середовище «FEM» (Скінченно-елементний аналіз): http://www.freecadweb.org/wiki/index.php?title=Fem_Workbench
- Репозиторій FreeCAD-addons: https://github.com/FreeCAD/FreeCAD-addons

Традиційне моделювання — метод CSG

Абревіатура CSG розшифровується як «конструктивна твердотільна геометрія» (Constructive Solid Geometry). Це базовий підхід до роботи з твердотільною 3D-геометрією, що полягає у створенні складних об'єктів шляхом додавання або видалення частин із твердих тіл за допомогою булевих операцій — об'єднання, віднімання або перетину.

Як ми вже бачили раніше в цьому посібнику, FreeCAD підтримує роботу з багатьма типами геометрії. Однак для створення 3D-об'єктів, що відповідають реальним фізичним предметам, найкраще підходить твердотільна геометрія типу BREP (Boundary Representation — представлення через межі). Робота з нею здійснюється переважно в середовищі (верстаті) Part. На відміну від полігональних сіток, які складаються лише з точок і трикутників, об'єкти BREP мають грані, визначені математичними кривими. Це забезпечує абсолютну точність незалежно від масштабу.

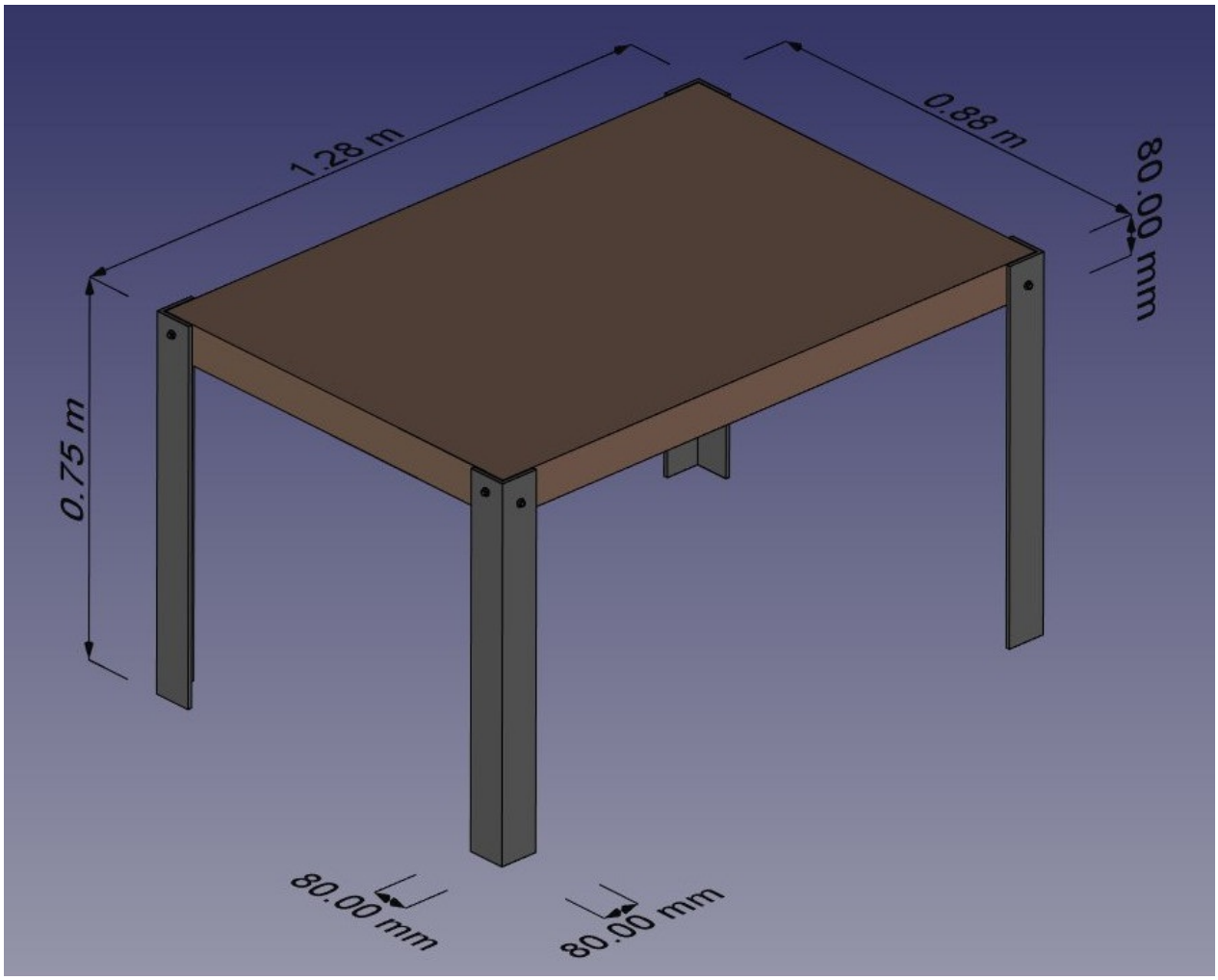


Різницю між цими двома підходами можна порівняти з відмінністю між растровими та векторними зображеннями. У полігональних сітках, подібно до растрових зображень, криволінійні поверхні розбиваються на набір точок. Якщо розглядати такий об'єкт зблизька або надрукувати його у великому розмірі, замість плавної кривої поверхні ви побачите грані (фасети). Натомість у векторних зображеннях і даних BREP положення будь-якої точки на кривій не зберігається як фіксоване значення, а обчислюється «на льоту» з абсолютною точністю.

У FreeCAD обробка всієї геометрії на основі BREP виконується за допомогою стороннього програмного забезпечення з відкритим вихідним кодом — OpenCasCade. Основним інтерфейсом взаємодії між FreeCAD і ядром OpenCasCade є середовище (верстат) Part. Більшість інших середовищ у FreeCAD будують свою функціональність на базі інструментів середовища Part.

Хоча інші середовища часто пропонують більш досконалі інструменти для побудови та редагування геометрії, усі вони, по суті, працюють з об'єктами типу Part. Тому дуже корисно розуміти, як ці об'єкти влаштовані зсередини, і вміти користуватися інструментами середовища Part. Оскільки вони простіші, то часто допомагають вирішити проблеми, з якими не можуть впоратися складніші, «розумніші» інструменти.

Щоб продемонструвати роботу верстака «Part» (Деталь), ми змоделюємо цей стіл, використовуючи виключно операції CSG (за винятком гвинтів, для яких ми скористаємося одним із додатків, а також розмірів, про які йтиметься в наступному розділі):

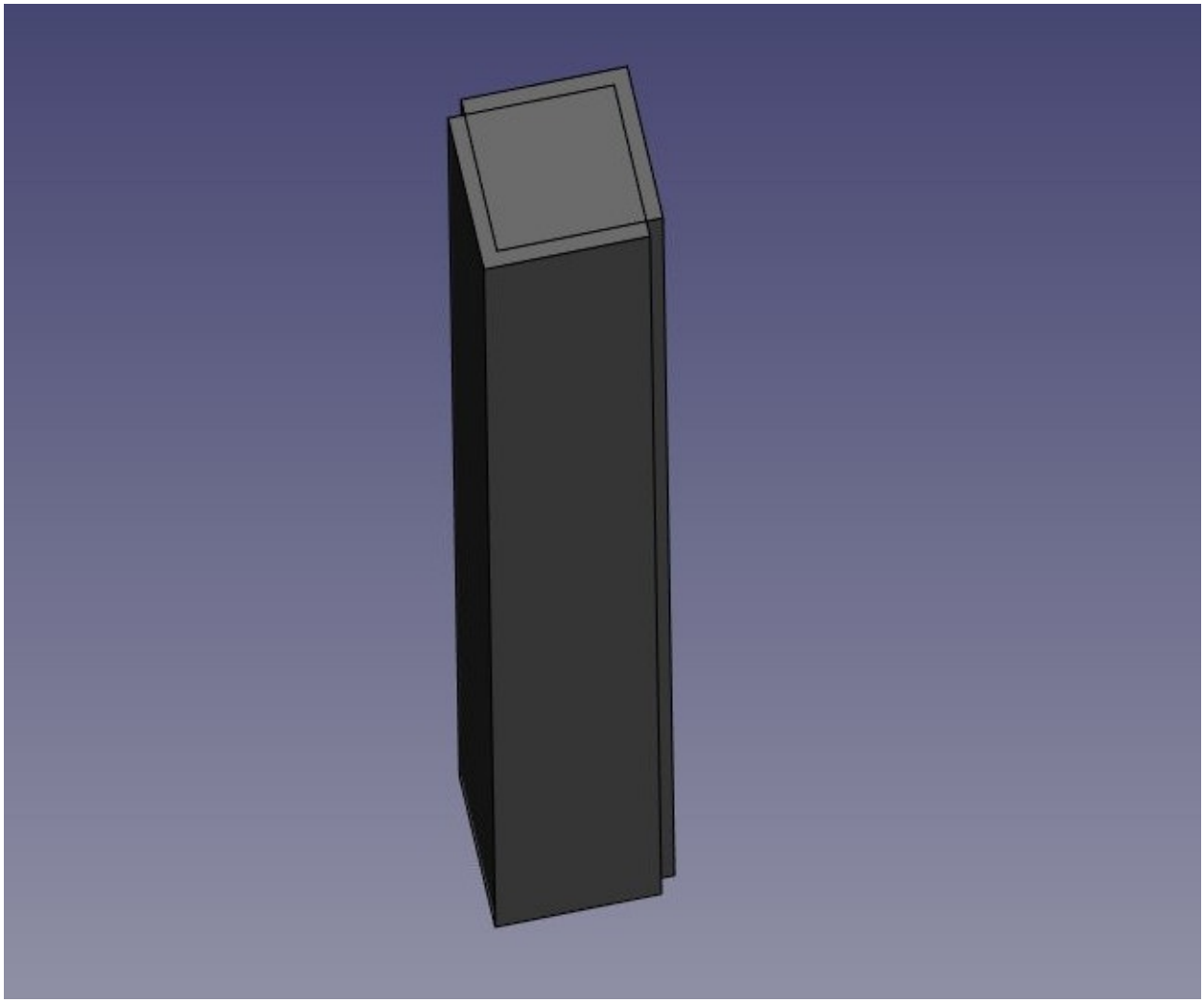


Створімо новий документ (Ctrl+N або меню «Файл» -> «Новий документ»), перейдімо в робоче середовище «Part» і почнімо з першої ніжки:

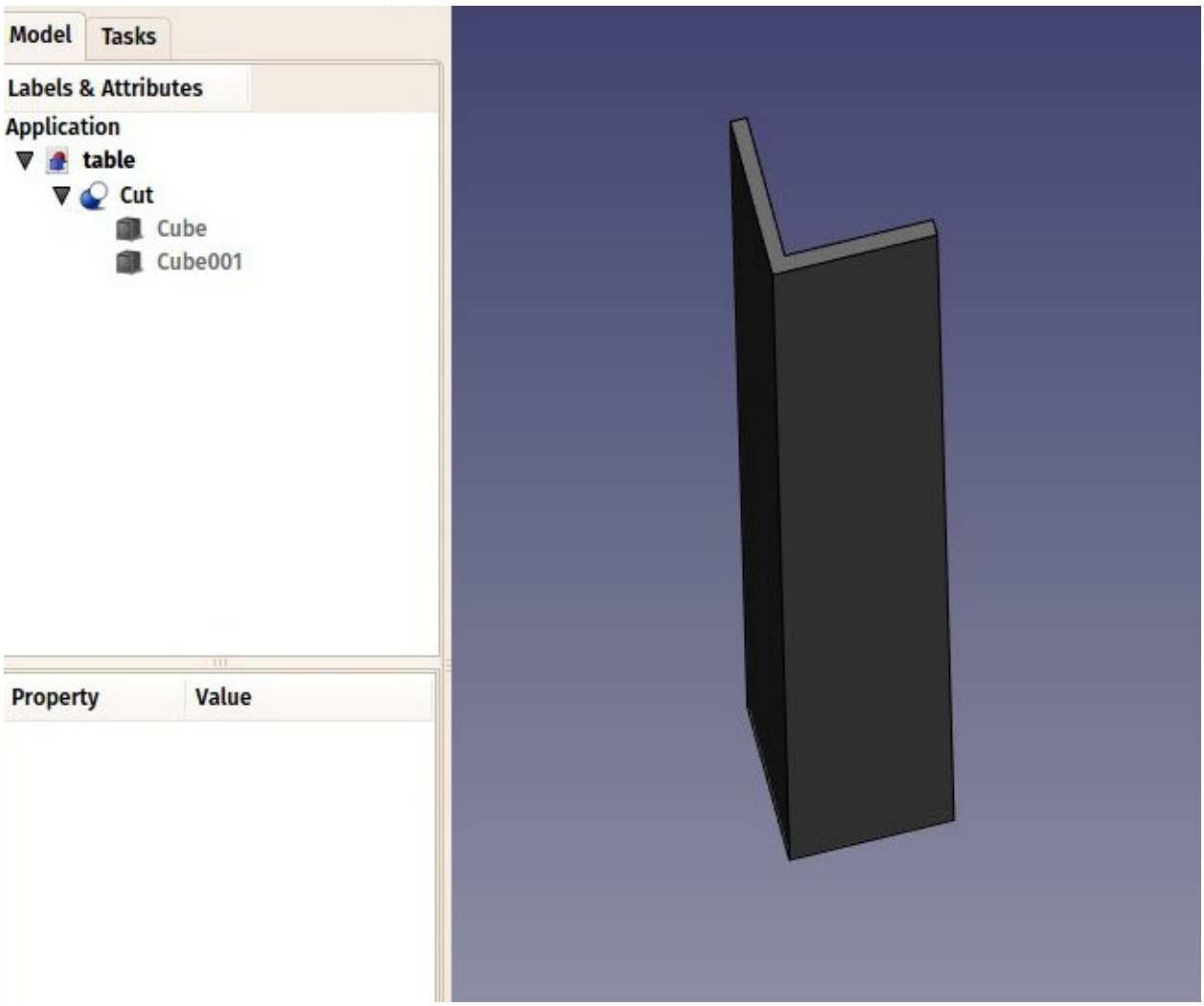
- Натисніть кнопку **«Паралелепіпед» (Box)**
- Виберіть створений паралелепіпед і задайте такі властивості (на вкладці **«Дані» / Data**):
  - Довжина: 80 мм (або 8 см, чи 0,8 м — FreeCAD працює з будь-якими одиницями виміру)
  - Ширина: 80 мм
  - Висота: 75 см
- Дублюйте цей об'єкт, натиснувши Ctrl+C, а потім Ctrl+V (або через меню «Правка» -> «Копіювати» та «Вставити»)
- Виберіть новий об'єкт, що з'явився в результаті

- Змініть його положення, відредагувавши властивість «Розміщення» (Placement):
  - Положення x: 8 мм
  - Положення y: 8 мм

У вас має вийти два високих паралелепіпеди, розташовані на відстані 8 мм один від одного:

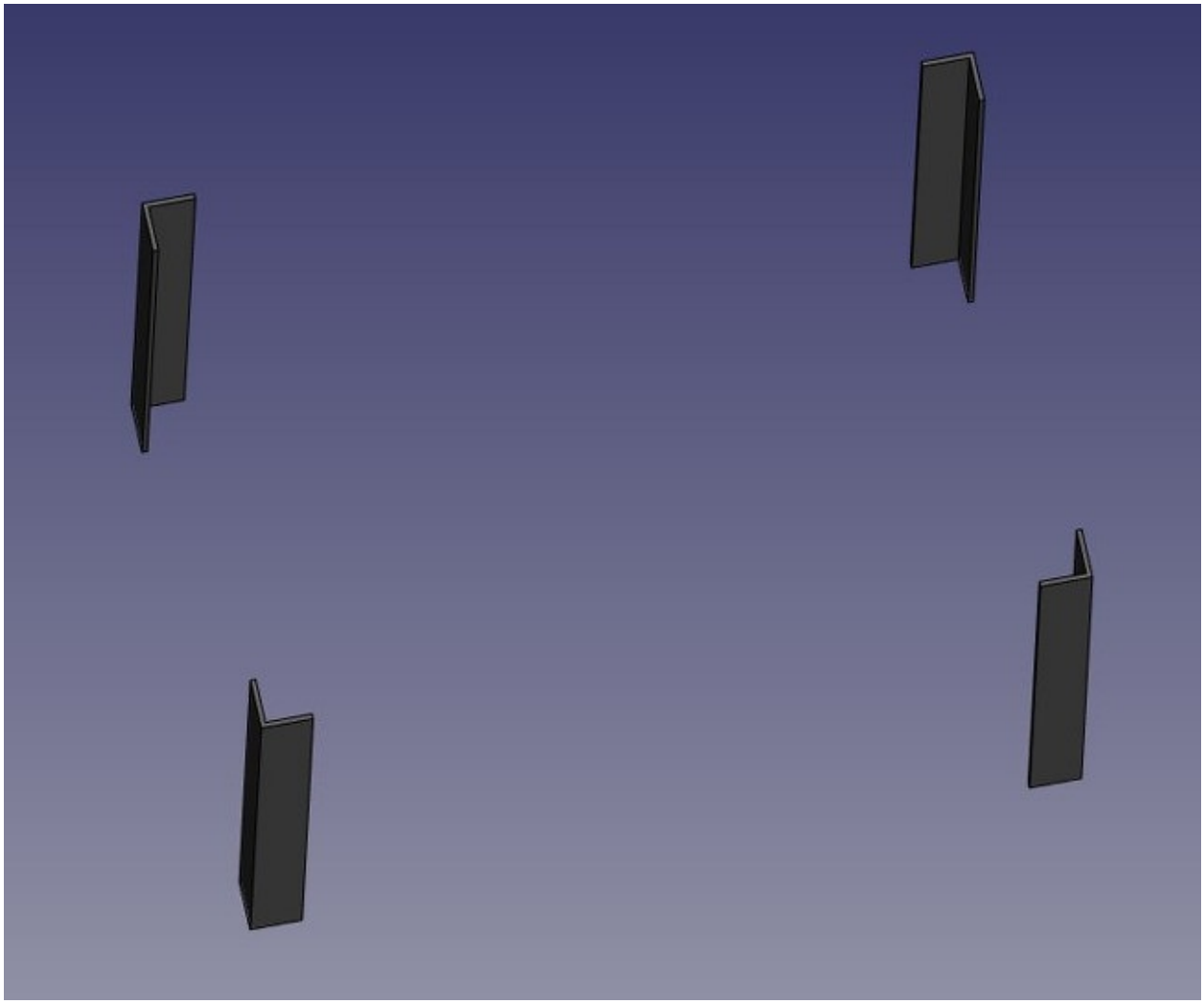


- Тепер можна відняти один об'єкт від іншого: виберіть перший — той, що залишиться, — а потім, утримуючи натиснутою клавішу CTRL, виберіть другий, який буде віднято (порядок має значення), і натисніть кнопку «Вирізати» (Cut):



Зверніть увагу, що новостворений об'єкт під назвою «Cut» («Виріз») усе ще містить два куби, які ми використовували як операнди. Фактично, ці два куби залишаються в документі — вони просто приховані та згруповані під об'єктом «Cut» у дереві об'єктів. Ви можете вибрати їх, розгорнувши список за допомогою стрілки поруч із об'єктом «Cut»; також, клацнувши правою кнопкою миші, можна знову зробити їх видимими або змінити будь-які їхні властивості.

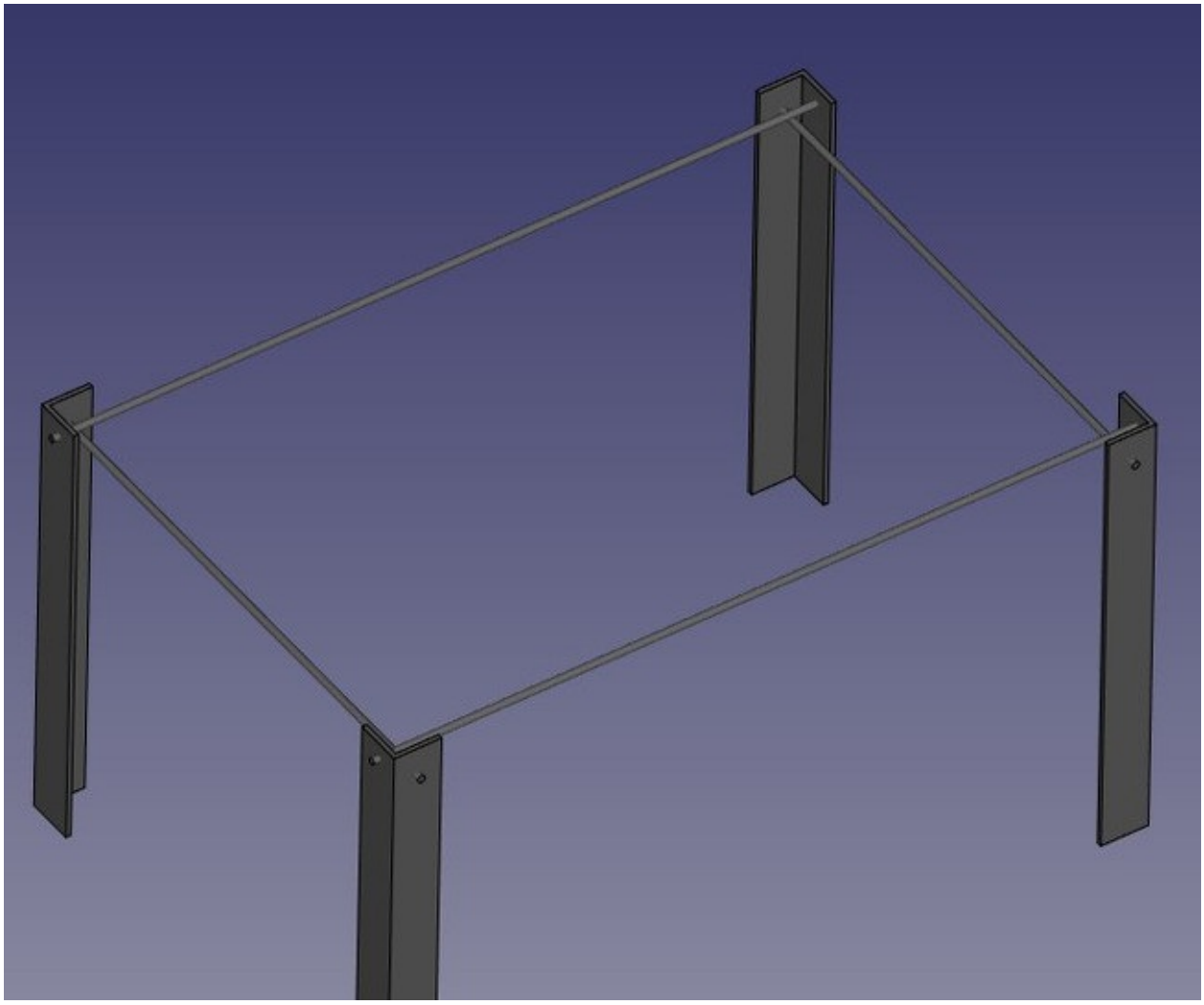
- Тепер створимо три інші ніжки, продублювавши наш базовий куб ще 6 разів. Оскільки він усе ще знаходиться в буфері обміну, ви можете просто вставити його (Ctrl+V) 6 разів. Змініть їхнє розташування наступним чином:
  - cube002: x: 0, y: 80 см
  - cube003: x: 8 мм, y: 79,2 см
  - cube004: x: 120 см, y: 0
  - cube005: x: 119,2 см, y: 8 мм
  - cube006: x: 120 см, y: 80 см
  - cube007: x: 119,2 см, y: 79,2 см
- Тепер виконаємо три інші операції вирізання, вибираючи спочатку «основний» куб, а потім — куб, який потрібно відсікти. Тепер у нас є чотири об'єкти «Cut»:



Можливо, ви подумали, що замість того, щоб дублювати базовий куб шість разів, можна було б тричі продублювати всю ніжку цілком. Це цілком слушно; як це часто буває у FreeCAD, одного й того ж результату можна досягти різними способами. Про це варто пам’ятати, адже під час роботи над складнішими об’єктами певні операції можуть не дати бажаного результату, тож часто доводиться шукати інші підходи.

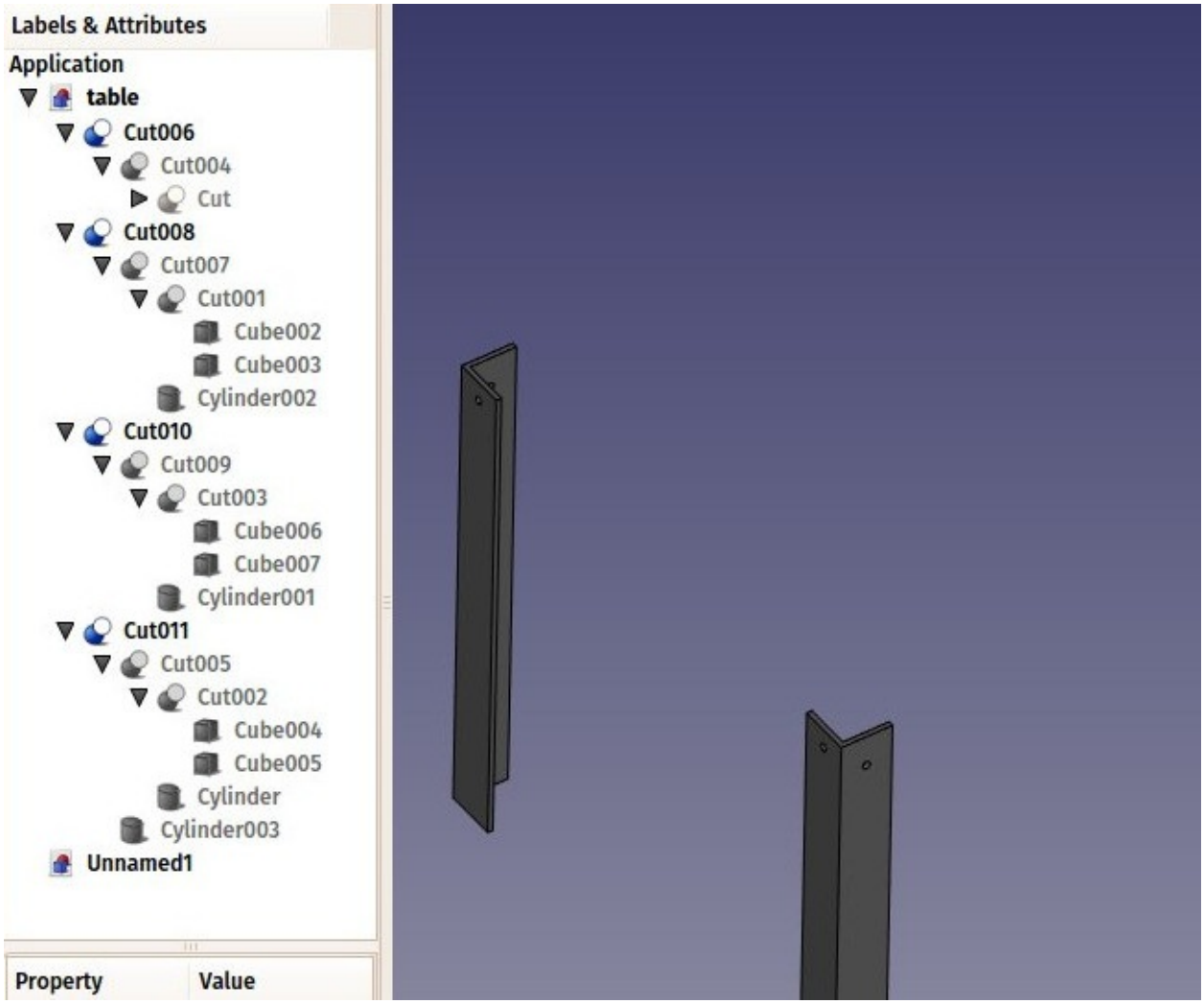
- Тепер зробимо отвори для гвинтів, скориставшись тим самим методом віднімання (Cut). Оскільки нам потрібно 8 отворів — по два в кожній ніжці, — можна було б створити 8 окремих об’єктів для віднімання. Проте спробуймо інший шлях: створимо 4 циліндри, кожен з яких використовуватиметься для двох ніжок. Отже, створимо чотири циліндри за допомогою інструмента «Циліндр» (Cylinder). Можна, як і раніше, створити лише один, а потім продублювати його. Встановіть радіус усіх циліндрів рівним 6 мм. Цього разу їх також доведеться повернути; це робиться через властивість «Розміщення» (Placement):
  - cylinder: висота: 130 см, кут: 90°, вісь: x:0, y:1, положення: x:-10 мм, y:40 мм, z:72 см
  - cylinder001: висота: 130 см, кут: 90°, вісь: x:0, y:1, положення: x:-10 мм, y:84 см, z:72 см
  - cylinder002: висота: 90 см, кут: 90°, вісь: x:-1, y:0, положення: x:40 мм, y:-10 мм, z:70 см

- cylinder003: висота: 90 см, кут: 90°, вісь: x:-1, y:0, положення: x:124 см, y:-10 мм, z:70 см



Ви помітите, що циліндри дещо довші, ніж потрібно. Це пояснюється тим, що, як і в усіх 3D-програмах, які працюють із твердими тілами, булеві операції у FreeCAD іноді надто чутливі до випадків, коли грані об'єктів збігаються (розташовані «грань до грані»), що може призвести до помилок. Роблячи циліндри довшими, ми убезпечуємо себе від таких проблем.

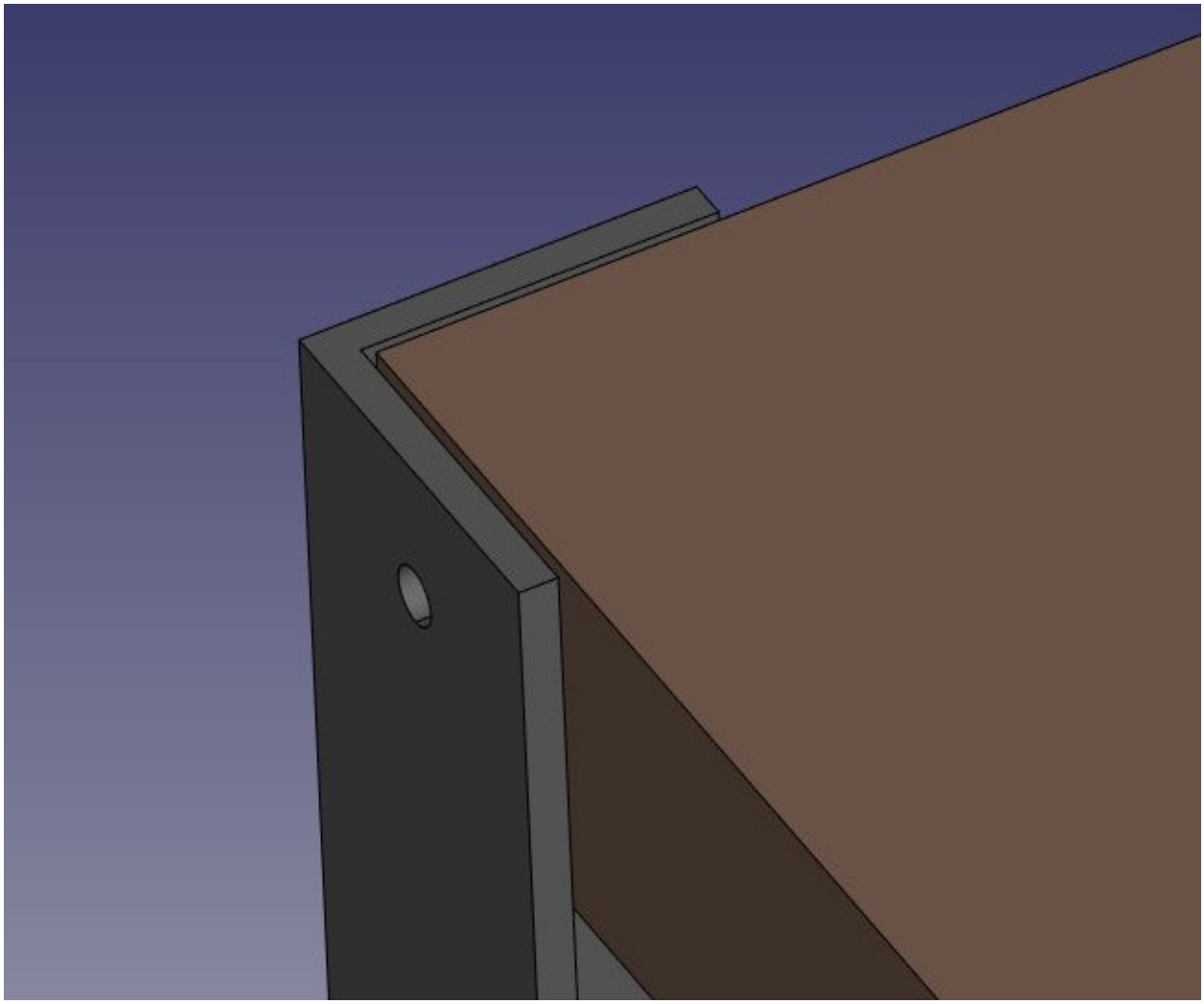
- Тепер виконаємо операції віднімання. Виберіть першу ніжку, потім, утримуючи клавішу CTRL, виберіть одну з трубок, що проходить крізь неї, і натисніть кнопку «Вирізати» (Cut). У ніжці з'явиться отвір, а трубка стане невидимою. Знайти її можна в дереві побудови, розгорнувши елемент, що відповідає за ніжку з отвором.
- Виберіть іншу ніжку, яку пронизує ця прихована трубка, і повторіть операцію. Цього разу виберіть трубку в дереві побудови (утримуючи CTRL), оскільки в 3D-вигляді вона прихована (ви також можете зробити її знову видимою та вибрати безпосередньо в 3D-просторі). Повторіть ці дії для решти ніжок, доки в кожній із них не з'явиться по два отвори:



Як бачите, кожна ніжка перетворилася на досить довгу послідовність операцій. Усе це залишається параметричним: ви будь-коли можете змінити будь-який параметр будь-якої з попередніх операцій. У FreeCAD цю сукупність часто називають «історією моделювання», оскільки вона фактично містить усю історію виконаних вами операцій.

Ще одна особливість FreeCAD полягає в тому, що поняття 3D-об’єкта та поняття 3D-операції тут фактично зливаються в одне ціле. Наприклад, операція «Вирізання» (Cut) є водночас і дією, і 3D-об’єктом, що утворюється в результаті цієї дії. У FreeCAD такі елементи називають «фічами» (features) — це поняття об’єднує в собі як об’єкти, так і операції.

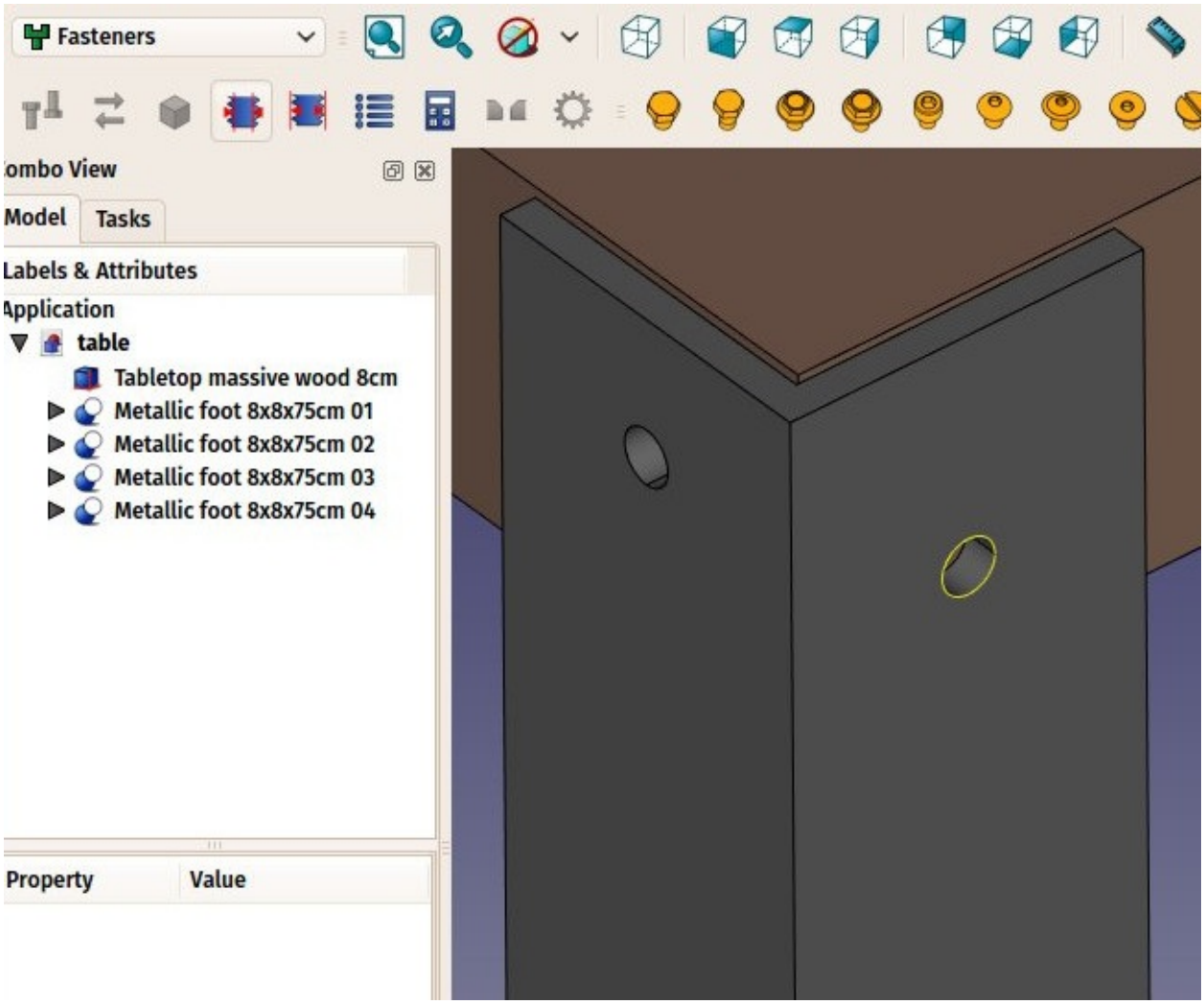
- Тепер створимо стільницю — це буде простий дерев’яний брусок. Для цього знову скористаємося інструментом «Паралелепіпед» (Box) із такими параметрами: довжина — 126 см, ширина — 86 см, висота — 8 см; розташування: x = 10 мм, y = 10 мм, z = 67 см. На вкладці «Вигляд» (View) можна надати об’єкту приємного коричневого відтінку, схожого на дерево, змінивши властивість «Колір фігури» (Shape Color).



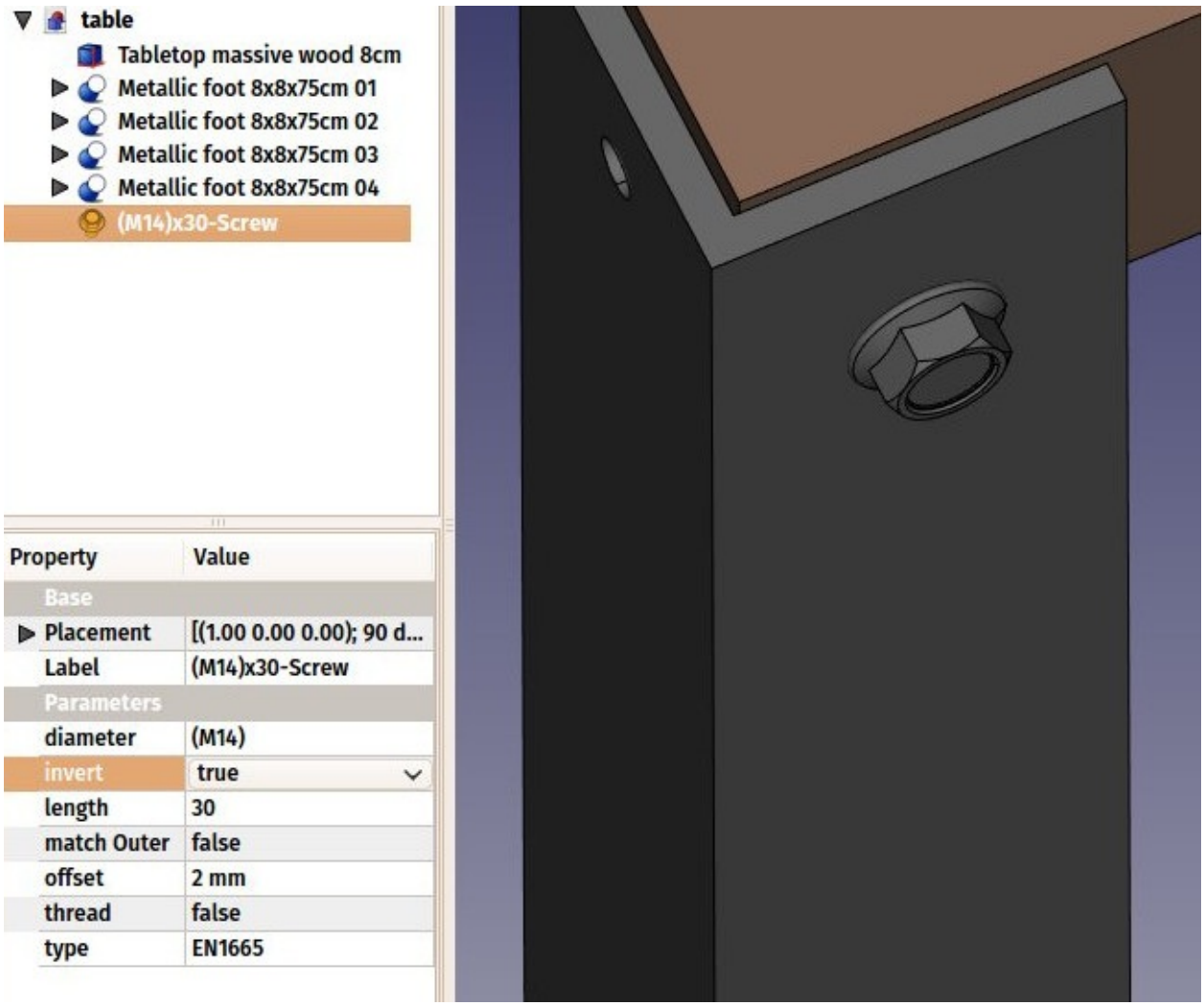
Зверніть увагу: хоча товщина ніжок становить 8 мм, ми розмістили їх на відстані 10 мм одна від одної, залишивши між ними зазор у 2 мм. Звісно, у реальному столі такого не буде, але в подібних «збірних» моделях це поширена практика: вона допомагає тому, хто розглядає модель, зрозуміти, що це окремі деталі, які згодом потрібно буде з'єднати вручну.

Тепер, коли всі п'ять деталей готові, саме час дати їм зрозуміліші назви, ніж «Cut015». Клацнувши правою кнопкою миші на об'єкті в дереві побудови (або натиснувши клавішу **F2**), ви можете перейменувати його на щось більш змістовне — як для себе, так і для іншої людини, яка відкриє ваш файл у майбутньому. Часто кажуть, що надання об'єктам влучних назв є навіть важливішим, ніж сам процес їх моделювання.

- Тепер додамо кілька гвинтів. Існує надзвичайно корисний додаток під назвою «Fasteners» (Кріплення), розроблений учасником спільноти FreeCAD; його можна знайти в репозиторії додатків FreeCAD. Він значно спрощує процес додавання гвинтів. Встановлення додаткових робочих середовищ (верстаків) — проста процедура, описана на сторінках відповідних додатків.
- Після встановлення робочого середовища «Fasteners» і перезапуску FreeCAD воно з'явиться у списку доступних середовищ, і ми зможемо перемкнутися на нього. Щоб додати гвинт в один із отворів, спочатку потрібно вибрати круговий край цього отвору:



- Після цього можна натиснути одну з кнопок для створення гвинтів у верстаку «Кріплення» (Fasteners Workbench) — наприклад, вибрати болт із шестигранною головкою та фланцем (стандарт EN 1665, посилена серія). Болт буде розміщено й вирівняно відносно отвору, а його діаметр підбереться автоматично відповідно до розміру цього отвору. Іноді болт може розміститися в перевернутому положенні; це легко виправити, змінивши значення властивості «invert» (інверсія). Також можна встановити зміщення (offset) на 2 мм — дотримуючись того самого правила, що ми застосували раніше для відстані між стільницею та ніжками:



- Повторіть це для всіх отворів — і наш стіл готовий!

Внутрішня структура об'єктів типу «Part»

Як ми бачили вище, у FreeCAD можна вибирати не лише цілі об'єкти, а й їхні окремі частини — наприклад, круговий контур отвору для гвинта. Саме час коротко розглянути, як влаштовані об'єкти типу «Part» (геометричні об'єкти) зсередини. Будь-який робочий верстак (workbench), що створює геометрію цього типу, базується на таких елементах:

- **Вершини (Vertices):** це точки (зазвичай кінцеві точки), на яких будується все інше. Наприклад, лінія має дві вершини.
- **Ребра (Edges):** це лінійна геометрія, така як відрізки, дуги, еліпси або криві NURBS. Зазвичай вони мають дві вершини, але в окремих випадках — лише одну (наприклад, замкнене коло).
- **Контури (Wires):** контур — це послідовність ребер, з'єднаних між собою кінцевими точками. Він може складатися з ребер будь-якого типу і бути як замкненим, так і розімкненим.
- **Грані (Faces):** грані можуть бути плоскими або криволінійними. Вони утворюються одним замкненим контуром (який слугує межею грані) або кількома контурами (якщо в грані є отвори).
- **Оболонки (Shells):** це просто група граней, з'єднаних між собою по ребрах.

Оболонка може бути відкритою або замкненою.

- **Тіла (Solids):** коли оболонка повністю замкнена — тобто не має жодних «прорізів» — вона стає твердим тілом (solid). Тіла мають поняття внутрішнього та зовнішнього простору. Багато робочих верстаків спираються на цю властивість, щоб гарантувати, що створені об'єкти можна виготовити в реальному світі.
- **Складені об'єкти (Compounds):** це просто сукупність інших геометричних форм (незалежно від їхнього типу), об'єднаних в один об'єкт.

У 3D-вигляді можна вибирати окремі вершини, ребра або грані. Вибір будь-якого з цих елементів автоматично виділяє й увесь об'єкт цілком.

Примітка щодо спільної розробки

Можливо, поглянувши на наведену вище модель, ви вирішите, що її конструкція не надто вдала. Наприклад, кріплення ніжок до стільниці може бути недостатньо міцним. Можливо, ви захочете додати підсилювальні елементи або маєте інші ідеї, як покращити виріб. Саме тут стає цікавою можливість спільної роботи: ви можете завантажити файл, створений під час цієї вправи (за посиланням нижче), і змінити його, щоб зробити кращим. А якщо ви поділитеся цим удосконаленим файлом, інші зможуть зробити його ще кращим або використати вашу грамотно розроблену таблицю у своїх проєктах. Ваш дизайн може наштовхнути інших людей на нові ідеї — і, можливо, ви бодай трохи долучитеся до того, щоб зробити світ кращим...

Завантаження

- Файл, створений у цьому практичному завданні: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/table.FCStd

Докладніше

- Робоче середовище Part: http://www.freecadweb.org/wiki/index.php?title=Part_Module
- Репозиторій додатків для FreeCAD: https://github.com/FreeCAD/FreeCAD-addons
- Робоче середовище Fasteners: https://github.com/shaise/FreeCAD_FastenersWB

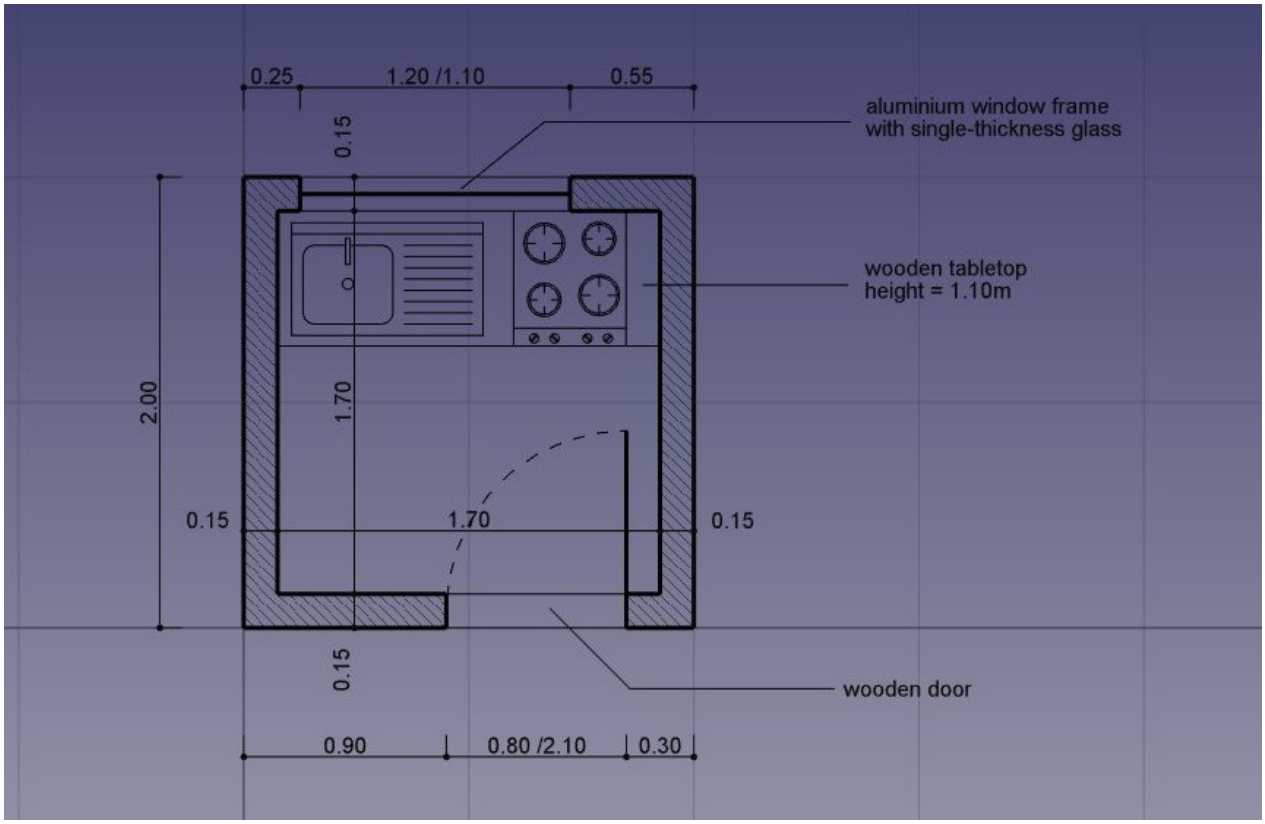
Традиційне 2D-креслення

Можливо, FreeCAD зацікавить вас, оскільки ви вже маєте певний досвід у створенні технічних креслень — наприклад, працювали в таких програмах, як AutoCAD. Або ж ви вже знайомі з основами проєктування чи просто віддаєте перевагу створенню креслень перед безпосереднім будівництвом об'єктів. У будь-якому разі, у FreeCAD є робоче середовище, що відповідає традиційним підходам і містить інструменти, притаманні більшості 2D CAD-систем — це верстак (середовище) «Draft» (Креслення).

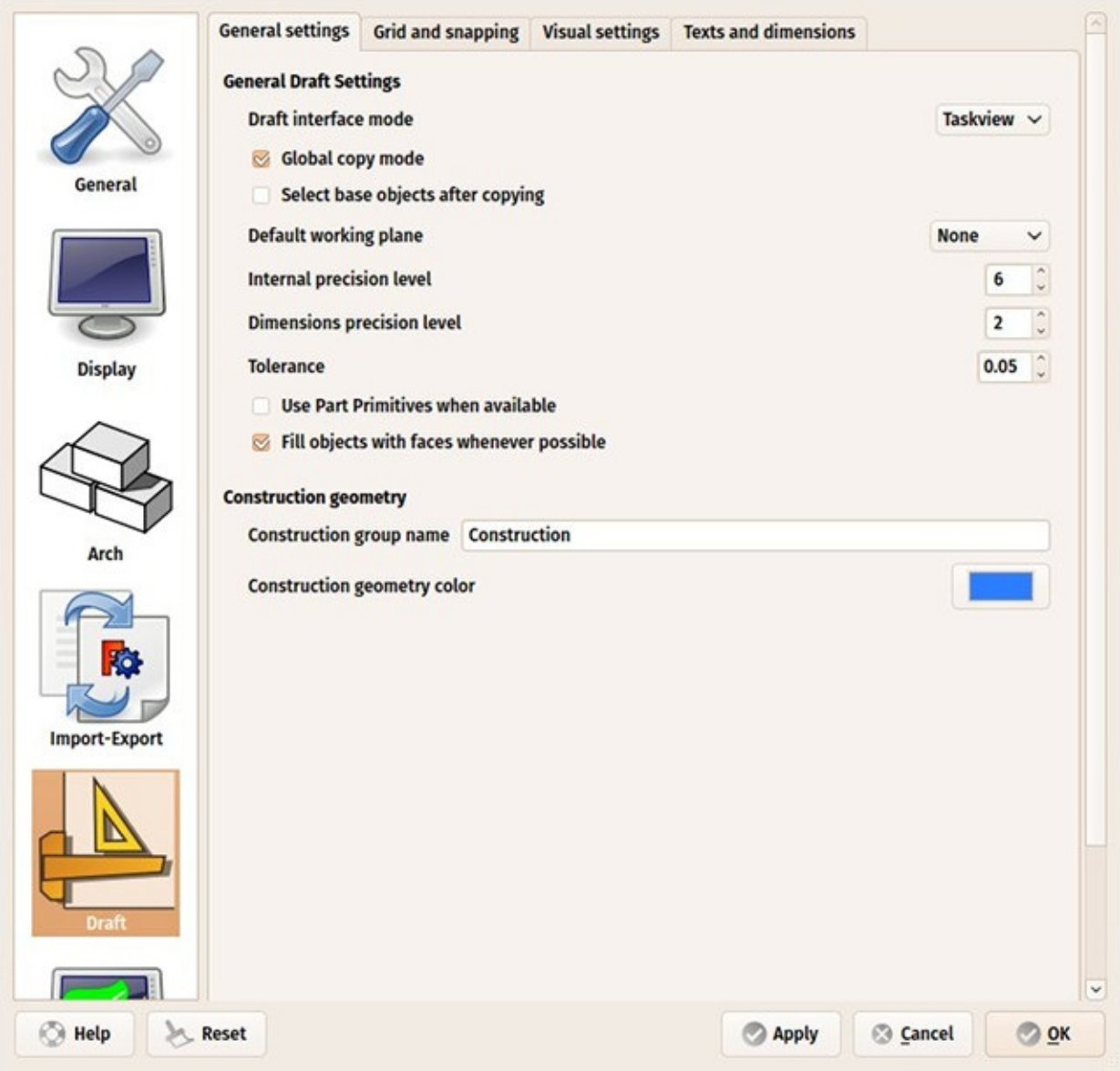
Хоча верстак «Draft» використовує методи роботи, успадковані від традиційного світу 2D CAD, він аж ніяк не обмежується лише двовимірним простором. Усі його інструменти працюють у повноцінному 3D-середовищі. Багато з них — наприклад, «Перемістити» (Move) або «Повернути» (Rotate) — активно використовуються в усьому FreeCAD, оскільки часто вони є більш інтуїтивно зрозумілими, ніж ручне налаштування параметрів розташування об'єктів.

Серед інструментів, доступних у верстаку «Draft», ви знайдете традиційні засоби для креслення, такі як «Лінія» (Line), «Коло» (Circle) або «Ламана лінія» (Wire/Polyline); інструменти для редагування, як-от «Перемістити» (Move), «Повернути» (Rotate) чи «Змістити» (Offset); систему робочих площин і сіток, що дозволяє точно визначити площину, у якій ви працюєте; а також повноцінну систему прив'язок (snapping), яка значно спрощує креслення та точне розміщення елементів відносно один одного.

Щоб продемонструвати принципи роботи та можливості верстака «Draft», ми виконаємо просту вправу. Її результатом стане невелике креслення — план поверху маленького будинку, де розміщено лише кухонну стільницю (план досить дивний, але ж ми тут вільні робити все, що забажаємо, чи не так?):



- Перехід до верстака **«Draft»**
- Як і в будь-якій програмі для технічного креслення, варто правильно налаштувати робоче середовище — це заощадить вам чимало часу. Ви можете налаштувати сітку, робочу площину, параметри тексту та розмірів на власний розсуд у меню **«Редагування» -> «Налаштування» -> «Draft» (Edit -> Preferences -> Draft)**. Однак у цій вправі ми припустимо, що всі ці параметри залишено за замовчуванням.



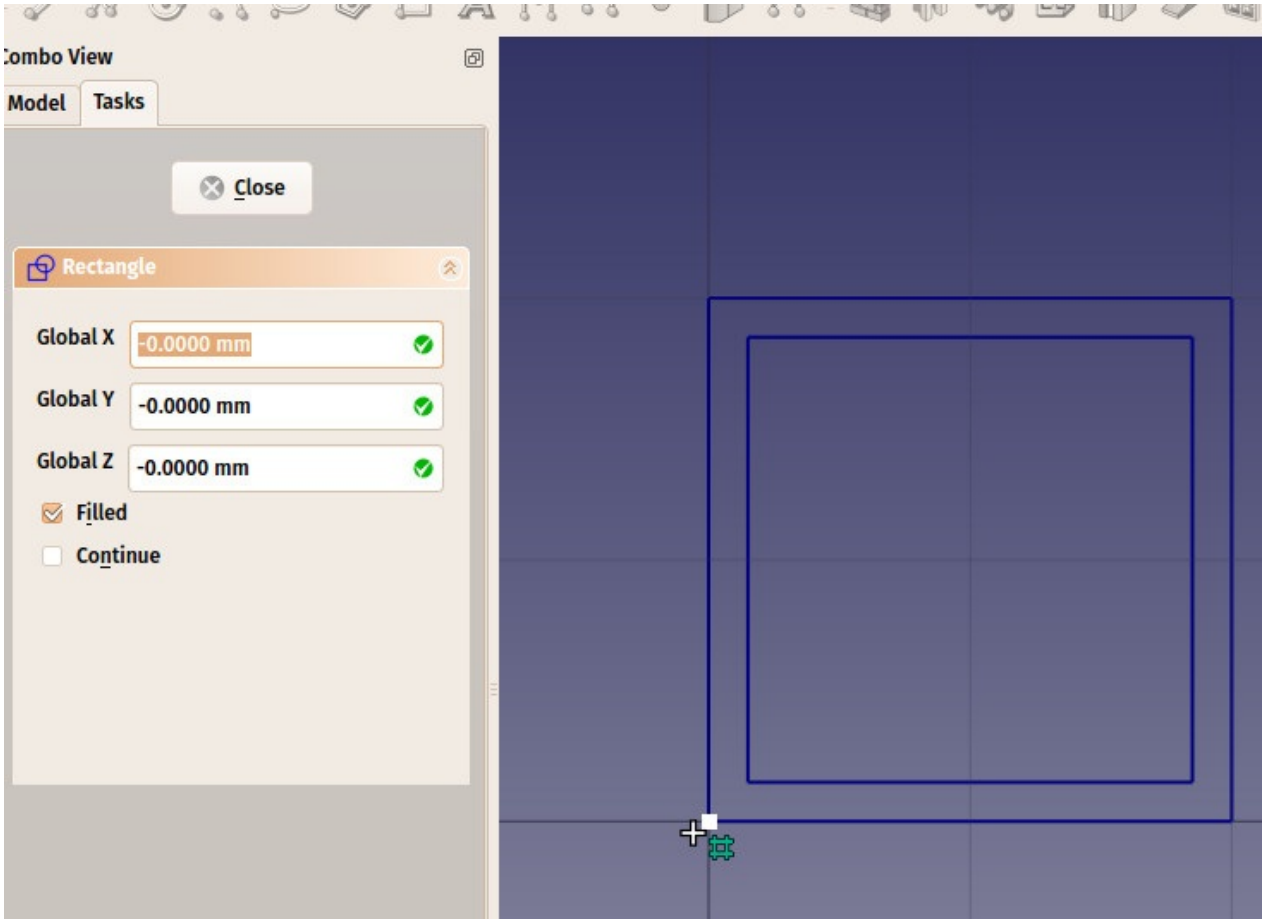
- У верстаку «Draft» (Креслення) також є дві спеціальні панелі інструментів: одна — з візуальними налаштуваннями (де можна змінити поточну робочу площину, увімкнути або вимкнути режим побудови, задати колір ліній і граней, товщину ліній та розмір тексту для нових об'єктів), а інша — з налаштуваннями прив'язки. На ній можна вмикати або вимикати сітку, а також активувати чи деактивувати окремі точки прив'язки.



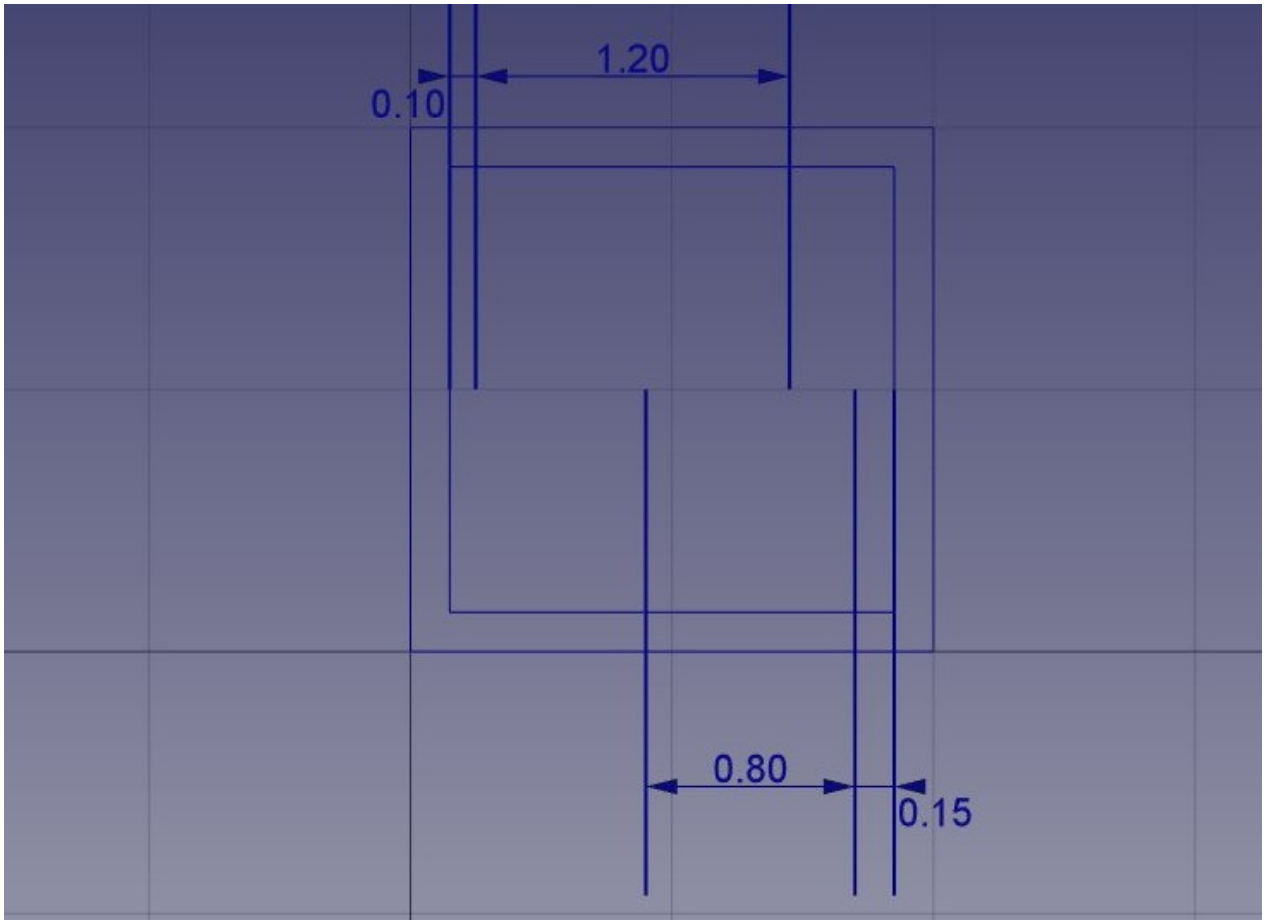
- Почнемо з увімкнення режиму побудови; це дозволить нам накреслити допоміжні лінії, на основі яких ми створимо остаточну геометрію.
- За бажанням, встановіть **робочу площину** на *XY. У такому разі вона залишатиметься незмінною незалежно від поточного вигляду (ракурсу). Якщо ж цього не зробити, робоча площина автоматично підлаштовуватиметься під поточний вигляд; тому, якщо ви хочете малювати на площині XY (площині землі), слідкуйте за тим, щоб у вас був увімкнений вигляд зверху.
- Далі виберіть інструмент «Прямокутник» (Rectangle) і накресліть прямокутник розміром 2 на 2 метри, почавши з точки (0,0,0) (значення Z залиште нульовим). Зауважте, що більшість команд у верстаку «Draft» можна виконувати повністю з клавіатури, не торкаючись миші, використовуючи дволітерні скорочення.

Наш перший прямокутник (2x2 м) можна створити так: `re` `0` `**Enter**` `0` `**Enter**` `0` `**Enter**` `2m` `**Enter**` `2m` `**Enter**` `0` `**Enter**`.

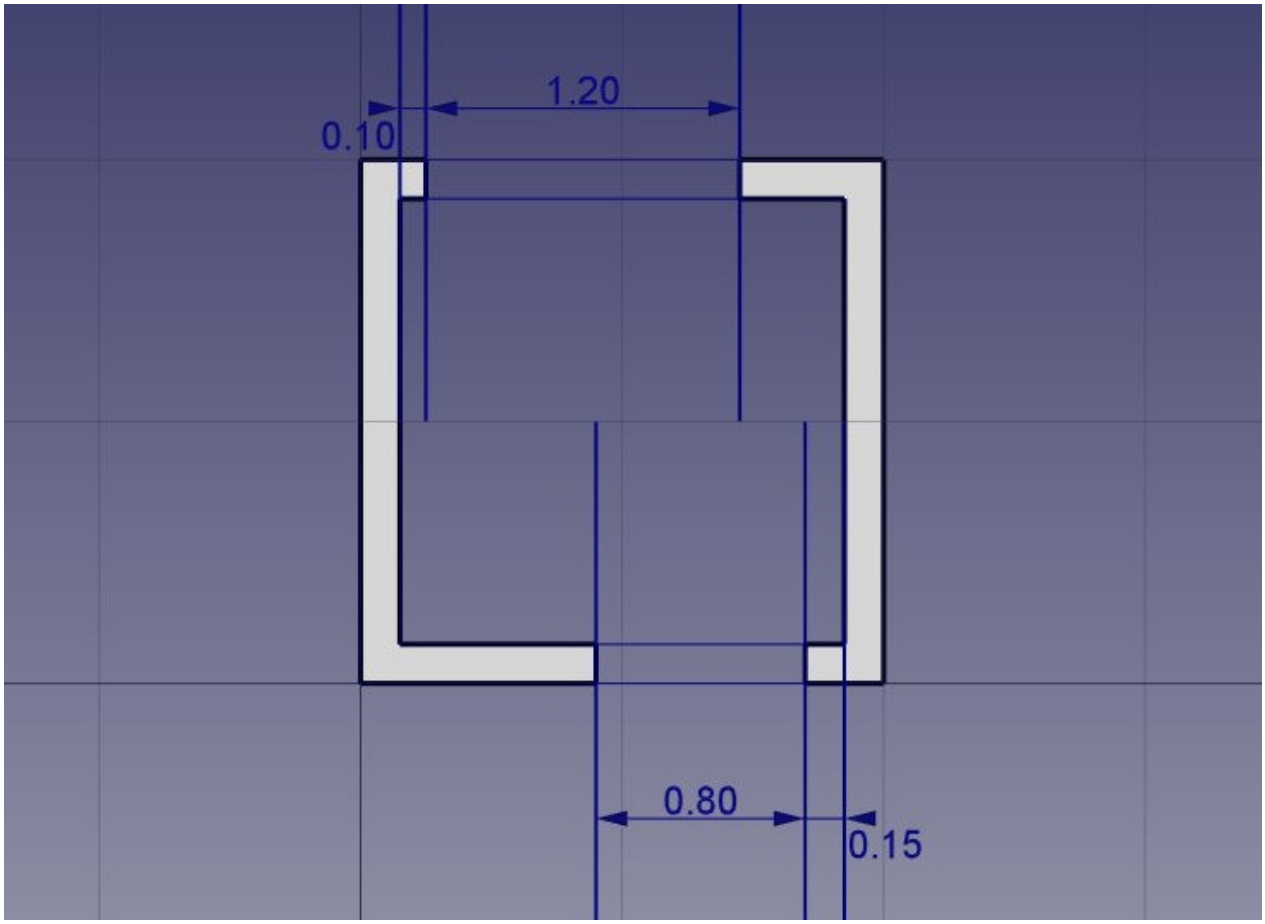
- Створіть копію цього прямокутника зі зміщенням 15 см всередину, скориставшись інструментом «Зміщення» (Offset). Увімкніть режим копіювання (Copy mode) і встановіть відстань зміщення — 15 см:



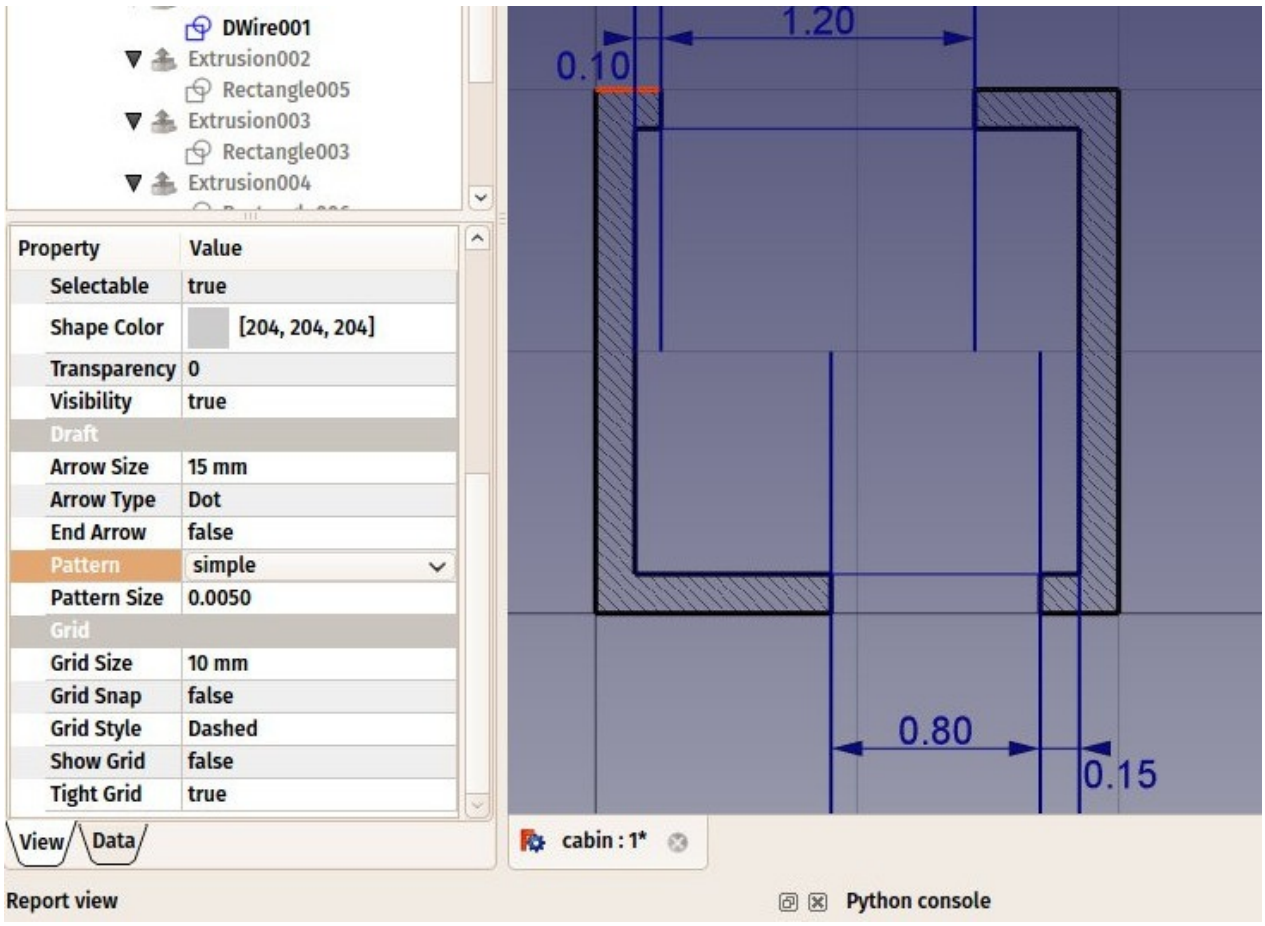
- Після цього за допомогою інструмента «Лінія» (Line) можна накреслити кілька вертикальних ліній, які визначать місця розташування дверей і вікон. Точки перетину цих ліній із нашими прямокутниками стануть у пригоді для прив'язки стін. Накресліть першу лінію від точки (15 см, 1 м, 0) до точки (15 см, 3 м, 0).
- Створіть 5 копій цієї лінії за допомогою інструмента «Переміщення» (Move), увімкнувши режим копіювання (Copy mode). Також увімкніть відносний режим (Relative mode) — це дозволить задавати переміщення відносно поточної позиції, що значно простіше, ніж щоразу розраховувати точні координати кожної лінії. Для кожної нової копії задайте будь-яку початкову точку (наприклад, можна залишити її в координатах (0,0,0)) та вкажіть наведені нижче відносні кінцеві точки:
  - line001: x: 10 см
  - line002: x: 120 см
  - line003: x: -55 см, y: -2 м
  - line004: x: 80 см
  - line005: x: 15 см



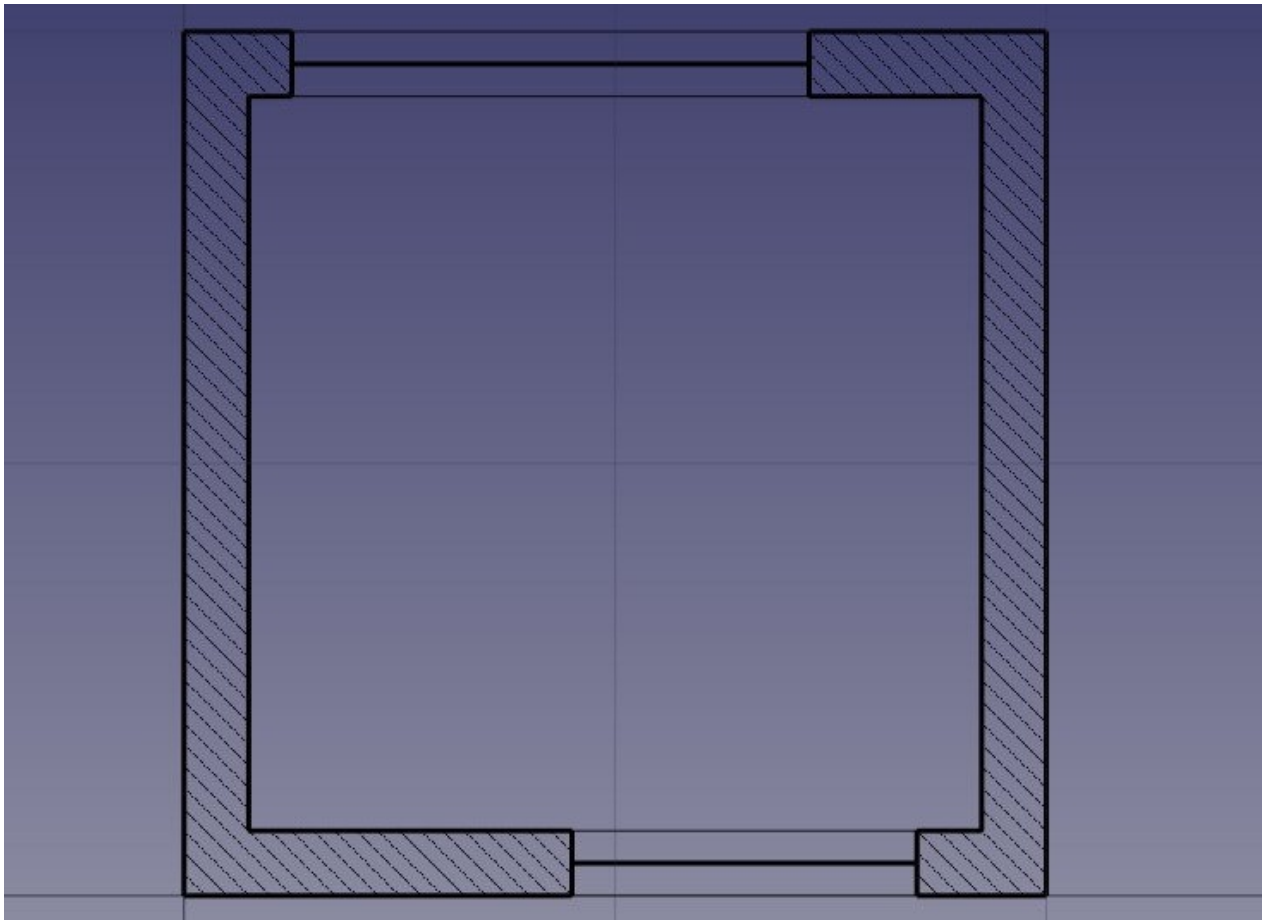
- Це все, що нам зараз потрібно, тому можна вимкнути режим побудови. Переконайтеся, що всі допоміжні геометричні елементи розміщені в групі «Construction» (Побудова) — це дозволить легко приховати їх усі одразу або навіть повністю видалити згодом.
- Тепер намалюймо два фрагменти стін за допомогою інструмента «Wire» (Лінія/Контур). Переконайтеся, що ввімкнено прив’язку до перетинів («Intersection snap»), оскільки нам знадобиться прив’язуватися до точок перетину ліній і прямокутників. Побудуйте два контури, як показано далі, послідовно вказуючи всі точки їхніх обрисів. Щоб замкнути контур, знову клацніть на першу точку або натисніть кнопку «Close» (Замкнути):



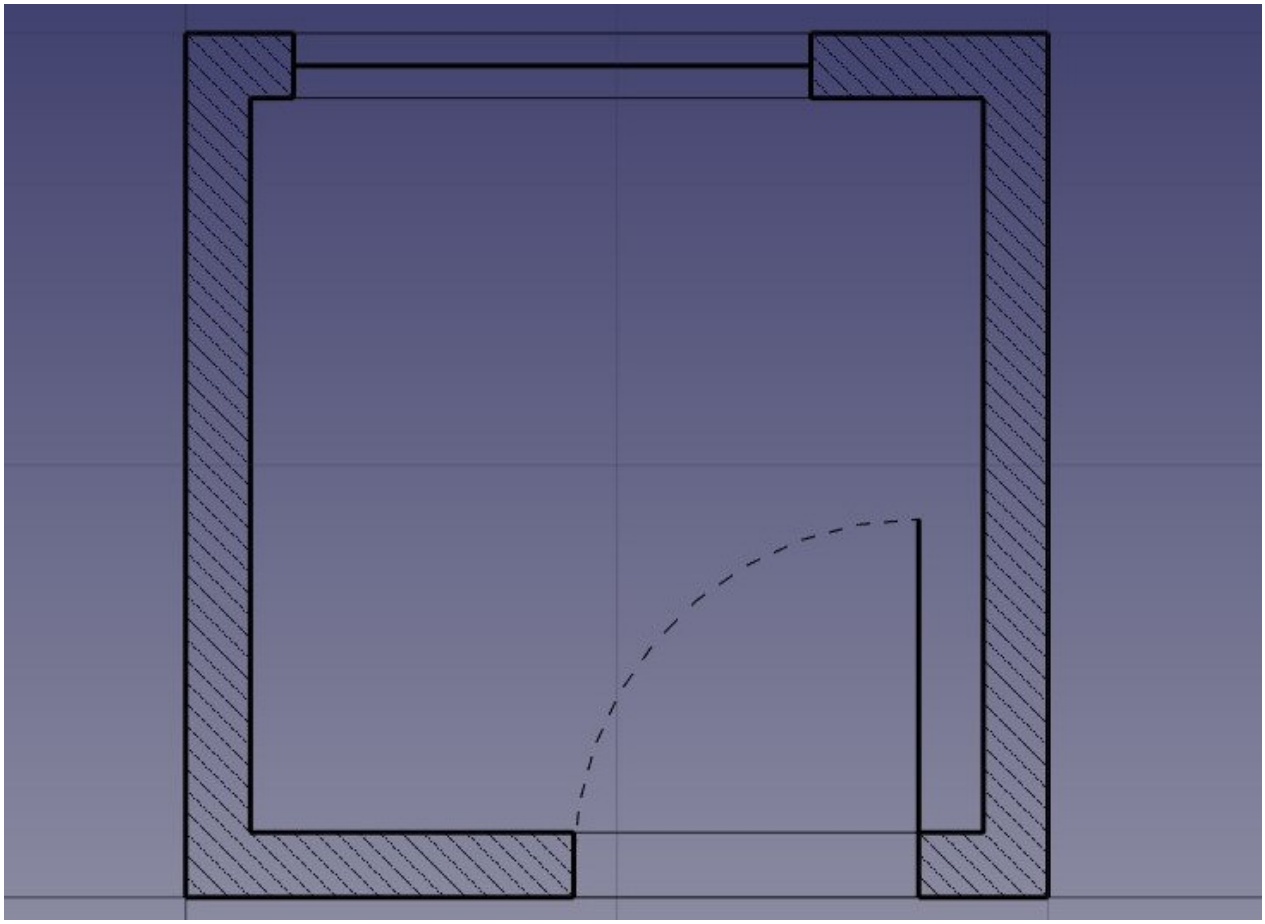
- Ми можемо змінити їхній стандартний сірий колір на гарний візерунок штрихування: для цього виберіть обидві стіни, встановіть для них властивість **«Візерунок» (Pattern)** на значення **«Простий» (Simple)**, а розмір візерунка — на бажане значення, наприклад 0,005.



- Тепер ми можемо приховати допоміжну геометрію: клацніть правою кнопкою миші на групі **«Construction» (Допоміжні елементи)** і виберіть **«Hide Selection» (Приховати вибране)**.
- Тепер намалюймо вікна та двері. Переконайтеся, що ввімкнено прив’язку до середньої точки (midpoint snap), і накресліть шість ліній, як показано нижче:

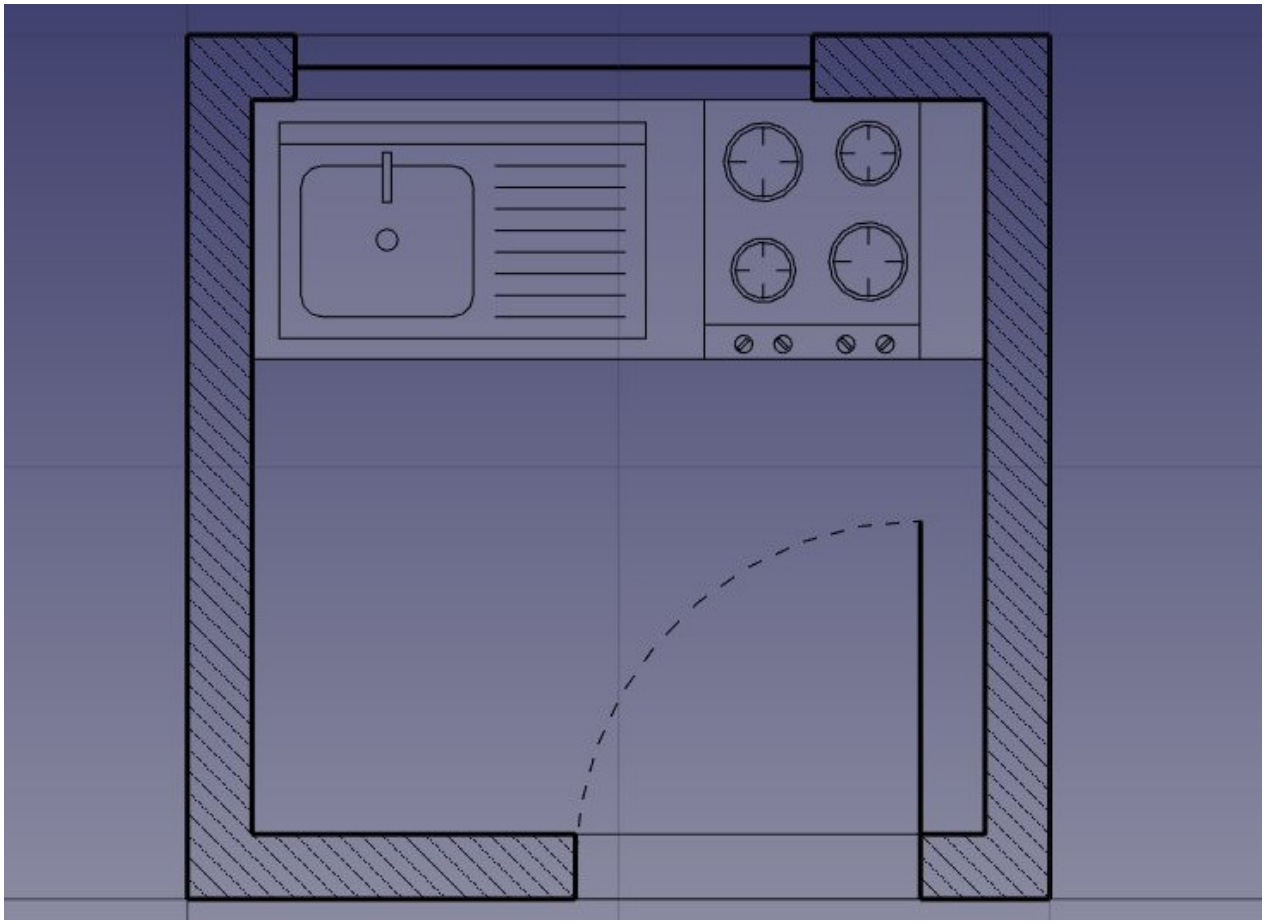


- Тепер ми змінимо лінію дверей, щоб створити умовне позначення відчинених дверей. Почніть із повороту лінії за допомогою інструмента «Поворот» (Rotate). Виберіть кінцеву точку лінії як центр обертання, задайте початковий кут **0**, а кінцевий кут — **-90**.
- Після цього створіть дугу, що позначає відчинення, за допомогою інструмента «Дуга» (Arc). Як центр дуги вкажіть ту саму точку, що використовувалася як центр обертання на попередньому кроці; клацніть на іншу кінцеву точку лінії, щоб визначити радіус, а потім укажіть початкову та кінцеву точки дуги, як описано далі:

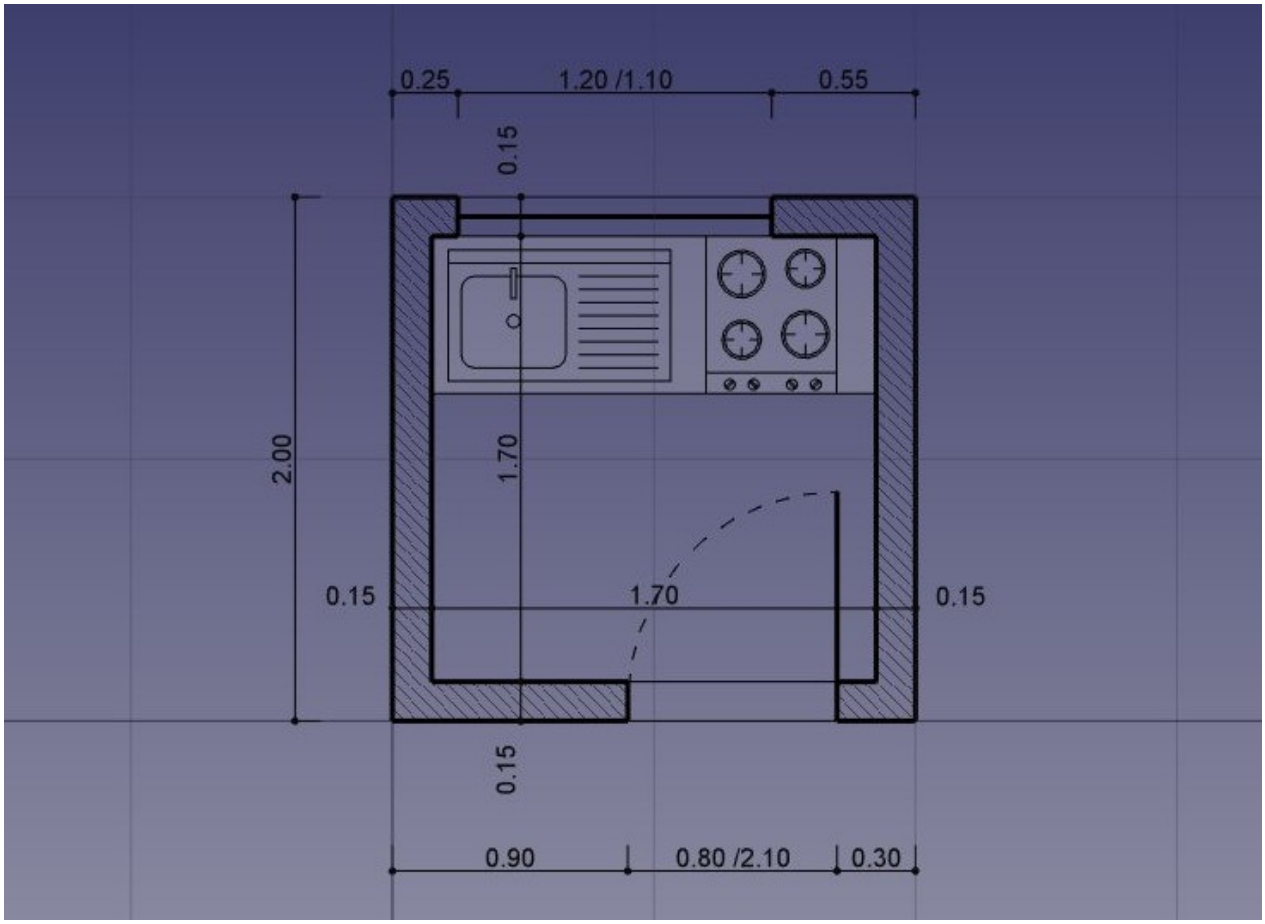


- Тепер можна розпочати розміщення меблів. Для початку створимо стільницю: намалюємо прямокутник, починаючи з внутрішнього верхнього лівого кута, задавши йому ширину 170 см і висоту -60 см. На зображенні нижче для цього прямокутника встановлено параметр **«Прозорість» (Transparency)** на рівні 80%, що надає йому вигляду, характерного для меблів.
- Далі додамо мийку та варильну поверхню. Самостійне креслення таких елементів може бути досить марудною справою, до того ж їх зазвичай легко знайти в інтернеті — наприклад, на сайті http://www.cad-blocks.net. Для зручності ми підготували окремі файли мийки та варильної поверхні з цього ресурсу у форматі DXF; ви можете завантажити їх за наведеними нижче посиланнями. Для цього клацніть правою кнопкою миші на кнопці **«Raw»** і виберіть опцію збереження файлу.

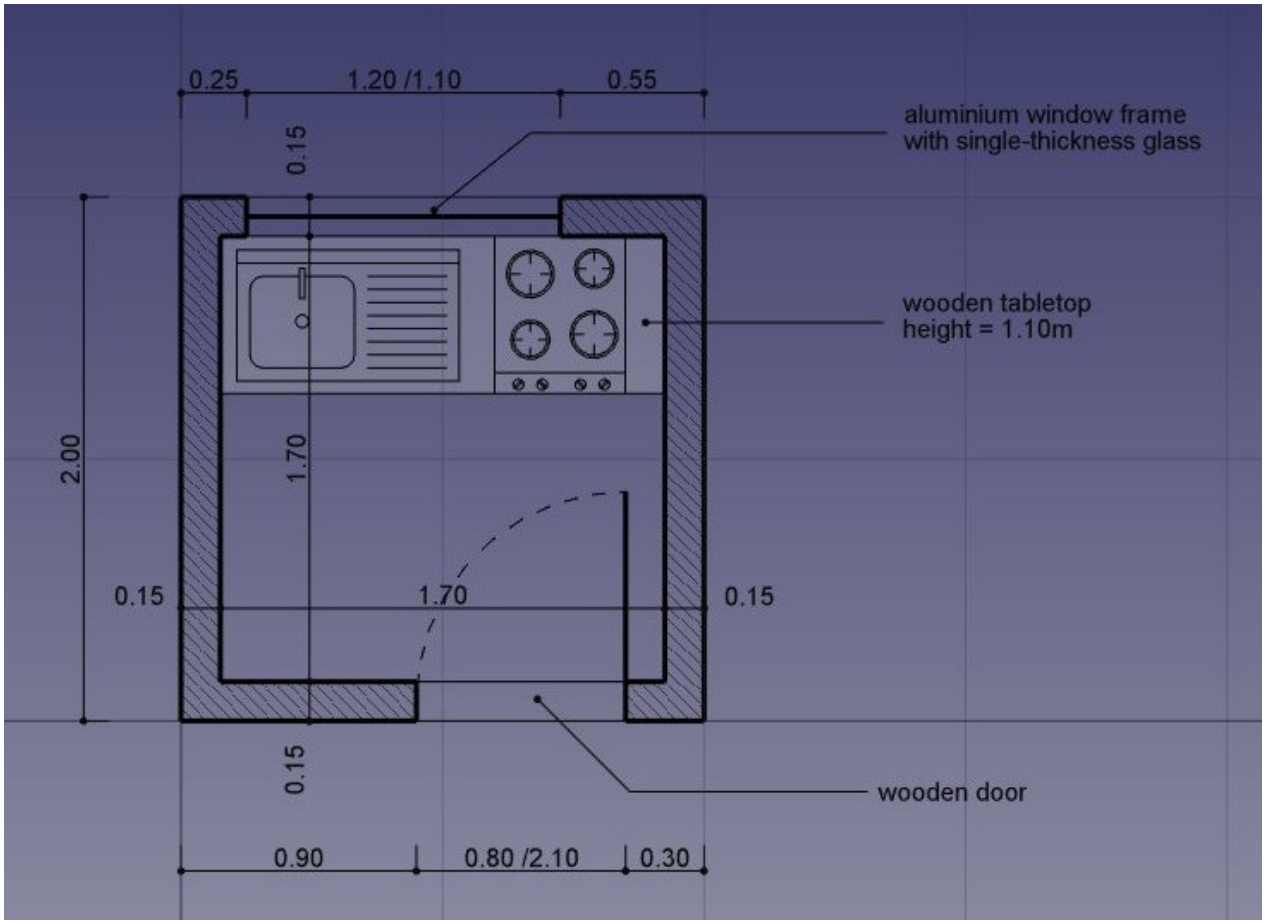
- Щоб вставити DXF-файл у відкритий документ FreeCAD, можна скористатися пунктом меню **«Файл» -> «Імпорт» (File -> Import)** або просто перетягнути файл із провідника вашої операційної системи безпосередньо у вікно програми FreeCAD. Вміст DXF-файлу може з'явитися не в центрі робочої області — це залежить від того, де саме об'єкти були розташовані у вихідному файлі. Щоб знайти імпортовані елементи, скористайтеся меню **«Вигляд» -> «Стандартні види» -> «Показати все» (View -> Standard views -> Fit all)** — це дозволить змінити масштаб огляду. Вставте обидва DXF-файли та розмістіть їх у відповідних місцях на стільниці:



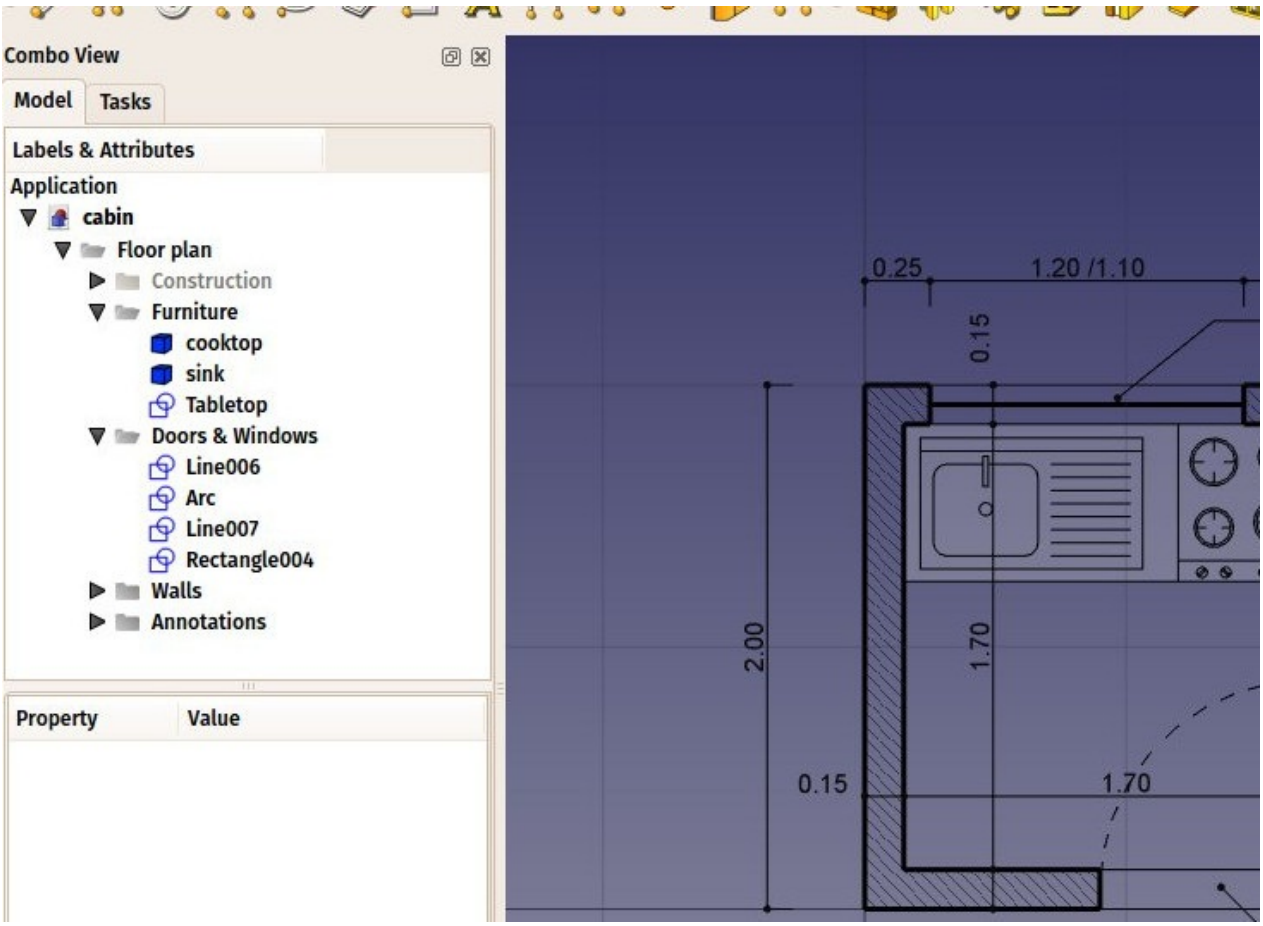
- Тепер ми можемо додати кілька розмірів за допомогою інструмента «Розмір» (Dimension). Розміри створюються шляхом вибору трьох точок: початкової, кінцевої та третьої точки, яка визначає розташування розмірної лінії. Щоб створити горизонтальний або вертикальний розмір (навіть якщо перші дві точки не розташовані на одній лінії), утримуйте клавішу **Shift** під час вибору другої точки.
- Ви можете змінити положення тексту розміру, двічі клацнувши на відповідний розмір у дереві об'єктів. З'явиться контрольна точка, за допомогою якої можна переміщувати текст у графічному режимі. У нашому прикладі написи «0,15» були зміщені вбік для кращої наочності.
- Вміст тексту розміру можна змінити, відредагувавши властивість **«Перевизначення» (Override)**. У нашому прикладі текст розмірів для дверей та вікон було змінено так, щоб вказати їхню висоту:



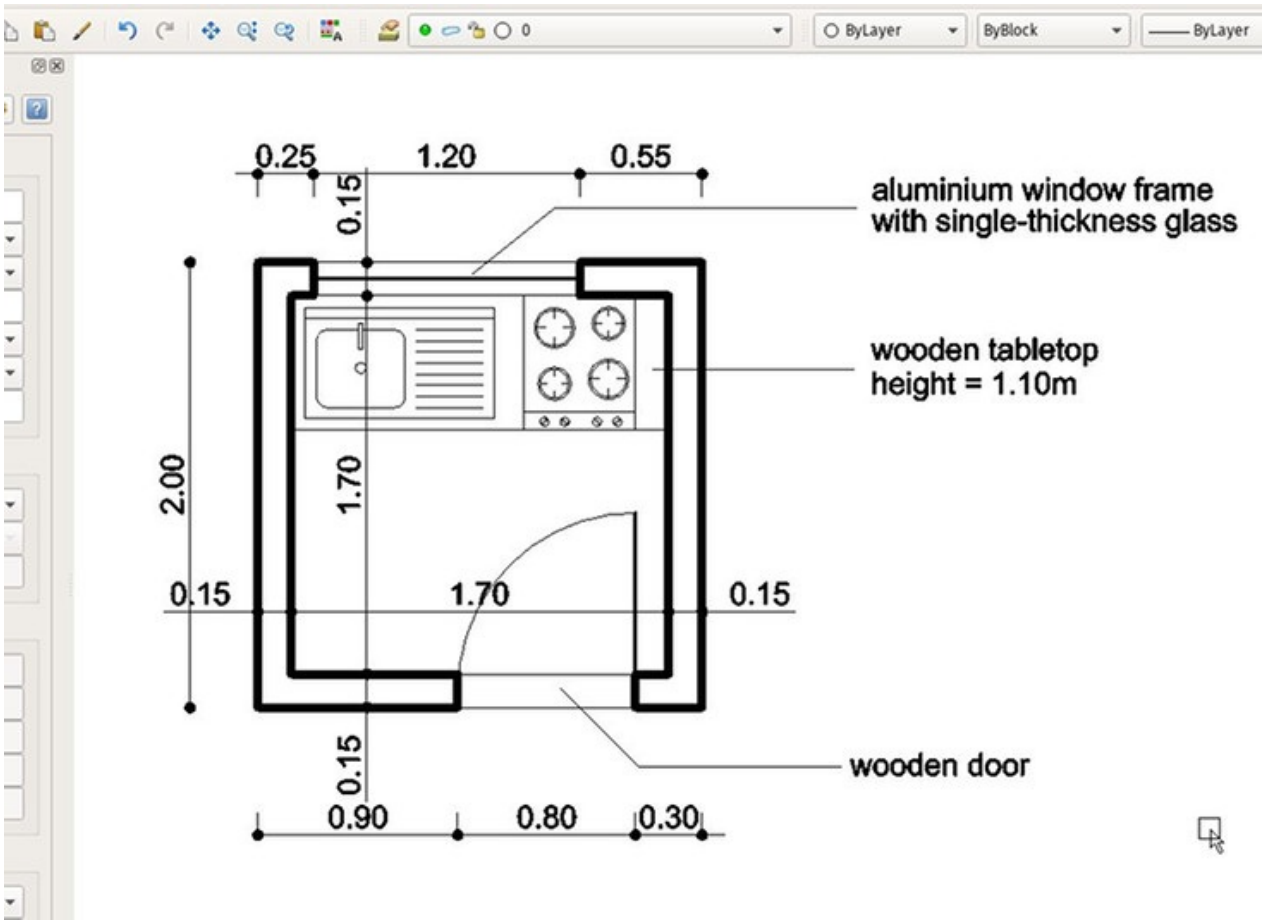
- Додамо кілька пояснювальних написів за допомогою інструмента «Текст» (Text). Клацніть у потрібному місці, щоб розмістити текст, а потім введіть рядки тексту, натискаючи клавішу Enter після кожного з них. Щоб завершити, натисніть Enter двічі.
- Лінії-вказівники (їх також називають «виносками»), що з’єднують написи з об’єктами, які вони описують, можна легко створити за допомогою інструмента «Лінія» (Wire). Проведіть лінії від місця розташування тексту до об’єкта, який потрібно описати. Після цього на кінцях ліній можна додати маркер або стрілку, встановивши для властивості **«Стрілка на кінці» (End Arrow)** значення **«Істина»** (True).



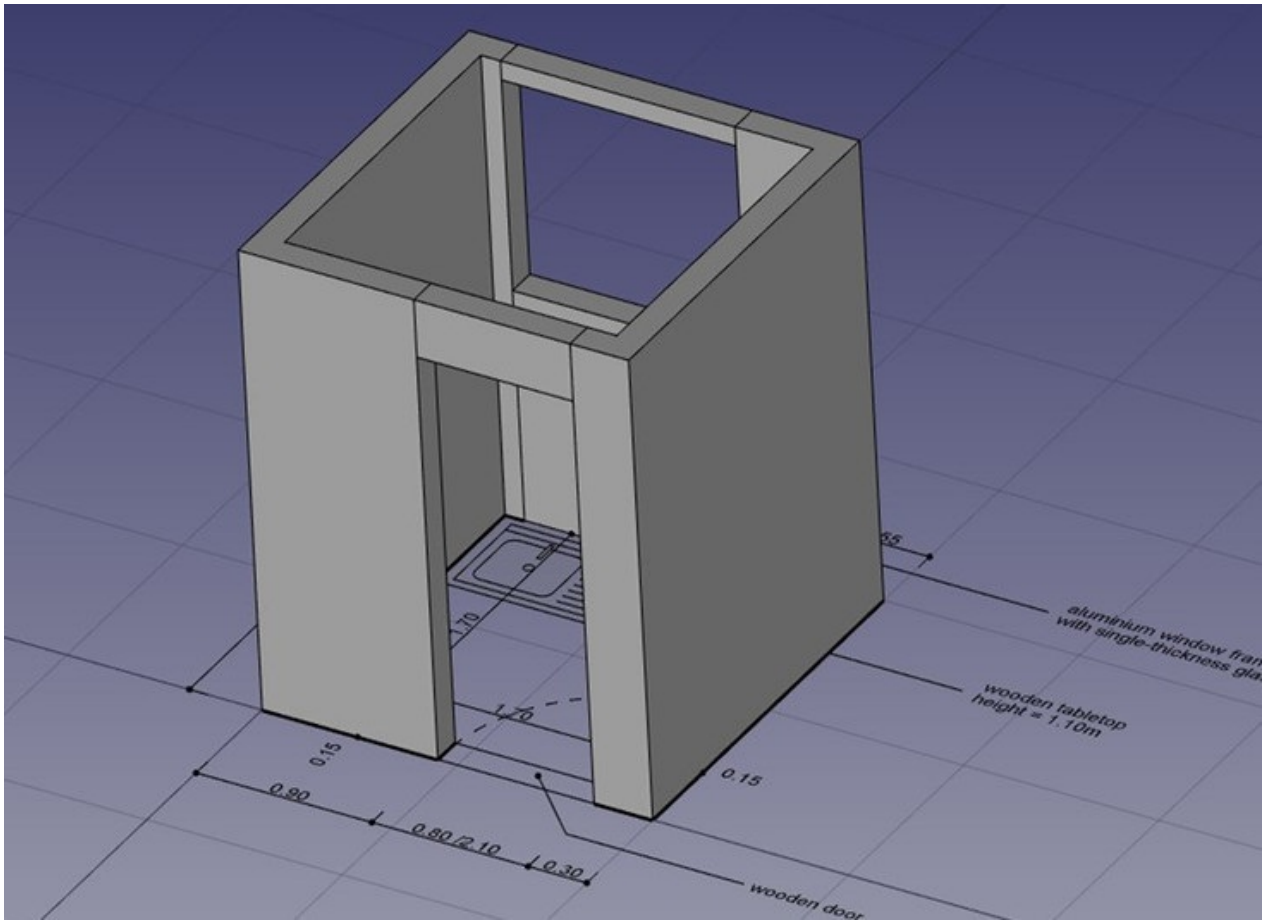
- Наше креслення тепер завершено! Оскільки об’єктів стало вже досить багато, варто навести лад і впорядкувати все за допомогою груп — це допоможе іншій людині легше розібратися у файлі:



- Тепер ми можемо роздрукувати нашу роботу, розмістивши її на аркуші креслення (про що йтиметься далі в цьому посібнику), або ж експортувати креслення в інші CAD-програми, зберігши його у форматі DXF. Для цього достатньо вибрати групу «План поверху» («Floor plan»), перейти в меню **«Файл» -> «Експорт» («File» -> «Export»)** і вибрати формат Autodesk DXF. Отриманий файл можна відкрити в будь-якій іншій 2D CAD-програмі, наприклад, у LibreCAD. Залежно від налаштувань конкретної програми, можуть спостерігатися певні відмінності у відображенні.



- Однак найважливіша особливість верстака Draft полягає в тому, що створену за його допомогою геометрію можна використовувати як основу або легко перетворити на 3D-об'єкти шляхом видавлювання. Для цього можна скористатися інструментом «Видавлювання» (Extrude) з верстака Part або — залишаючись у середовищі Draft — інструментом Trimex («Обрізати/Подовжити/Видавити»). Останній, по суті, виконує операцію видавлювання з верстака Part, але робить це «у стилі Draft»: дозволяє графічно вказати довжину видавлювання та прив'язати її до певних точок. Спробуйте виконати видавлювання наших стін, як показано нижче.
- Вибравши грань об'єкта й натиснувши кнопку «Робоча площина» (Working plane), ви можете розмістити цю площину в будь-якому місці. Це дозволяє створювати об'єкти Draft у різних площинах — наприклад, поверх стін. Згодом їх можна видавити, щоб отримати нові 3D-тіла. Спробуйте встановити робочу площину на одну з верхніх граней стін, а потім накресліть там кілька прямокутників.



- Будь-які отвори можна так само легко створити, намалювавши об'єкти в середовищі Draft на гранях стін, виконавши їх видавлювання, а потім застосувавши булеві операції з верстака Part, щоб відняти їх від іншого твердотільного об'єкта — як ми вже бачили в попередньому розділі.

По суті, середовище Draft надає графічні засоби для виконання базових операцій, доступних у верстаку Part. Якщо в середовищі Part ви зазвичай розміщуєте об'єкти, вручну задаючи їхні параметри розташування (властивість «placement»), то в Draft це можна робити безпосередньо на екрані. Бувають ситуації, коли зручніше використовувати один підхід, а бувають — коли краще підходить інший. Пам'ятайте: ви можете створити власні панелі інструментів в одному з цих верстаків, додати до них інструменти з іншого — і таким чином отримати переваги обох середовищ.

Завантаження

- Файл, створений під час виконання цієї вправи: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/cabin.FCStd
- DXF-файл із зображенням раковини: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/sink.dxf
- DXF-файл із зображенням варильної поверхні: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/cooktop.dxf
- Кінцевий DXF-файл, отриманий у результаті цієї вправи: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/cabin.dxf

Додаткова інформація

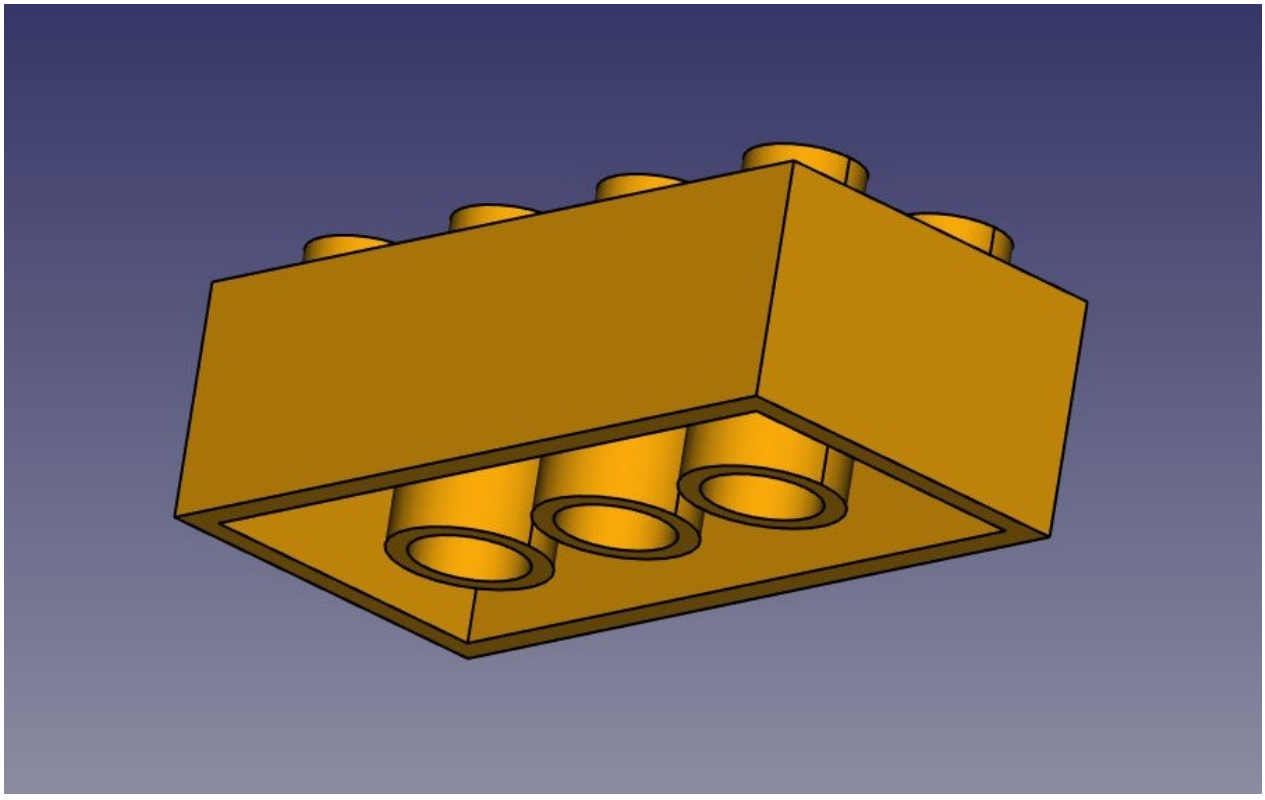
- Середовище Draft: http://www.freecadweb.org/wiki/index.php?title=Draft_Module
- Прив'язка (Snapping): http://www.freecadweb.org/wiki/index.php?title=Draft_Snap
- Робоча площина в Draft: http://www.freecadweb.org/wiki/index.php?title=Draft_SelectPlane

Моделювання для промислового дизайну

Термін «промисловий дизайн» (product design) походить зі сфери комерції, проте у світі 3D-технологій він часто означає створення моделі об'єкта з метою його подальшого виготовлення — чи то за допомогою 3D-принтера, чи то на верстаті з ЧПК (числовим програмним керуванням).

При створенні об'єктів за допомогою 3D-друку вкрай важливо, щоб вони були **«суцільними» (solid)**. Це цілком логічно, адже в результаті ви отримаєте реальні фізичні предмети. Звісно, вони можуть бути порожнистими всередині, проте ви завжди повинні чітко розуміти, де проходить межа між матеріалом і порожнечею. 3D-принтер або верстат із ЧПК має точно знати, які ділянки мають бути заповнені матеріалом, а які — ні. Саме тому в програмі FreeCAD для створення таких деталей ідеально підходить робоче середовище «Part Design» (Проектування деталей): воно автоматично гарантує, що ваші об'єкти залишатимуться суцільними та придатними для виготовлення.

Щоб продемонструвати роботу середовища «Part Design», змоделюємо добре відомий елемент конструктора Lego:



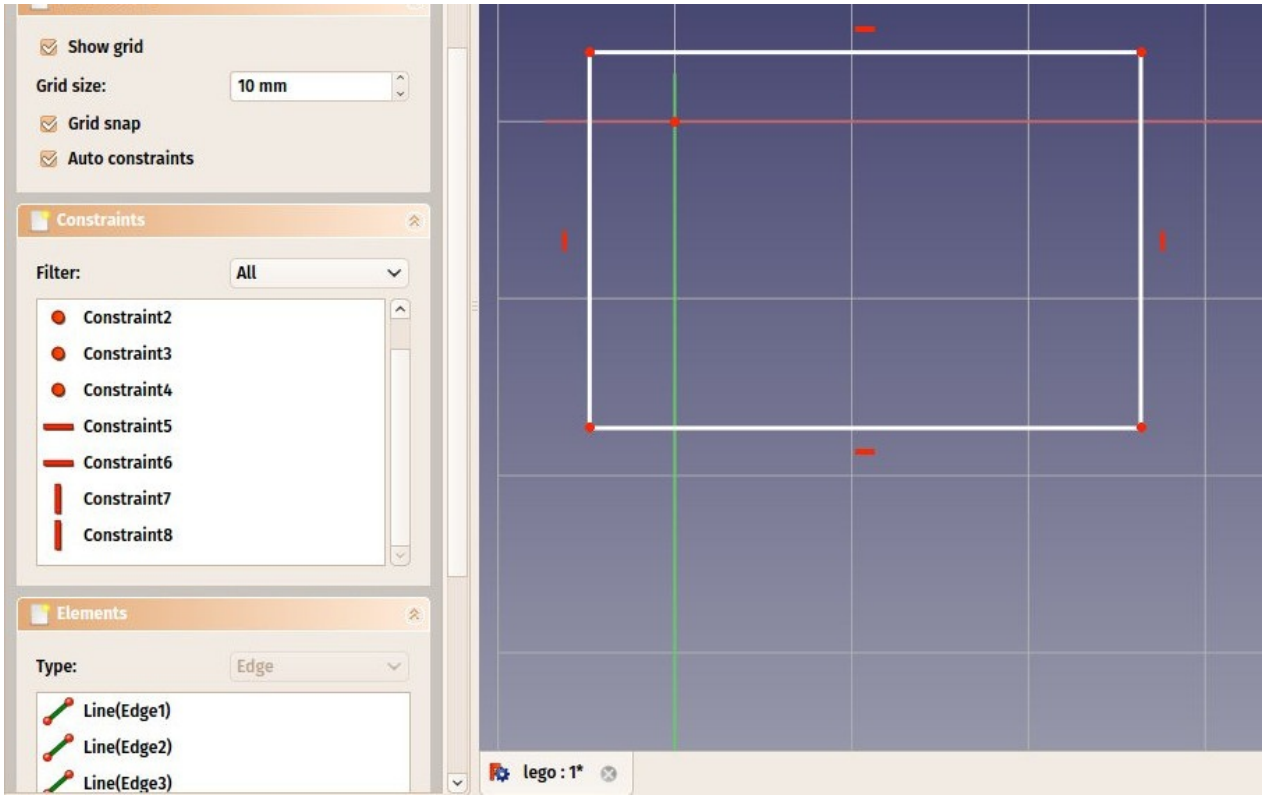
Перевага деталей Lego полягає в тому, що їхні розміри легко знайти в інтернеті (принаймні для стандартних елементів). Такі деталі досить просто змоделювати та надрукувати на 3D-принтері. Якщо набратися трохи терпіння (адже 3D-друк часто потребує ретельного налаштування та коригування), можна створити елементи, які будуть повністю сумісні з оригінальними блоками Lego та ідеально з'єднуватимуться з ними. У наведеному нижче прикладі ми створимо деталь, яка в 1,5 раза більша за оригінал.

Ми використовуватимемо виключно інструменти середовищ «Sketcher» (Ескіз) та «Part Design». Оскільки всі інструменти з «Sketcher» також доступні в «Part Design», нам не доведеться постійно перемикатися між ними — ми зможемо працювати виключно в середовищі «Part Design».

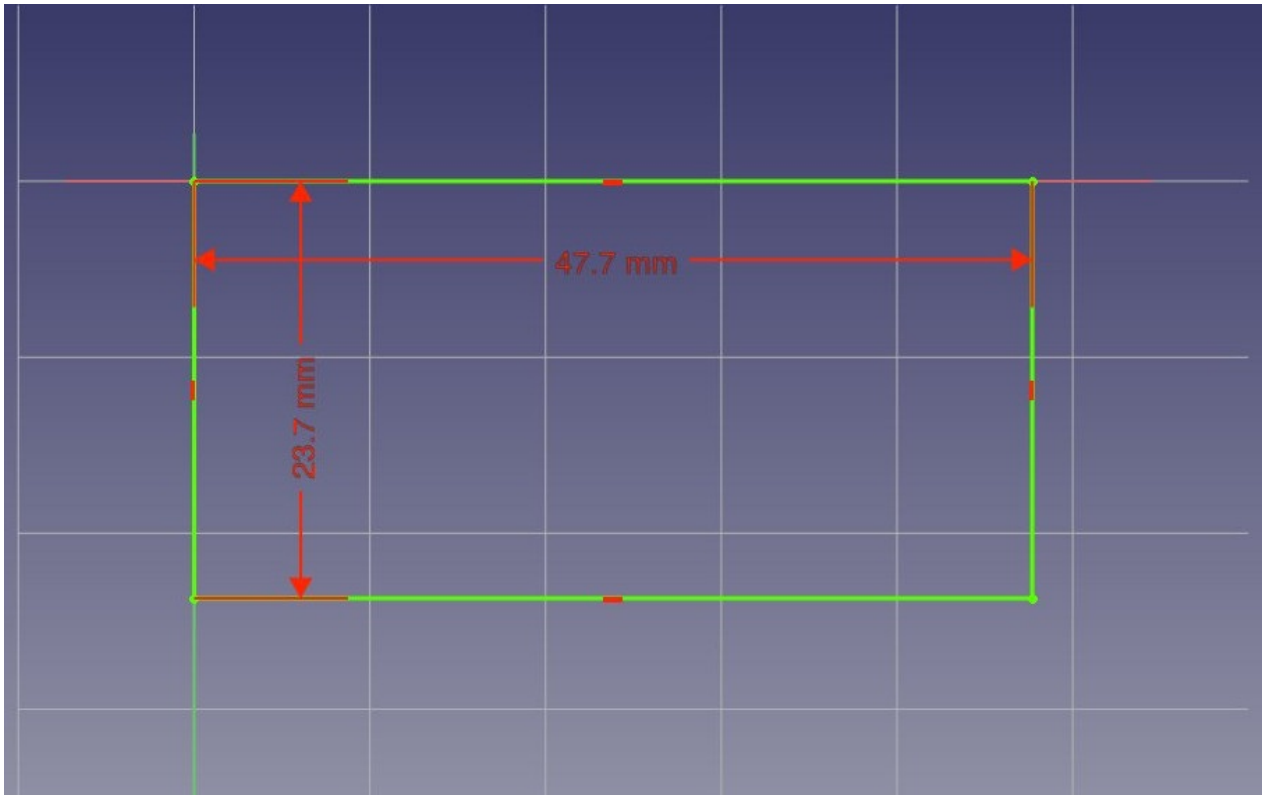
Об'єкти в «Part Design» повністю базуються на **ескізах (sketches)**. Ескіз — це 2D-об'єкт, що складається з лінійних сегментів (відрізків, дуг кіл або еліпсів) та обмежень (constraints). Ці обмеження застосовуються до лінійних сегментів, їхніх кінцевих точок або центрів; вони змушують геометрію підпорядковуватися певним правилам. Наприклад, до відрізка можна застосувати обмеження вертикальності, щоб він завжди залишався вертикальним, або обмеження фіксації (блокування) до кінцевої точки, щоб унеможливити її переміщення. Якщо ескіз має рівно стільки обмежень, скільки необхідно, щоб жодна його точка не могла змінити своє положення, такий ескіз називають повністю визначеним. Якщо ж присутні зайві обмеження, які можна видалити без ризику зміни геометрії, ескіз вважається надмірно визначеним. Цього слід уникати; FreeCAD повідомить вас, якщо виникне така ситуація.

Ескізи мають режим редагування, у якому можна змінювати їхню геометрію та обмеження. Після завершення редагування та виходу з цього режиму ескіз поводиться як будь-який інший об'єкт FreeCAD. Його можна використовувати як основу для побудови моделей за допомогою інструментів верстака Part Design, а також в інших середовищах, таких як Part або Arch. У верстаку Draft також є інструмент для перетворення об'єктів Draft в ескізи (і навпаки).

- Розпочнімо з моделювання кубічної форми, яка стане основою нашого цеглинки Lego. Згодом ми сформуємо внутрішню порожнину та додамо 8 виступів (шипів) на верхній грані. Отже, почнімо зі створення прямокутного ескізу, який ми потім перетворимо на об'ємне тіло (видавимо):
- Перейдіть в середовище Part Design.
- Натисніть кнопку «Новий ескіз» (New Sketch). З'явиться діалогове вікно із запитанням, у якій площині розмістити ескіз — виберіть площину XY (площину «основи»). Ескіз буде створено, програма автоматично перейде в режим редагування, а вид повернеться так, щоб ви бачили ескіз ортогонально (прямо перед собою).
- Тепер можна накреслити прямокутник: виберіть інструмент «Прямокутник» (Rectangle) і вкажіть дві точки, що відповідають його кутам. Точки можна розмістити довільно, оскільки їхнє точне положення буде задано на наступному етапі.
- Ви помітите, що до прямокутника автоматично додалося кілька обмежень: вертикальні відрізки отримали обмеження вертикальності, горизонтальні — обмеження горизонтальності, а в кожному куті з'явилося обмеження збігу точок, яке «склеює» відрізки між собою. Ви можете спробувати перемістити прямокутник, перетягуючи його лінії мишею — уся геометрія залишатиметься підпорядкованою заданим обмеженням.

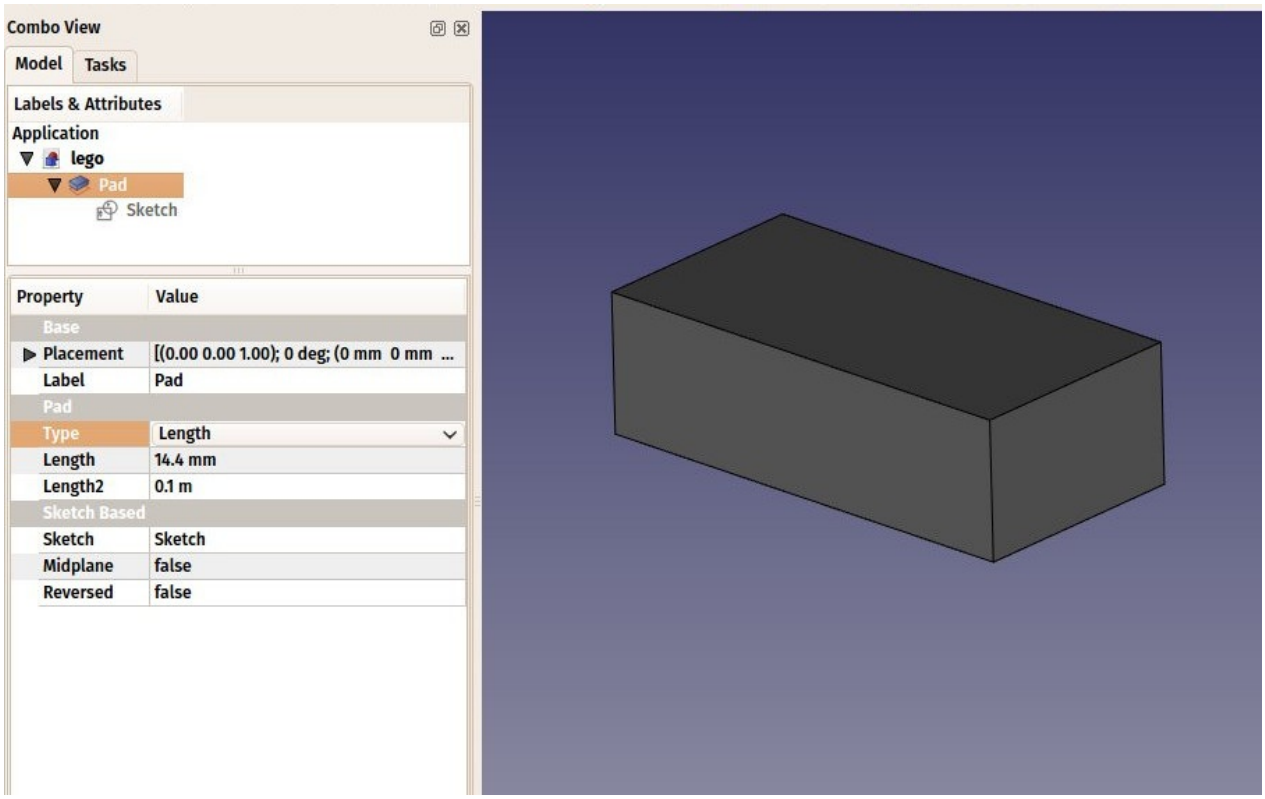


- Тепер додамо ще три обмеження:
  - Виберіть один із вертикальних відрізків і додайте обмеження «Вертикальна відстань». Встановіть його розмір рівним 23,7 мм.
  - Виберіть один із горизонтальних відрізків і додайте обмеження «Горизонтальна відстань». Встановіть значення 47,7 мм.
  - Нарешті, виберіть одну з кутових точок, а потім — початкову точку (точку на перетині червоної та зеленої осей) і додайте обмеження «Точка на точці». Після цього прямокутник переміститься в початкову точку, а ваш ескіз стане зеленим — це означає, що він повністю визначений (повністю обмежений). Якщо ви спробуєте перемістити його лінії або точки, нічого не зрушиться з місця.



Зауважте, що останнє обмеження «точка до точки» не було суворо необхідним. Вас ніхто не зобов’язує створювати ескізи, в яких усі елементи повністю визначені обмеженнями. Однак, якщо ми плануємо надрукувати цей блок на 3D-принтері, важливо, щоб деталь розташовувалася поблизу початку координат (який відповідатиме центру робочої зони, де переміщується друкувальна головка принтера). Додавши це обмеження, ми гарантуємо, що деталь залишатиметься «прив’язаною» до початку координат.

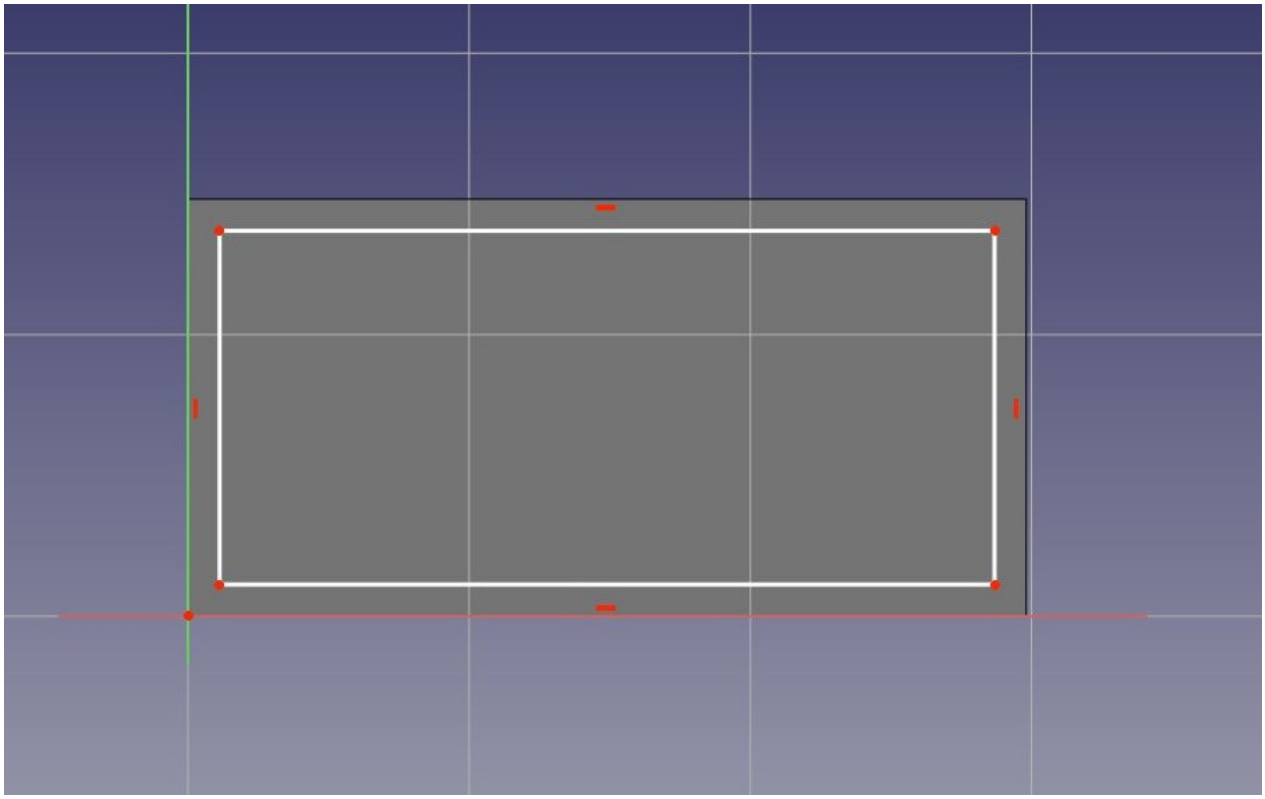
- Тепер наш базовий ескіз готовий. Вийти з режиму редагування можна, натиснувши кнопку **«Закрити» (Close)** у верхній частині панелі завдань або просто клавішу **Escape**. Якщо згодом виникне потреба, повернутися до редагування можна в будь-який момент, двічі клацнувши на ескіз у дереві побудови.
- Виконаємо операцію видавлювання за допомогою інструмента «Пад» (Pad), задавши відстань 14,4 мм. Інші параметри можна залишити без змін — зі значеннями за замовчуванням:



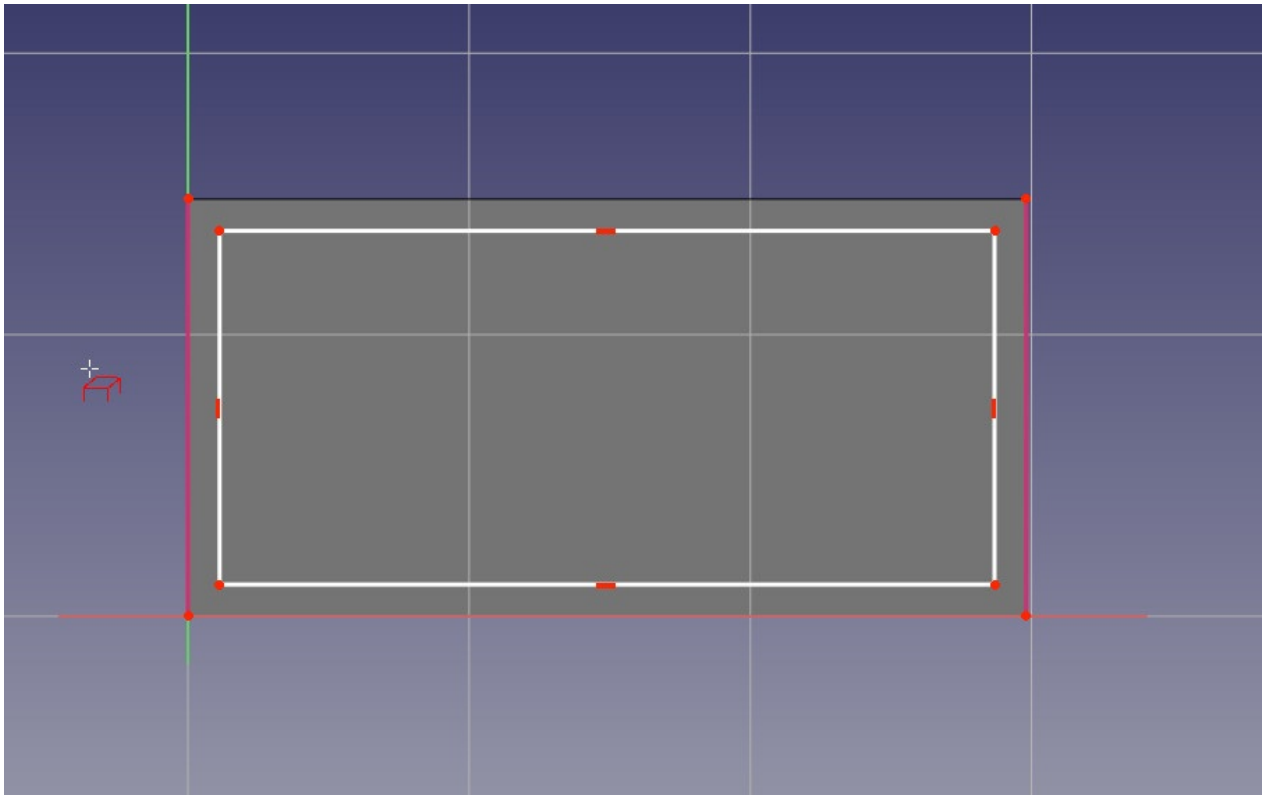
Інструмент **«Pad» (Видавлювання)** працює майже так само, як інструмент «Extrude» (Видавлювання) із середовища «Part», який ми використовували в попередньому розділі. Проте є кілька відмінностей; головна з них полягає в тому, що об’єкт, створений за допомогою «Pad», не можна перемістити — він назавжди прив’язаний до свого ескізу. Якщо ви хочете змінити положення цього елемента, вам доведеться перемістити базовий ескіз. У нашому випадку, коли важливо, щоб жоден елемент не змістився зі свого місця, це слугує додатковою гарантією стабільності.

- Тепер ми зробимо заглиблення всередині блоку за допомогою інструмента «Pocket» (Виріз) — це аналог інструмента «Cut» (Вирізання) із середовища «Part», але адаптований для середовища «PartDesign». Щоб створити заглиблення, ми побудуємо ескіз на нижній грані нашого блоку; цей ескіз визначить частину
- матеріалу, яку потрібно видалити.
- Вибравши нижню грань, натисніть кнопку «Новий ескіз» (New Sketch).

  Намалюйте прямокутник на цій грані.



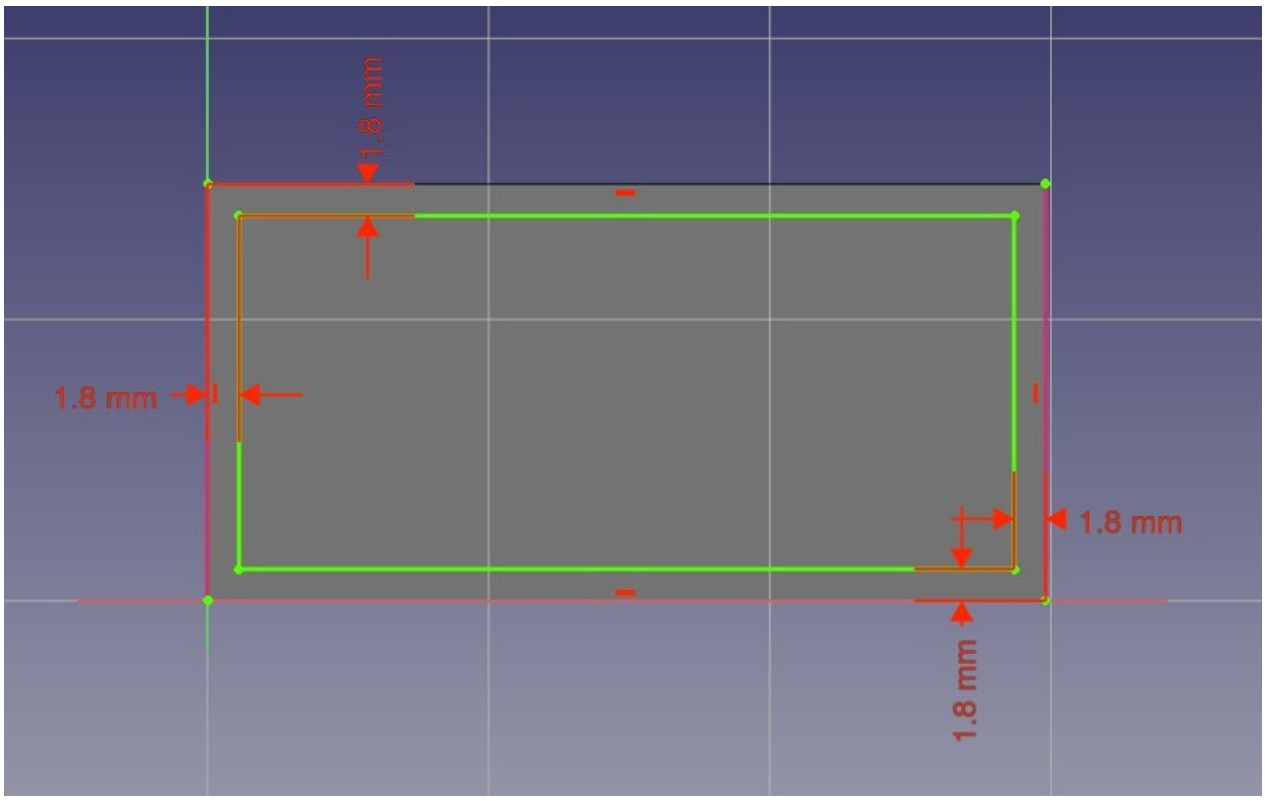
- Тепер ми прив’яжемо прямокутник відносно нижньої грані. Для цього потрібно «імпортувати» деякі ребра цієї грані за допомогою інструмента «Зовнішня геометрія» (External geometry). Застосуйте цей інструмент до двох вертикальних ліній нижньої грані:



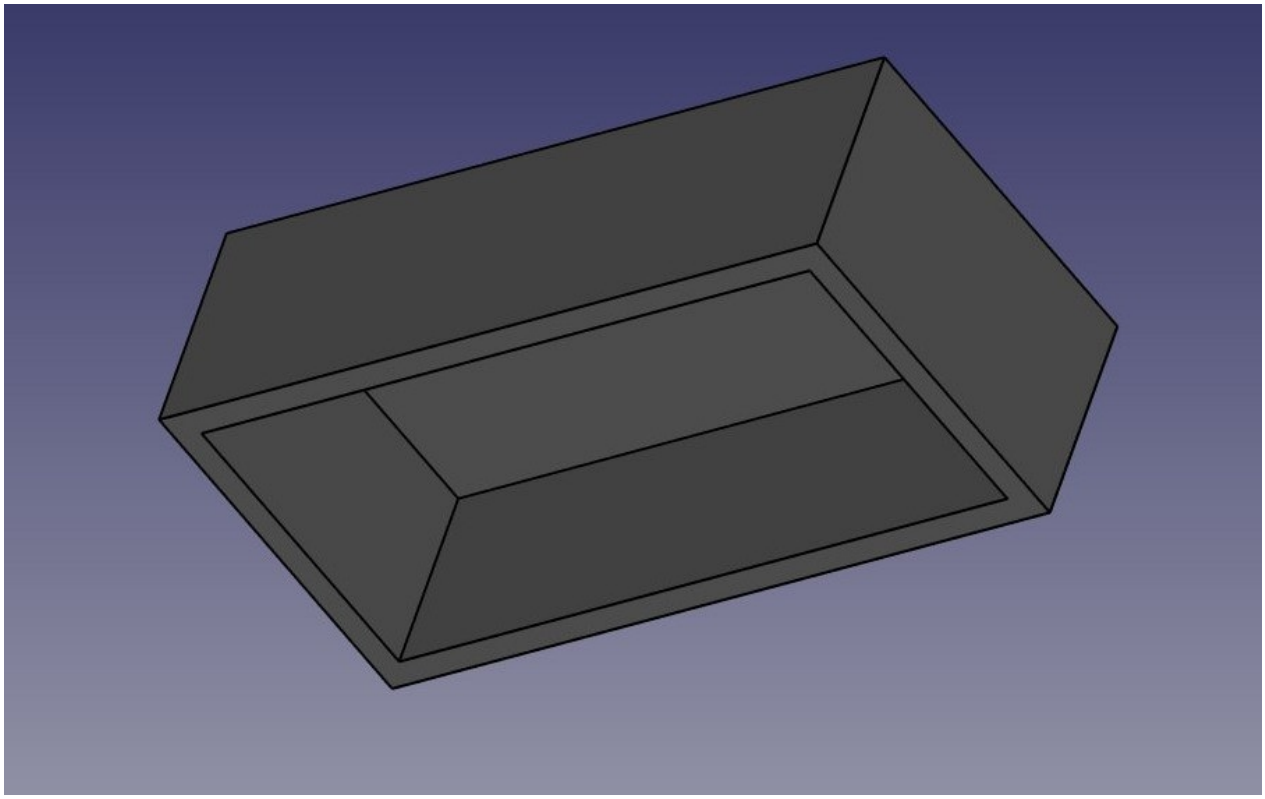
Ви помітите, що за допомогою цього інструмента можна додати лише ребра, які належать базовій грані. Коли ви створюєте ескіз із вибраною гранню, між цією гранню та ескізом встановлюється зв’язок, що важливо для подальших операцій. Згодом ви завжди зможете переприв’язати ескіз до іншої грані за допомогою

інструмента «Прив'язати ескіз» (Map Sketch).

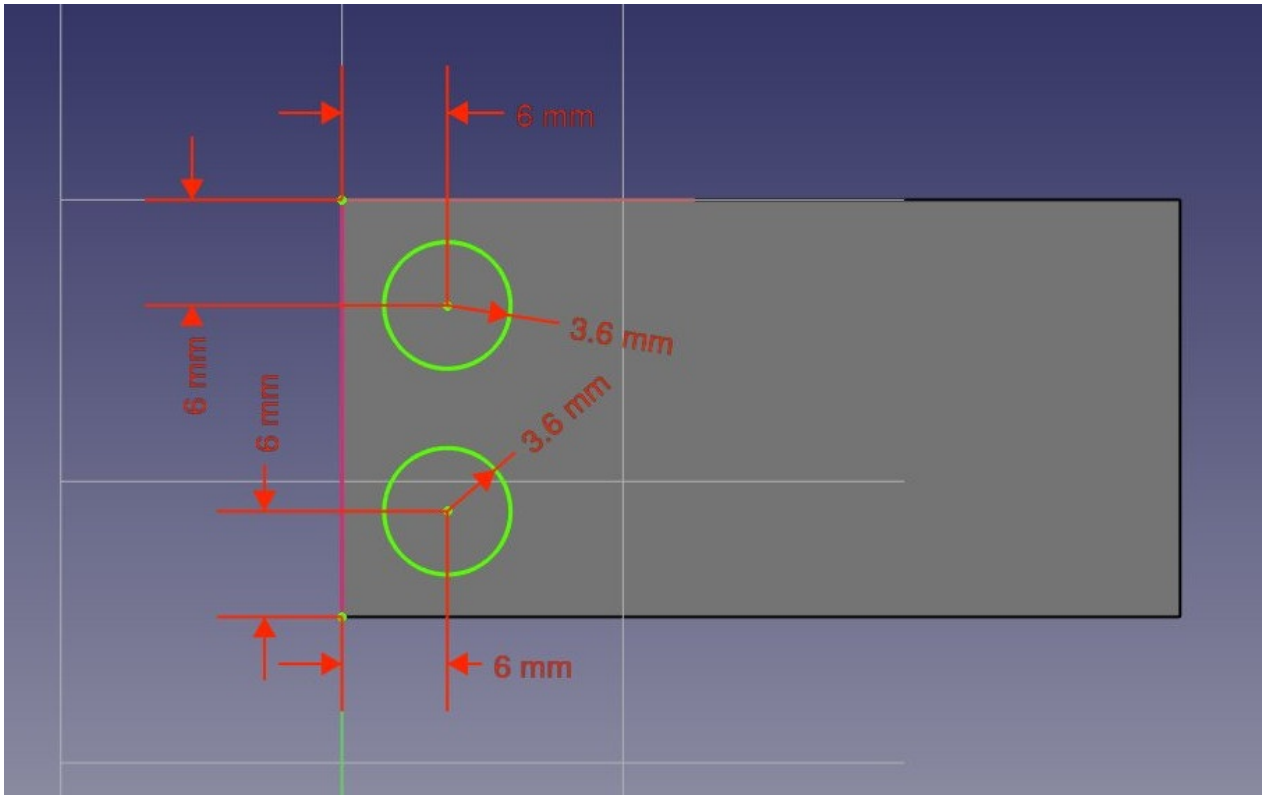
- The external geometry is not "real", it will be hidden when we leave edit mode. But we can use it to place constraints. Place the 4 following constraints:
  - Select the two upper left points of the rectangle and the left imported line and add a Horizontal Distance Constraint of 1.8mm
  - Select again the two upper left points of the rectangle and the left imported line and add a Vectical Distance Constraint of 1.8mm
  - Select the two lower right points of the rectangle and the right imported line and add a Horizontal Distance Constraint of 1.8mm
  - Select again the two lower right points of the rectangle and the right imported line and add a Vectical Distance Constraint of 1.8mm



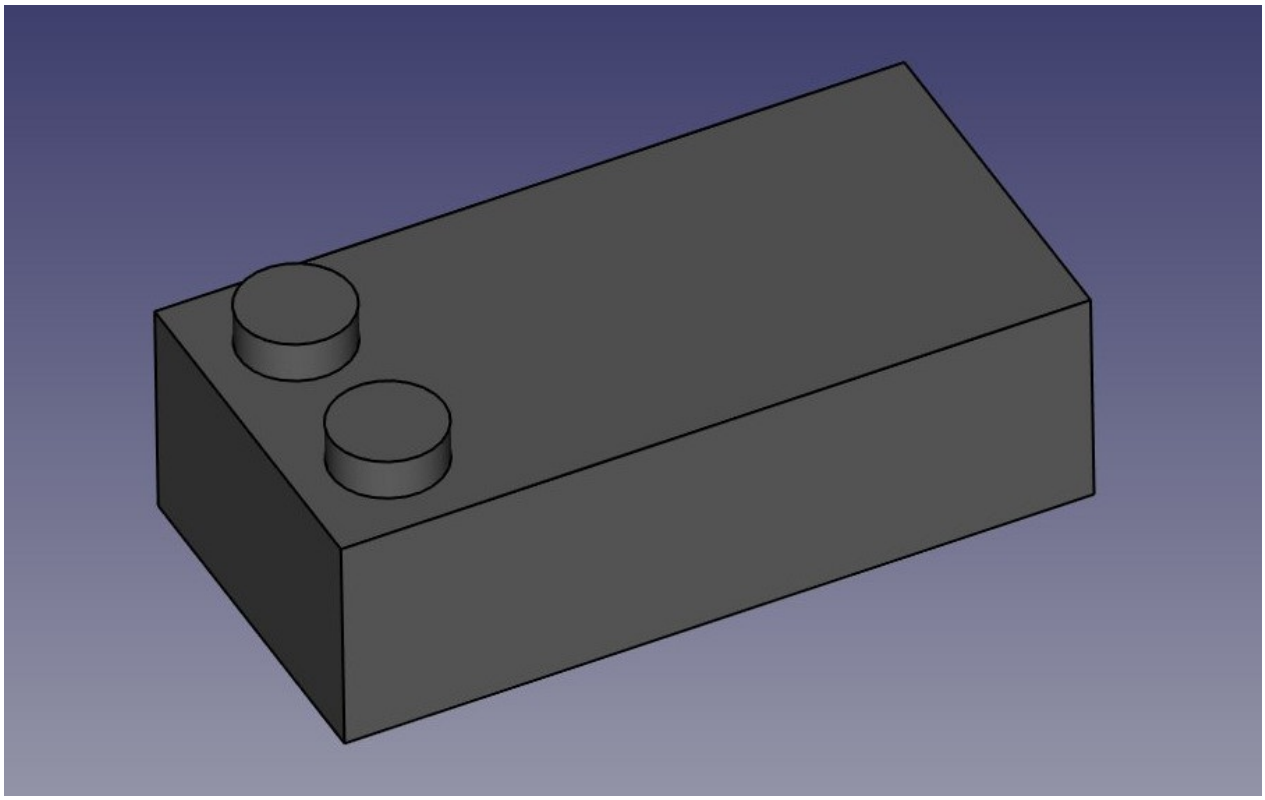
- Leave edit mode and we can now perform the pocket operation: With the sketch selected, press the Pocket buttton. Give it a length of 12.6mm, which will leave the upper face of our pad with a thickness of 1.8mm (remember, the total height of our pad was 14.4mm).



- We will now attack the 8 dots on the top face. To do this, since they are a repetition of a same feature, we will use the handy Linear Pattern tool of the Part Design Workbench, which allows to model once and repeat the shape.
- Start by selecting the top face of our block
- Create a New Sketch.
- Create two circles.
- Add a Radius Constraint of 3.6mm to each of them
- Import the left edge of the base face with the External geometry tool.
- Place two vertical constraints and two horizontal constraints of 6mm between the center point of each circle and the corner points of the imported edge, so each circle has its center at 6mm from the border of the face:



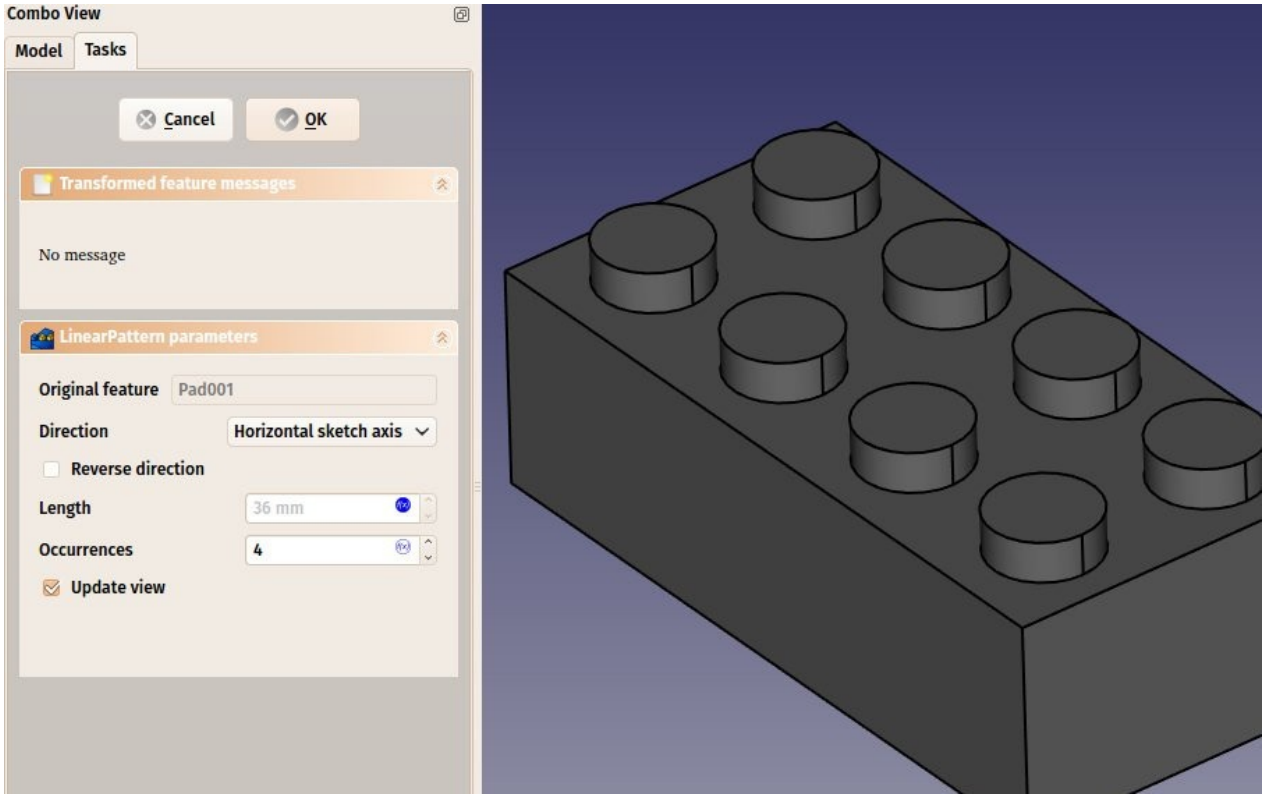
- Notice how, once again, when you lock the position and dimension of everything in your sketch, it becomes fully constrained. This always keeps you on the safe side. You could change the first sketch now, everything we did afterwards would keep tight.
- Leave edit mode, select this new sketch, and create a Pad of 2.7mm:



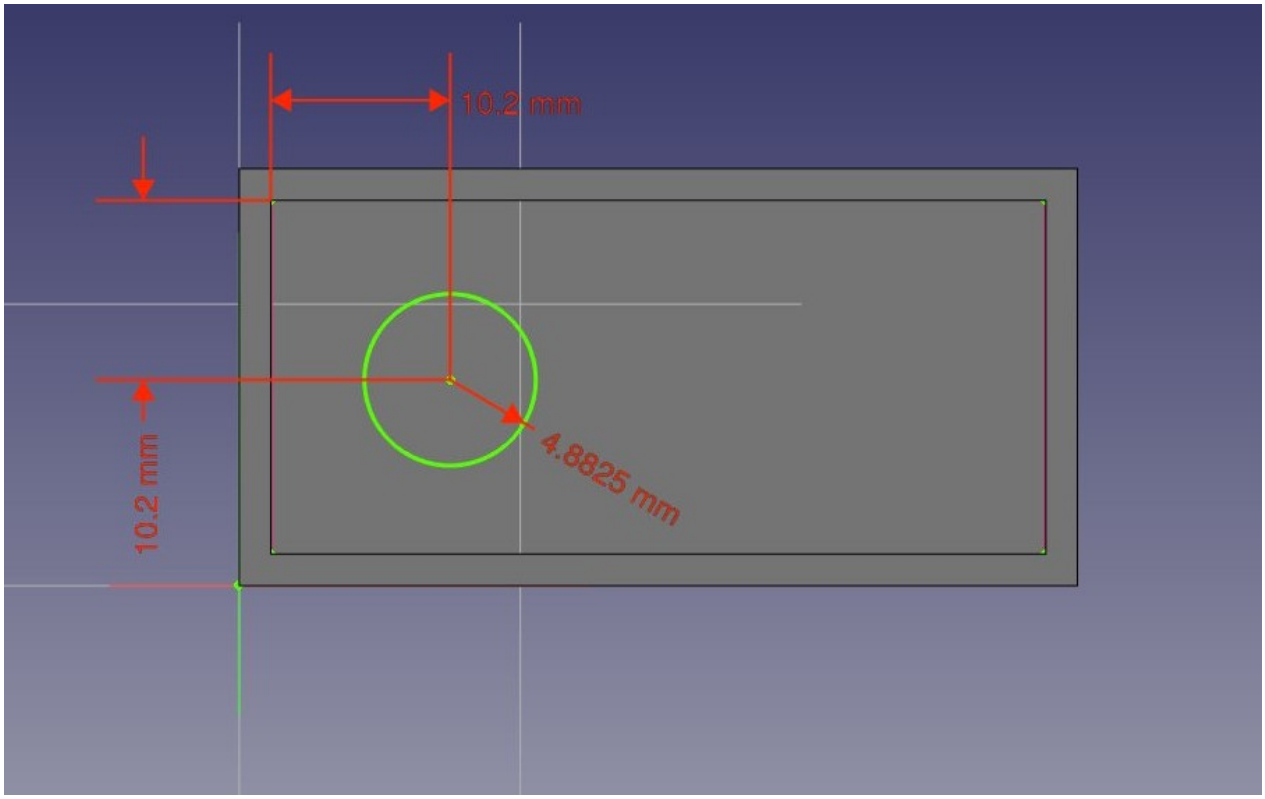
- Notice that, as earlier with the pocket, since we used the top face of our base block as a base for this latest sketch, any PartDesign operation we do with this sketch will correctly

be built on top of the base shape: The two dots are not independent objects, they have been extruded directly from our brick. This is the great advantage of working with the Part Design Workbench, as long as you take care of always building one step on top of the previous one, you are actually building one final solid object.

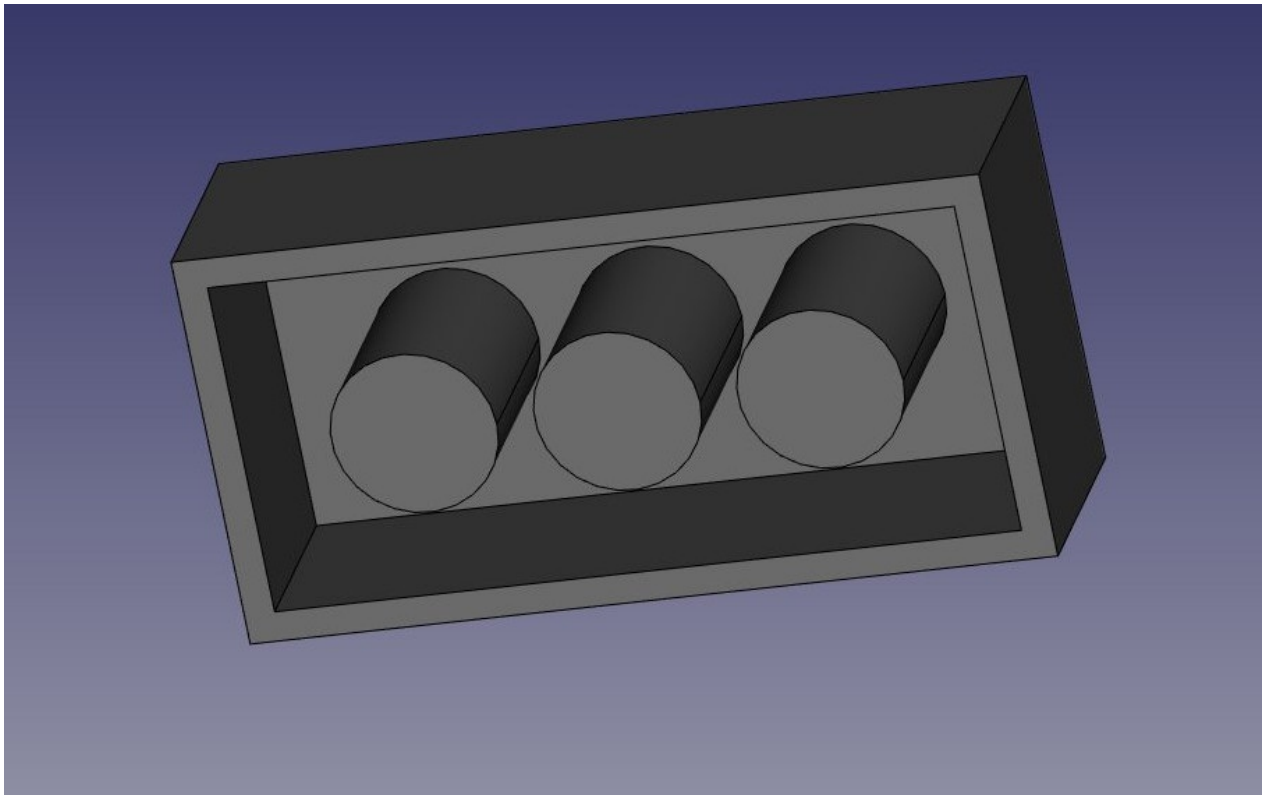
- We can now duplicate our two dots four times, so we get eight. Select the latest Pad we just created.
- Press the Linear Pattern button.
- Give it a length of 36mm (which is the total "span" we want our copies to fit in), in the "horizontal sketch axis" direction, and make it 4 occurences:



- Once again, see that this is not just a duplication of an object, it is a *feature* of our shape that has been duplicated, the final object is still only one solid object.
- Now let's work on the three "tubes" that fill the void we created on the bottom face. We have several possibilities: create a sketch with three circles, pad it then pocket it three times, or create a base sketch with one circle inside the other and pad it to form the complete tube already, or even other combinations. Like always in FreeCAD, there are many ways to do achieve a same result. Sometimes one way will not work the way we want, and we must try other ways. Here, we will take the safest approach, and do things one step at a time.
- Select the face that is at the bottom of the hollow space we carved earlier inside the block.
- Create a new sketch, add a circle with a radius of 4.8825mm, import the left border of the face, and constrain it vertically and horizontally at 10.2mm from the upper corner of the face:

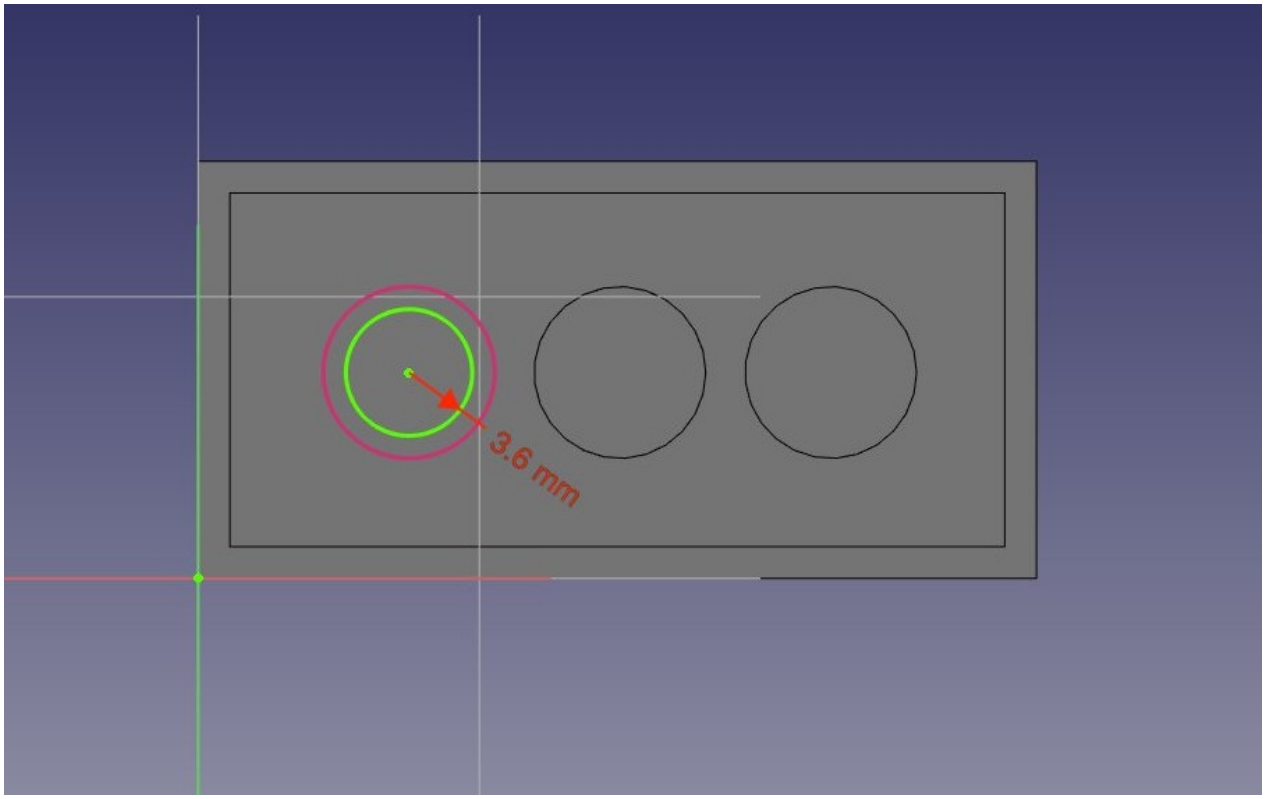


- Leave edit mode, and pad this sketch with a distance of 12.6mm
- Create a linear pattern from this last pad, give it a length of 24mm and 3 occurences. We now have three filled tubes filling the hollow space:

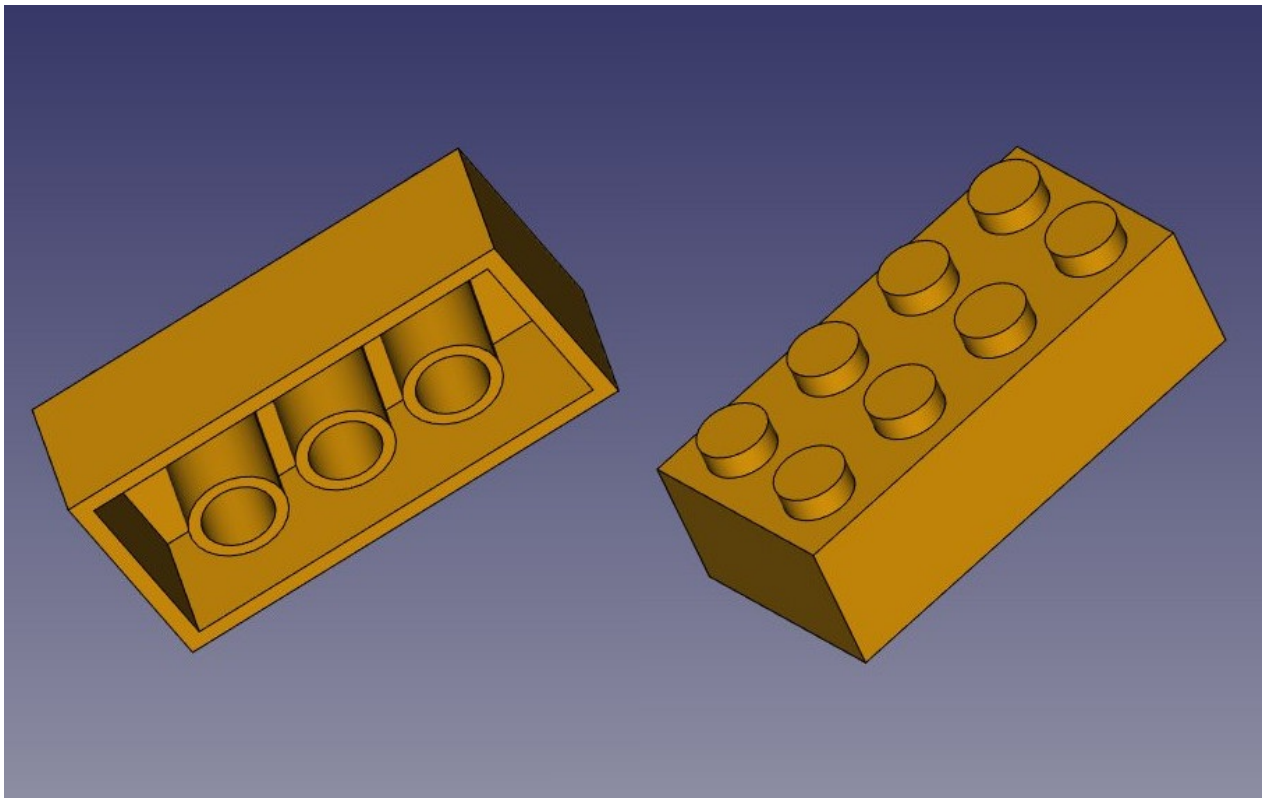


- Now let's make the final holes. Select the circular face of the first of our three "pins"
- Create a new sketch, import the circular boder of our face, create a circle with a radius constraint of 3.6mm, and add a Point-on-Point Constraint between the center of the

imported circle and our new circle. We now have a perfectly centered circle,and once again fully constrained:



- Leave edit mode, and create a pocket from this sketch, with a length of 12.6mm
- Create a linear pattern from this pocket, with a length of 24mm and 3 occurences. That's the last step, our piece of lego is now complete, we can give it a nice color of Victory!



You will notice that our modeling history (what appears in the tree view) has become quite long. This is of precious because every single step of what we did can be changed later on. Adapting this model for another kind of brick, for example one with 2x2 dots, instead of 2x4, would be a piece of cake, we would just need to change a couple of dimensions and the number of occurences in linear patterns. We could as easily create bigger pieces that don't exist in the original Lego game.

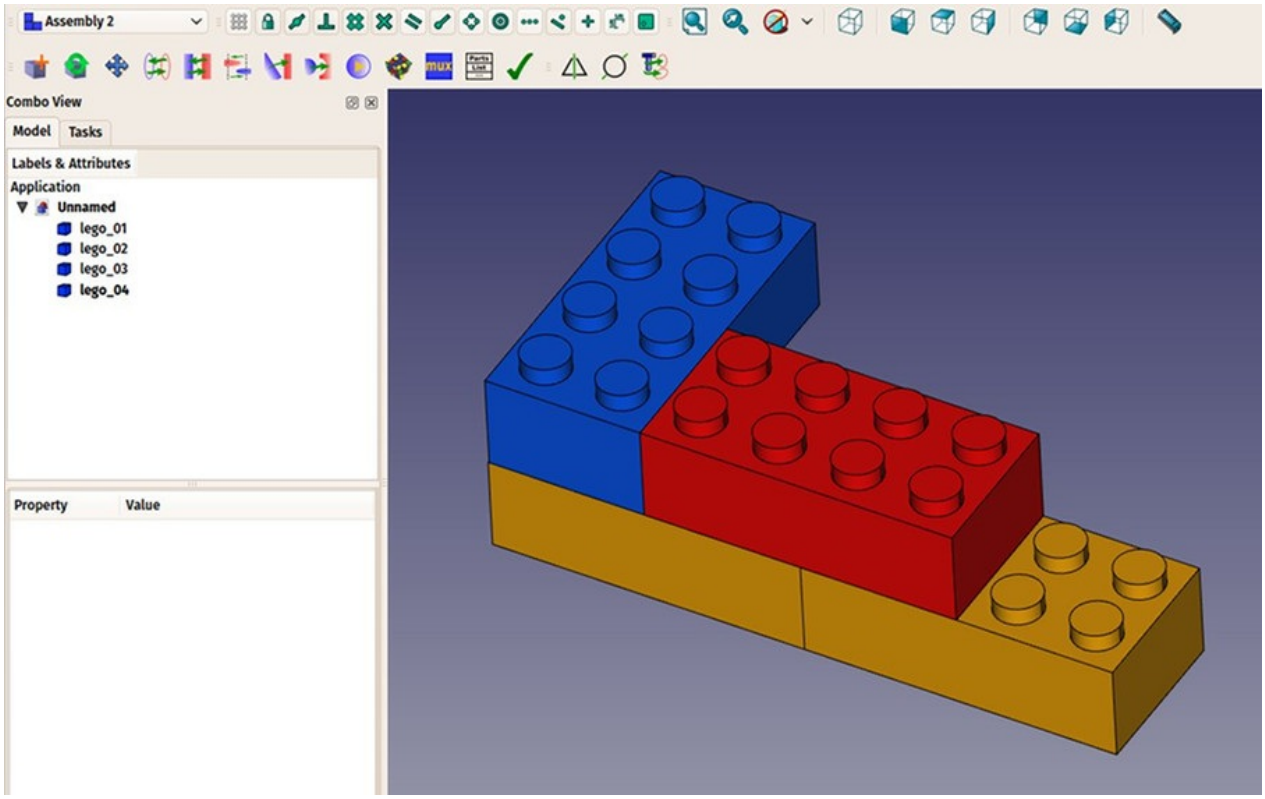
But we could also need to get rid of the history, for example if we are going to model a castle with this brick, and we don't want to have this whole history repeated 500 times in our file.

There are two simple ways to get rid of the history, one is using the Create simple copy tool from the Part Workbench, which will create a copy of our piece that doesn't depend anymore on the history (you can delete the whole history afterwards), the other way is exporting the piece as a STEP file and reimporting it.

Assembling

But the best of both worlds also exists, which is the Assembly2 Workbench, an addon that can be installed from the FreeCAD-addons repository. This Workbench is named "2" because there is also an official built-in Assembly Workbench in development, which is not ready yet. The Assembly2 Workbench, however, already works very well to construct assemblies, and also features a couple of object-to-object constraints which you can use to constrain the position of one object in relation to another. In the example below, however, it will be quicker and easier to position the pieces using Draft Move and Draft Rotate than using the Assembly2 constraints.

- Save the file we did until now
- Install the Assembly2 Workbench and restart FreeCAD
- Create a new empty document
- Switch to the Assembly2 workbench
- Press the **Import a part from another FreeCAD document** button
- Select the file we saved above
- The final piece will be imported in the current document. The Assembly2 workbench will determine automatically what is the final piece in our file that needs to be used, and the new object stays linked to the file. If we go back and modify the contents of the first file, we can press the **Update parts imported into the assembly** button to update the pieces here.
- By using the **Import a part from another FreeCAD document** button several times, and moving and rotating the pieces (with the Draft tools or by manipulating their Placement property), we can quickly create a small assembly:



Downloads

- The model produced during this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/lego.FCStd

Read more

- The Sketcher: http://www.freecadweb.org/wiki/index.php?title=Sketcher_Module
- The Part Design Workbench: http://www.freecadweb.org/wiki/index.php?title=PartDesign_Workbench
- The Assembly2 Workbench: https://github.com/hamish2014/FreeCAD_assembly2

Preparing models for 3D printing

One of the main uses of FreeCAD is to produce real-world objects. These can be designed in FreeCAD, and then made real by different ways, such as communicated to other people who will then build them, or, more and more fequently, sent directly to a 3D printer or a CNC mill. This chapter will show you how to get your models ready to send to these machines.

If you have been cautious while modeling, most of the difficulty you might encounter when printing your model in 3D has already been avoided. This involves basically:

- Making sure that your 3D objects are **solid**. Real-world objects are solid, the 3D model must be solid too. We saw in earlier chapters that FreeCAD helps you a lot in that regard, and that the Part Design Workbench will notify you if you do an operation that prevents your model to stay solid. The Part Workbench also contains a Check Geometry tool that is handy to check further for possible defects.
- Making sure about the **dimensions** of your objects. One millimeter will be one millimeter in real-life. Every dimension matters.
- Controlling the **degradation**. No 3D printing or CNC milling system can take FreeCAD files directly. Most of them will only understand a machine language called G-Code. G-code has dozens of different dialects, each machine or vendor usually using its own. The conversiom of your models into G-Code can be easy and automatic, but you can also do it manually, with total control over the output. In any case, some loss of quality of your model will unavoidably occur during the process. When printing in 3D, you must always make sure this loss of quality stays below your minimal requirements.

Below, we will assume that the first two criterias are met, and that by now you are able to produce solid objects with correct dimensions. We will now see how to address the third point.

Exporting to slicers

This is the technique most commonly used for 3D printing. The 3D object is exported to another program (the slicer) which will generate the G-code from the object, by slicing it into thin layers (hence the name), which will reproduce the movements that the 3D printer will do. Since many of those printers are home-built, there are often small differences from one to the other. These programs usually offer advanced configuration possibilities that allow to tailor the output exactly for the particularities of your 3D printer.

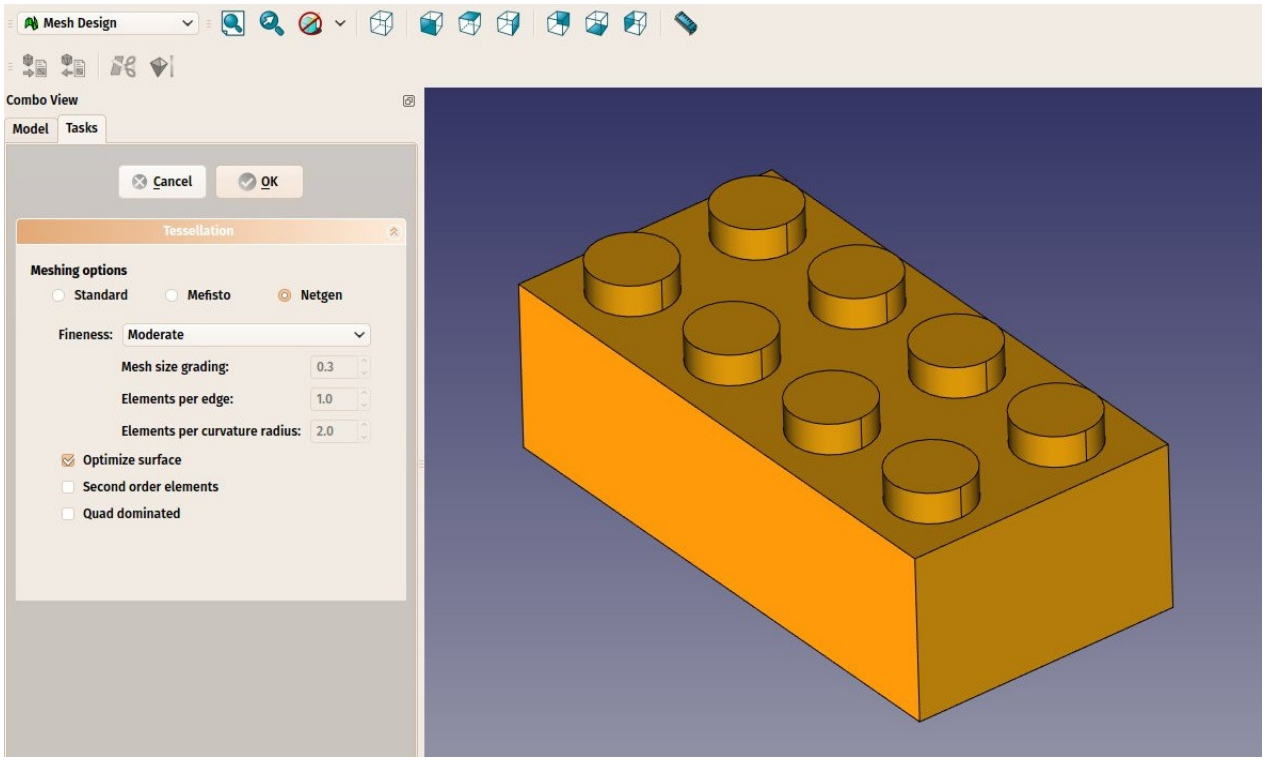
Actual 3D printing, however, is a too vast subject for this manual. But we will see how to export and use these slicers to check that the output is correct.

Converting objects to meshes

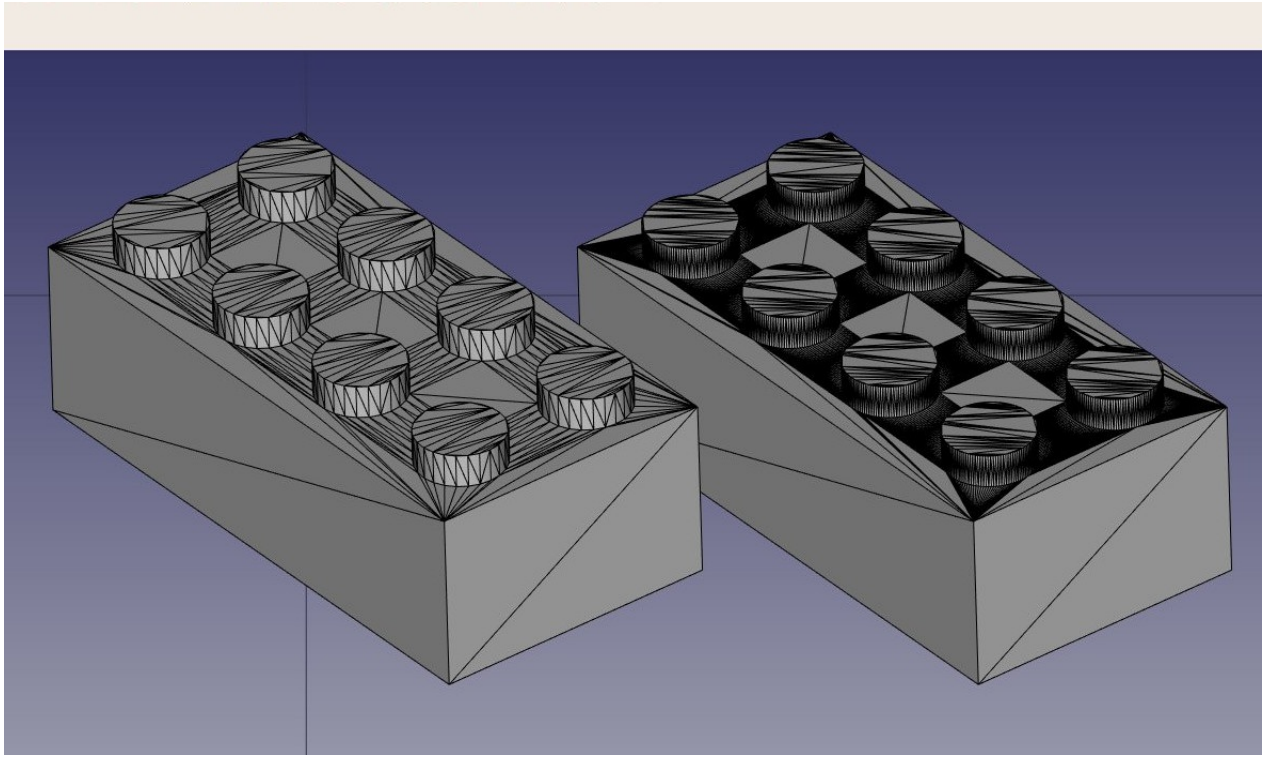
None of the slicers will, at this date, take directly solid geometry as we produce in FreeCAD. So we will need to convert any object we want to 3D print into a mesh first, that the slicer can open. Fortunately, converting a solid to a mesh, is very straightforward. All we need to be careful about, is that it is now that the degradation we mentioned above will occur. We need to check that the degradation stays inside acceptable limits.

All the mesh handling, in FreeCAD, is done by another specific workbench, the Mesh Workbench. This workbench contains, the tools that convert between Part and Mesh objects, several utilities meant to analyze and repair meshes. Although working with meshes is not the focus of FreeCAD, when working with 3D modeling, you often need to deal with mesh objects, since their use is very widespread among other applications. This workbench allows you to handle them fully in FreeCAD.

- Let's convert one of the objects we modelled in the previous chapters, such as the lego piece (which can be downloaded from the end of the previous chapter).
- Open the FreeCAD file containing the lego piece.
- Switch to the Mesh Workbench
- Select the lego brick
- Select menu **Meshes -> Create Mesh from Shape**
- A task panel will open with several options. Some additional meshing algorithms (Mefisto or Netgen) might not be available, depending on how your version of FreeCAD was compiled. The Standard meshing algorithm will always be present. It offers less possibilities than the two others, but is totally sufficient for small objects that fit into the maximum print size of a 3D printer.



- Select the **Standard** mesher, and leave the deviation value to the default value of **0.10**. Press **Ok**.
- A mesh object will be created, exactly on top of our solid object. Either hide the solid, or move one of the objects apart, so you can compare both.
- Change the **View -> Display Mode** property of the new mesh object to **Flat Lines**, in order to see how the triangulation occured.
- If you are not happy, and think that the result is too coarse, you can repeat the operation, lowering the deviation value. In the example below, the left mesh used the default value of **0.10**, while the right one uses **0.01**:



In most cases, though, the default values will give a satisfying result.

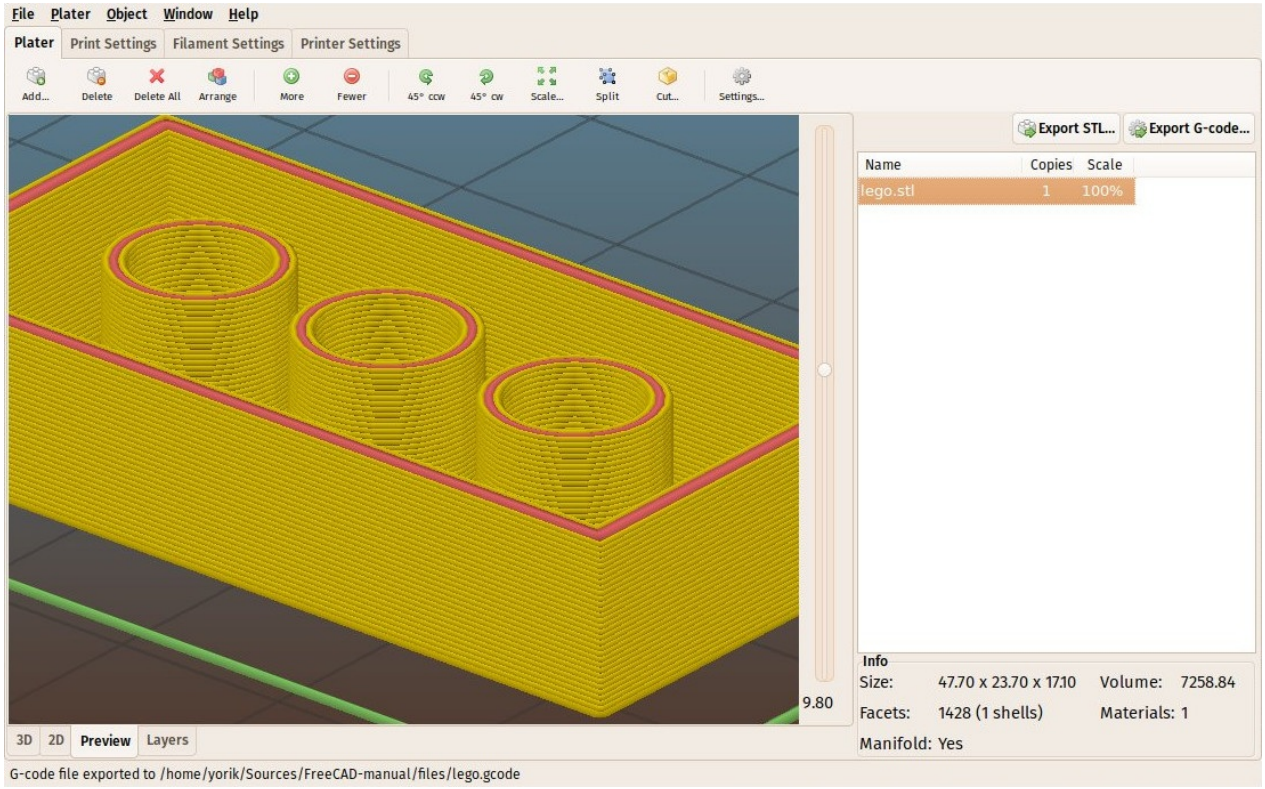
- We can now export our mesh to a mesh format, such as STL, which is currently the most widely used format in 3D printing, by using menu **File -> Export** and choosing the STL file format.

If you don't own a 3D printer, it is usually very easy to find commercial services that will print and send you the printed objects by mail. Among the famous ones are Shapeways and Sculpteo, but you will also usually find many others in your own city. In all major cities, you will also nowadays find Fab labs, which are workshops equipped with a range of 3D manufacturing machines, almost always including at least one 3D printer. Fab labs are usually community spaces, that will let you use their machines, for a fee or for free depending on the Fab lab, but also teach you how to use them, and promote other activities around 3D manufacturing.

Using Slic3r

Slic3r is an application that converts STL objects into G-code that can be sent directly to 3D printers. Like FreeCAD, it is free, open-source and runs on Windows, Mac OS and Linux. Correctly configurating things for 3D printing is a complicated process, where you must have a good knowledge of your 3D printer, so it is not very useful to generate G-code before actually going to print (your G-code file might not work well on another printer), but it is useful for us anyway, to check that our STL file will be printable without problems.

This is our exported STL file opened in Slic3r. By using the **preview** tab, and moving the right slider, we can visualize the path that the 3D printer head will follow to construct our object.



Using the Cura addon

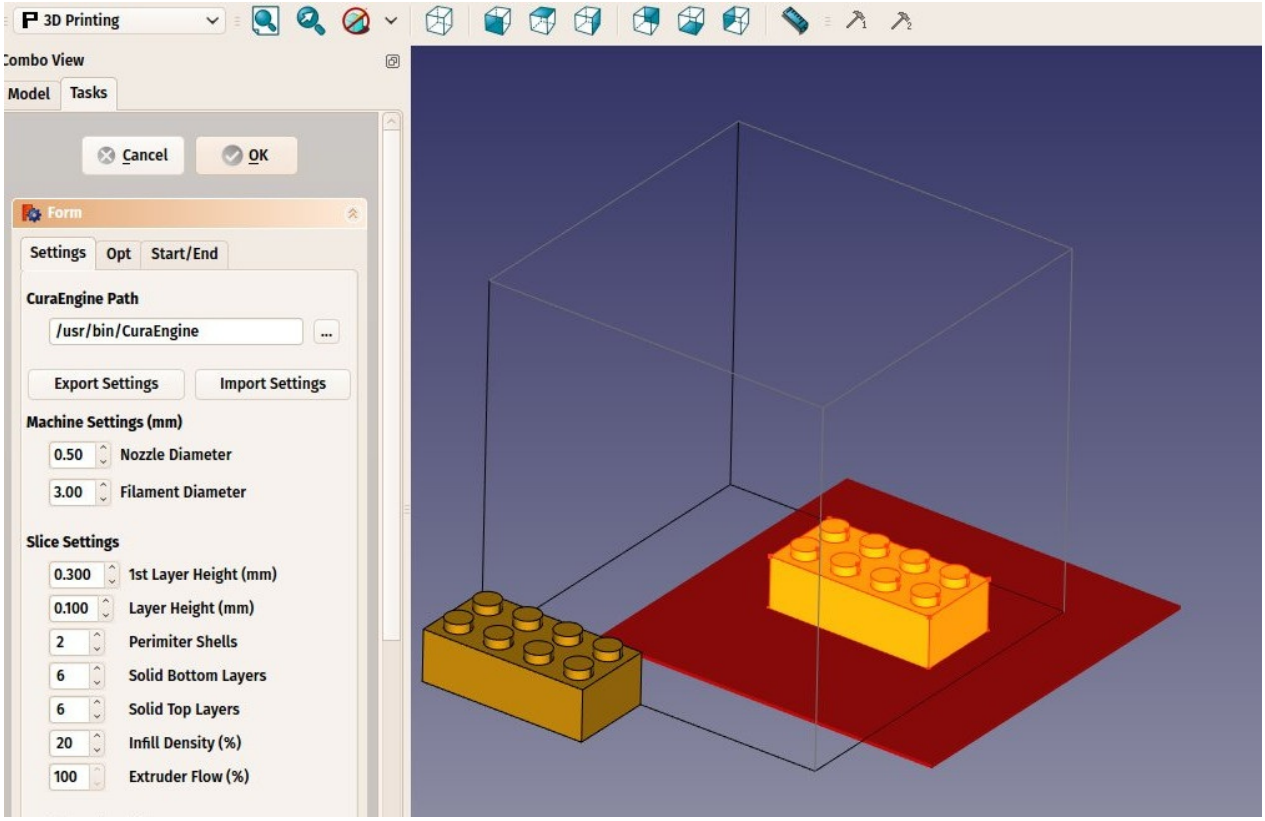
Cura is another free and open-source slicer application for Windows, Mac and Linux, maintained by the 3D printer maker Ultimaker. Some FreeCAD users have created a Cura Workbench that uses cura internally. The Cura Workbench is available from the FreeCAD addons repository. To use the Cura Workbench, you also need to install Cura itself, which is not included in the workbench.

Once you have installed both Cura and the Cura Workbench, you will be able to use it to produce the G-code file directly from Part objects, without the need to convert them to meshes, and without the need to open an external application. Producing another G-code file from our Lego brick, using the Cura Workbench this time, is done as follows:

- Load the file containing our Lego brick (it can be downloaded at the end of the previous chapter)
- Switch to the Cura Workbench
- Setup the printer space by choosing menu **3D printing -> Create a 3D printer definition**. Since we aren't going to print for real, we can leave the settings as they are. The geometry of the printing bed and available space will be shown in the 3D view.
- Move the Lego brick to a suitable location, such as the center of the printing bed.

Remember that PartDesign objects cannot be moved directly, so you need either to move its very first sketc (the first rectangle), or to move (and print) a copy, which can be made with the Part -> Create Simple Copy tool. The copy can be moved, for example with Draft -> Move.

- Select the object to be printed, and select menu **3D printing -> Slice with Cura Engine**.
- In the task panel that will open, make sure the path to the Cura executable is correctly set. Since we are not going to really print, we can leave all other options as they are. Press **Ok**. Two files will be generated in the same directory as your FreeCAD file, an STL file and a G-code file.



- The generated G-code can also be reimported into FreeCAD (using the slic3r preprocessor) for checking.

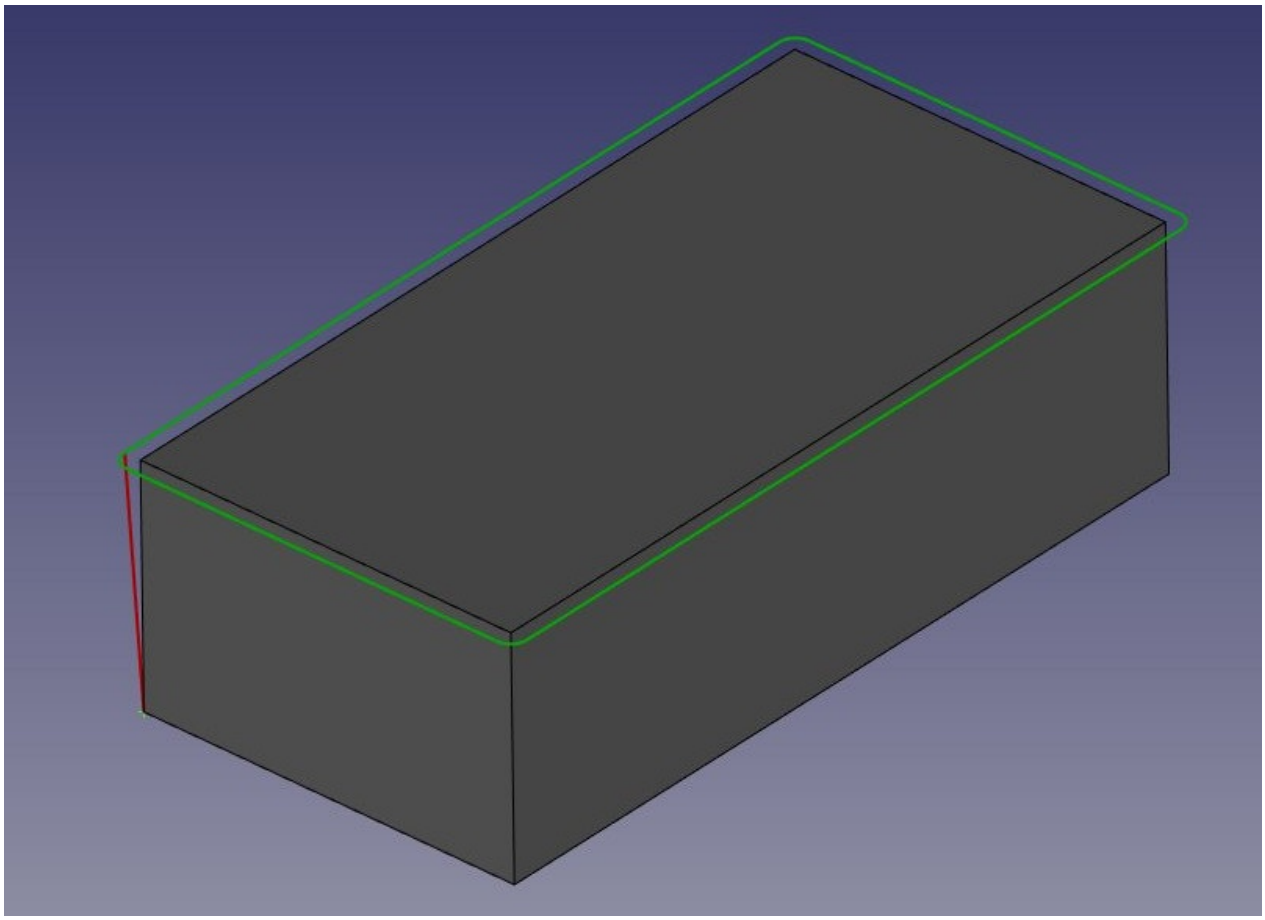
Generating G-code

FreeCAD also offers more advanced ways to generate G-code directly. This is often much more complicated than using automatic tools as we saw above, but has the advantage to let you fully control the output. This is usually not needed when using 3D printers, but becomes very important when dealing with CNC milling, as the machines are much more complex.

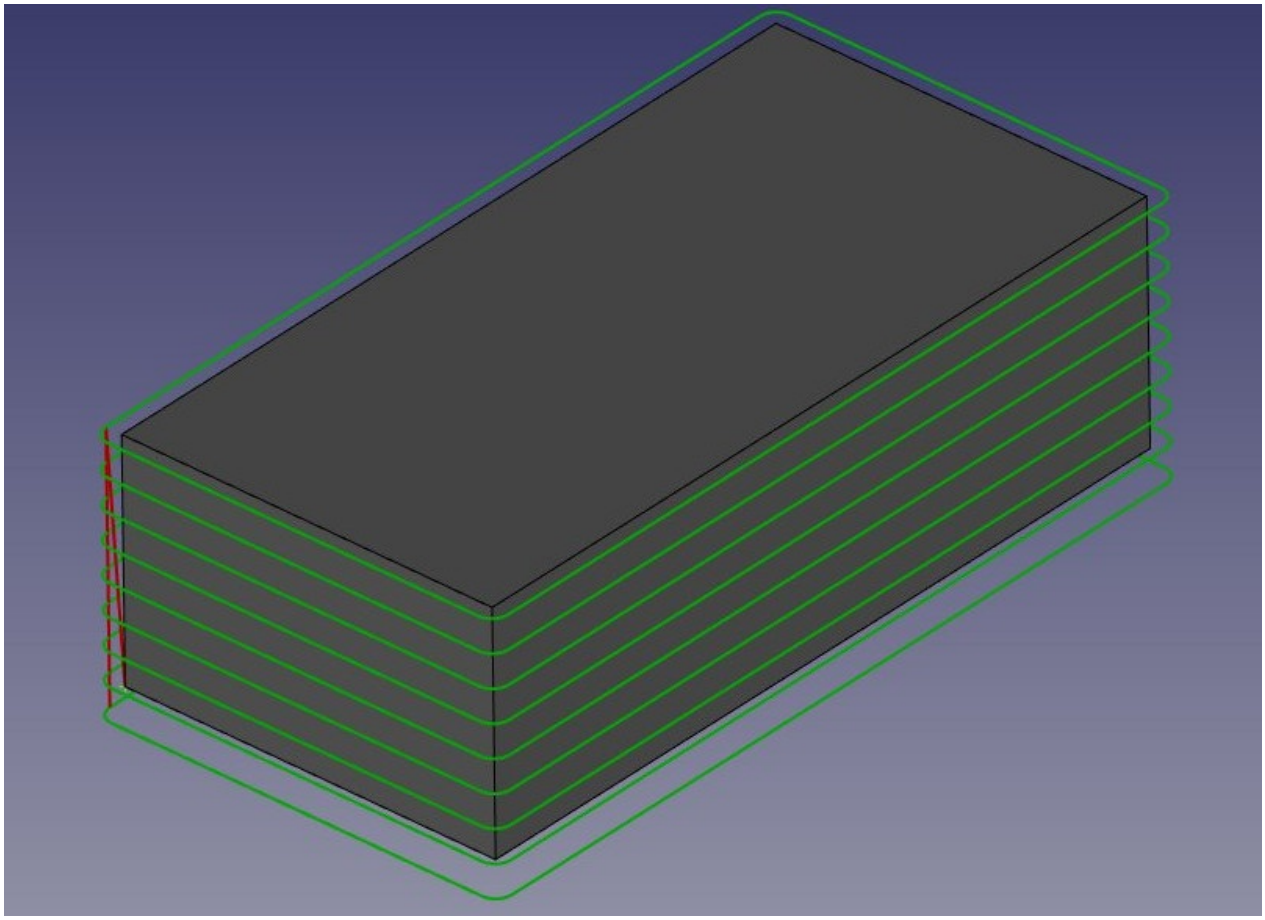
G-code path generation in FreeCAD is done with the Path Workbench. It features tools that generate full machine paths and others that generate only parts of a G-code project, that can be assembled to form a whole milling operation.

Generating CNC milling paths is another subject that is much too vast to fit in this manual, so we are going to show how to build a simple Path project, without caring much about most of the details of real CNC machining.

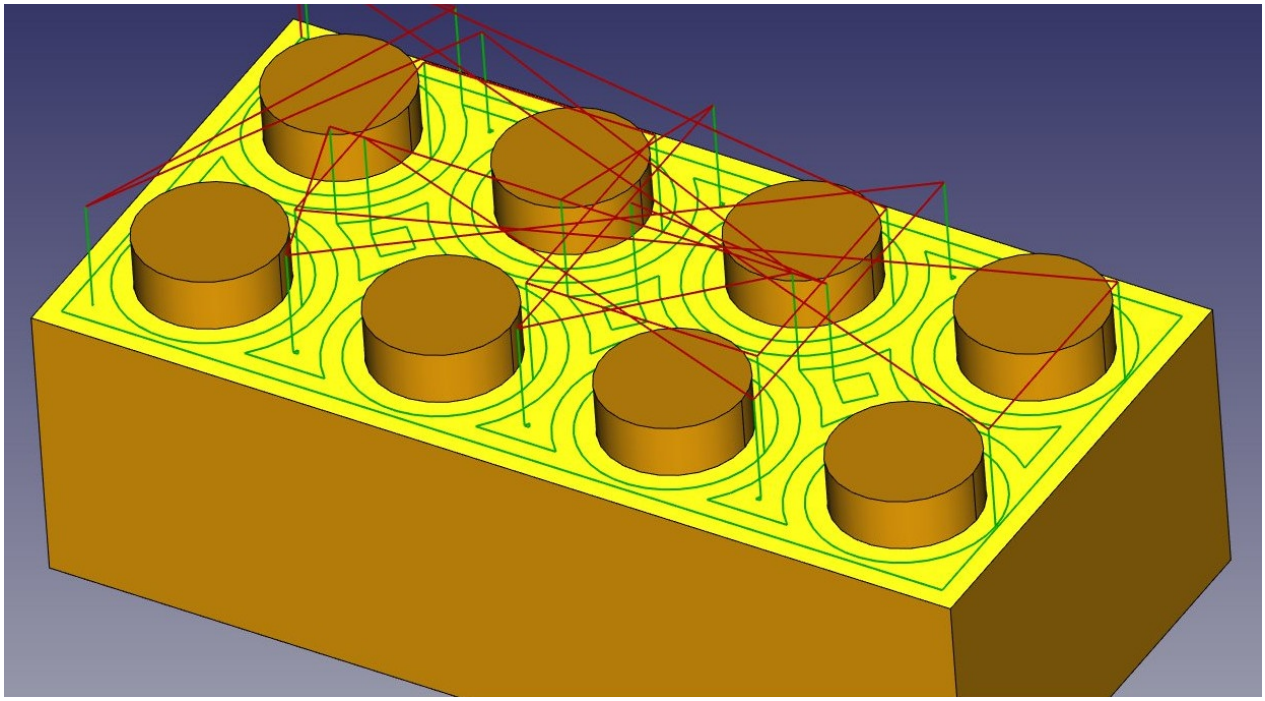
- Load the file containing our lego piece, and switch to the Path Workbench.
- Since the final piece doesn't contain anymore a rectangular top face, hide the final lego piece, and show the first cubic pad that we did, which has a rectangular top face.
- Select the top face and press the Face Profile button.
- Set its **Offset** property to 1mm.



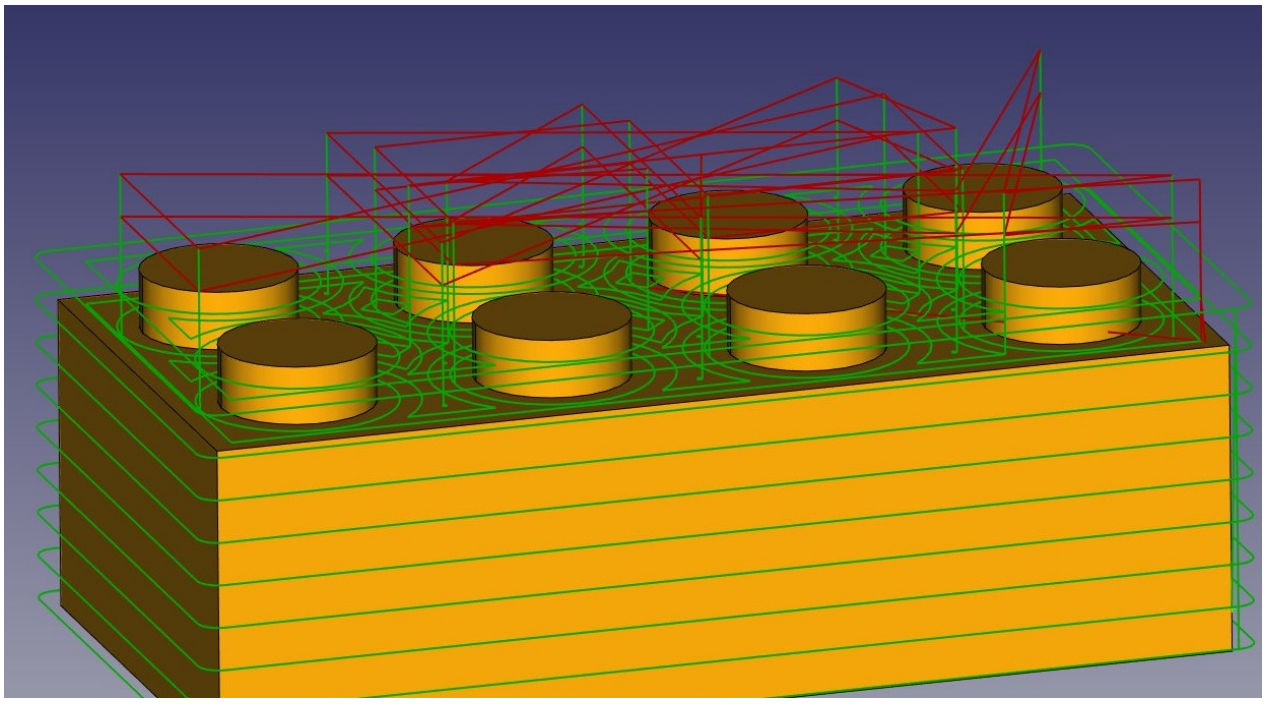
- Then, let's duplicate this first loop a couple of times, so the tool will carve out the whole block. Select the FaceProfile path, and press the Array button.
- Set the **Copies** property of the array to 8, and its **Offset** to -2mm in the Z direction, and move the placement of the array by 2mm in the Z direction, so the cutting will start a bit above the pad, and include the height of the dots too.



- Now we have defined a path that, when followed by the milling machine, will carve a rectangular volume out of a block of material. We now need to carve out the space between the dots, in order to reveal them. Hide the Pad, and show the final piece again, so we can select the face that lies between the dots.
- Select the top face, and press the Face Pocket button. Set the **Offset** property to 1mm, and the **retraction height** to 20mm. That is the height to where the cutter will travel when switching from one loop to another. Otherwise, the cutter might cut right through one of our dots:



- Once again, make an array. Select the FacePocket object, and press the Array button. Set the **Copies** number to 1 and the **offset** to -2mm in the Z direction. Move the placement of the array by 2mm in the Z direction. Our two operations are now done:



- Now all that is left to do is to join these two operations into one. This can be done with a Path Compound or a Path Project. Since we will need nothing more and will be ready to export already, we will use the project. Press the Project button.
- Set the **Use Placements** property of the project is to True, because we changed the placement of the arrays, and we want that to be taken into account in the project.
- In the tree view, drag and drop the two arrays into the project. You can reorder the arrays inside the project if needed, by double-clicking it.

- The project can now be exported to G-code, by selecting it, choosing menu **File -> Export**, selecting the G-code format, and in the pop-up dialog that will open, selecting a post-procesing script according to your machine.

There are many applications available to simulate the real cutting, one of them that is also multi-platform and open-source, like FreeCAD, is Camotics.

Downloads

- The STL file generated in this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/lego.stl
- The file generated during this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/path.FCStd
- The G-code file generated in this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/lego.gcode

Read more

- The Mesh Workbench: http://www.freecadweb.org/wiki/index.php?title=Mesh_Module
- The STL file format: https://en.wikipedia.org/wiki/STL_%28file_format%29
- Slic3r: http://slic3r.org/
- Cura: https://ultimaker.com/en/products/cura-software
- The Cura Workbench: https://github.com/cblt2l/FreeCAD-CuraEngine-Plugin
- The Path Workbench: http://www.freecadweb.org/wiki/index.php?title=Path_Workbench
- Camotics: http://camotics.org/

Generating 2D drawings

When your model cannot be printed or milled directly by a machine, for example it is too big (a building) or it requires manual assembly after the pieces are ready, you will usually need to explain to another person how to do that. In technical fields (engineering, architecture, etc), this is usually done with drawings, that are handed over to the person responsible for assembling the final product, that will explain how to do it.

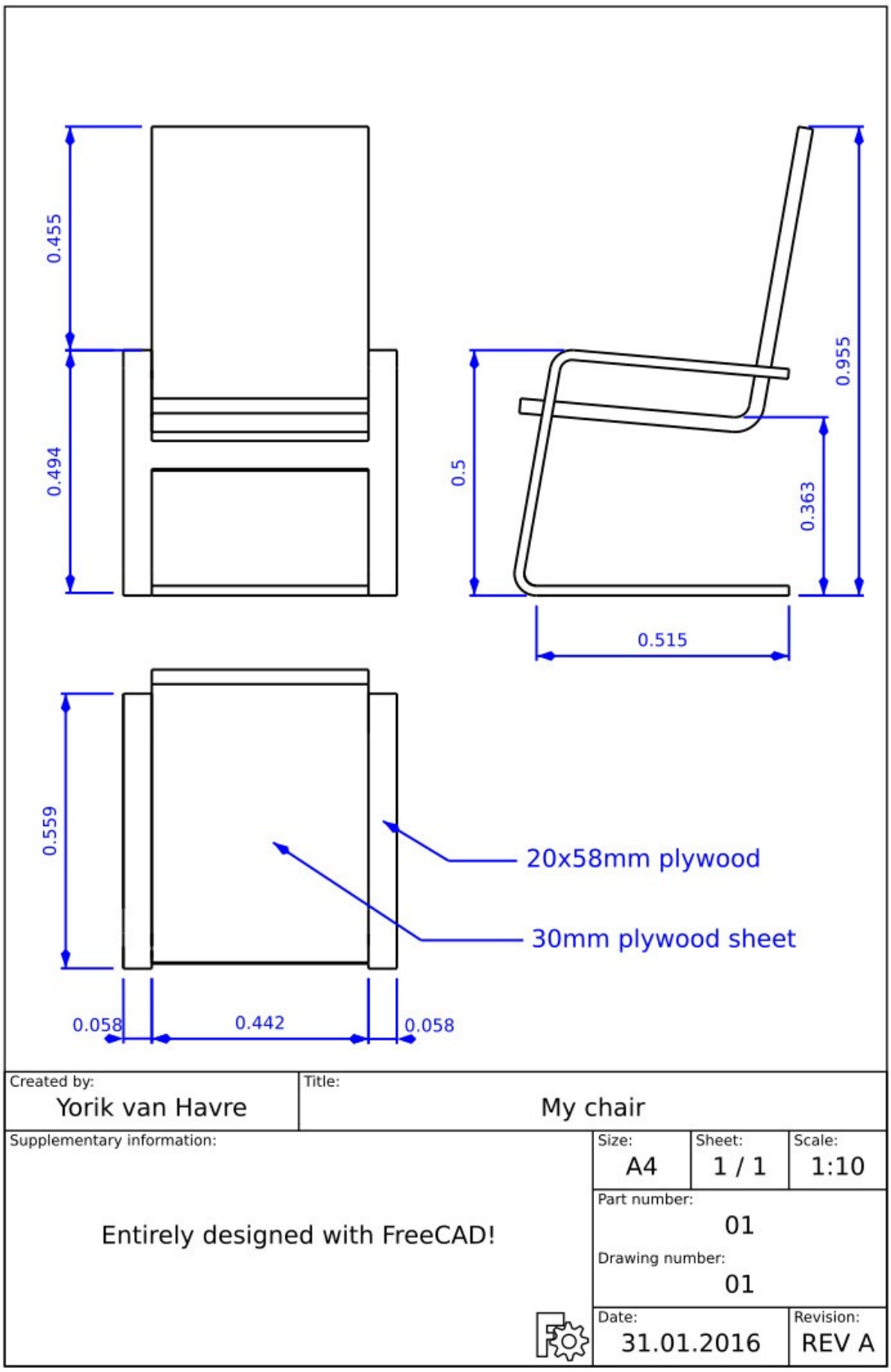
Typical examples are Ikea instructions, architectural drawings or blueprints. These drawings usually contain not only the drawing itself, but also many annotations, such as texts, dimensions, numbers, symbols that will help other people to understand what needs to be done and how.

In FreeCAD, the workbench responsible for making such drawings is the Drawing Workbench.

The Drawing Workbench allows you to create sheets, which can be blank or use a pre-made template to already have a series of items on the sheet, such as borders and title. On these sheets, you can then place views of the 3D objects you modeled previously, and configure how these views must appear on the sheet. Finally, thanks to an addon called Drawing Dimensioning Workbench, you can also place all kinds of annotations on the sheet, such as dimensions, texts, and other usual symbols commonly used in technical drawings.

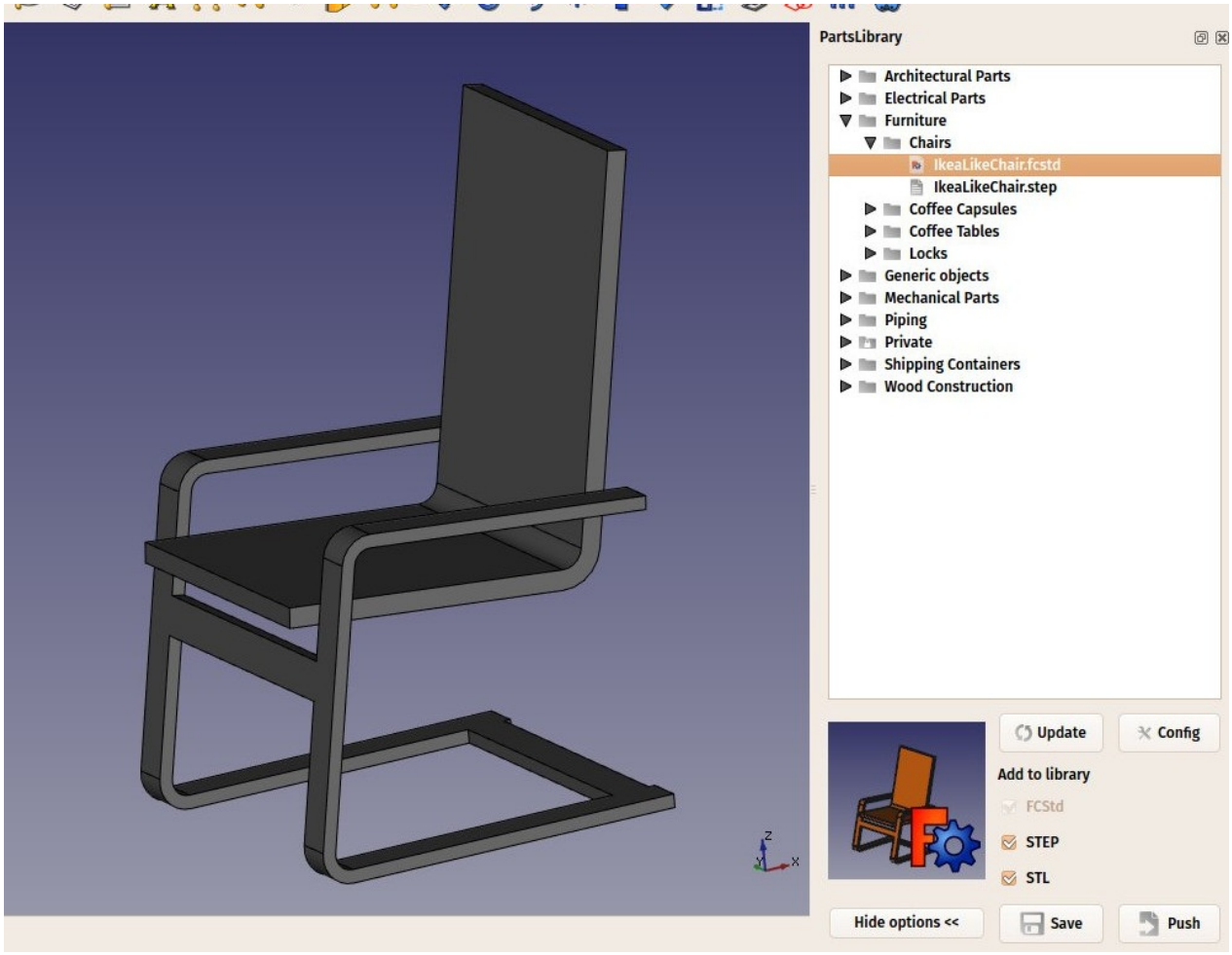
Drawing sheets, once complete, can be printed or exported as SVG, PDF or DXF files.

In the following exercise, we will see how to create a simple drawing of a chair model found in the FreeCAD library (Furniture -> Chairs -> IkeaChair). The FreeCAD library can easily be added to your FreeCAD installation (refer to the installing chapter of this manual), or you can simply download the model from the library webpage, or via the direct link provided at the bottom of this chapter.

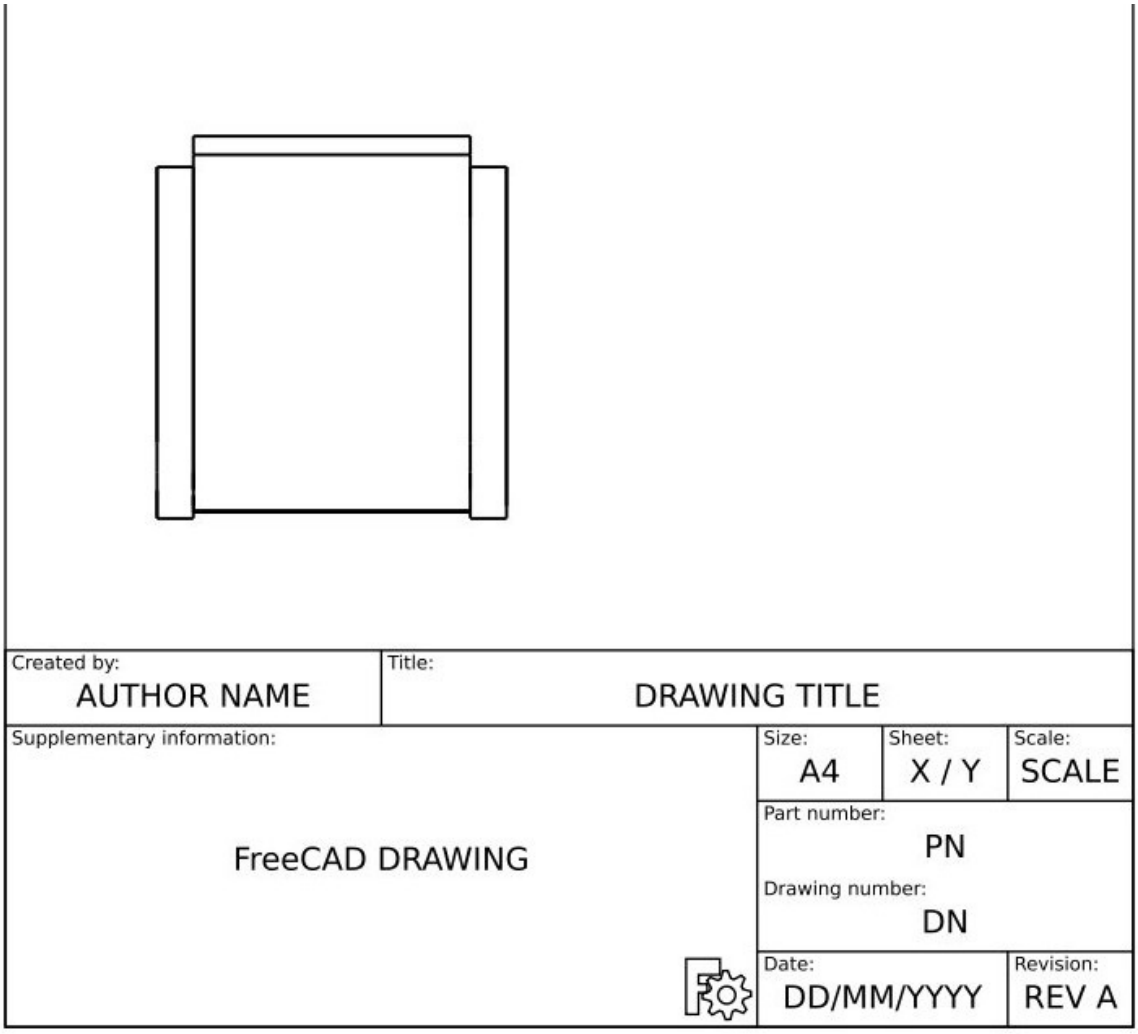


- Load the IkeaChair file from the library. You can choose between the .FCStd version, which will load the full modeling history, or the .step version, which will create only one object, without the history. Since we won't need to model any further now, it is best to

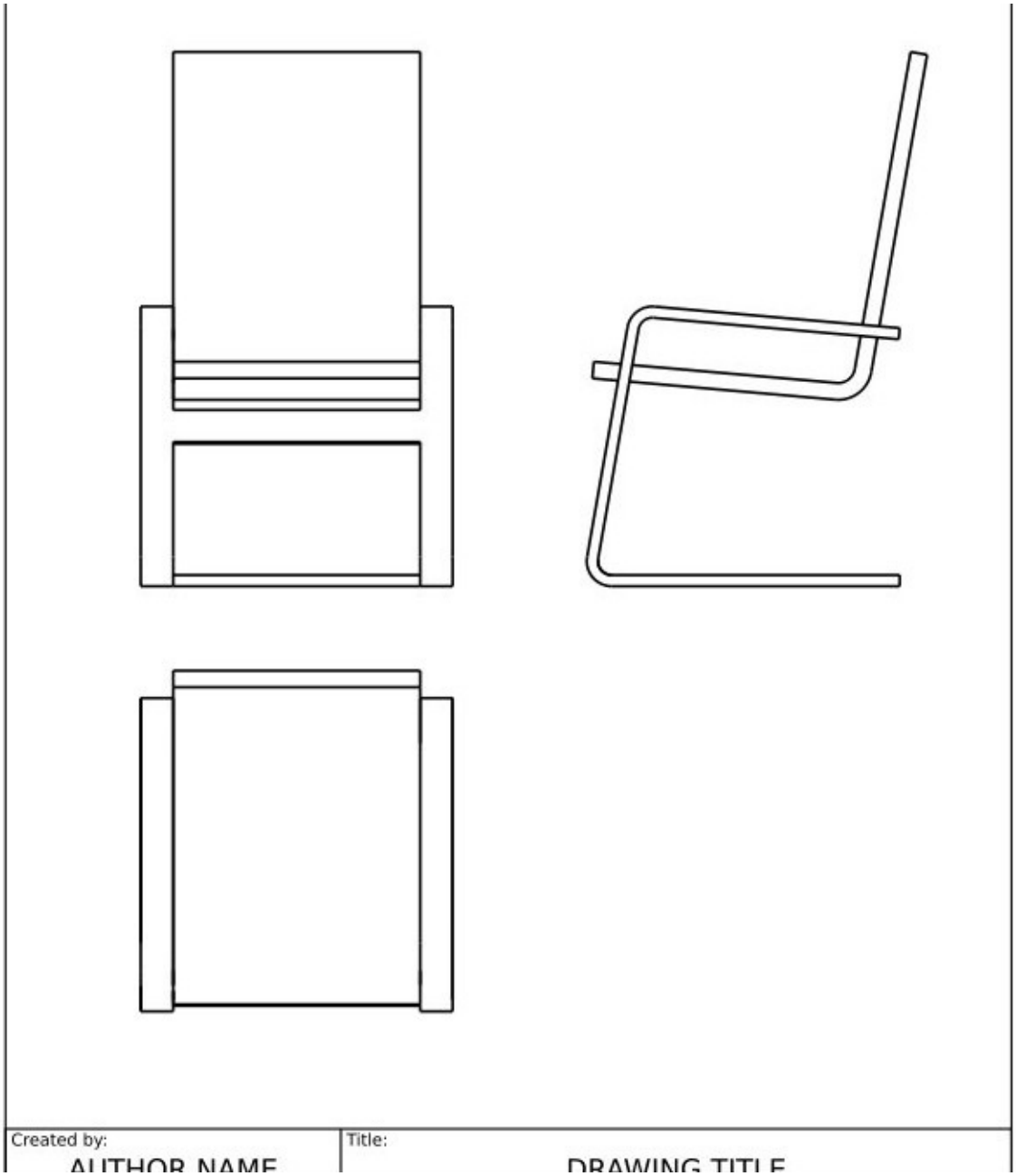
choose the .step version, as it will be easier to manipulate.



- Switch to the Drawing Workbench
- Press the little arrow next to the New Drawing Page button.
- Select the **A4 Portrait / ISO7200** template. A new tab will open in your FreeCAD window, showing the new page.
- In the tree view (or in the model tab), select the chair model.
- Press the Insert view button.
- A View object will be created on our page. Give the view the following properties:
  - X: 100
  - Y: 150
  - Scale: 0.1
  - Rotation: 270
- We now have a nice top view (which is the default projection) of our chair:



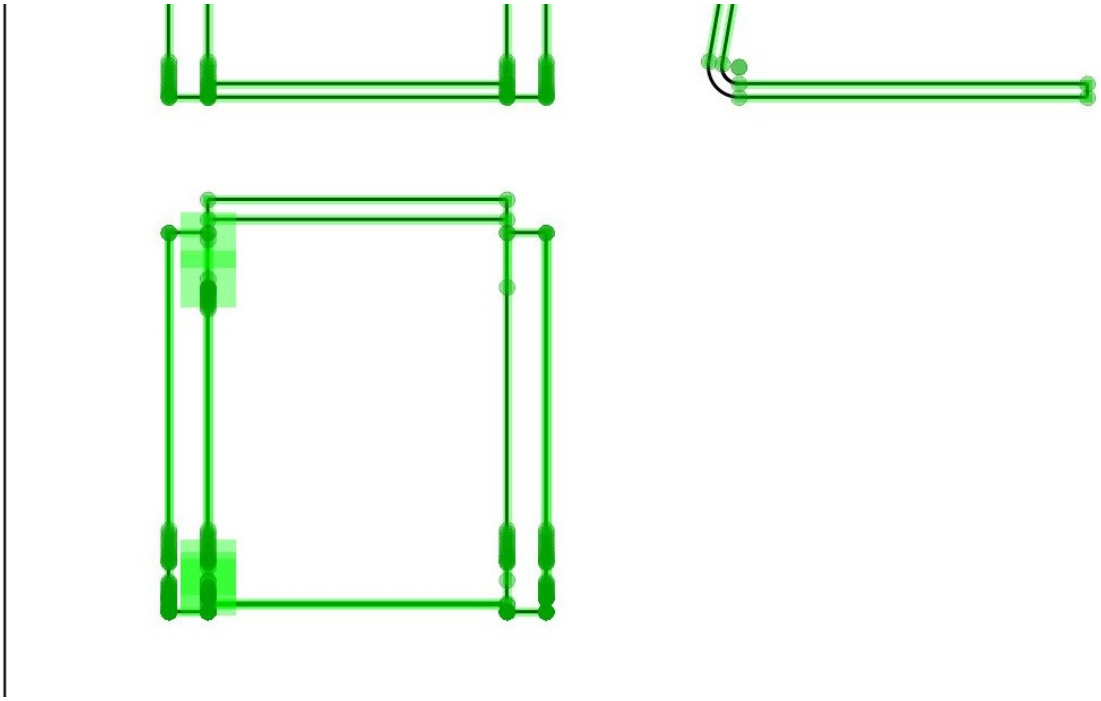
- Let's repeat the operation twice, to create two more views. We will set their X and Y values, which indicate the position of the view on the page, in order to show them apart from the top view, and their direction, to create different view orientations. Give each new view the following properties:
  - View001 (front view): X: 100, Y: 130, Scale: 0.1, Rotation: 90, Direction: (-1,0,0)
  - View002 (side view): X: 180, Y: 130, Scale: 0.1, Rotation: 90, Direction: (0,-1,0)
    - After that, we obtain the following page:



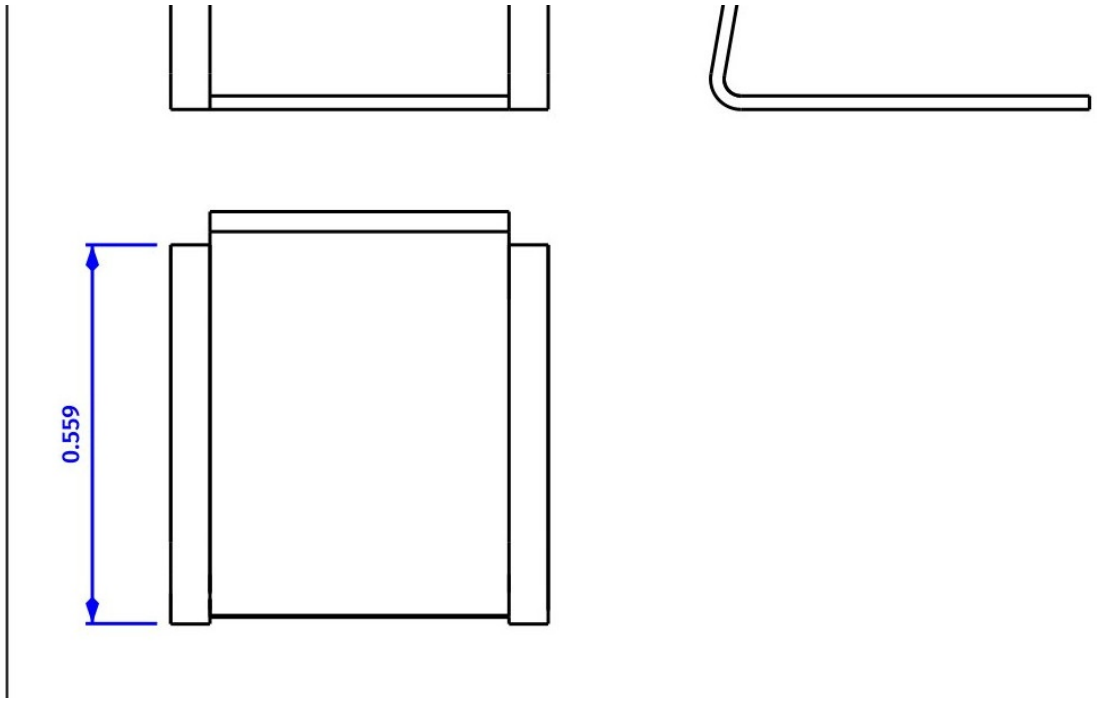
- We can tweak a bit the aspect of our views if we want, for example we can raise their **Line Width** property to 0.5.

We will now place dimensions and indications on our drawing. There are two ways to add dimensions to a model, one is placing the dimensions inside the 3D model, using the Dimension tool of the Draft Workbench, and then place a view of these dimensions on our sheet with the Draft View tool (which can be used with a single dimension or a group containing dimensions), or we can do things directly on the Drawing sheet, using the Drawing Dimensioning Workbench, which is installable from the FreeCAD addons. We will use here this latter method.

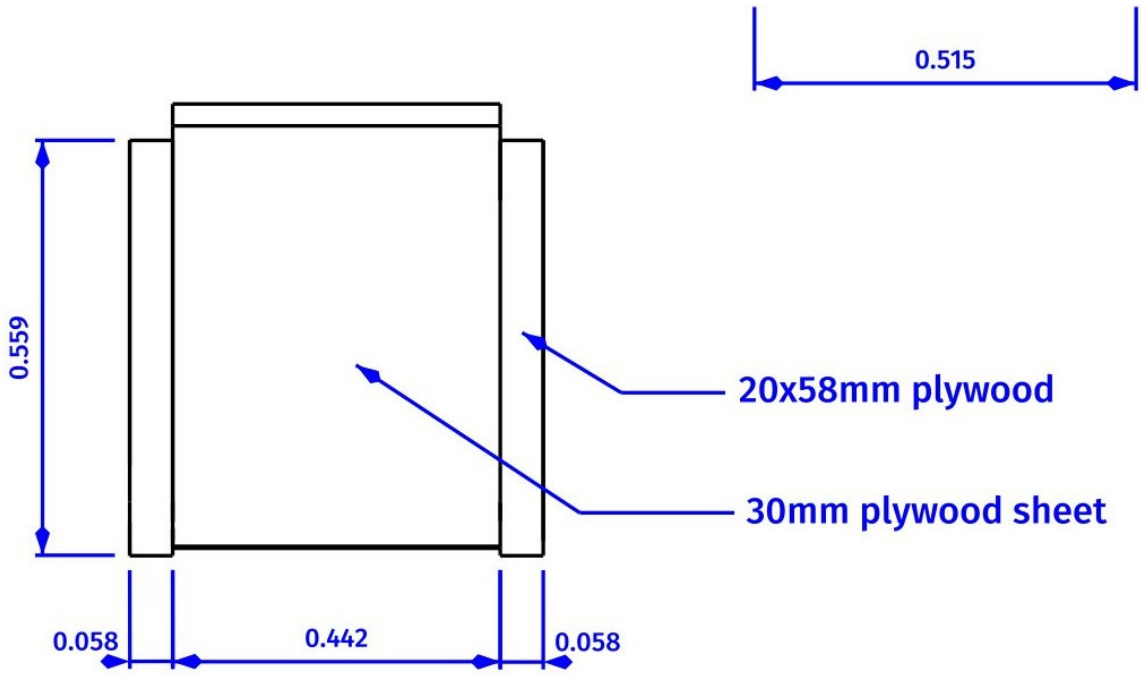
- Switch to the Drawing Dimensioning Workbench
- Press the **Add Linear Dimension** button. Available nodes are highlighted in green on the drawing page:



- Click two of these points, then click a third point to place the dimension line:



- The Linear Dimension tool, as most of the other Drawing Dimensioning tools, will not exit after you finished, allowing you to place more dimensions. When you are done, simply click the **Close** button in the Task panel.
- Repeat the operation, until all the dimensions you wish to indicate are placed. Take a minute to browse through the different options proposed in the Linear Dimension's task panel. For example, by unticking the **auto place text** option, you will be able to place the text of the dimension elsewhere, like on the image below:



- We will now place two indications, using the **Welding/Groove symbols** tool, selecting the default one (no groove symbol). Draw the two lines like on the image above.
- Now place two texts using the **Add text** tool, and change their **text** property to the contents of your likings.
- Our drawing is now complete, all that is left to do is to fill in the informations of the sheet titleblock. With most of the default FreeCAD templates, this can be done easily, by changing the **Editable Texts** property of the page.

Our page can now be exported to SVG to be worked further in graphical applications like inkscape, or to DXF by selecting menu **File -> Export**. The Drawing Dimensioning workbench also features its own **DXF export** tool, which also supports the annotations added with that workbench. The DXF format is importable in almost all existing 2D CAD applications. Drawing pages can also be directly printed or exported to PDF.

Downloads

- The chair model: https://github.com/FreeCAD/FreeCAD-library/blob/master/Furniture/Chairs/IkeaLikeChair.step
- The file created during this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/drawing.FCStd
- The SVG sheet produced from that file: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/drawing.svg

Read more

- The Drawing Workbench: http://www.freecadweb.org/wiki/index.php?title=Drawing_Module
- The Drawing Dimensioning Workbench:

https://github.com/hamish2014/FreeCAD_drawing_dimensioning

- The FreeCAD library: https://github.com/FreeCAD/FreeCAD-library
- Inkscape: http://www.inkscape.org

BIM modeling

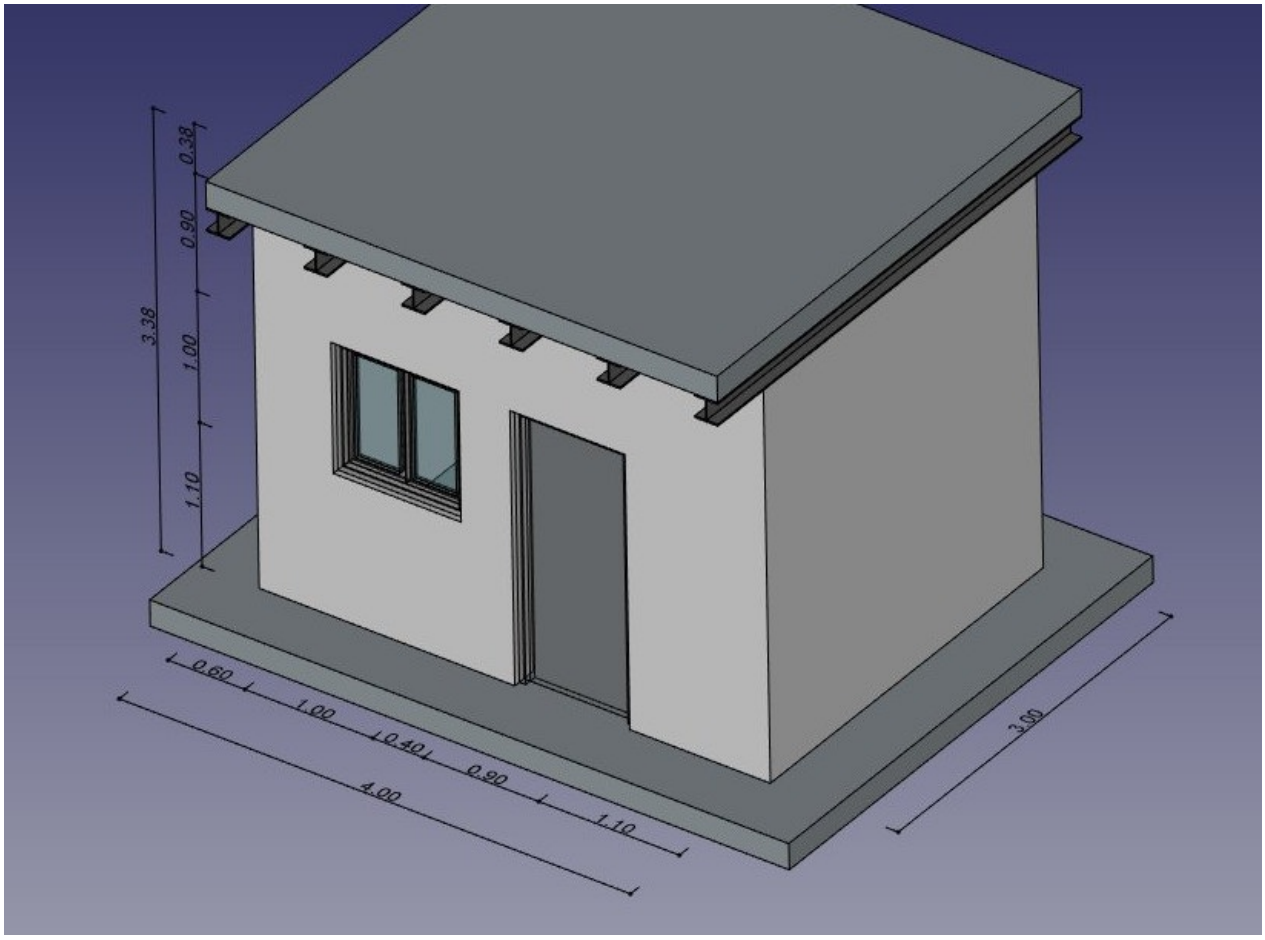
BIM stands for Building Information Modeling. The exact definition of what it is varies, but we can say simply that is how buildings and other large structures like bridges, tunnels, etc... are modeled today. BIM models are usually based on 3D models, and also include a series of additional layers of information, such as materials information, relationships to other objects or models, or special instructions for building or maintenance. This extra information permits all kinds of advanced analyses of the model, such as structural resistance, cost and construction time estimations, or calculaitons of energy consumption.

The Arch Workbench of FreeCAD implements a series of tools and facilities for BIM modeling. Although it has a different purpose, it is made to work in tight integration with the rest of FreeCAD: Anything made with any other workbench of FreeCAD can become an Arch object, or be used as a base for an Arch object.

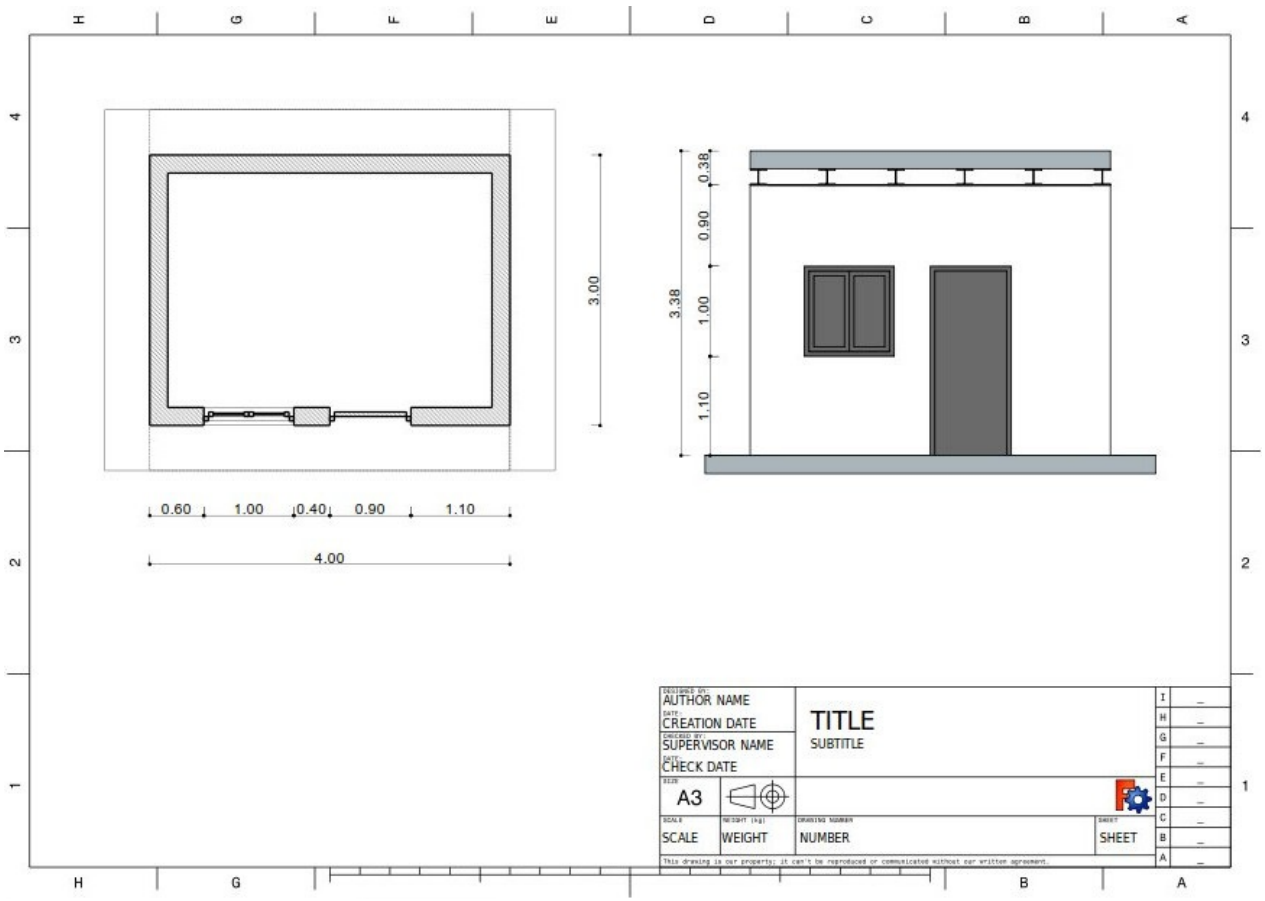
As in the PartDesign Workbench, the objects produced by the Arch Workbench are meant to be built in the real world. Therefore, they need to be **solid**. The Arch tools usually take care of that automatically, and also provide utility tools to help you check the validity of objects.

The Arch Workbench also includes all the tools from the Draft Workbench, and uses its grid and snapping system. Before beginning, it is always a good idea to browse through the preferences pages of both Draft and Arch and set the default settings to your likings.

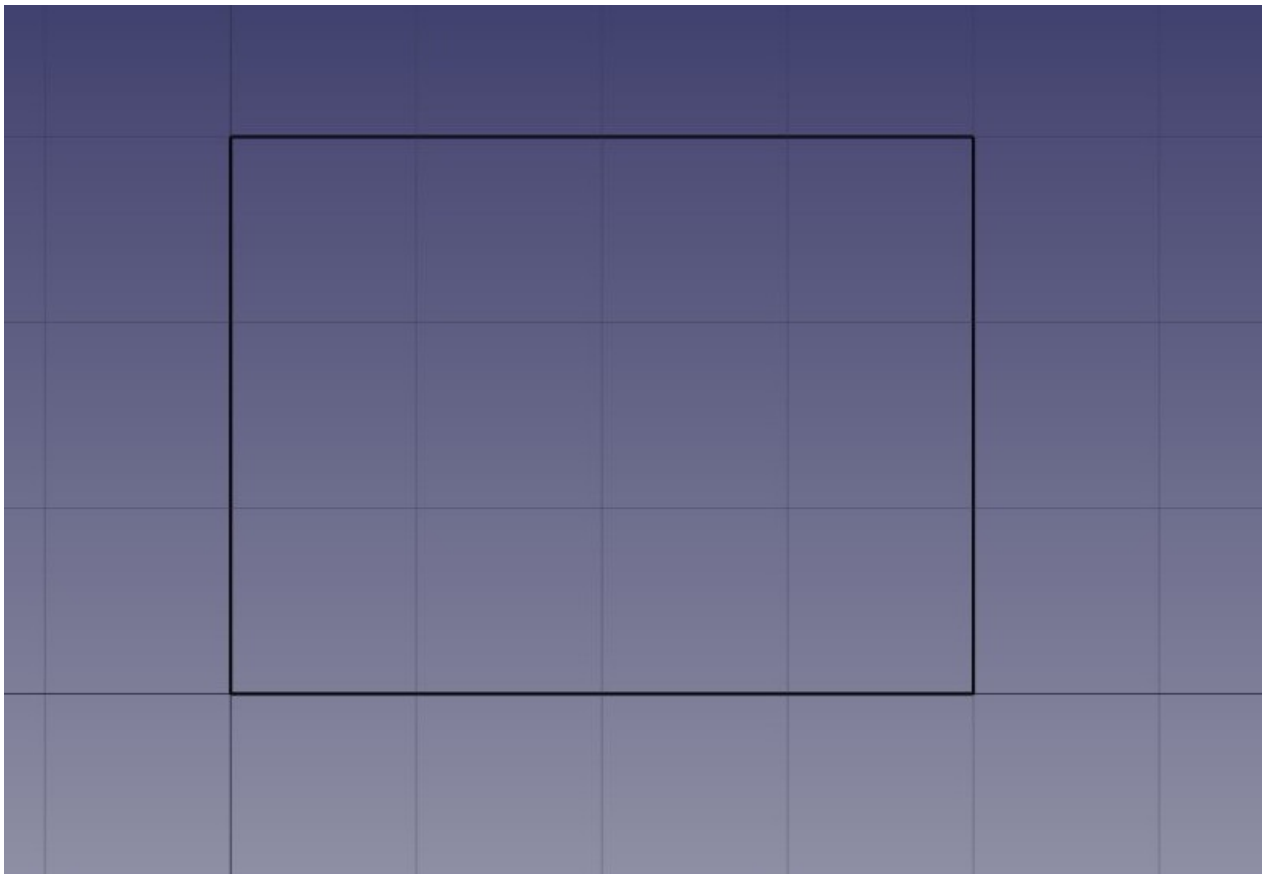
In this chapter, we will see how to model this small building:



and produce a plan and a section view from it:



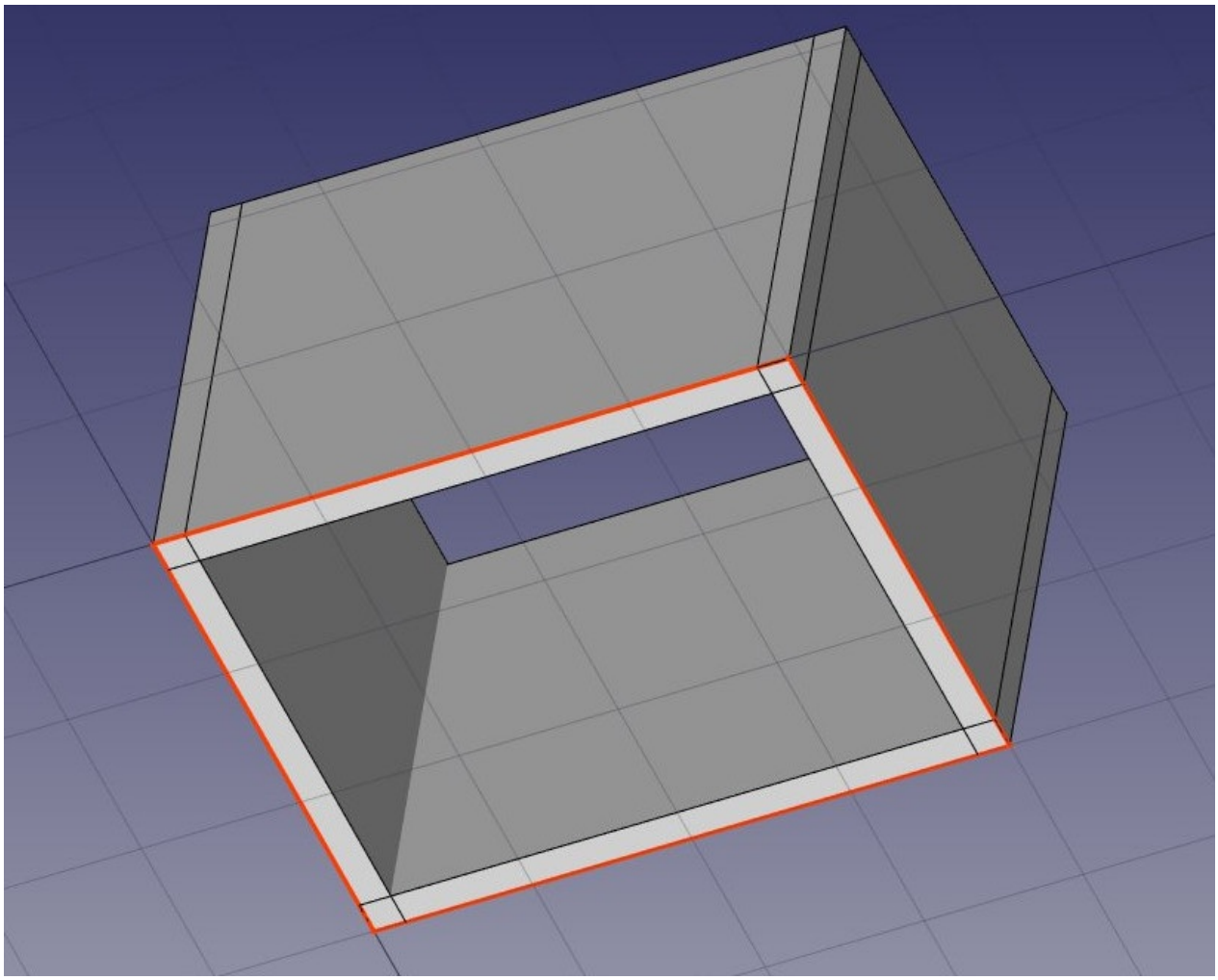
- Create a new document, and switch to the Arch Workbench.
- Open menu **Edit -> Preferences -> Draft -> Grid and Snapping** and set the **grid spacing** setting to 1000mm, so we have a one meter-based grid, which will be convenient for the size of our buiding.
- On the **snapping toolbar**, make sure the grid snap button is enabled, so we can use the grid as much as possible.
- Set the Working Plane to **XY** plane
- Draw four lines with the Draft Line tool. You can enter coordinates manually, or simply pick the points on the grid with the mouse:
  - From point (0,0) to point (0,3)
  - From point (0,3) to point (4,3)
  - From point (4,3) to point (4,0)
  - From point (4,0) to point (0,0)



Notice that we drew always in the same direction (clockwise). This is not necessary, but will ensure that the walls that we will build next all have the same left and right directions. You might also think we could simply have drawn a rectangle here, which is true. But the four lines will allow us to illustrate better how to add one object into another.

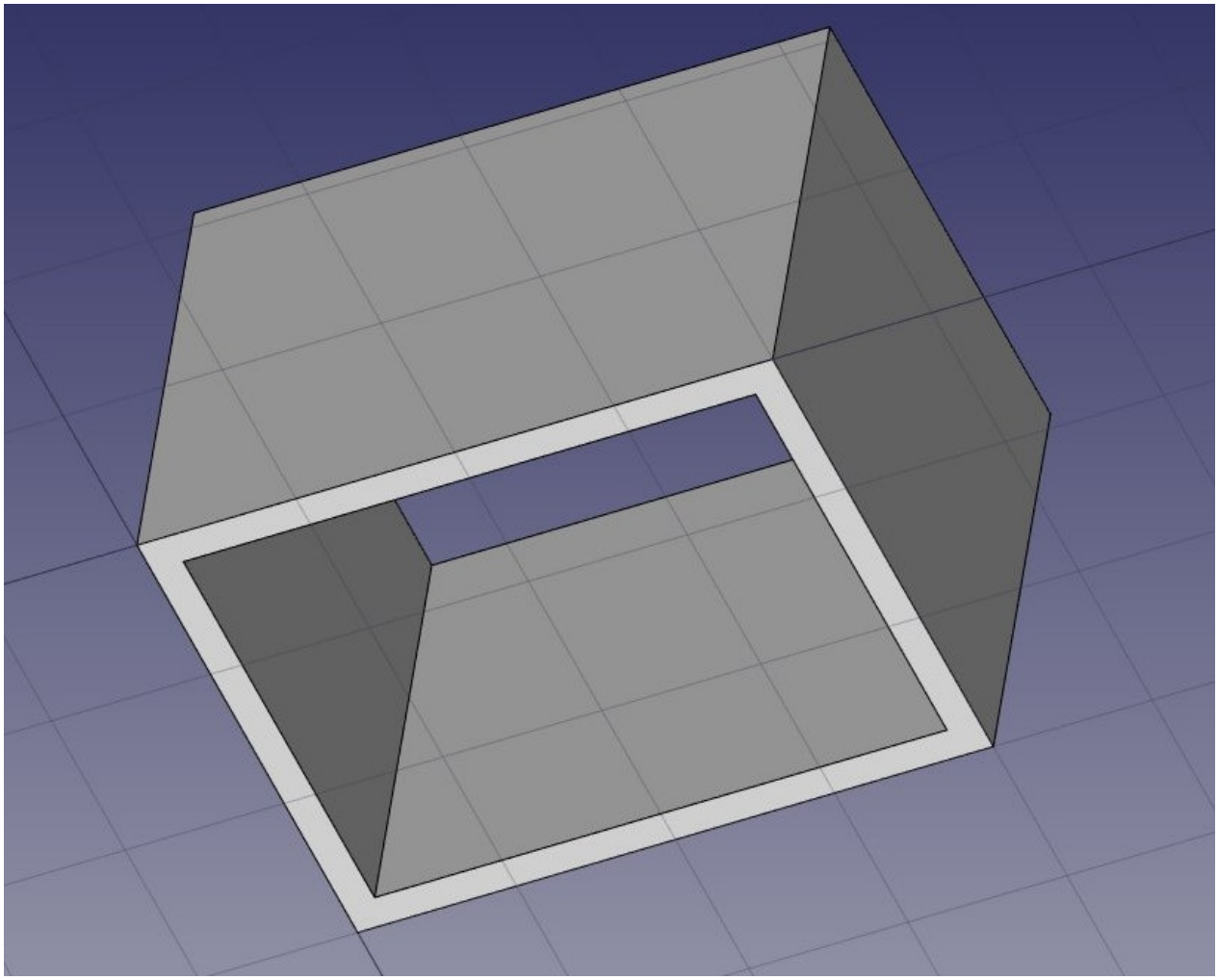
- Select the first line, then press the Arch Wall button.
- Repeat this for the 3 other lines, untilyou have 4 walls.
- Select the four walls, and set their **Height** property to **3.00m** and their **Alignment**

property to **left**. If you didn't draw the lines in the same order as we did above, some of the walls might have their left and right directions flipped, and might need to be set to **right** instead. You will obtain four intersecting walls, on the inside of the baselines:



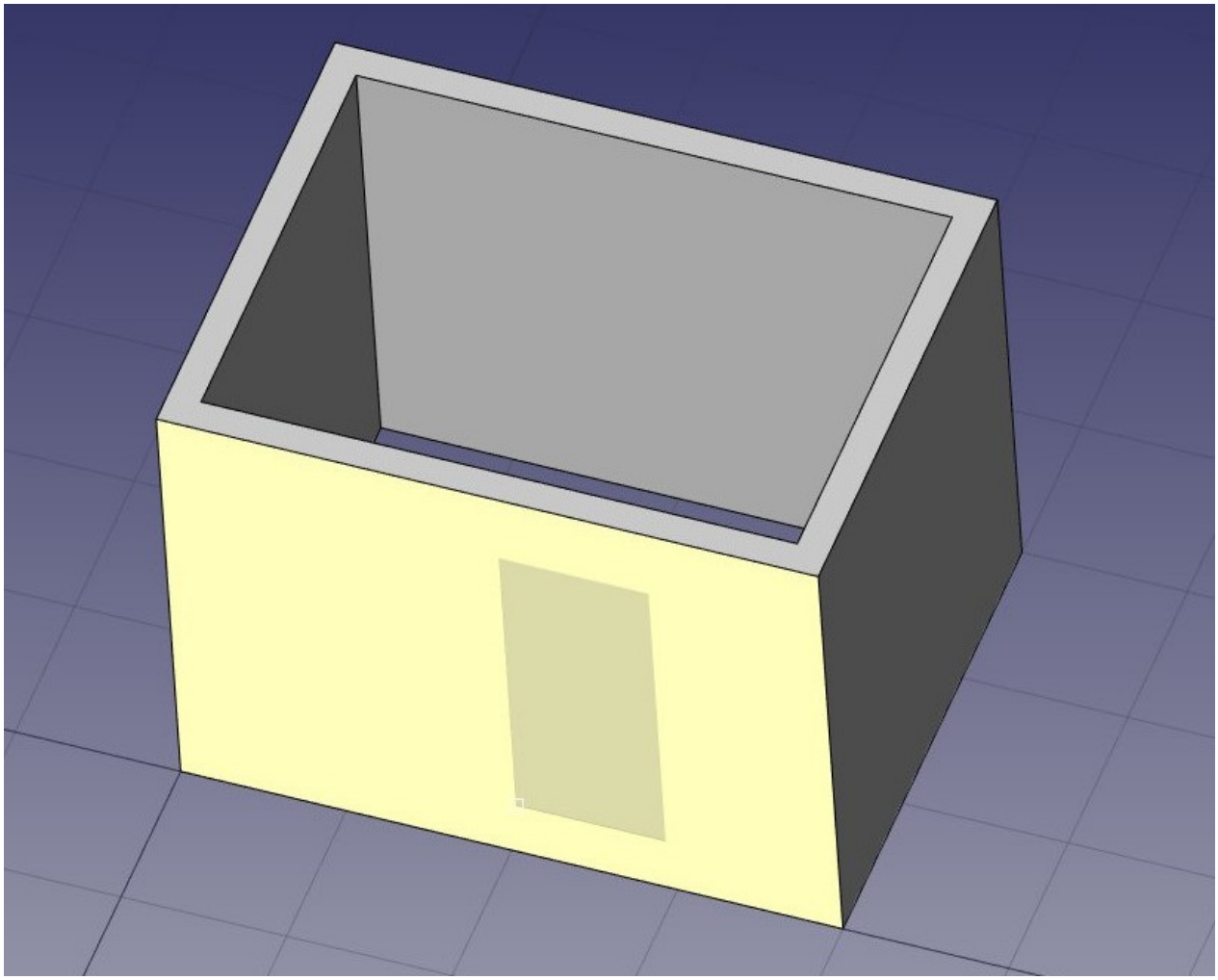
Now we need to join these walls together, so they intersect properly. This is not necessary when your walls are drawn in a way that they already connect cleanly, but here we need to, since they are intersecting. In Arch, this is done by electing one of the walls to be the "host", and adding the others to it, as "additions". All arch objects can have any number of additions (objects whose geometry will be added to the host's geometry), and subtractions (objects whose geometry will be subtracted). The additions and subtractions of an object can be managed anytime by double-clicking the object in the tree.

- Select the four walls with **Ctrl** pressed, the last one being the wall that you chose to become the host
- Press the Arch Add button. The four walls have now been turned into one:

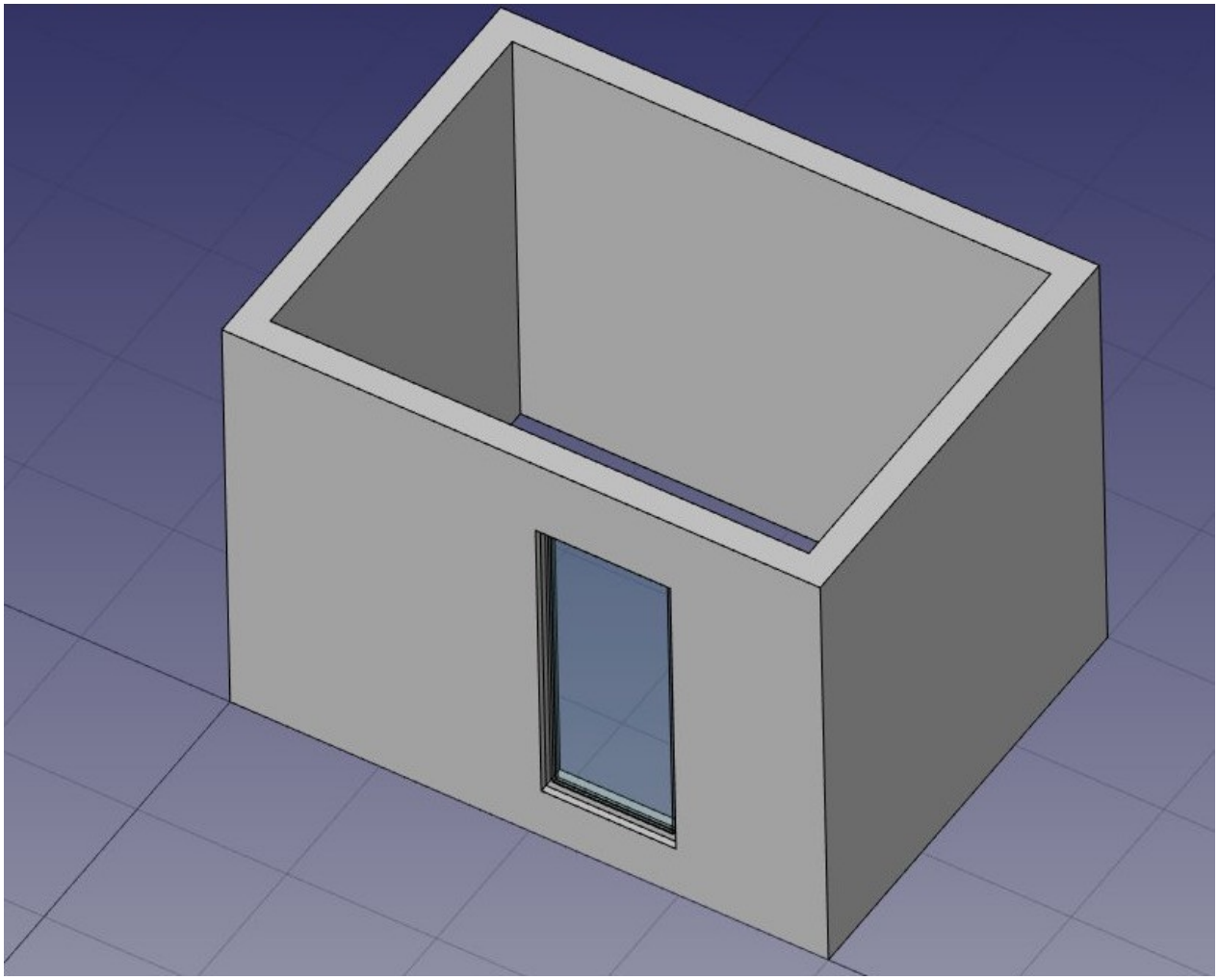


The individual walls are however still accessible, by expanding the wall in the tree view.

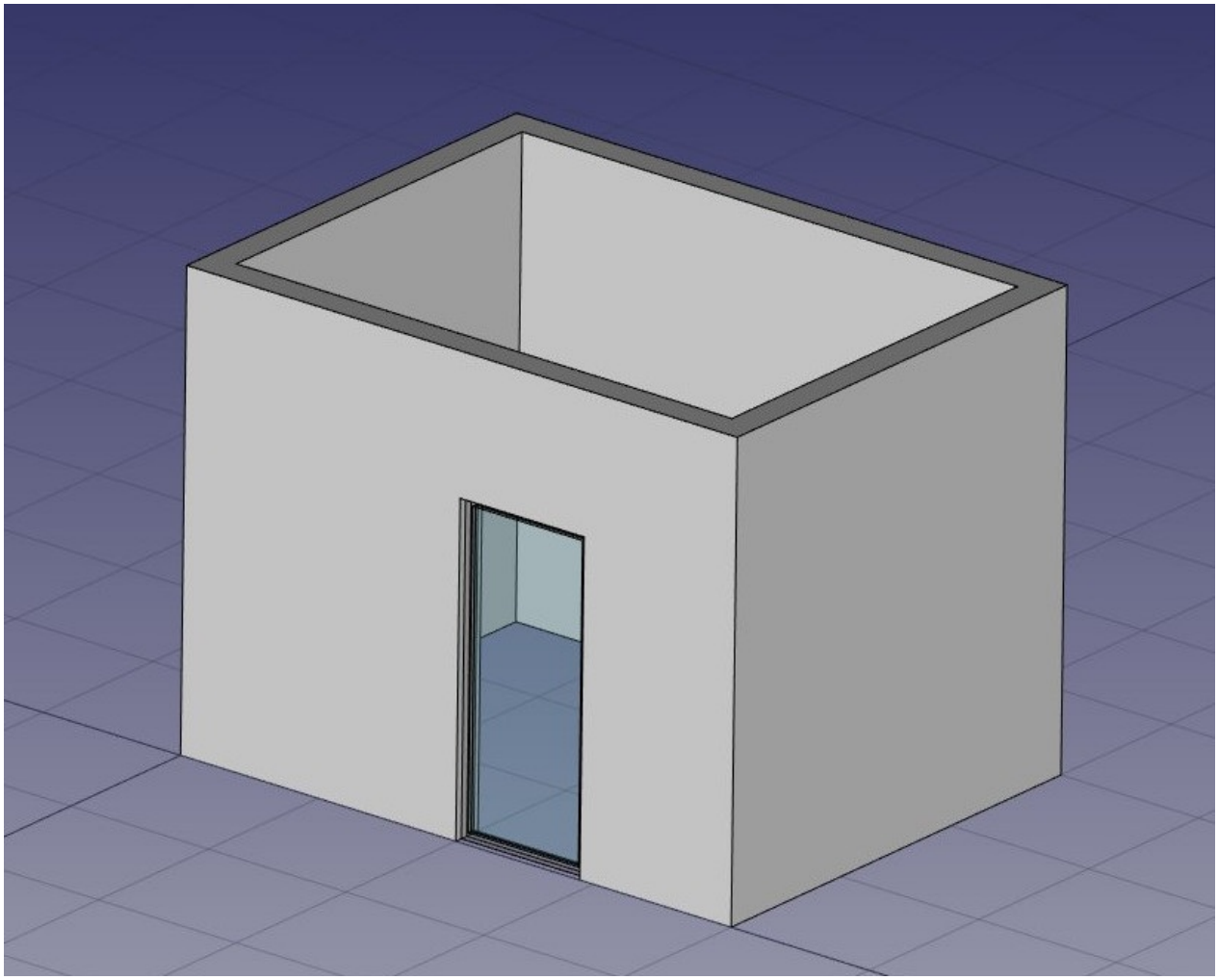
- Let's now place a door. In FreeCAD, doors are considered a special case of windows, so this is done using the Window tool.
- Start by selecting the wall. This is not necessary, but a good habit to take. If an object is selected when starting the window tool, you will force the window to be inserted in that object, even if you snap to another object.
- Set the Working Plane to **auto** so we are not restricted to the ground plane
- Press the Window button.
- In the window creation panel, select the **Simple door** preset, and set its **Width** to 0.9m and its **Height** to 2.1m
- Make sure the Near snap location is turned on, so we can snap on faces
- Place your window roughly on the middle of the front face of the wall:



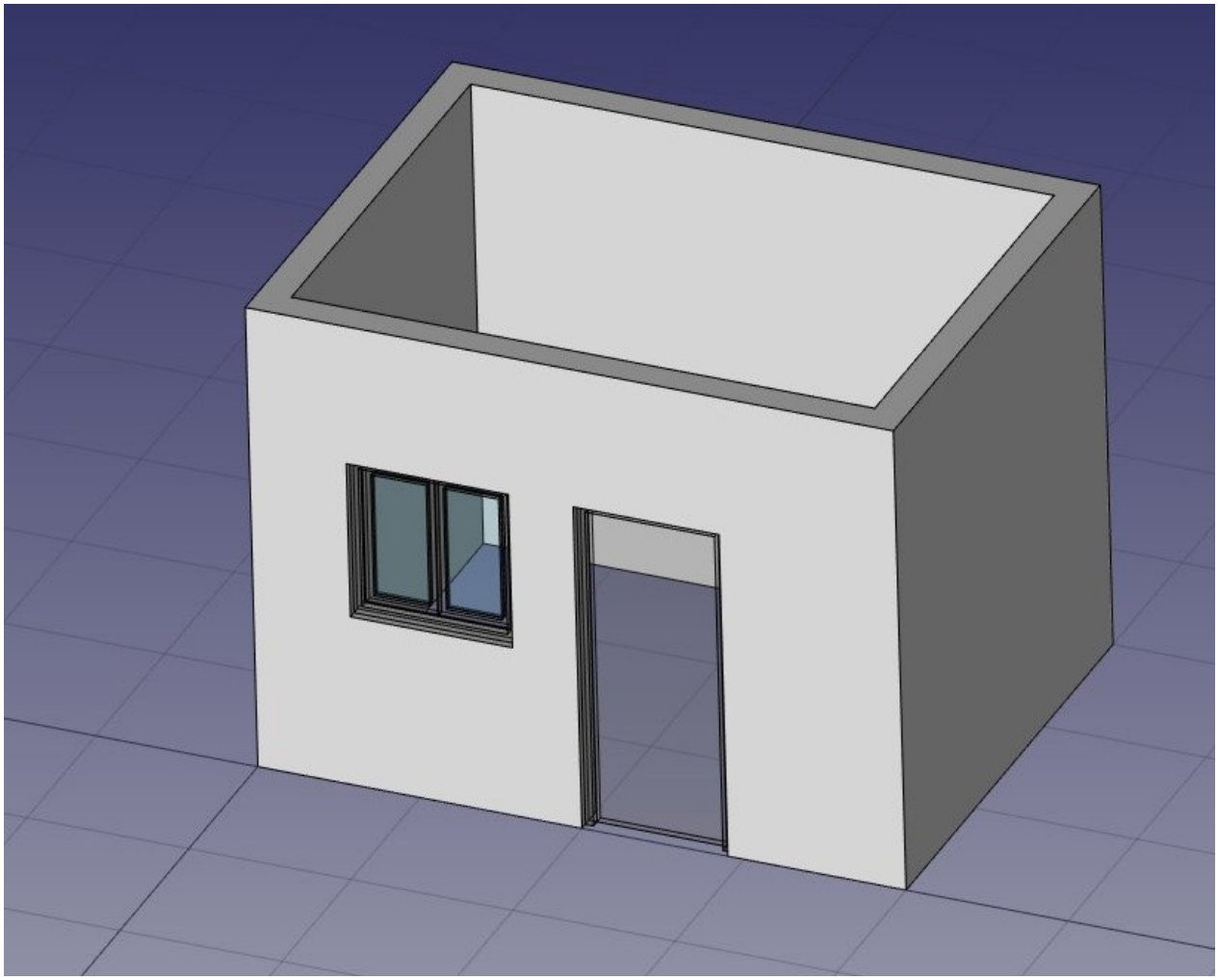
- After clicking, our window is placed on the correct face, but not exactly where we want:



- We can now set the precise location by expanding the wall and the window objects in the tree view, and changing the **Placement** property of the base sketch of our door. Set its position to **x = 2m, y = 0, z = 0**. Our window is now exactly where we want it:

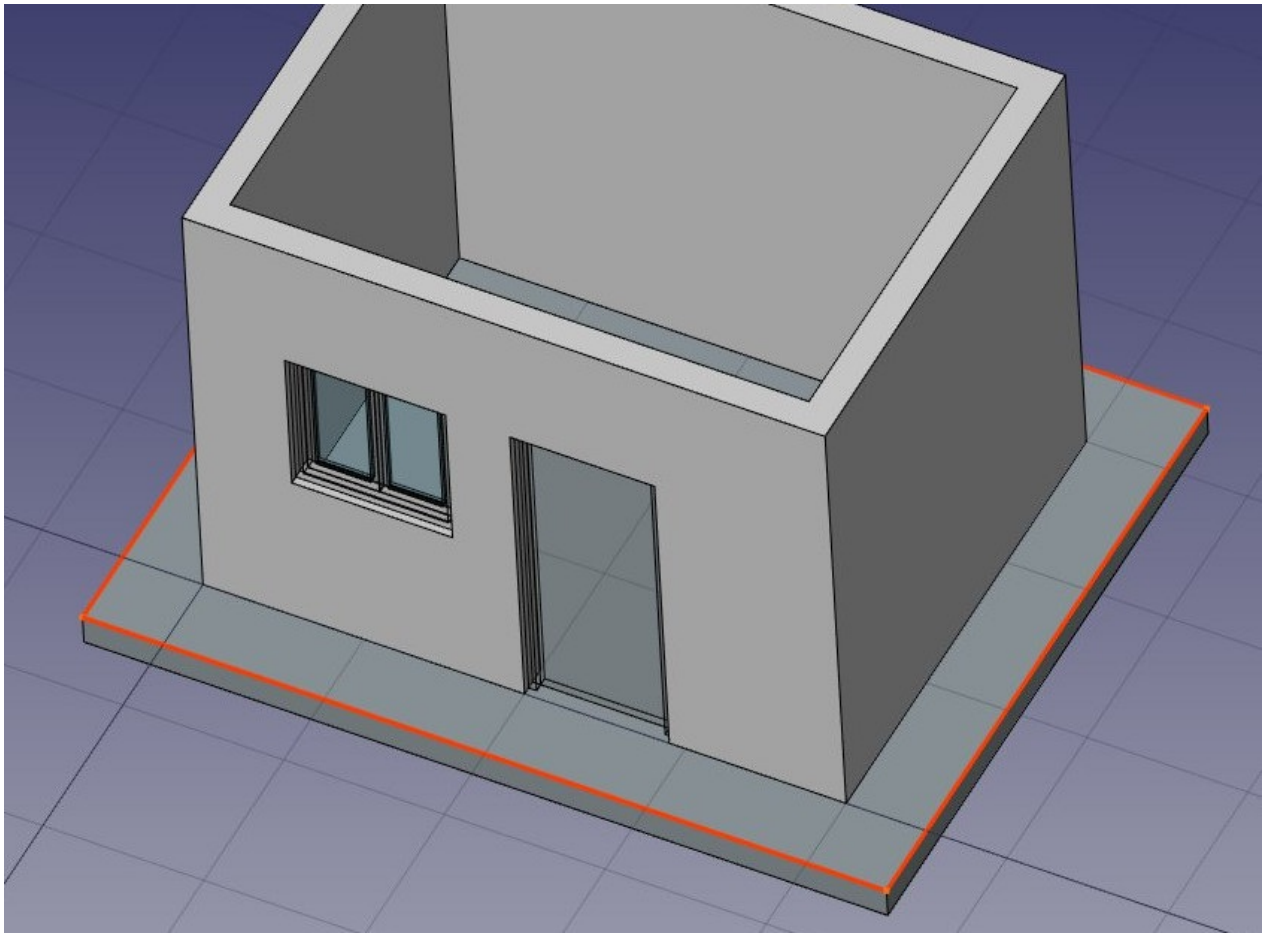


- Repeat the operation to place a window: Select the wall, press the window tool, select the **Open 2-pane** preset, and place a 1m x 1m window in the same face as the door. Set the placement of the underlying sketch to position **x = 0.6m, y = 0, z = 1.1m**, so the upper line of the window is aligned to the top of the door.

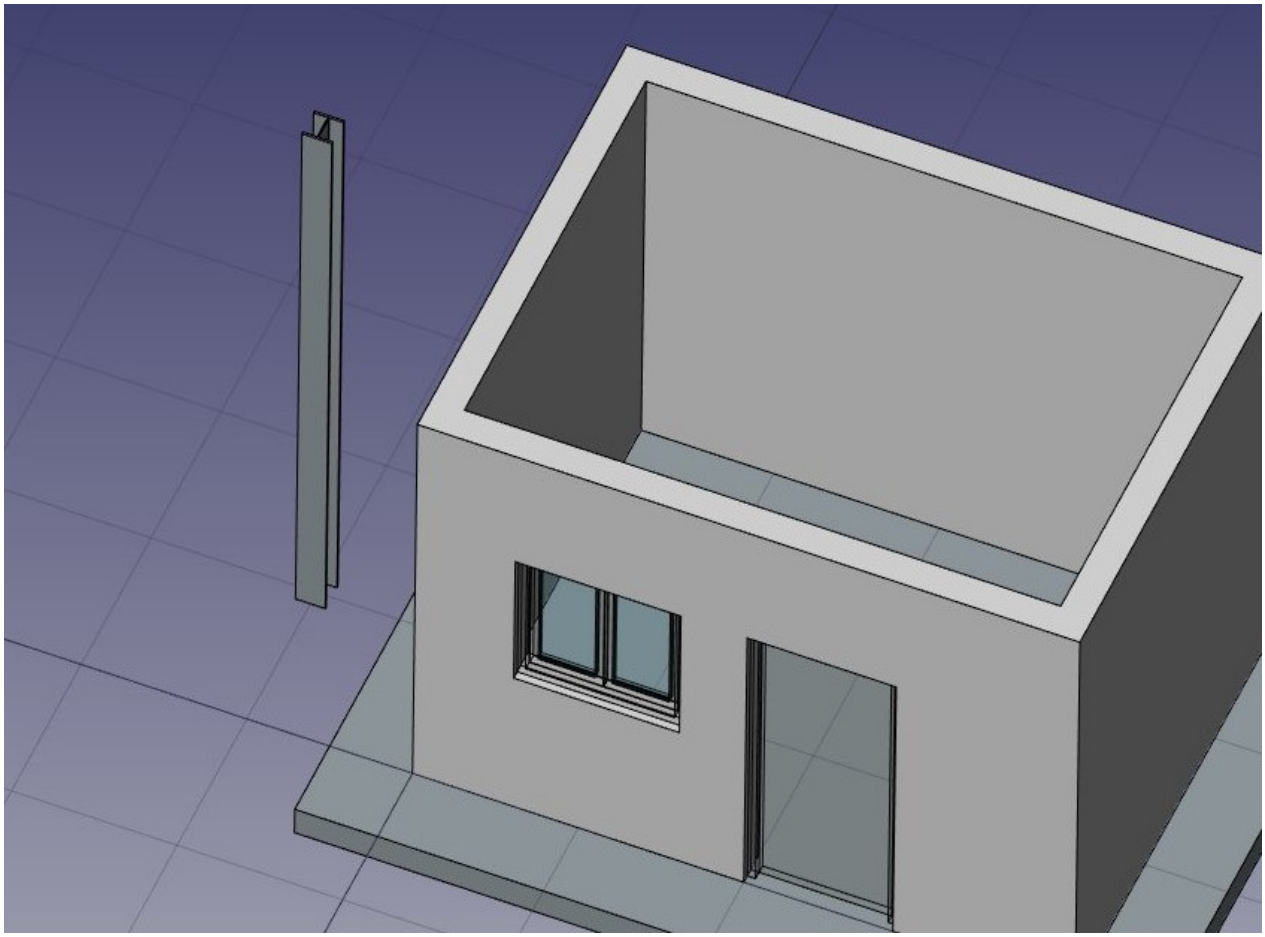


Windows are always built on sketches. It is easy to create custom windows by first creating a sketch on a face, then turning it into a window by selecting it, then pressing the window button. Then, the window creation parameters, that is, which wires of the sketch must be extruded and how much, can be defined by double-clickingthe window in the tree view. Let's now create a slab:

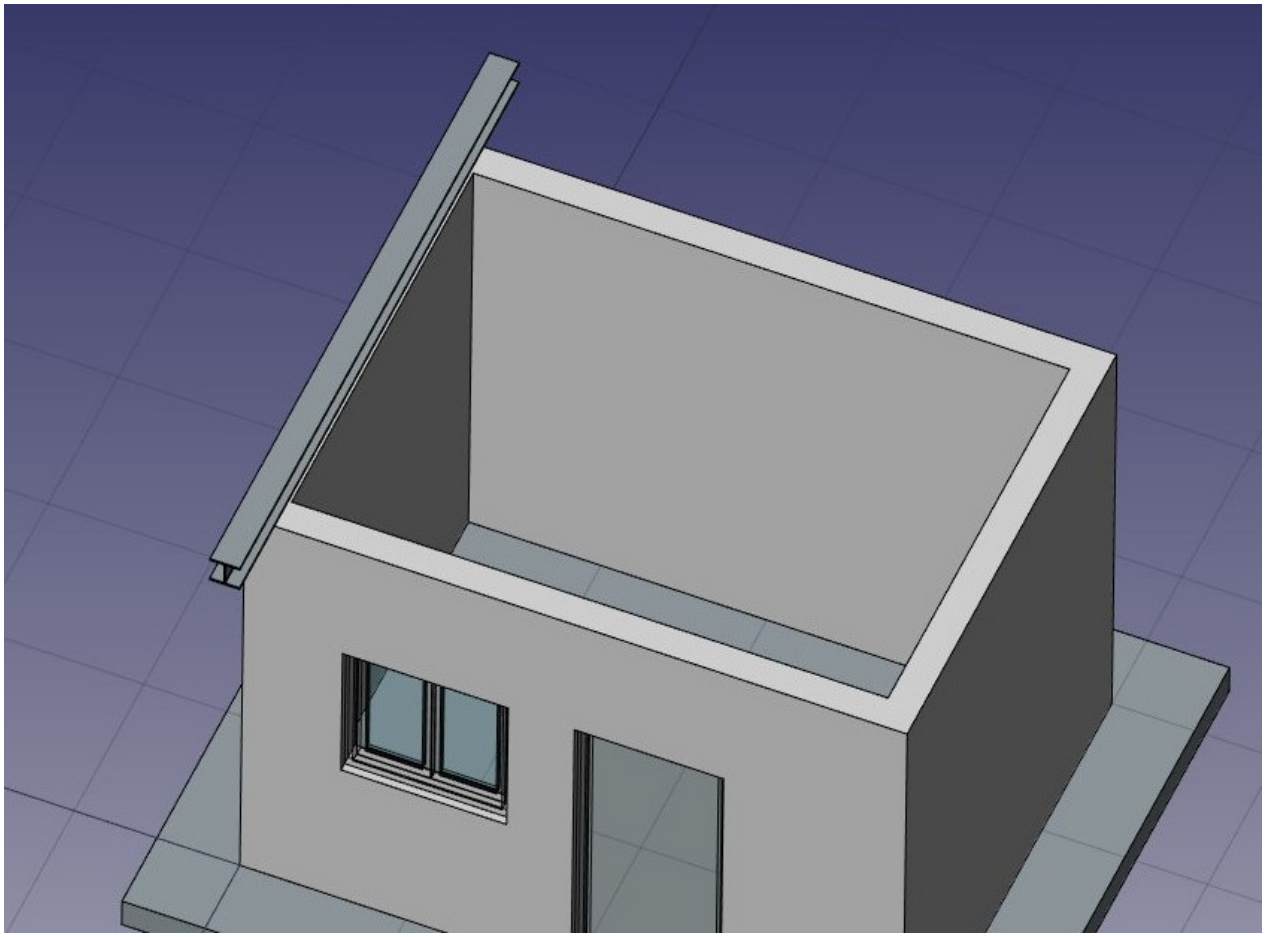
- Set the Working Plane to **XY** plane
- Create a rectangle with a **length** of 5m, a height of **4m**, and place it at position x:-0.5m, y:-0.5m, z:0.
- Select the rectangle
- Click the structure tool to create a slab from the rectangle
- Set the **height** property of the slab to 0.2m and its **normal** direction to (0,0,-1) because we want it to extrude downwards. We could also simply have moved it 0.2m down, but it is always good practice to keep extruded objects at the same place as their base profile.
- Set the **Role** property of the slab to **slab**. This is not necessary in FreeCAD, but is important for IFC export, as it will ensure that the object is exported with the correct IFC type.



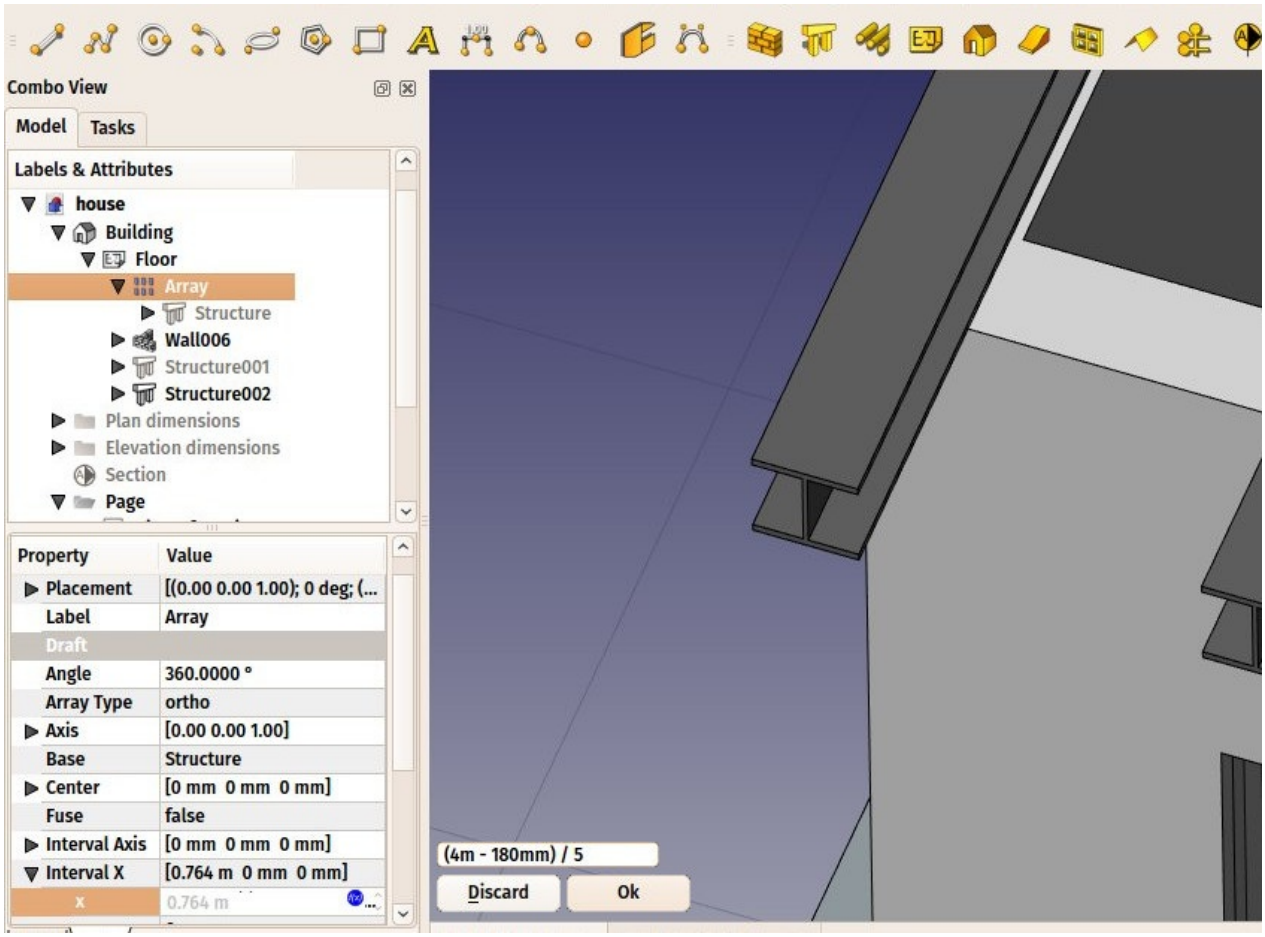
- Let's now use one of the structural presets to make a metallic beam. Click the structure button, select a **HEB 180** preset, and set its height to **4m**. Place it anywhere:



- Adjust its **placement** by setting its **rotation** to 90° in the (1,0,0) axis, and its **position** to x:90mm, y:3.5m, z:3.09m. This will position the beam exactly on one of the side walls:



- We need now to duplicate this beam a couple of times. We could do that one by one using the clone tool, but there is a better way, to do all the copies at once using an array:
- Select the beam
- Press the Array button
- Set the **Number X** property of the array to 6, leave the Y and Z numbers to 1
- Expand the **interval X** property, and press the small **expression** icon at the right side of the X field. This will open an expressions editor:

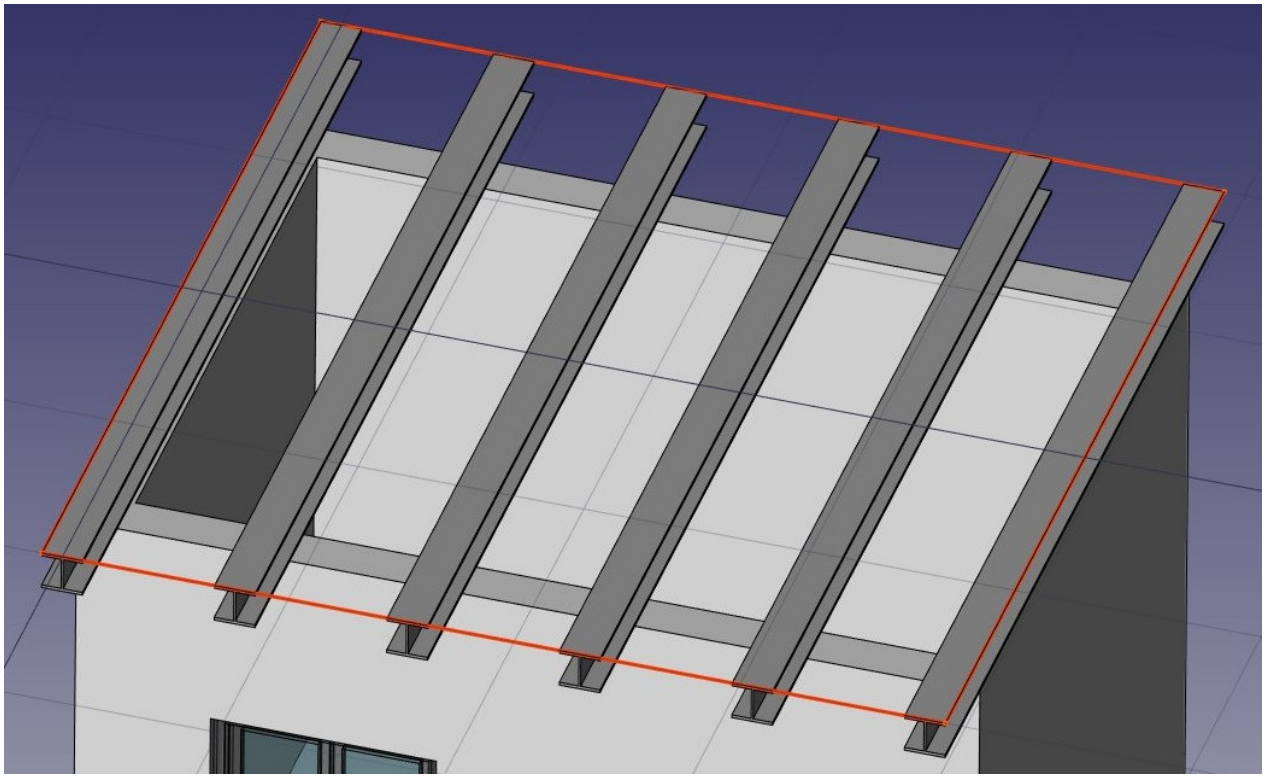


- Write **(4m-180mm)/5** in the expression field, and press **OK**. This will set the x value to 0.764 (4m is the total length of our front wall, 180mm is the width of the beam, which is why it is called HEB180, and we want a fifth of that space as interval between each beam):



- We can now easily build a simple slab on top of them, by drawing a rectangle directly on the top plane of the beams. Select a top face of one of the beams

- Press the working plane button. The working plane is now set to that face.
- Create a rectangle, snapping to two opposite points of the border beams:

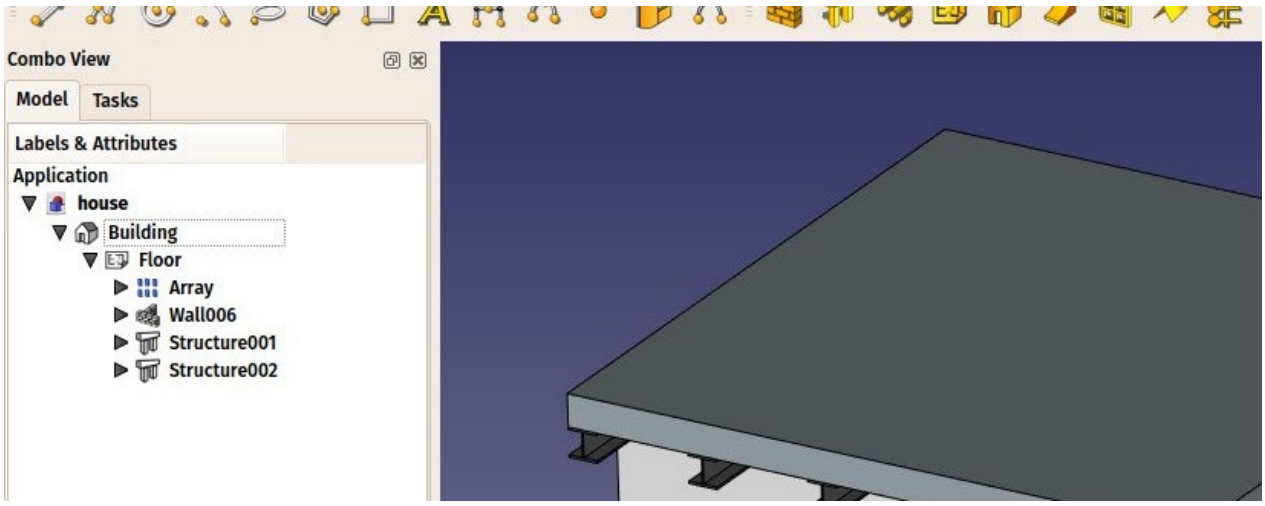


- Select the rectangle
- Click the structure button and create a slab with a height of **0.2m**.

That's it, our model is now complete. We should now organize it so it exports correctly to IFC. The IFC format requires that all objects of a building are inside a building object, and optionally, inside a storey. It also requires that all buildings are placed on a site, but the IFC exporter of FreeCAD will add a default site automatically if needed, so we don't need to add one here.

- Select the two slabs, the wall, and the array of beams
- Press the Floor button
- Select the floor we just created
- Press the Building button

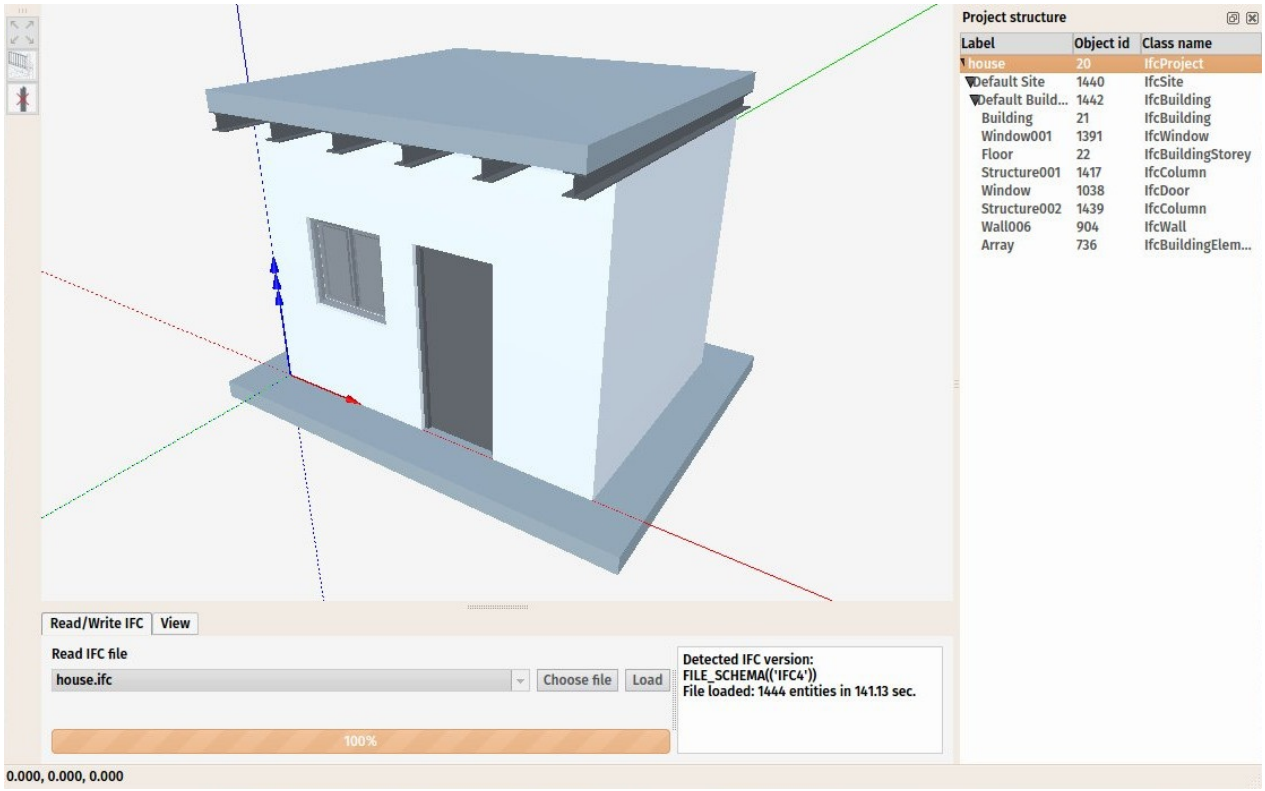
Our model is now ready to export:



The IFC format is one of the most precious assets in a free BIM world, because it allows the exchange of data between any application and actor of the construction world, in an open manner (the format is open, free and maintained by an independent consortium). Exporting your BIM models as IFC ensures that anyone can see and analyze them, no matter the application used.

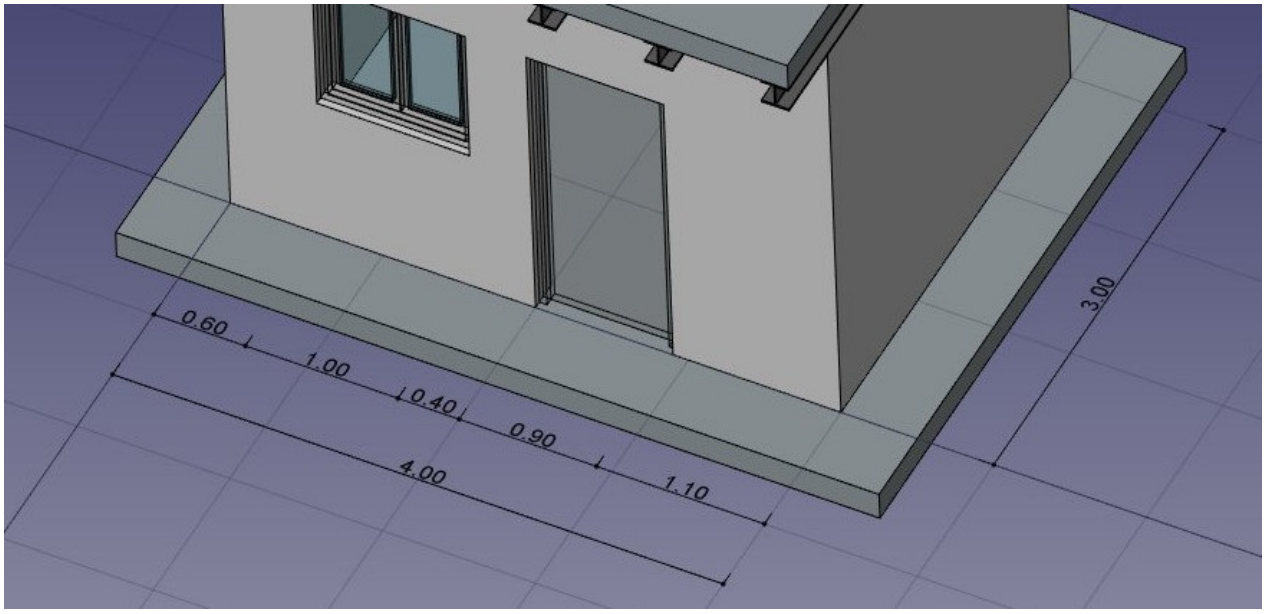
In FreeCAD, IFC import and export is done by interfacing with another piece of software, called IfcOpenShell. To be able to export to IFC from FreeCAD, the IfcOpenShell-python package must be installed on your system. Be sure to select one which uses the same python version as FreeCAD. The python version that FreeCAD uses is informed when opening the **View -> Panels -> Python console** panel in FreeCAD. When that is done, we can now export our model:

- Select the top object you want to export, that is, the Building object.
- Select menu **File -> Export -> Industry Foundation Classes** and save your file.
- The resulting IFC file can now be opened ina wide range of applications and viewers (the image below shows the file opened in the free IfcPlusPlus viewer. Checking the exported file in such a viewer application before distributing it to other people is important to check that all the data contained in the file is correct. FreeCAD itself can also be used to re-open the resulting IFC file.

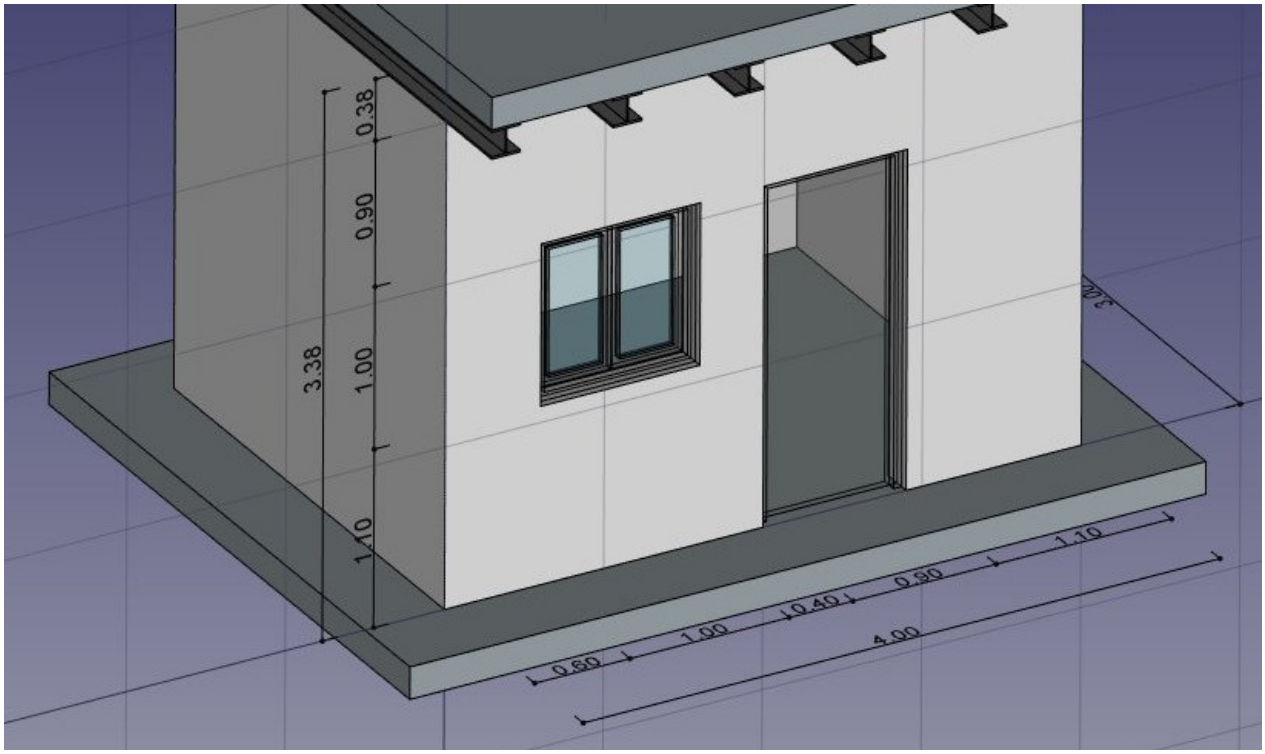


We will now place some dimensions. Unlike the previous chapter, where we drew all the dimensions directly on the Drawing sheet, we will use another method here, and place Draft dimensions directly in the 3D model. These dimensions will then be placed on the Drawing sheet. We will first make two groups for our dimensions, one for the dimensions that will appear in the plan view, and another for those that appear on the elevation.

- Right-click the "house" document in the tree view, and create two new groups: **Plan dimensions** and **Elevation dimensions**.
- Set the Working Plane to **XY** plane
- Make sure the restrict snap location is turned on, so everything you draw stays on the working plane.
- Draw a couple of dimensions, for example as on the image below. Pressing **Shift** and **Ctrl** while snapping the dimension points will give you additional options.

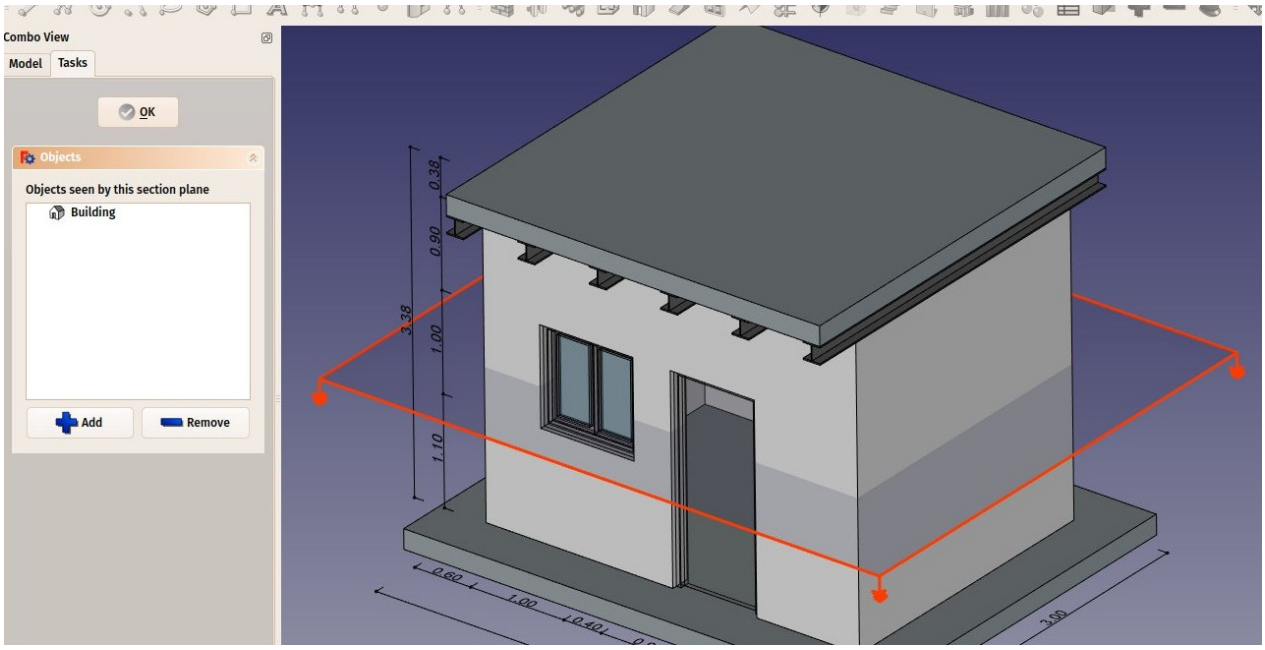


- Select all your dimensions, and drag them to the **Plan dimensions** group in the tree view
- Set the Working Plane to **XZ** plane, that is, the frontal vertical plane.
- Repeat the operation, draw a couple of dimensions, and place them in the **Elevation dimensions** group.

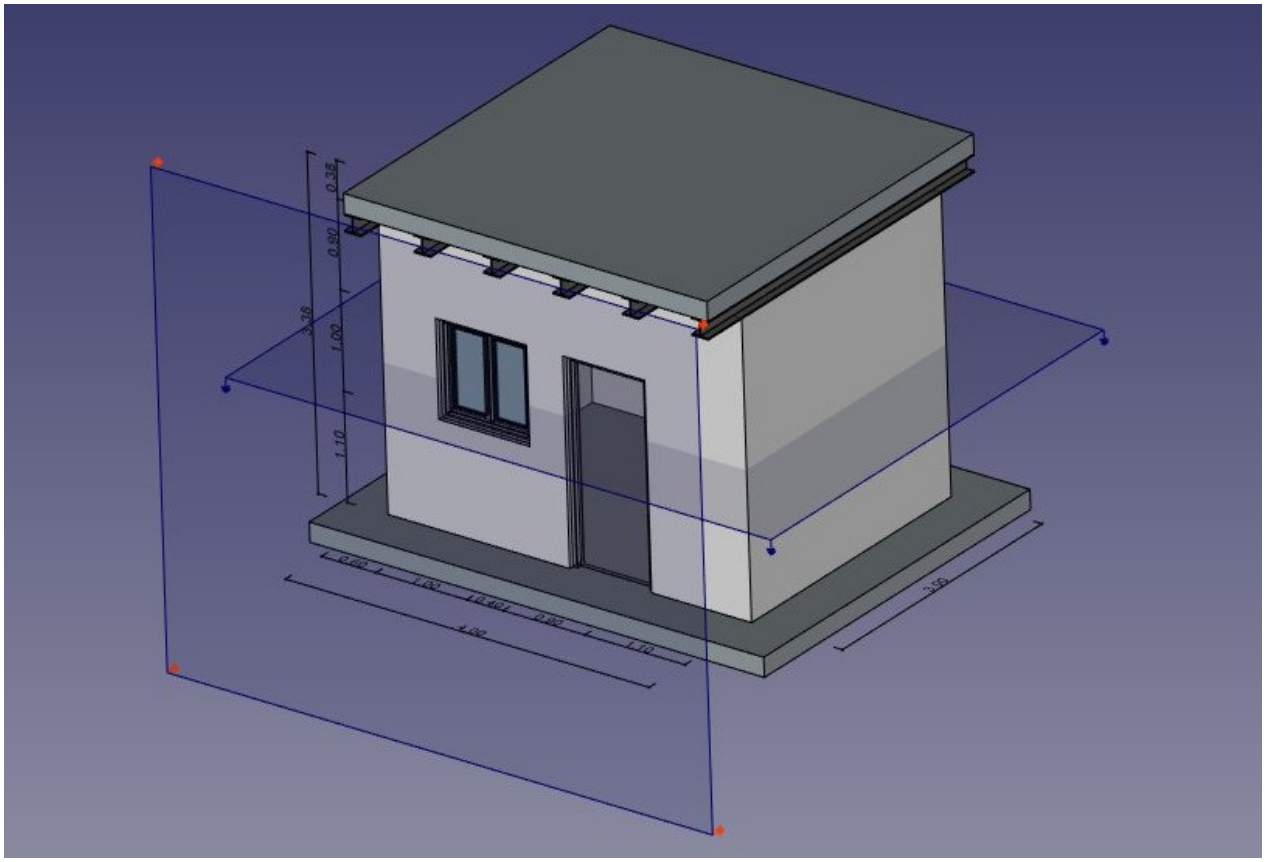


We will now prepare a set of views from our model, to be paced on a Drawing page. We can do that with the tools from the Drawing Workbench, as we have seen in the previous chapter, but the Arch Workbench also offers an all-in-one advanced tool to produce plan, section and elevation views, called Section Plane. We will now add two of these section planes, to create a plan view and an elevation view.

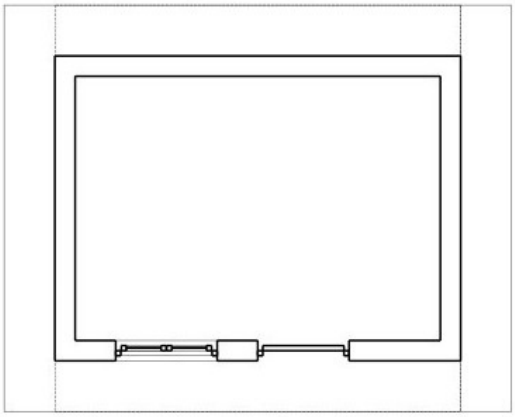
- Select the building object in the tree view
- Press the Section Plane button.
- Set its **Display Height** property to 5m, its **Display Length** to 6m, so we encompass our house (this is not needed, but will look better, as it will show naturally what it is used for), and its **Placement** position at x:2m, y:1.5m, z:1.5m.
- Check the list of objects considered by the Section Plane by double-clicking it in the tree view. Section Planes only render specified objects from the model, not all of them. The objects considered by the Section Plane can be changed here.

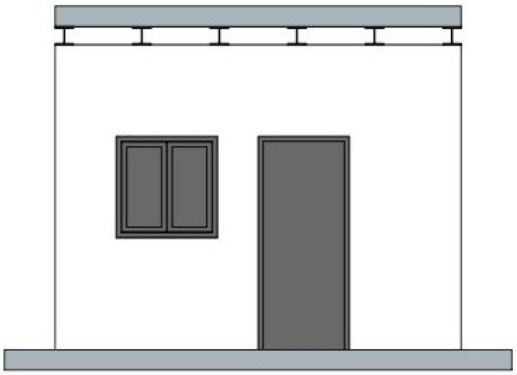


- Repeat the operation to create another section plane, give it the same display length and height, and give it the following **Placement**: position: x:2m, y:-2m, z:1.5m, angle: 90°, axis: x:1, y:0, z:0. Make sure this new section plane also considers the building object.



- Now we have everything we need, and we can create our Drawing page. Start by switching to the Drawing Workbench, and create a new default A3 page (or select another template if you wish).
- Select the first section plane, used for the plan view
- Press the Draft View button. This tool offers a couple of additional features over the standard Drawing View tool, and supports the Section Planes from the Arch Workbench.
- Give the new view the following properties:
  - X: 50
  - Y: 140
  - Scale: 0.03
  - Line width: 0.15
  - Show Cut True
  - Show Fill: True
- Select the other section plane, and create a new Draft View, with the followng properties:
  - X: 250
  - Y: 150
  - Scale: 0.03
  - Rendering: Solid







We will now create two more Draft Views, one for each group of dimensions.

- Select the Plan dimensions group
- Press the Draft View button.
- Give the new view the following properties:
  - X: 50
  - Y: 140
  - Scale: 0.03
  - Line width: 0.15
  - Font size: 10mm
- Repeat the operation for the other group, with the following settings:
  - X: 250
  - Y: 150
  - Scale: 0.03
  - Line width: 0.15
  - Font size: 10mm
  - Direction: 0,-1,0
  - Rotation: 90°

Our page is now ready, and we can export it to SVG or DXF formats, or print it. The SVG format allows to open the file illustration applications such as inkscape, with which you can quickly enhance technical drawings and turn them into much nicer presentation drawings. It offers many more possibilities than the DXF format.

Downloads

- The file produced during this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/house.FCStd
- The IFC file exported from the above file: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/house.ifc
- The SVG file exported from the above file: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/house.svg

Read more

- The Arch Workbench: http://www.freecadweb.org/wiki/index.php?title=Arch_Module
- The Draft working plane: http://www.freecadweb.org/wiki/index.php?title=Draft_SelectPlane
- The Draft snapping settings: http://www.freecadweb.org/wiki/index.php?title=Draft_Snap
- The expressions system: http://www.freecadweb.org/wiki/index.php?title=Expressions
- The IFC format: https://en.wikipedia.org/wiki/Industry_Foundation_Classes
- IfcOpenShell: http://ifcopenshell.org/
- IfcPlusPlus: http://ifcplusplus.com/
- Inkscape: http://www.inkscape.org

Using spreadsheets

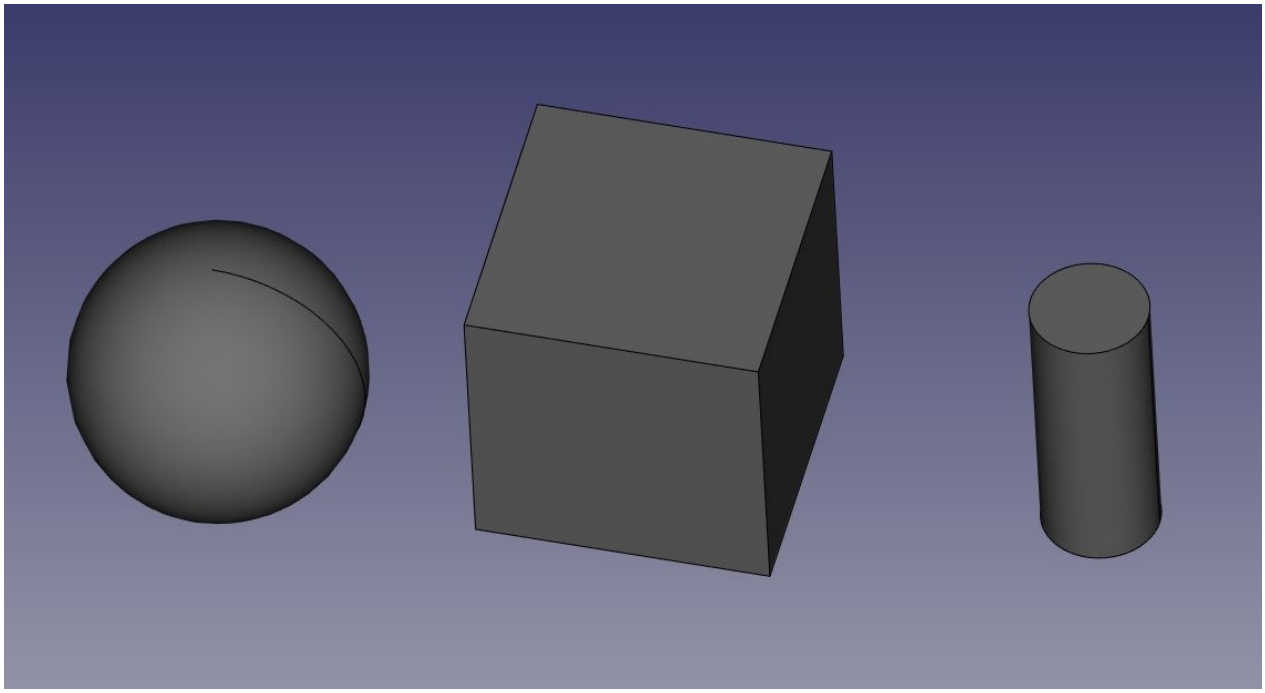
FreeCAD features another interesting workbench to explore: the Spreadsheet Workbench. This workbench allows to create spreadsheets such as those made with Excel or LibreOffice directly in FreeCAD. These spreadsheets can then be populated with data extracted from your model, and can also perform a series of calculations between values. Spreadsheets can be exported as CSV files, which can be imported in any other spreadsheet application.

In FreeCAD, however, spreadsheets have an additional utility: Their cells can receive a name, and can then be referenced by any field supported by the expressions engine. This turns spreadsheets into powerful control structures, where the values inserted in specific cells can drive dimensions of the model. There is only one thing to keep in mind, as FreeCAD prohibits circular dependencies between objects, a same spreadsheet cannot be used to set a property of an object and at the same time retrieve a property value from the same object. That would make the spreadsheet and the object depending on each other.

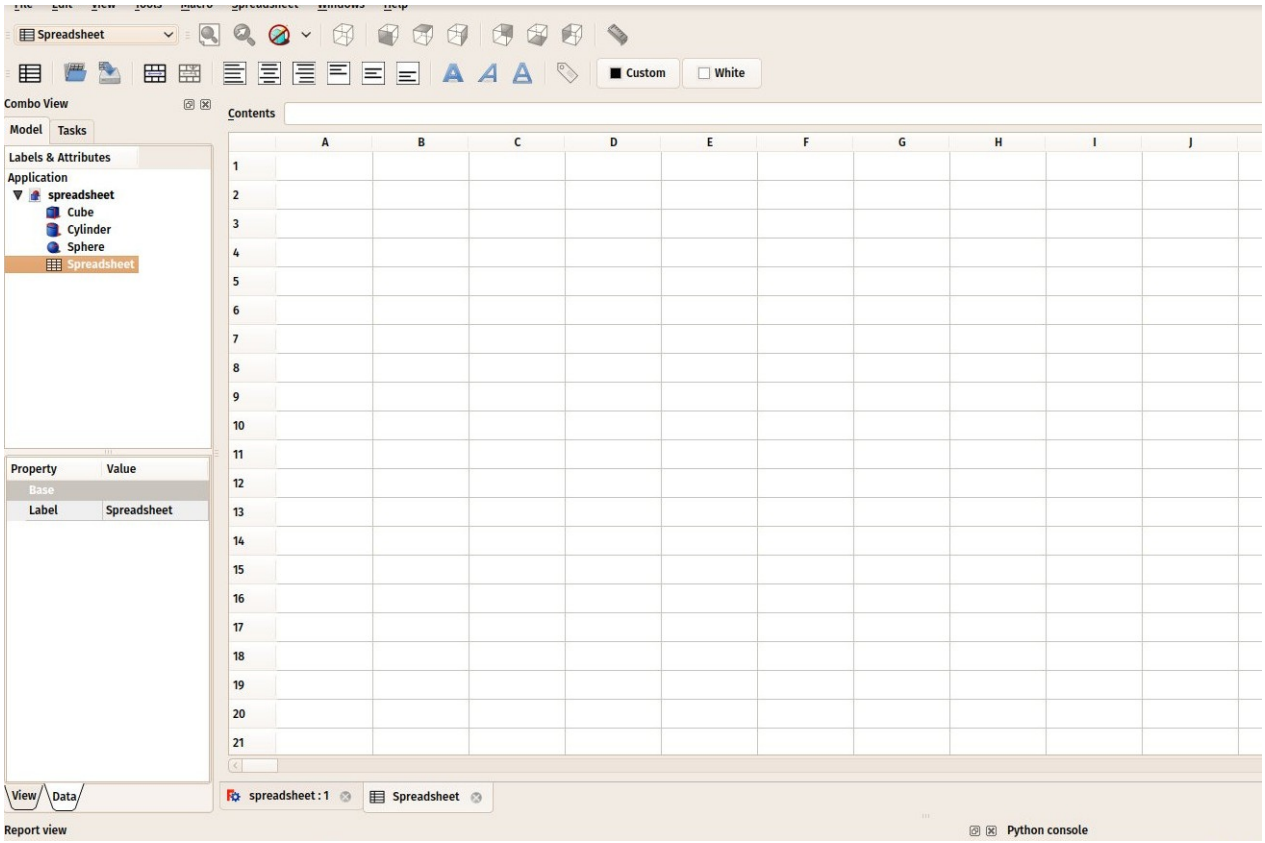
In the following example, we will create a couple of objects, retrieve some of their properties in a spreadsheet, then use the spreadsheet to directly drive properties of other objects. se

Reading properties

- Start by switching to the Part Workbench, and create a couple of objects: a box, a cylinder and a sphere.
- Edit their **Placement** property (or use the Draft Move tool) to place them a little apart, so we can watch better the effects of what we'll do:



- Now, It's extract some information about these objects. Switch to the Spreadsheet Workbench
- Press the **New Spreadsheet** button
- Double-click the new Spreadsheet object in the tree view. The spreadsheet editor opens:



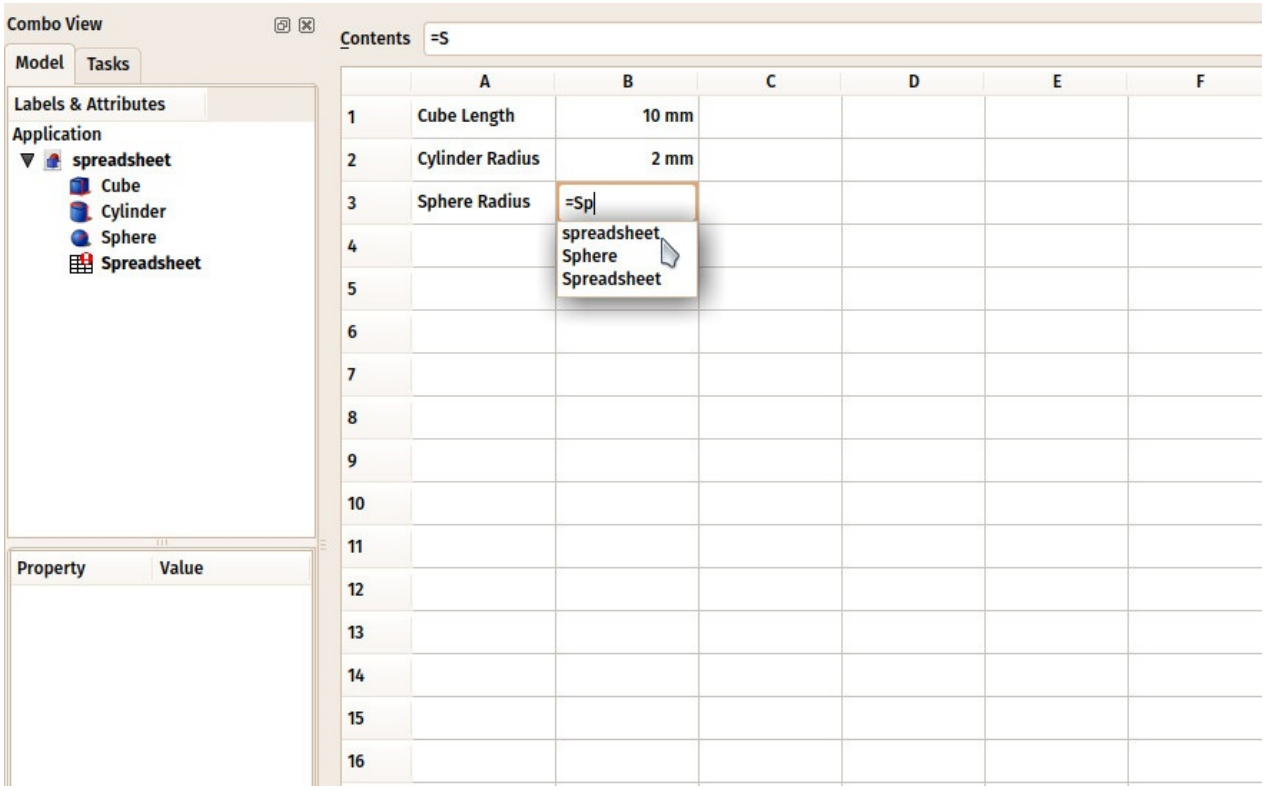
The spreadsheet editor of FreeCAD, although it is not as complete and powerful as the more complete spreadsheet applications we listed above, has nevertheless most of the basic tools and functions that are commonly used, such as the possibility to change the aspect of the cells (size, color, alignment), join and split cells, use formulas such as **=2+2**, or reference other cells with **=B1**.

In FreeCAD, to these common behaviours, has been added one very interesting: The possibility to reference not only other cells, but other objects from the document, and retrieve values from their properties. For example, let's retrieve a couple of properties from the 3 objects we created above. Properties are what we can see in the properties editor window, under the **Data** tab, when an object is selected.

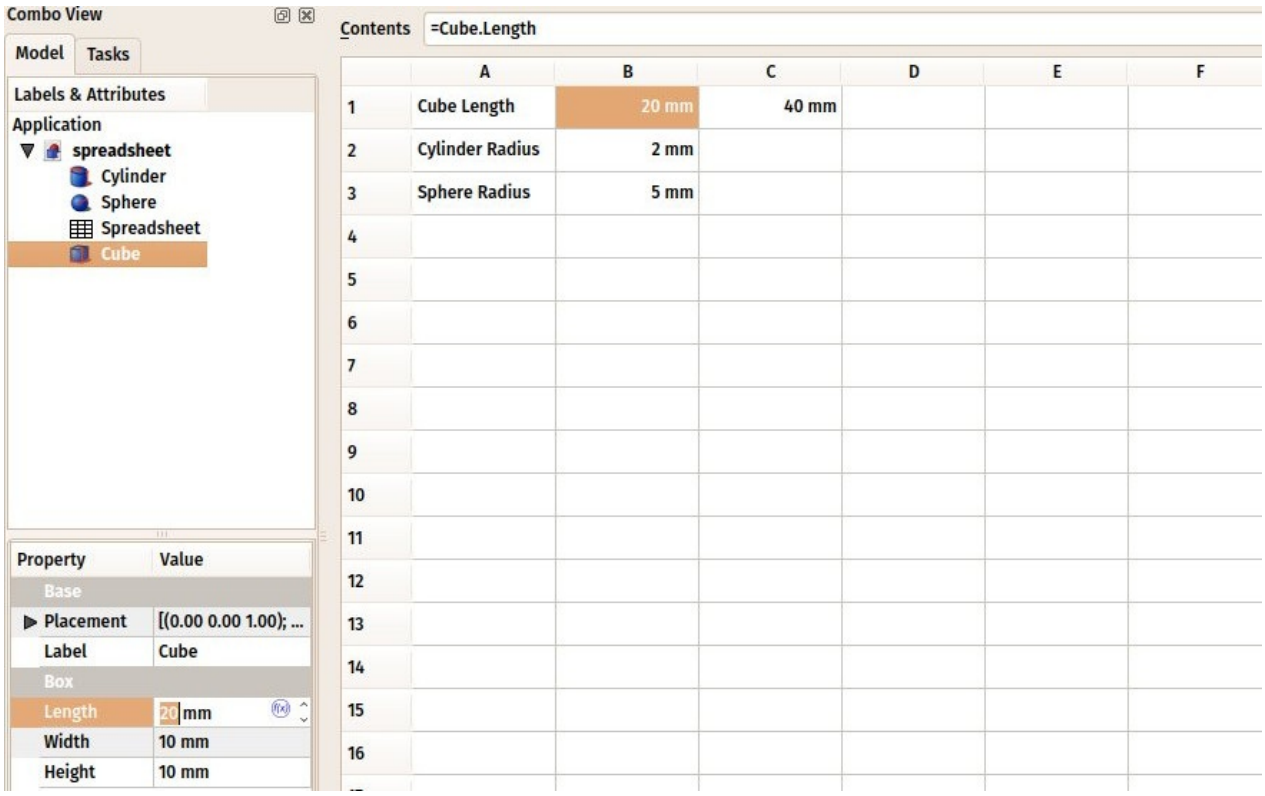
- Let's start by entering a couple of texts in the cells A1, A2 amd A3, so we remember what is what later on, for example **Cube Length**, **Cylinder Radius** and **Sphere Radius**. To enter text, just write in the "Contents" filed above the spreadsheet, or double-click a cell.
- Now let's retrieve the actual length of our cube. In cell B1, type **=Cube.Length**. You will notice that the spreadhseet has an autocompletion mechanism, which is actually the

same as the expression editor we used in the previous chapter.

- Do the same for cell B2 (**=Cylinder.Radius**) and B3 (**=Sphere.Radius**).



- Although these results are expressed with their units, the values can be manipulated as any number, try for example entering in cell C1: **=B1*2**.
- We can now change one of these values in the propertties editor, and the change will be immediately reflected in the spreadsheet. For example, let's change the length of our cube to **20mm**:



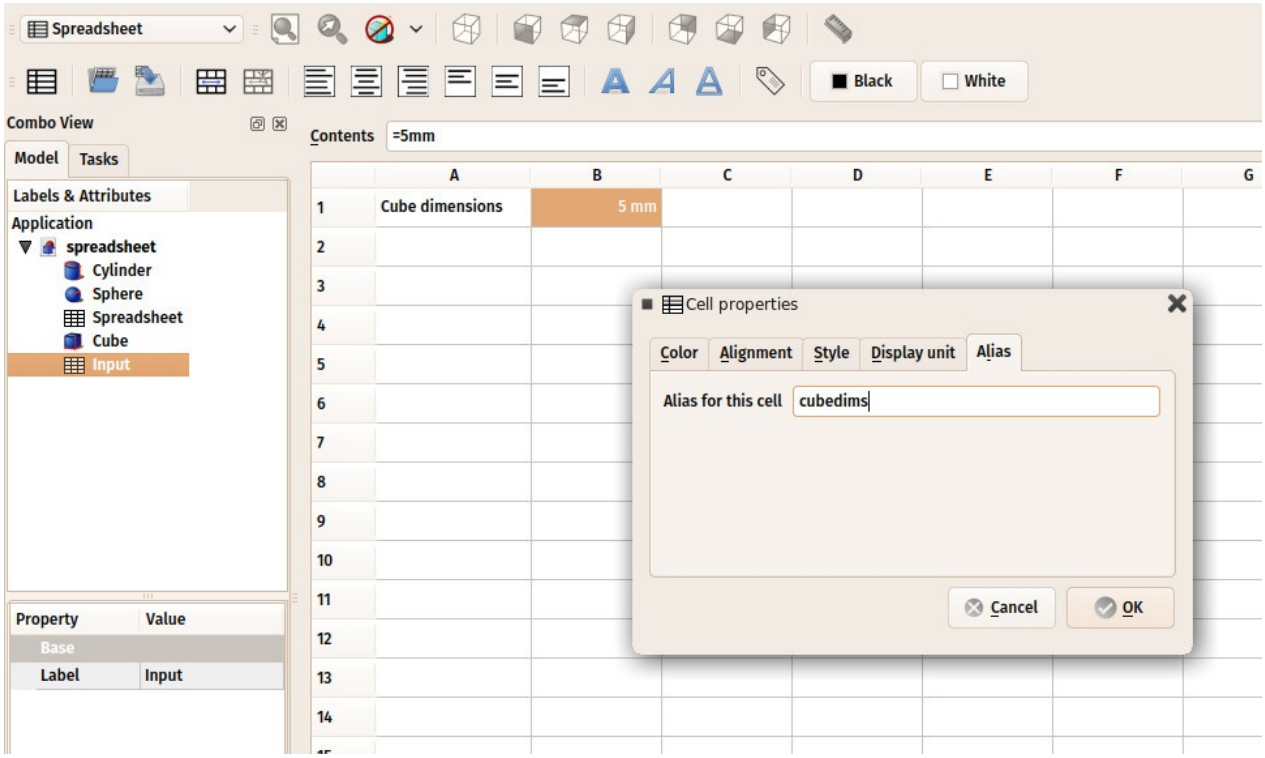
The Spreadsheet Workbench page will describe more in detail all the possible operations and functions that you can use in spreadsheets.

Writing properties

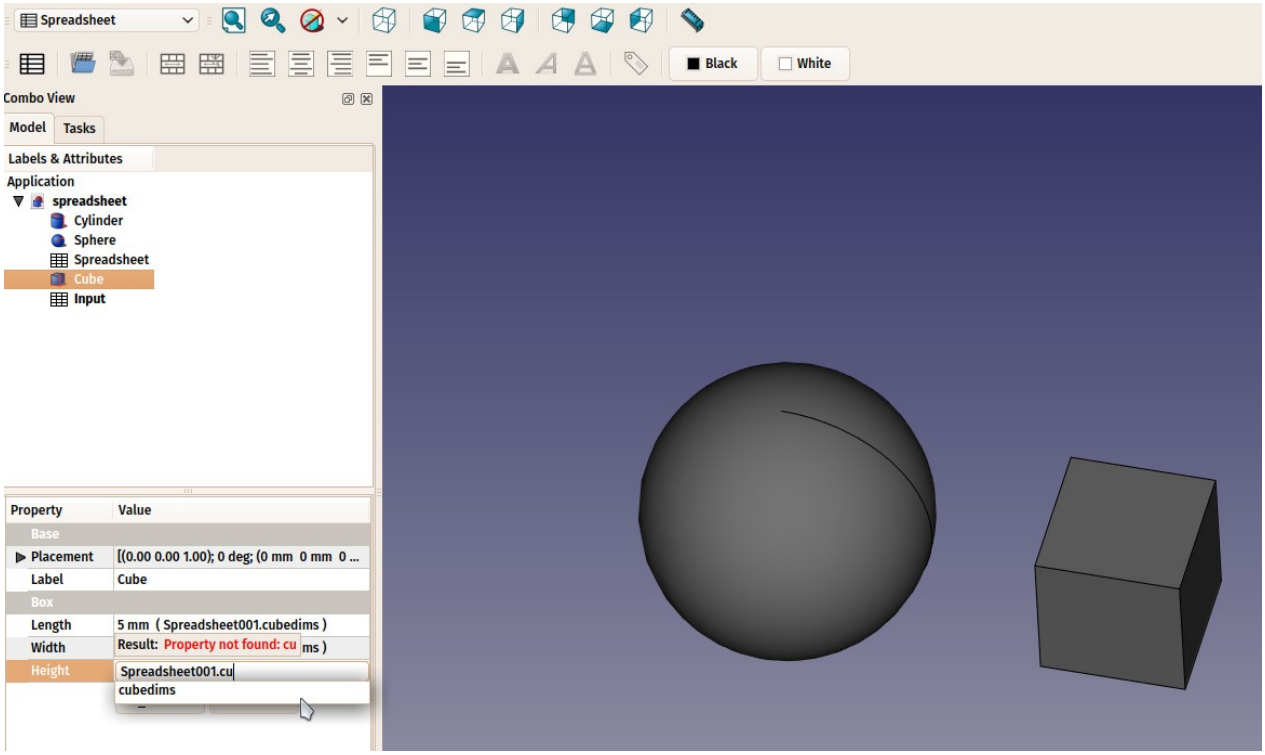
Another very interesting use of the Spreadsheet Workbench in FreeCAD is to do the contrary of what we have been doing until now: Instead of reading the values of properties of 3D objects, we can also assign values to these objects. Remember, however, one of the fundamental rules of FreeCAD: Circular dependencies are forbidden. We can therefore not use the same spreadsheet to read **and** write values to a 3D object. That would make the object depend on the spreadsheet, which would also depend on the object. Instead, we will create another spreadsheet.

- We can now close the spreadsheet tab (under the 3D view). This is not mandatory, there is no problem in keeping several spreadsheet windows open.
- Press the **New Spreadsheet** button again
- Change the name of the new spreadsheet to something more meaningful, suchas **Input** (do this by right-clicking the new spreadsheet object, and choosing **Rename**).
- Double-click the Input spreadsheet to open the spreadsheet editor.
- In cell A1, let's put a descriptive text, for example: "Cube dimensions"
- In cell B1, write **=5mm** (using the = sign makes sure the value is interpreted as a unit value, not a text).
- Now to be able to use this value outside the spreadsheet, we need to give a name, or alias, to the B1 cell. Right-click the cells, click **Properties** and select the **Alias** tab. Give

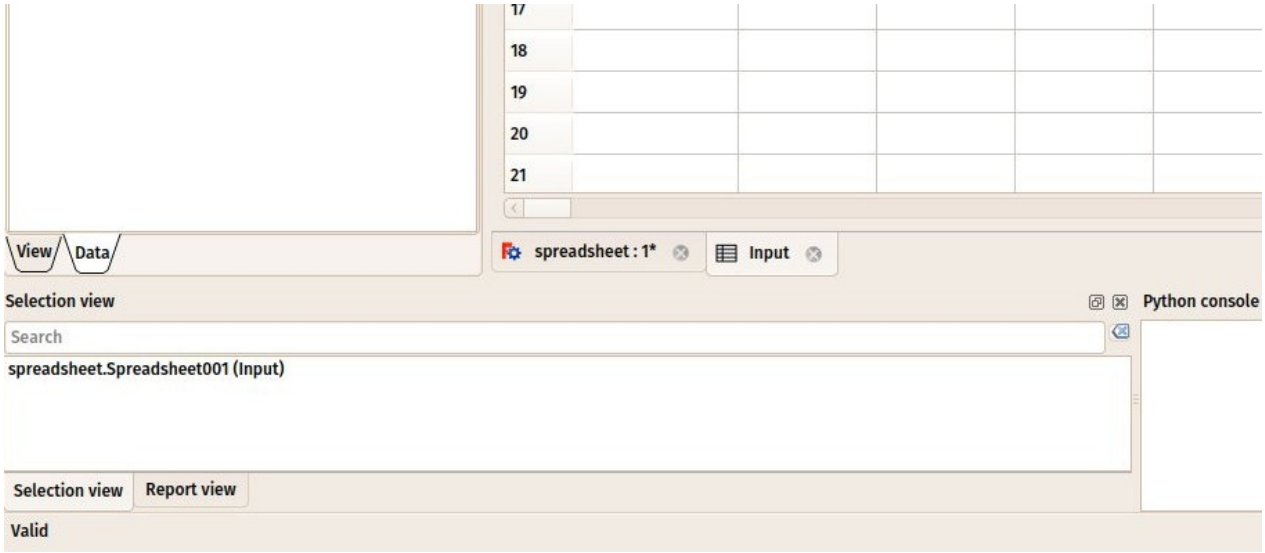
it a name, such as **cubedims**:



- Press **OK**, then close the spreadsheet tab
- Select the cube object
- In the properties editor, click the little **expression** icon at the right side of the **Length** field. This will openthen expressions editor, where you can write **Spreadsheet001.cubedims**. Repeat this for Height and Width:

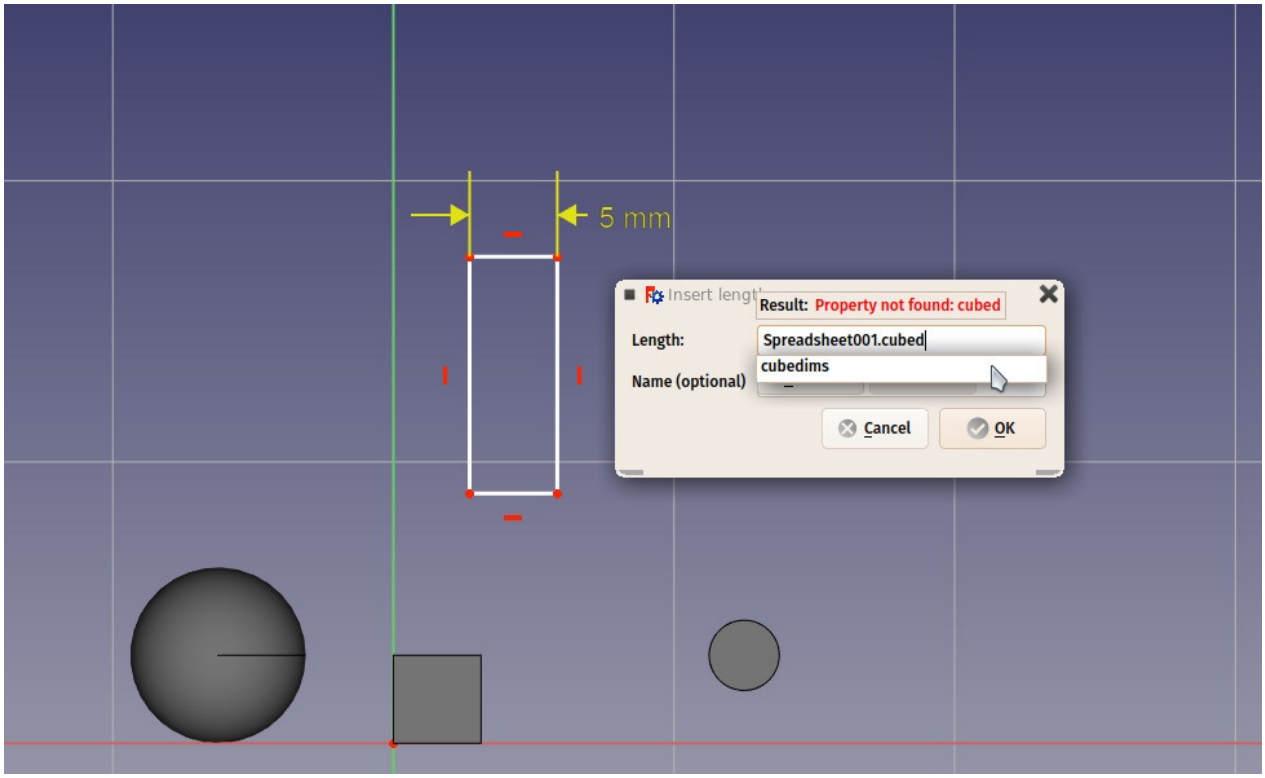


You might wonder why we had to use "Spreadsheet001" instead of "Input" in the expression above. This is beacause each object, in a FreeCAD document, has an **internal name**, which is unique in the document, and a **label**, which is what appears in the tree view. If you uncheck the appropriate option in the preferences settings, FreeCAD will allow you to give the same label to more than one object. This is why all operations that must identify an object with absolutely no doubt, will use the internal name instead of the label, which could designate more than one object. The easiest way to know the internal name of an object is by keeping the **selection panel** (menu Edit->Panels) open, it will always indicate the internal name of a selected object:

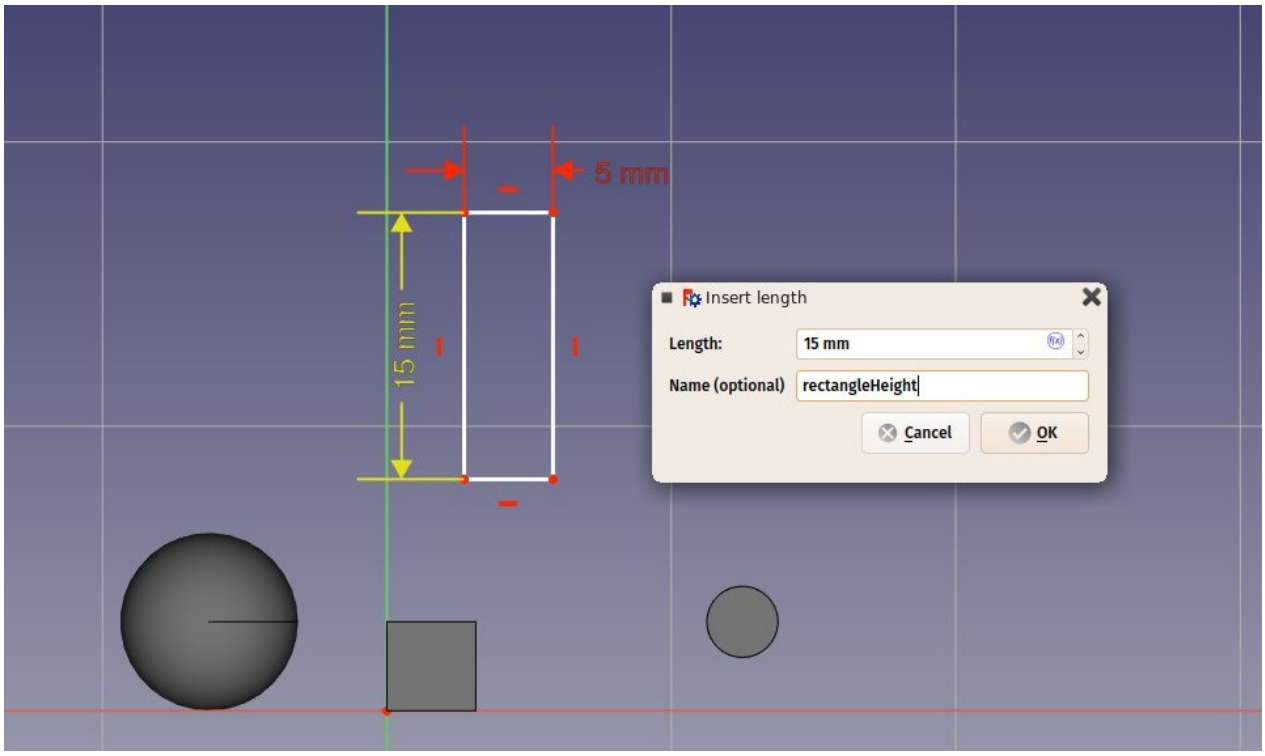


By using cell aliases in spreadsheets, we are able to use a spreadsheet to store "master values" in a FreeCAD document. This can be used, for example, to have a model of a piece of certain dimensions, and to store these dimensions in a spreadsheet. It becomes then very easy to produce another model with different dimensions, it is just a matter of opening the file and changing a couple of dimensions in the spreadsheet.

Finally, note that the constraints inside a sketch can also receive the value of a spreadsheet cell:



You can also give aliases to dimensional constraints (horizontal, vertical or distance) in a sketch (you can then use that value from outside the sketch as well):



Download

- The file produced in this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/spreadsheet.FCStd

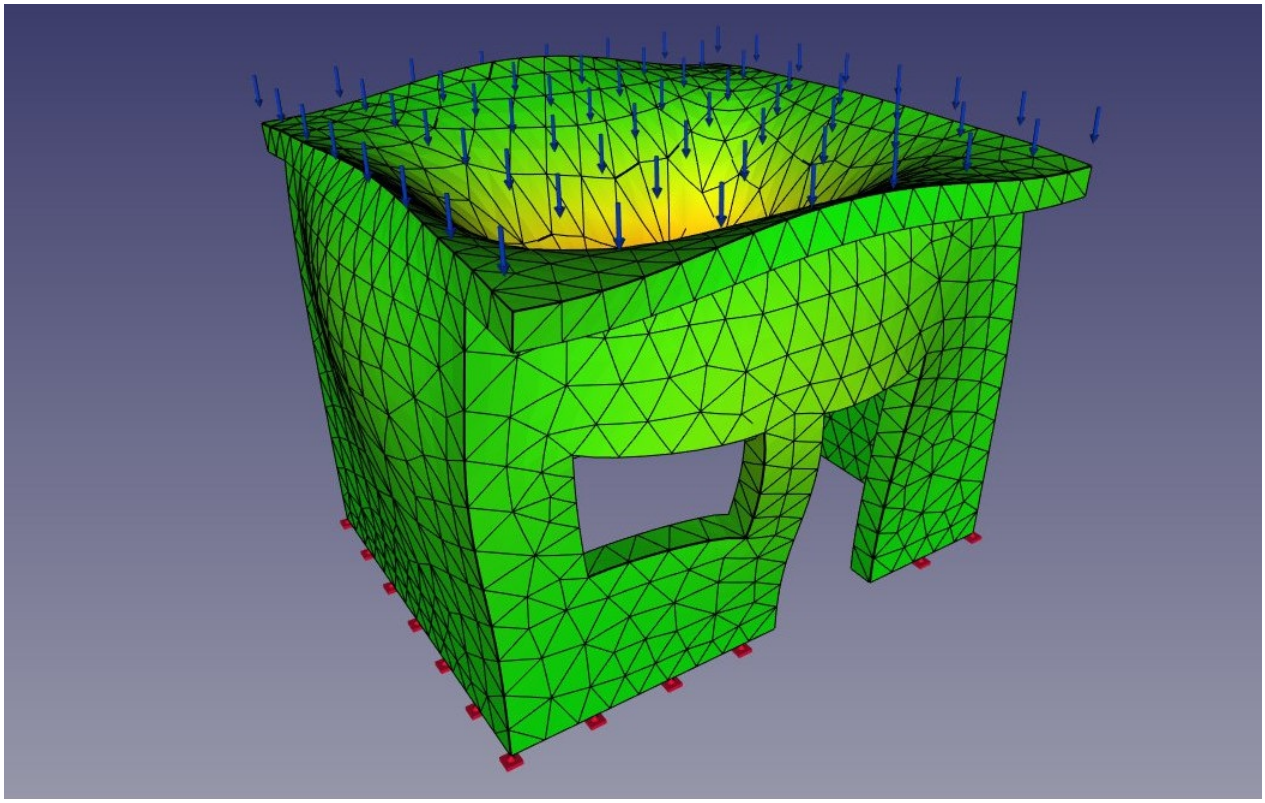
Read more

- The Spreadsheet Workbench: http://www.freecadweb.org/wiki/index.php?title=Spreadsheet_Module
- The Expressions engine: http://www.freecadweb.org/wiki/index.php?title=Expressions

Creating FEM analyses

FEM stands for Finite Element Method. It is a vast mathematical subject, but in FreeCAD we can resume it as a way to calculate propagations inside a 3D object, by cutting it into small pieces, and analyzing the impact of each small piece over its neighbours. This has several uses in the engineering and electomagnetism fields, but we will look here more in depth at one use that is already well developed in FreeCAD, which is simulating deformations in objects which are submitted to forces and weights.

Obtaining such simulation is done in FreeCAD with the FEM Workbench. It involves different steps: Preparing the geometry, setting its material, performing the meshing (division into smaller parts, like we did in the Preparing objects for 3D printing chapter, and finally calculating the simulation.



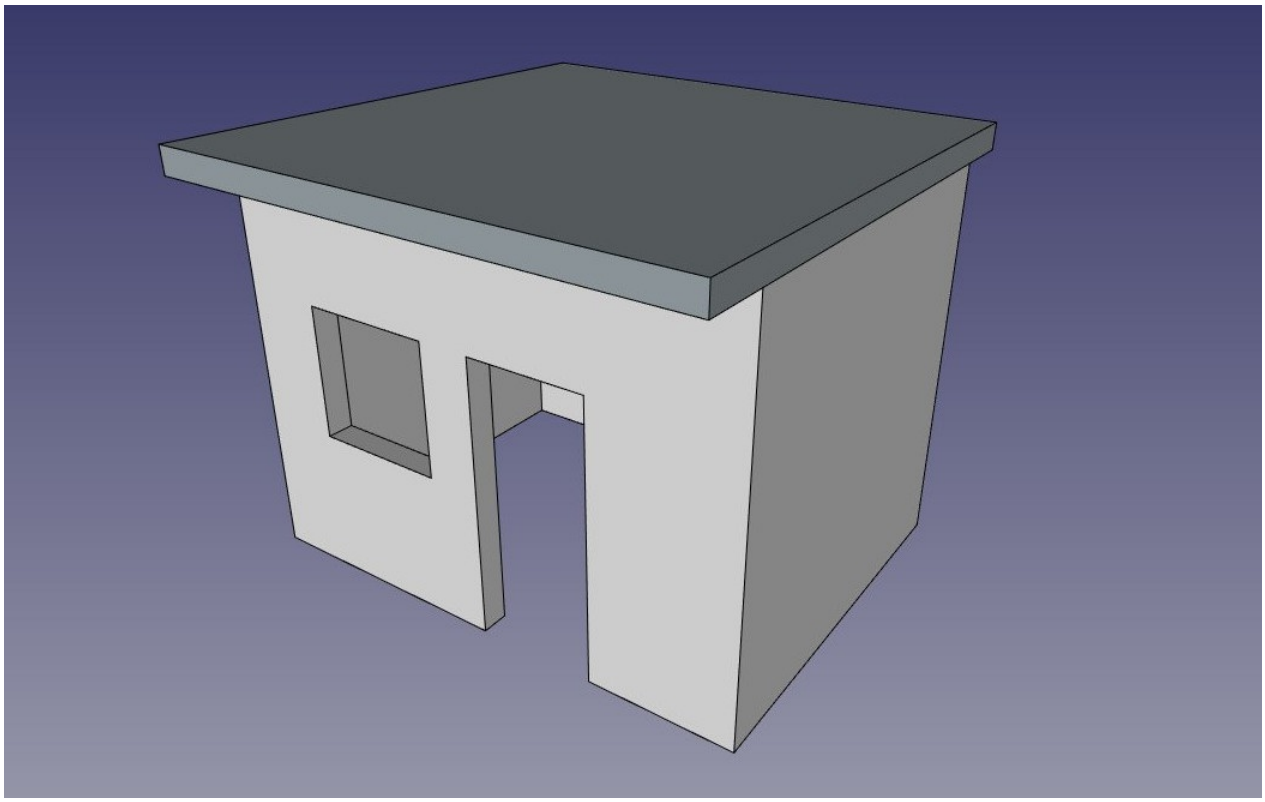
Preparing FreeCAD

The simulation itself is done by another piece of software, that is used by FreeCAD to obtain the results. As there are several interesting open-source FEM simulation applications available, the FEM Workbench has been made to be able to use more than one. However, currently only CalculiX is fully implemented. Another piece of software, called NetGen, which is responsible for generating the subdivision mesh, is also required. Detailed instructions to install these two components are provided in the FreeCAD documentation.

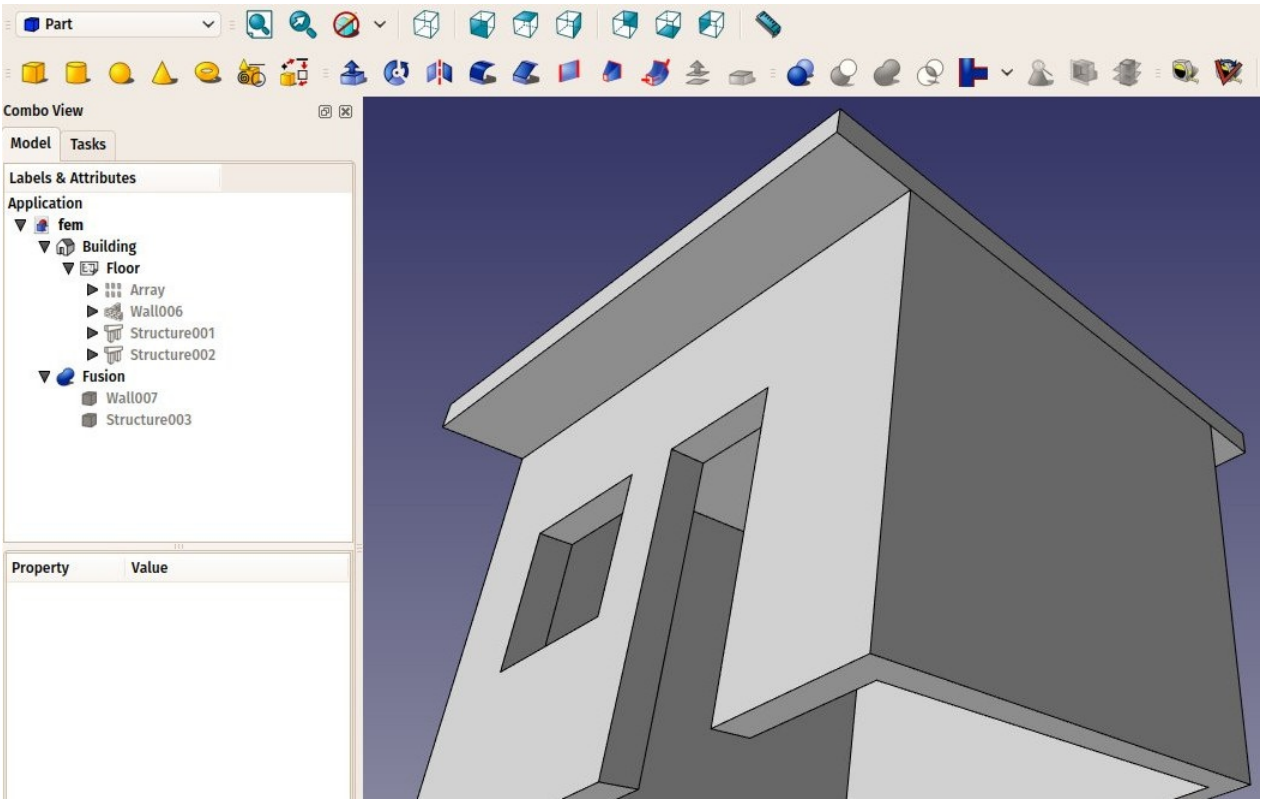
Preparing the geometry

We will start with the house we modelled in the BIM modeling chapter. However, some changes have to be made to make the model suitable for FEM calculations. This involves, basically, discarding the objects that we don't want to include in the calculaiton, such as the door and window, and joining all the remaining objects into one.

- Load the house model we modeled earlier
- Delete or hide the page object, the section planes and the dimensions, so we stay only with our model
- Hide the window, the door and the ground slab
- Also hide the metal beams from the roof. Since they are very different objects from the rest of the house, we will simplify our calculation by not including it. Instead, we will consider that the roof slab is directly placed on top of the wall.
- Now move the roof slab down so it rests on top of the wall: Edit the **Rectangle** object that we used as a base of the roof slab, and change it's **Placement->Position->X** value from 3.18m to 3.00m
- Our model is now clean:

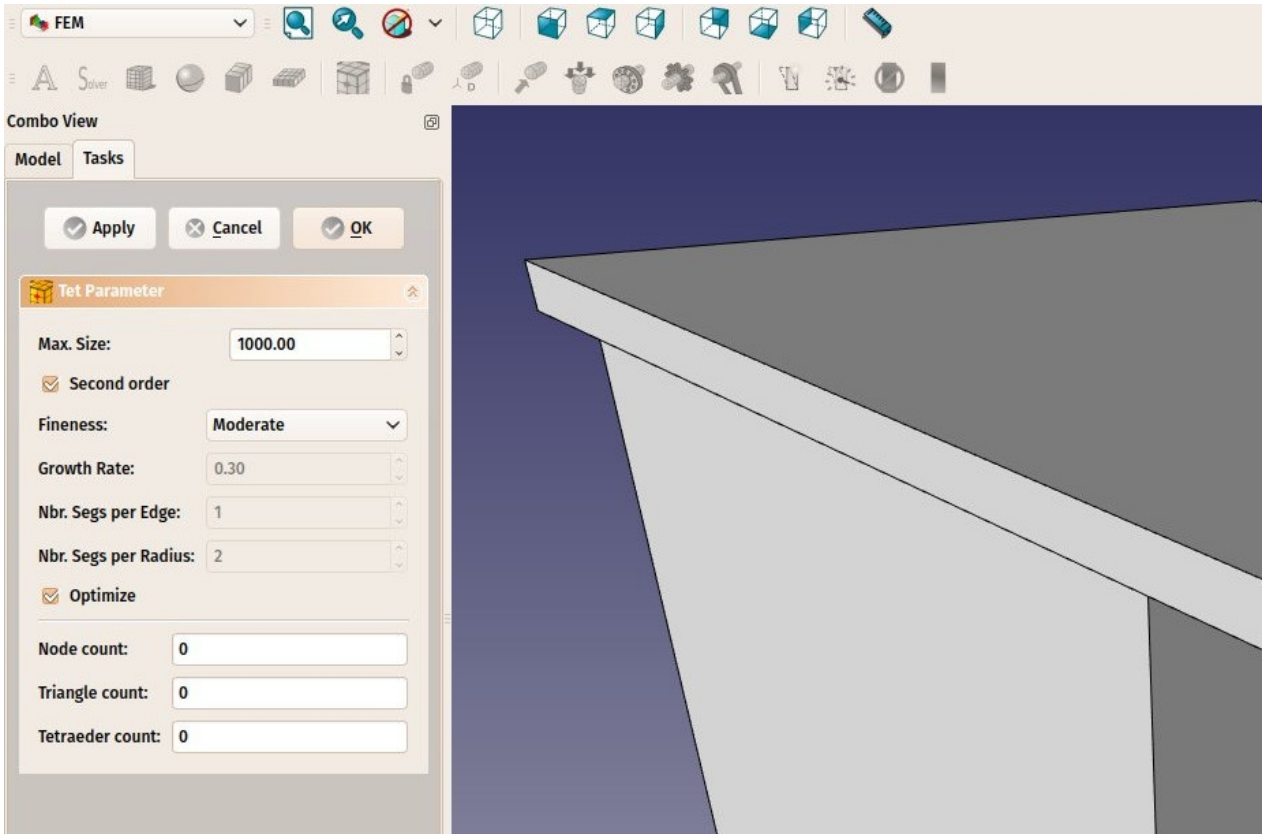


- The FEM Workbench can currently calculate deformations on one single object only. Therefore, we need to join our two objects (the wall and the slab). Switch to the Part Workbench, select the two objects, and press the Fuse. We now have obtained one fused object:

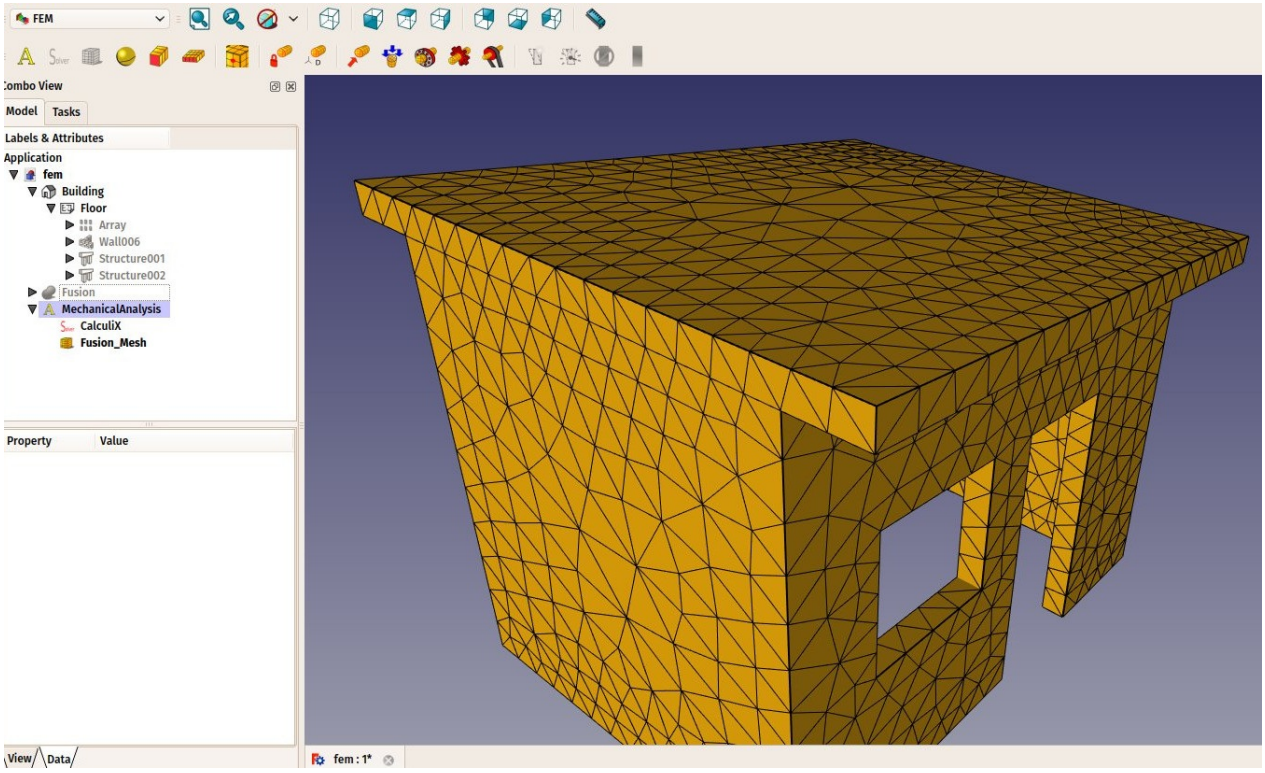


Creating the analysis

- We are now ready to start a FEM analysis. Let's switch to the FEM Workbench
- Select the fusion object
- Press the New Analysis button
- A new analysis will be created and a settings panels opened. Here you can define the meshing parameters to be used to produce the FEM mesh. The main setting to edit is the **Max Size** which defines the maximum size (in millimeters) of each piece of the mesh. For now, we can leave the default value of 1000:

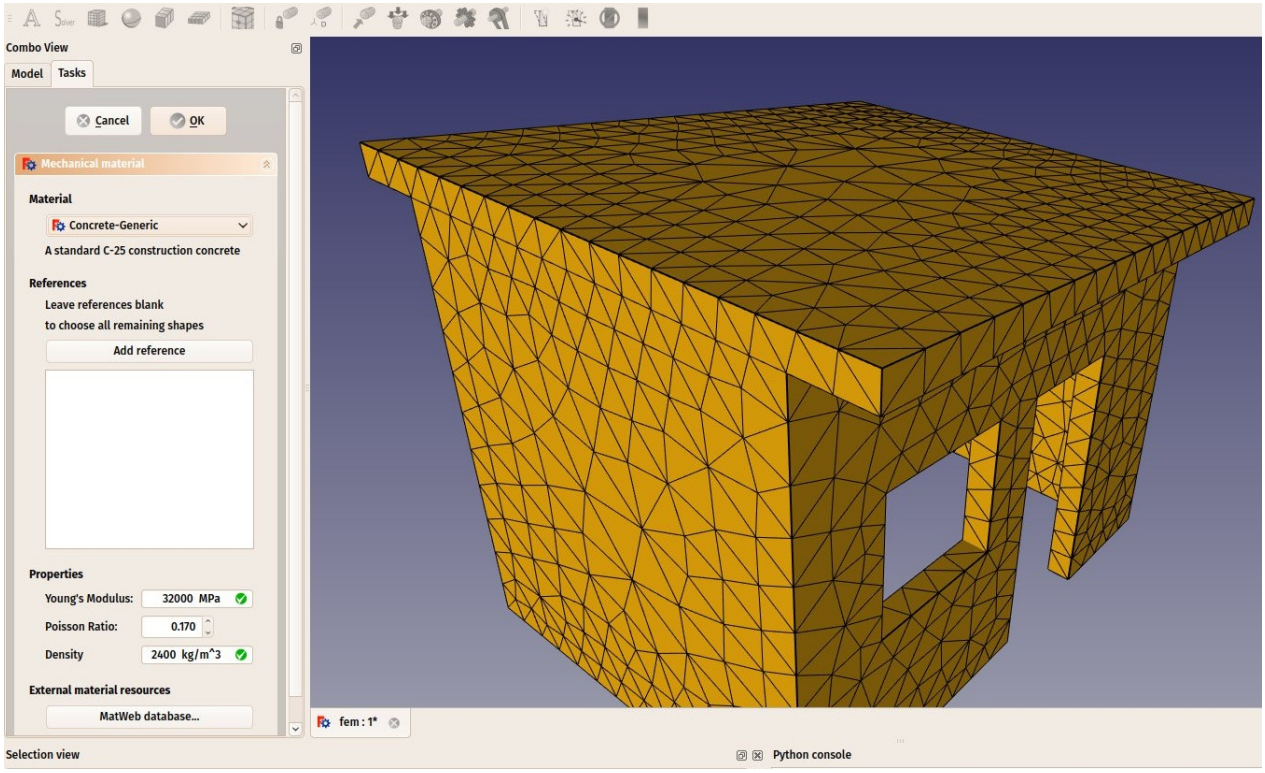


- After pressing OK and a few seconds of calculaiton, our FEM mesh is now ready:

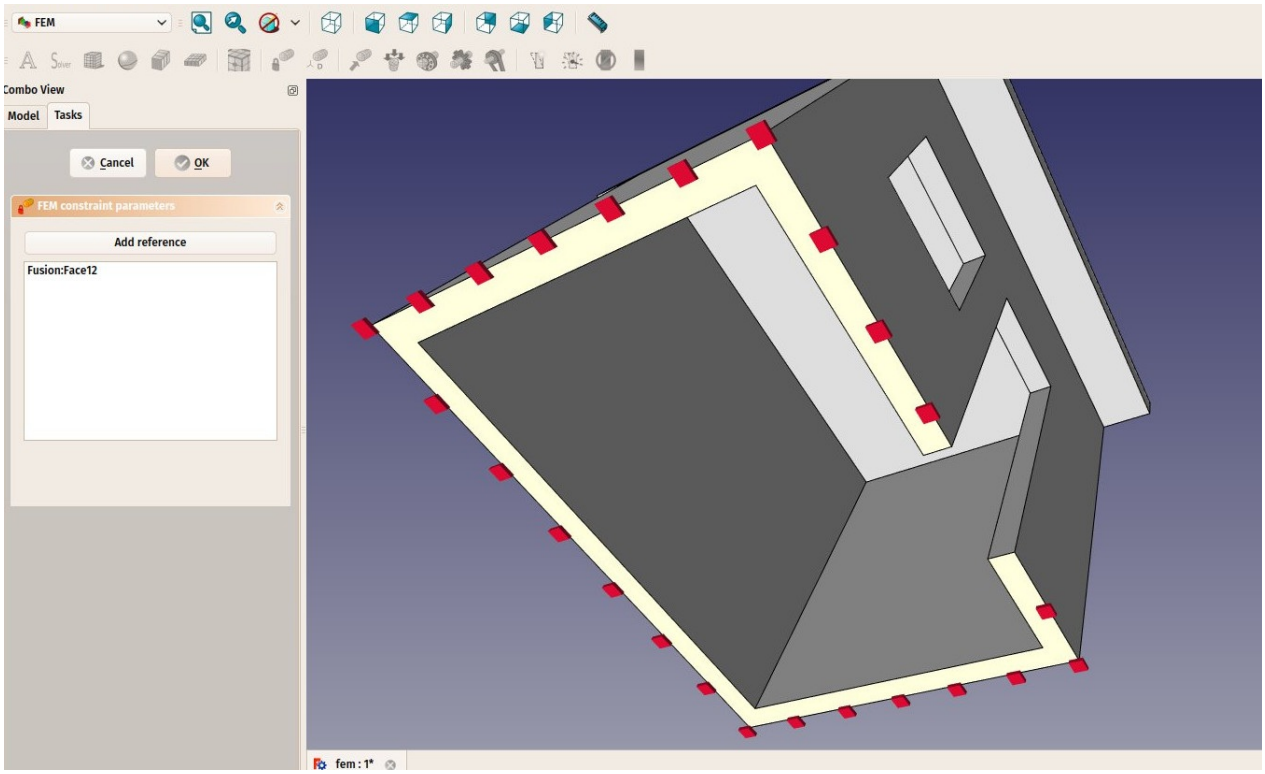


- We can now define the material to be applied to our mesh. This is important because depending on the material strength, our object will react differently to forces applied to it. Select the analysis object, and press the New Material button.
- A task panel will open to allow us to choose a material. In the Material drop-down list,

choose the **Concrete-generic** material, and press OK.



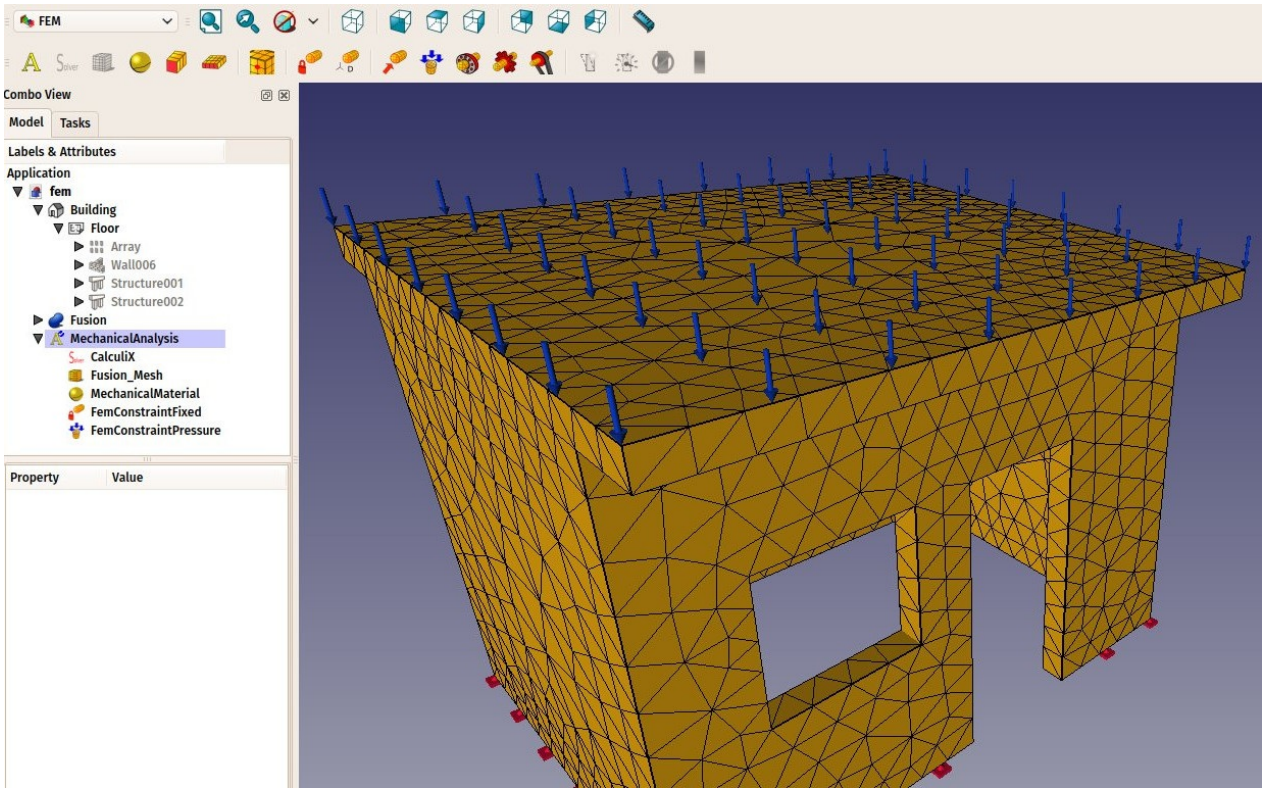
- We are now ready to apply forces. Let's start by specifying which faces are fixed into the ground and can therefore not move. Press the Fixed Constraint button.
- Click on the bottom face of our building and press OK. The bottom face is now indicated as unmovable:



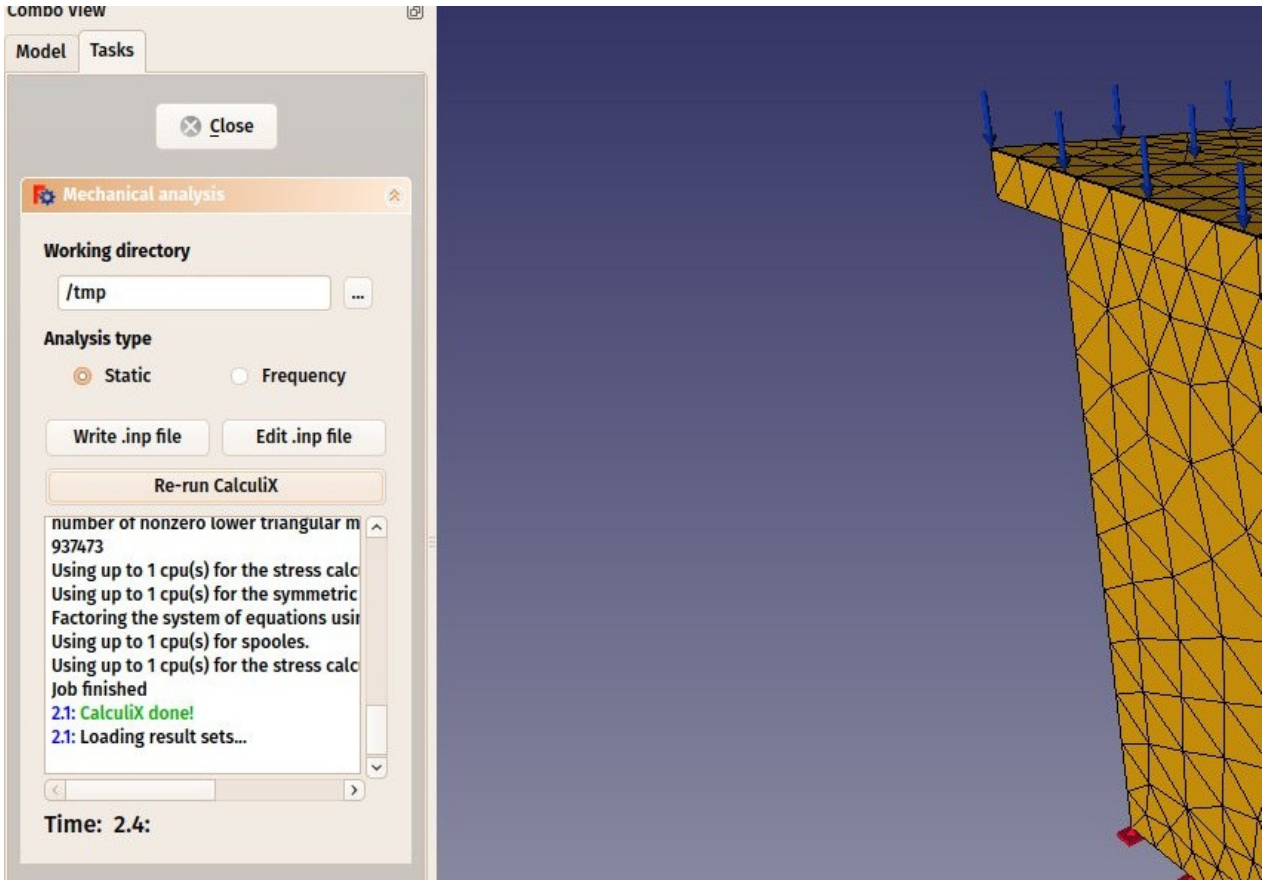
- We will now add a load on the top face, that could represente, for example, a massive

weight being spread on the roof. For this we will use a pressure constraint. Press the Pressure Constraint button.

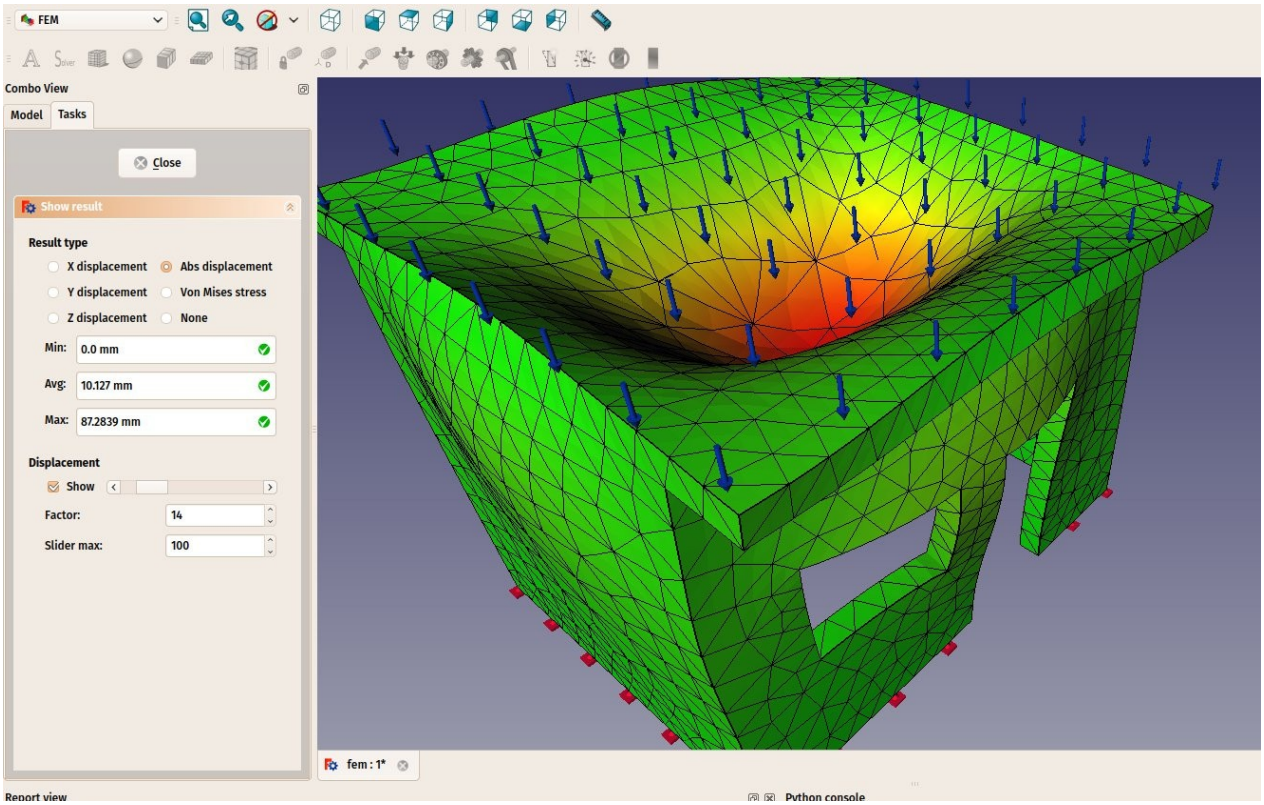
- Click the top face of the roof, set the pressure to **10MPa** (the pressure is applied by square millimeter) and click the OK button. Our force is now applied:



- We are now ready to start the calculation. Select the **CalculiX** object in the tree view, and press the Start Calculation button.
- In the task panel that will open, click first the **Write .inp file** button to create the input file for CalculiX, then the **Run CalculiX** button. A few moments later, the calculation will be done:



- We can now look at the results. Close the task panel, and see that a new **Results** object has been added to our analysis.
- Double-click the Results object
- Set the type of result that you want to see on the mesh, for example "absolute displacement", tick the **show** checkbox under **Displacement**, and move the slider next to it. You will be able to see the deformation growing as you apply more force:



The results displayed by the FEM workbench are of course currently not enough to perform real-life decisions about structures dimensionning and materials. However, they can already give precious information about how the forces flow through a structure, and which are the weak areas that will bear the more stress.

Downloads

- The file created during this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/fem.FCStd

Read more

- The FEM Workbench: http://www.freecadweb.org/wiki/index.php?title=Fem_Workbench
- Installing required FEM components: http://www.freecadweb.org/wiki/index.php?title=FEM_Install
- CalculiX: http://www.calculix.de/
- NetGen: https://sourceforge.net/projects/netgen-mesher/

Creating renderings

In computer talk, a rendering is a word used to describe a nice image produced from a 3D model. Of course, we could say that what se see in the FreeCAD 3D view is already nice. But anybody who saw a recent Hollywood movie knows that it is possible to produce images with a computer that are almost undistinguishable from a photograph.

Of course, producing such photo-realistic images requires a lot of work, and a 3D application that offers specific tools for that, such as precise controls for materials and lighting. FreeCAD being an application more geared towards technical modeling, it doesn't feature any advanced rendering tool.

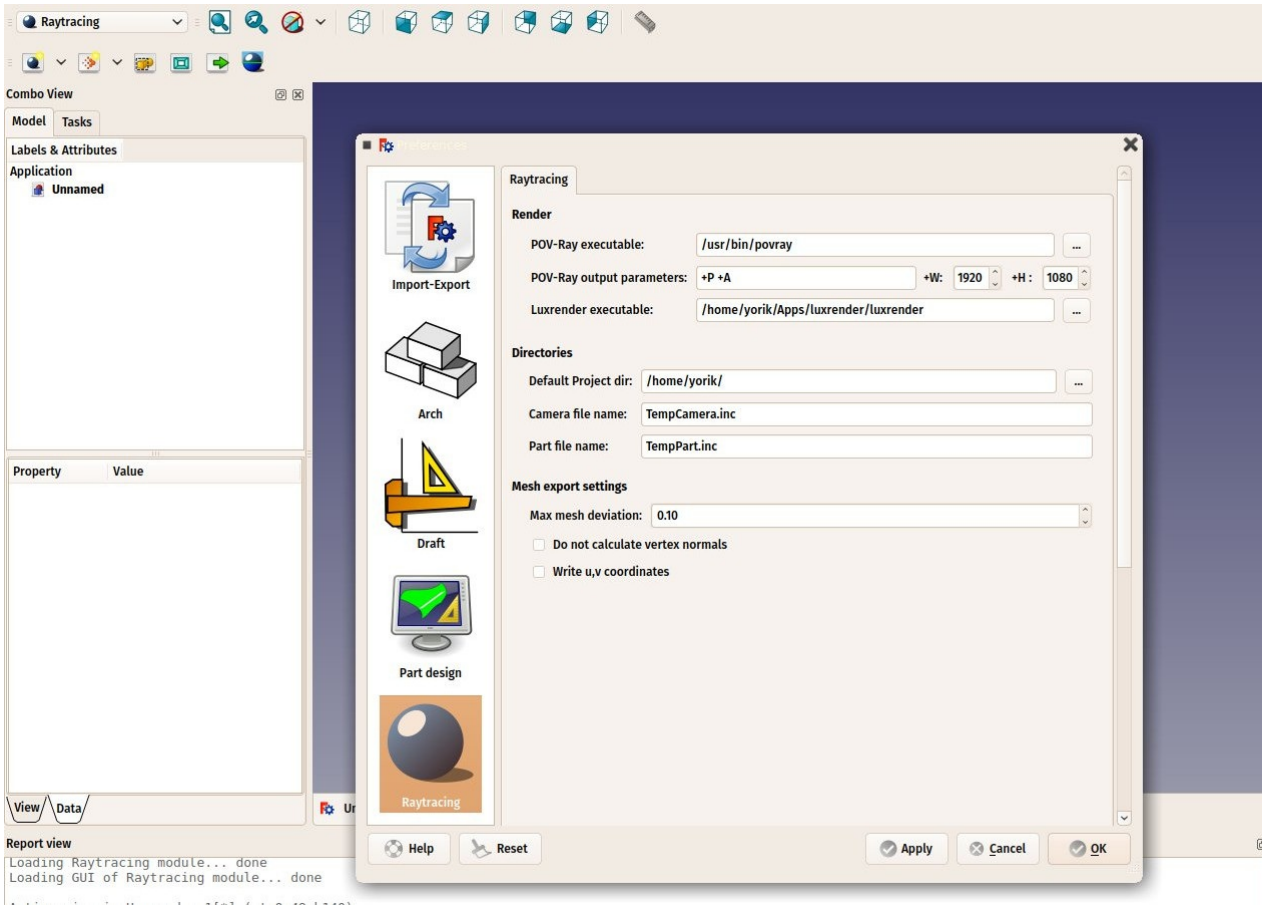
Fortunately, the opens-source world offers many applications to produce realistic images. Probably the most famous one is Blender, which is very popular and widely used in the movies and gaming industries. 3D models can very easily and faithfully be exported from FreeCAD and imported into Blender, where you can add realistic materials and illumination, and produce the final images or even animations.

Some other open-source rendering tools are made to be used inside another application, and will take care of doing the complex calculations to produce realistic images. Through its Raytracing Workbench, FreeCAD can use two of these rendering tools: POV-Ray and Luxrender. POV-Ray is a very old project, and is considered a classical raytracing engine, while Luxrender is much newer, and is categorized as an unbiased renderer. Both have their strengths and weaknesses, depending on the type of image one wants to render. The best way to know is to look at examples on both engines websites.

Installation

Before being able to use the Raytracing Workbench in FreeCAD, one of these two rendering applications needs to be installed on your system. This is usually very straightforward, both provide installers for many platforms or are usually included in the software repositories of most Linux distributions.

Once POV-Ray or Luxrender is installed, we need to set the path to their main executable in the FreeCAD preferences. This is usually only required on Windows and Mac. On Linux FreeCAD will pick it from the standard locations. The location of the povray or luxrender executables can be found by simply searching your system for files named povray (or povray.exe on Windows) and luxrender (or luxrender.exe on Windows).

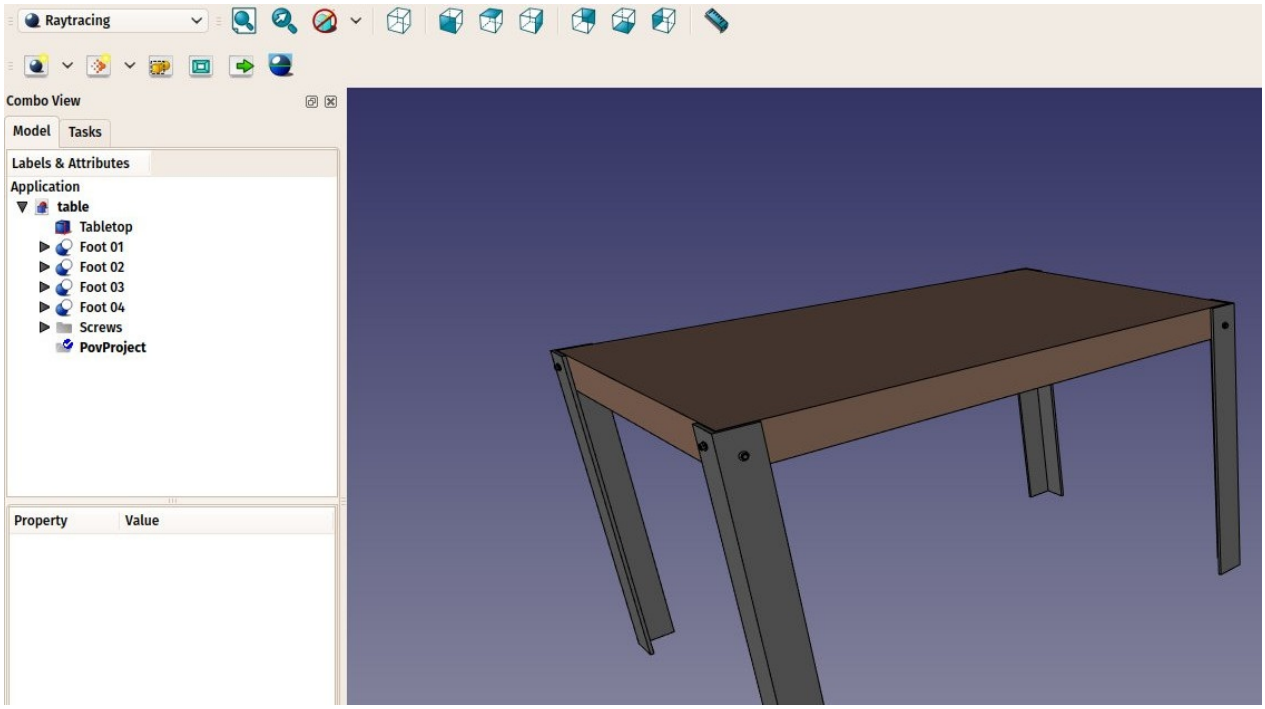


In this preferences screen we can also set the desired image size we want to produce.

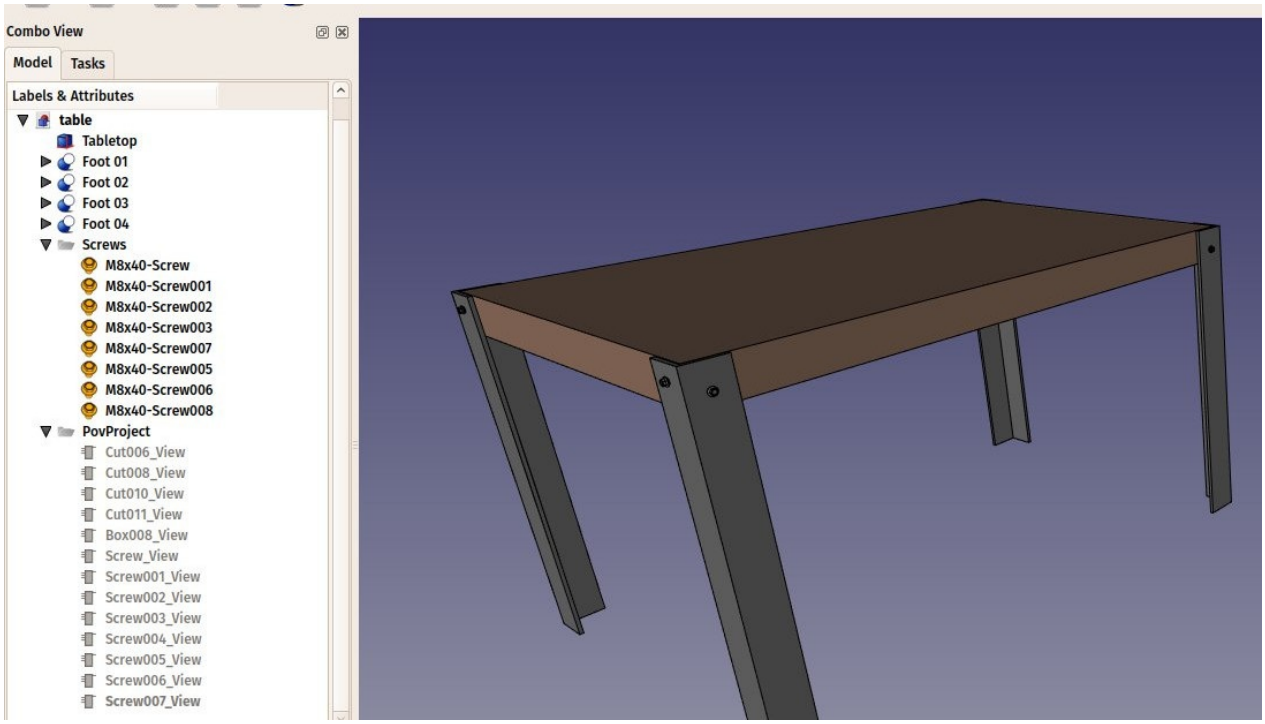
Rendering with PovRay

We will use the table we have been modeling in the tradtional modeling chapter to produce renderings with PovRay and Luxrender.

- Start by loading the table.FCStd file that we modeled earlier or from the link at the bottom of this chapter.
- Press the small down arrow next to the New Povray project button, and choose the **RadiosityNormal** template
- A warning message might appear telling you that the current 3D view is not in perspective mode and the rendering will therefore differ. Correct this by choosing **No**, choosing menu **View->Perspective view** and choosing the RadiosityNormal template again.
- You might also try other templates after you created a new project, simply by editing its **Template** property.
- A new project has now been created:

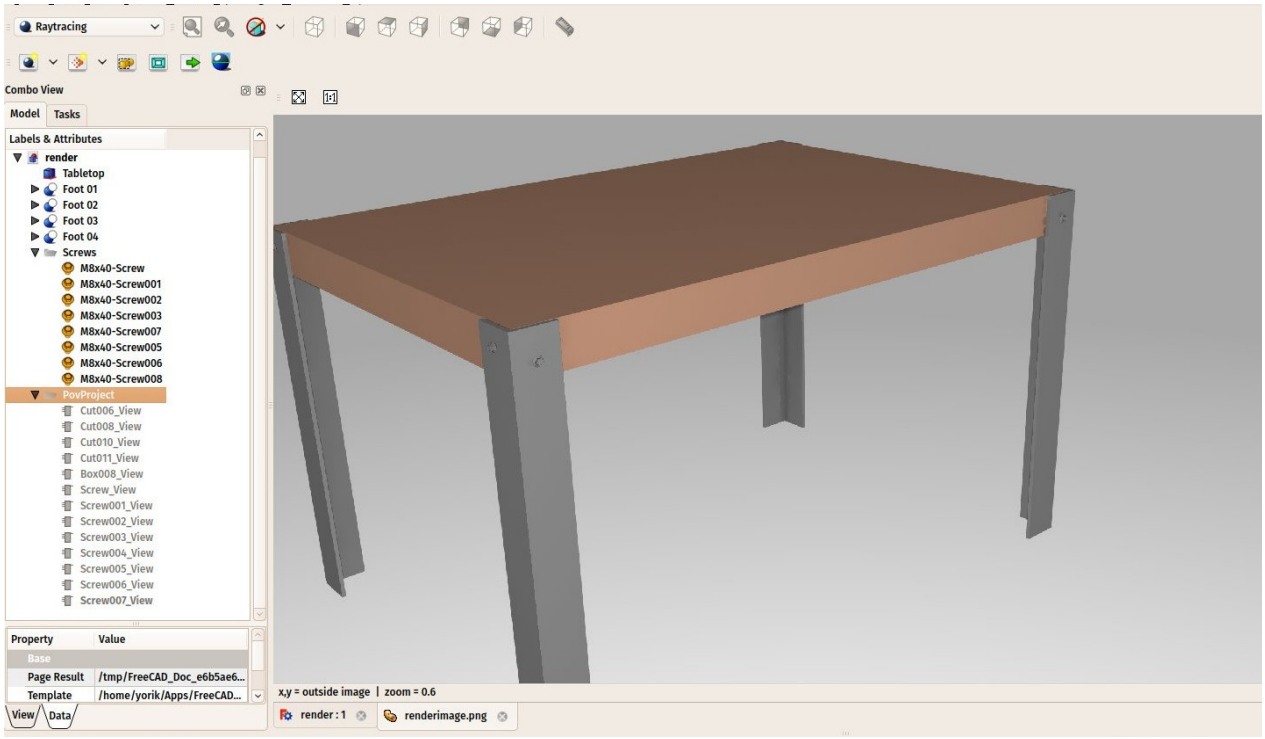


- The new project has adopted the point of view of the 3D view as it was at the moment we pressed the button. We can change the view, and update the view position stored in the povray project anytime, by pressing the Reset camera button.
- The Raytracing Workbench works the same way as the Drawing Workbench: Once a project folder is created, we must add **Views** of our objects to it. We can now do that by selecting all the objects that compose the table, and press the Insert part button:



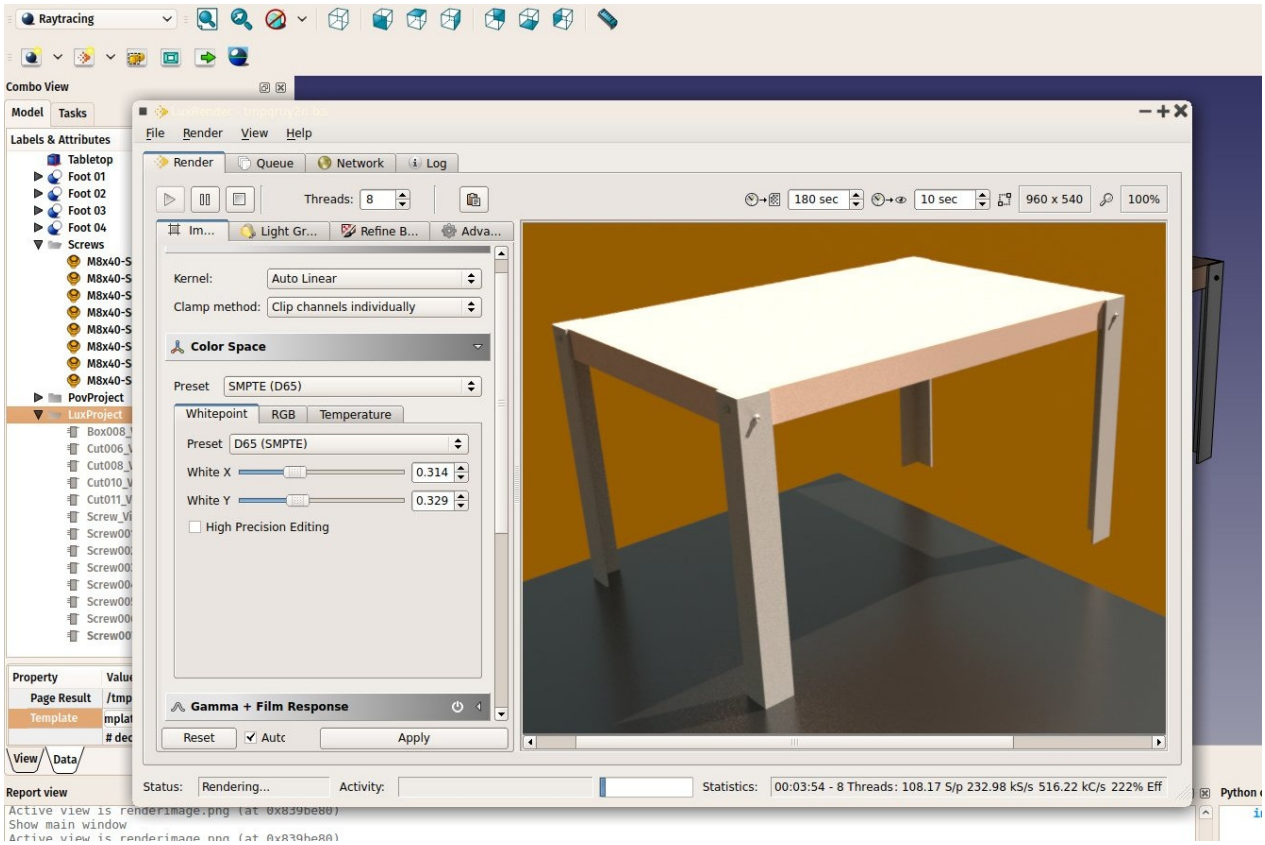
- The views have taken the color and transparency values from their original parts, but you can change that in the properties of each individual view if you wish.
- We are now ready to produce our first povray render. Press the Render button.

- You will be asked to give a file name and path for the .png image that will be saved by povray.
- The povray will then open and calculate the image.
- When this is done, simply click the image to close the povray window. The resulting image will be loaded in FreeCAD:



Rendering with LuxRender

- Rendering with Luxrender works almost the same way. We can leave our file open and create a new Luxrender project in the same file, or reload it to start from scratch.
- Press the little down arrow next to the New Luxrender project button and choose the **LuxOutdoor** template.
- Select all the components of the table. If you still have the povray project in your document, be sure to also select the lux project itself, so the views created in the next step won't go in the wrong project by mistake.
- Press the Insert part button.
- Select the luxrender project, and press the Render button.
- Luxrender works differently than povray. When you start the render, the luxrender application will open and immediately start rendering:



- If you leave that window open,Luxrender will continue calculating and rendering forever, progressively refining the image. It is up to you to decide when the image has reach a sufficient quality for your needs, and stop the render.
- There are also many controls to play with, on the left panel. All these controls will change the aspect of the image being rendered on the fly, without stopping the rendering.
- When you feel the quality is good enough, simply press **Render->stop**, and then **File->Export to image->Tonemapped low dynamic range** to save the rendered image to a png file.

You can extend greatly the render possibilities of FreeCAD by creating new templates for povray or luxrender. This is explained in the Raytracing Workbench documentation.

Downloads

- The table model: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/table.FCStd
- The file produced during this exercise: https://github.com/yorikvanhavre/FreeCAD-manual/blob/master/files/render.FCStd

Read more

- The Raytracing Workbench: http://www.freecadweb.org/wiki/index.php?title=Raytracing_Module
- Blender: http://www.blender.org

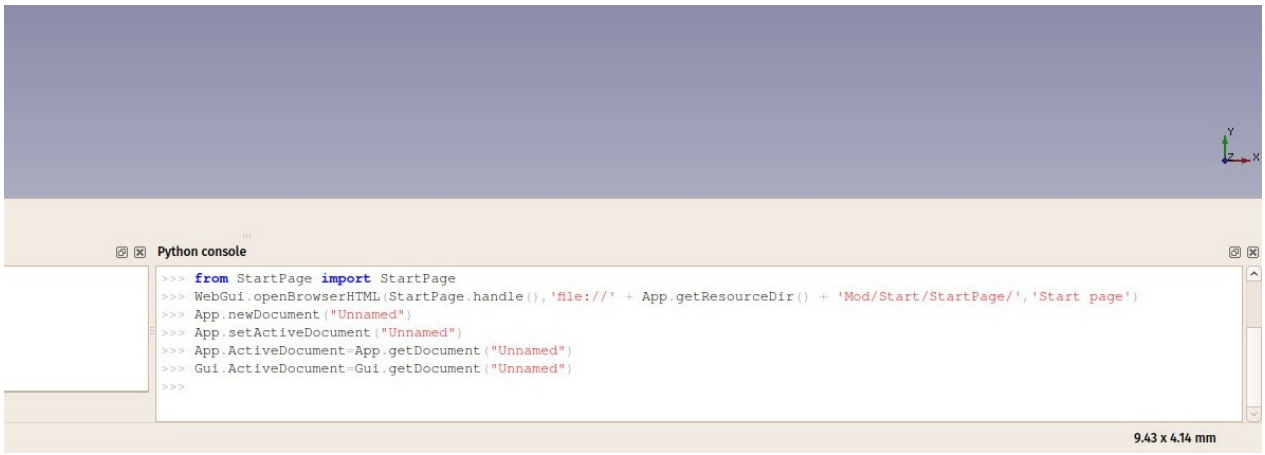
- POV-Ray: http://www.povray.org
- Luxrender: http://www.luxrender.net

Python scripting

A gentle introduction

Python is a widely popular, open-source programming language, very often used as a scripting language, embedded in applications, as this is the case with FreeCAD. It also has a series of features that makes it specially interesting for us FreeCAD users: It is very easy to learn, specially for people who had never programmed before, and it is embedded in many other applications, which makes it a very valuable tool to learn, as you will be able to use it in many other applications, such as Blender, Inkscape or GRASS.

FreeCAD makes an extensive use of Python. With it, you can access and control almost any feature of FreeCAD. You can for example create new objects, modify their geometry, analyze their contents, or even create new interface controls, tools and panels. Some workbenches of FreeCAD and most of the addon workbenches are fully programmed in python. FreeCAD has an advanced python console, available from menu **View->Panels->Python console**. It is often useful to perform operations for which there is no toolbar button yet, or to check shapes for problems, or to perform repetitive tasks:



But the python console also has another very important use: Everytime you press a toolbar button, or perform other operations in FreeCAD, some python code is printed in the console and executed. By leaving the Python console open, you can litterally see the python code unfold as you work, and in no time, almost without knowing it, you will be learning some python language.

FreeCAD also has a macros system, which allows you to record actions to be replayed later. This system also uses the Python console, by simply recording everything that is done in it.

In this chapter, we will discover very generally the Python language. If you are interested in learning more, the FreeCAD documentation wiki has an extensive section related to python programming.

Writing python code

There are two easy ways to write python code in FreeCAD: From the python console (menu **View -> Panels -> Python Console**), or from the Macro editor (menu **Tools -> Macros -> New**). In the console, you write python commands one by one, which are executed when you press return, while the macros can contain a more complex script made of several lines, which is executed only when the macro is launched from the same Macros window.

In this chapter, you will be able to use both methods, but it is highly recommended to use the Python Console, since it will immediately inform you of any error you could do while typing.

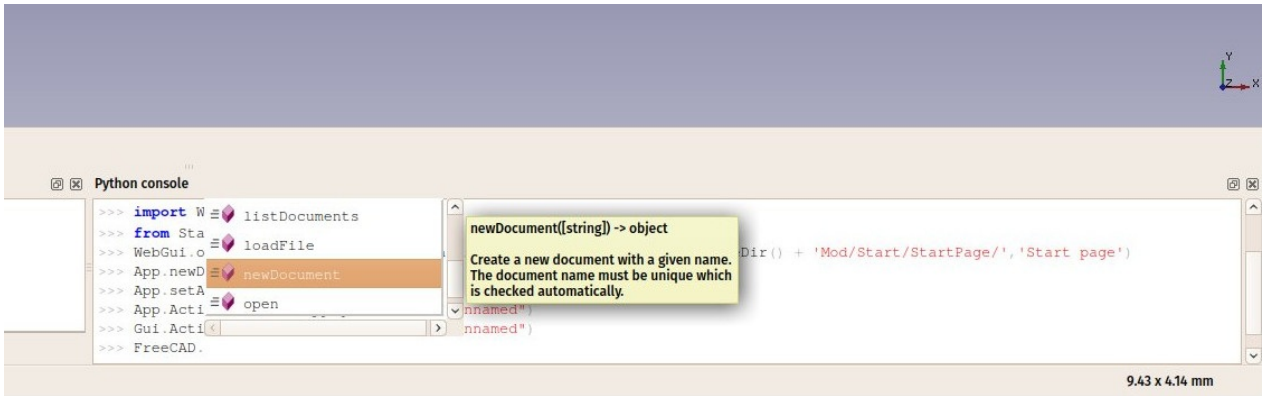
If this is the first time you are doing Python coding, consider reading this short introduction to Python programming before going further, it will make the basic concepts of Python clearer.

Manipulating FreeCAD objects

Let's start by creating a new empty document:

```
doc = FreeCAD.newDocument()
```

If you type this in the FreeCAD python console, you will notice that as soon as you type "FreeCAD." (the word FreeCAD followed by a dot), a windows pops up, allowing to quickly autocomplete the rest of your line. Even better, each entry in the autocomplete list has a tooltip explaining what it does. This makes it very easy to explore the functionality available. Before choosing "newDocument", have a look at the other options available.



As soon as you press **Enter** our new document will be created. This is similar to pressing the "new document" button on the toolbar. In Python, the dot is used to indicate something that is contained inside something else (newDocument is a function that is inside the FreeCAD module). The window that pops up therefore shows you everything that is contained inside "FreeCAD". If you would add a dot after newDocument, instead of the parentheses, it would show you everything that is contained inside the newDocument function. The parentheses are mandatory when you are calling a Python function, such as this one. We will illustrate that better below.

Now let's get back to our document. Let's see what we can do with it:

```
doc.
```

Explore the available options. Usually names that begin with a capital letter are attributes, they contain a value, while names that begin with small letter are functions (also called methods), they "do something". Names that begin with an underscore are usually there for the internal working of the module, and you shouldn't care about them. Let's use one of the methods to add a new object to our document:

```
box = doc.addObject("Part::Box","myBox")
```

Our box is added in the tree view, but nothing happens in the 3D view yet, because when working from Python, the document is never recomputed automatically. We must do that manually, whenever we need:

```
doc.recompute()
```

Now our box appeared in the 3D view. Many of the toolbar buttons that add objects in FreeCAD actually do two things: add the object, and recompute. If you turned on the "show script commands in python console" option above, try now adding a sphere with the appropriate button in the Part Workbench, and you will see the two lines of python code being executed one after the other.

You can get a list of all possible object types like Part::Box:

```
doc.supportedTypes()
```

Now let's explore the contents of our box:

```
box.
```

You'll immediately see a couple of very interesting things such as:

```
box.Height
```

This will print the current height of our box. Now let's try to change that:

```
box.Height = 5
```

If you select your box with the mouse, you will see that in the properties panel, under the **Data** tab, our **Height** property appears with the new value. All properties of a FreeCAD object that appear in the **Data** and **View** tabs are directly accessible by python too, by their names, like we did with the Height property. Data properties are accessed directly from the object itself, for example:

```
box.Length
```

View properties are stored inside a **ViewObject**. Each FreeCAd object possesses a ViewObject, which stores the vieual properties of the object. When running FreeCAD without its Graphical Interface (for example when launching it from a terminal with the -c command line option, or using it from another Python script), the ViewObject is not available, since there is no visual at all.

For example, to access the line color of our box:

```
box.ViewObject.LineColor
```

Vectors and Placements

Vectors are a very fundamental concept in any 3D application. It is a list of 3 numbers (x, y and z), describing a point or position in the 3D space. A lot of things can be done with vectors, such as additions, subtractions, projections and much more. In FreeCAD vectors work like this:

```
myvec = FreeCAD.Vector(2,0,0)
print(myvec)
prnimarkdownt(myvec.x)
print(myvec.y)
othervec = FreeCAD.Vector(0,3,0)
sumvec = myvec.add(othervec)
```

Another common feature of FreeCAD objects is their **Placement**. As we saw in earlier chapters, each object has a Placement property, which contains the position (Base) and orientation (Rotation) of the object. It is easy to manipulate from Python, for example to move our object:

```
print(box.Placement)
print(box.Placement.Base)
box.Placement.Base = sumvec
otherpla = FreeCAD.Placement()
otherpla.Base = FreeCAD.Vector(5,5,0)
box.Placement = otherpla
```

Read more

- Python: https://www.python.org/
- Working with Macros: http://www.freecadweb.org/wiki/index.php?title=Macros
- Introduction to Python scripting: http://www.freecadweb.org/wiki/index.php?title=Introduction_to_Python
- Using Python in FreeCAD: http://www.freecadweb.org/wiki/index.php?title=Python_scripting_tutorial

- The Python scripting wiki hub: http://www.freecadweb.org/wiki/index.php?title=Power_users_hub

Creating and manipulating geometry

In the previous chapters, we learned about the different workbenches of FreeCAD, and that each of them implements its own tools and geometry types. The same concepts applies when working from Python code.

We also saw that the big majority of the FreeCAD workbenches depend on a very fundamental one: the Part Workbench. In fact, may other workbenches, such as Draft or Arch, do exactly what we will do in this chapter: They use Python code to create and manipulate Part geometry.

So the first thing we need to do to work with Part geometry, is to do the Python equivalent to switching to the Part Workbench: import the Part module:

```
import Part
```

Take a minute to explore the contents of the Part module, by typing `Part.` and browsing through the different methods offered there. The Part module offers several convenience fuctions such as makeBox, makeCircle, etc... which will instantly build an object for you. Try this, for example:

```
Part.makeBox(3,5,7)
```

When you press Enter after typing the line above, nothing will appear in the 3D view, but something like this will be printed on the Python Console:

```
<Solid object at 0x5f43600>
```

This is where an important concept takes place. What we created here is a Part Shape. It is not a FreeCAD document object (yet). In FreeCAD, objects and their geometry are independent. Think of a FreeCAD document object as a container, that will host a shape. Parametric objects will also have properties such as Length and Width, and will **recalculate** their Shape on-the-fly, whenever one of the properties changes.What we did here is calculate a shape manually.

We can now easily create a "generic" document object in the current document (make sure you have at least one new document open), and give it a box shape like we just made:

```
boxShape = Part.makeBox(3,5,7)
myObj = FreeCAD.ActiveDocument.addObject("Part::Feature","MyNewBox")
myObj.Shape = boxShape
FreeCAD.ActiveDocument.recompute()
```

Note how we handled `myObj.Shape`, see that it is done exactly like we did in the previous chapter, when we changed other properties of an object, such as `box.Height = 5`. In fact, **Shape** is also a property, just like **Height**. Only it takes a Part Shape, not a number. In next chapter we will have a deeper look at how those parametric objects are constructed.

For now, let's explore our Part Shapes more in detail. At the end of the chapter about traditional modeling with the Part Workbench we showed a table that explains how Part Shapes are constructed, and their different components (Vertices, edges, faces, etc). The exact same components exist here and can be retrieved from Python. All Part Shape always have the following attributes:Vertexes, Edges, Wires, Faces, Shells and Solids. All of them are lists, that can contain any number of elements or be empty:

```
print(boxShape.Vertexes)
print(boxShape.Edges)
print(boxShape.Wires)
print(boxShape.Faces)
print(boxShape.Shells)
print(boxShape.Solids)
```

For example, let's find the area of each face of our box shape above:

```
for f in boxShape.Faces:
  print(f.Area)
```

Or, for each edge, its start point and end point:

```
for e in boxShape.Edges:
  print("New edge")
  print("Start point:")
  print(e.Vertexes[0].Point)
  print("End point:")
  print(e.Vertexes[1].Point)
```

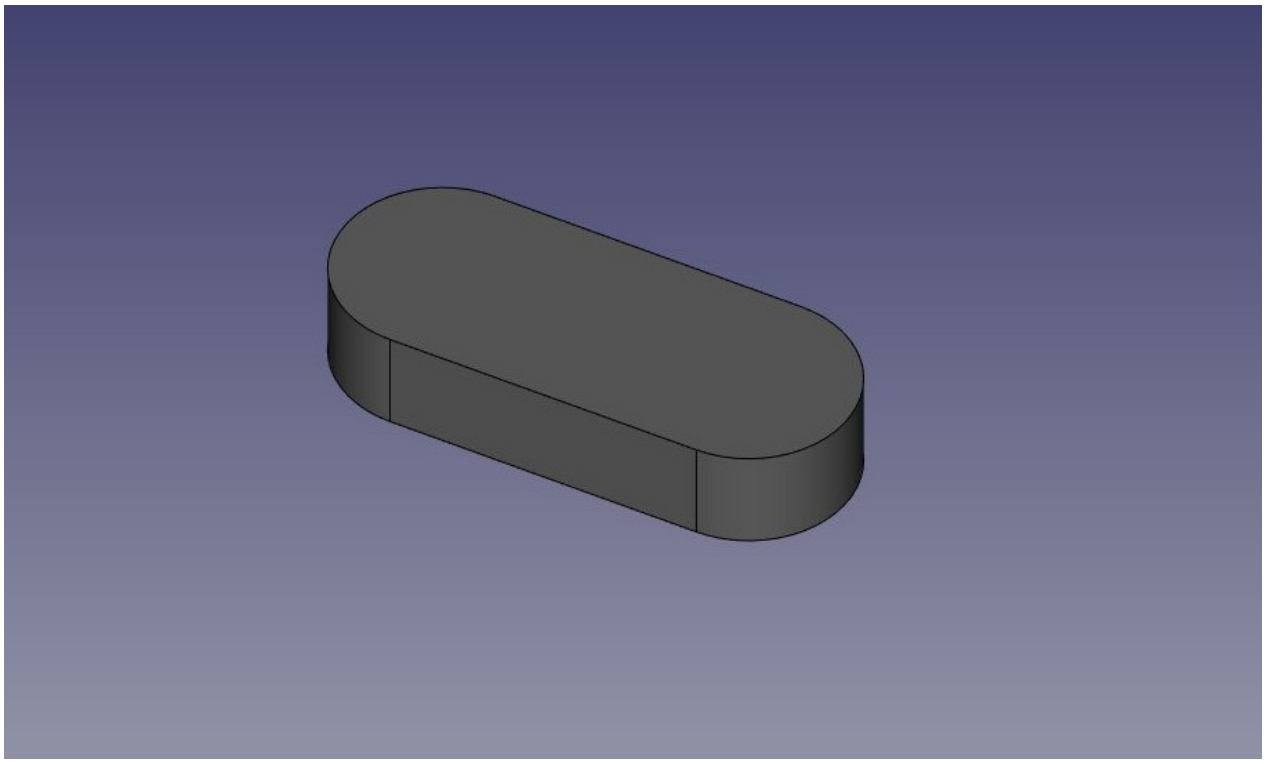
As you see, if our boxShape has a "Vertexes" attribute, each Edge of the boxShape also has a "Vertexes" attribute. As we can expect, the boxShape will have 8 vertices, while the edge will only have 2, which are both part of the list of 8.

We can always check what is the type of a shape:

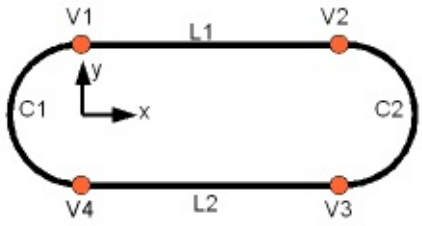
```
print(boxShape.ShapeType)
print(boxShape.Faces[0].ShapeType)
print (boxShape.Vertexes[2].ShapeType)
```

So to resume the whole diagram of Part Shapes: Everything starts with Vertices. With one or two vertices, you form an Edge (full circles have only one vertex). With one or more Edges, you form a Wire. With one or more closed Wires, you form a Face (the additional Wires become "holes" in the Face). With one or more Faces, you form a Shell. When a Shell is fully closed (watertight), you can form a Solid from it. And finally, you can join any number of Shapes of any types together, which is then called a Compound.

We can now try creating complex shapes from scratch, by constructing all their components one by one. For example, let's try to create a volume like this:



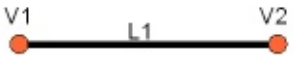
We will start by creating a planar shape like this:



First, let's create the four base points:

```
V1 = FreeCAD.Vector(0,10,0)
V2 = FreeCAD.Vector(30,10,0)
V3 = FreeCAD.Vector(30,-10,0)
V4 = FreeCAD.Vector(0,-10,0)
```

Then we can create the two linear segments:

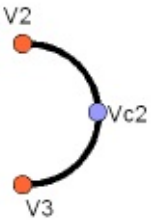



```
L1 = Part.Line(V1,V2)
L2 = Part.Line(V4,V3)
```

Note that we didn't need to create Vertices? We could immediately create Part.Lines from FreeCAD Vectors. This is because here we haven't created Edges yet. A Part.Line (as well as Part.Circle, Part.Arc, Part.Ellipse or PArt.BSpline) does not create an Edge, but rather a base geometry on which an Edge will be created. Edges are always made from such a base geometry, which is stored its Curve attribute. So if you have an Edge, doing:

`print(Edge.Curve)`

will show you what kind of Edge this is, that is, if it is based on a line, an arc, etc... But let's come back to our exercise, and build the arc segments. For this, we will need a third point, so we can use the convenient Part.Arc, which takes 3 points:




```
VC1 = FreeCAD.Vector(-10,0,0)
C1 = Part.Arc(V1,VC1,V4)
VC2 = FreeCAD.Vector(40,0,0)
C2 = Part.Arc(V2,VC2,V3)
```

Now we have 2 lines (L1 and L2) and 2 arcs (C1 and C2). We need to turn them into edges:

```
E1 = Part.Edge(L1)
E2 = Part.Edge(L2)
E3 = Part.Edge(C1)
E4 = Part.Edge(C2)
```

Alternatively, base geometries also have a toShape() function that do exactly the same thing:

```
E1 = L1.toShape()
E2 = L2.toShape()
...
```

Once we have a series of Edges, we can now form a Wire, by giving it a list of Edges. We don't need to take care of the order. OpenCasCade, the geometry "engine" of FreeCAD, is extraordinarily tolerant to unordered geometry. It will sort out what to do:

```
W = Part.Wire([E1,E2,E3,E4])
```

And we can check if our Wire was correctly understood, and that it is correclty closed:

```
print( W.isClosed() )
```

Which will print "True" or "False". In order to make a Face, we need closed Wires, so it is always a good idea to check that before creating the Face. Now we can create a Face, by giving it a single Wire (or a list of Wires if we had holes):

```
F = Part.Face(W)
```

Then we extrude it:

```
P = F.extrude(FreeCAD.Vector(0,0,10))
```

Note that P is already a Solid:

```
print(P.ShapeType)
```

Because when extruding a single Face, we always get a Solid. This wouldn't be the case, for example, if we had extruded the Wire instead:

```
S = W.extrude(FreeCAD.Vector(0,0,10))
print(s.ShapeType)
```

Which will of course give us a hollow shell, with the top and bottom faces missing.

Now that we have our final Shape, we are anxious to see it on screen! So let's create a generic object, and attribute it our new Solid:

```
myObj2 = FreeCAD.ActiveDocument.addObject("Part::Feature","My_Strange_Solid")
myObj2.Shape = P
FreeCAD.ActiveDocument.recompute()
```

Altenatively, the Part module also provides a shortcut that does the above operation quicker (but you cannot choose the name of the object):

```
Part.show(P)
```

All of the above, and much more, is explained in detail on the Part Scripting page, together with examples.

Read more:

- The Part Workbench: http://www.freecadweb.org/wiki/index.php?title=Part_Workbench
- Part scripting: http://www.freecadweb.org/wiki/index.php?title=Topological_data_scripting

Creating parametric objects

In the previous chapter, we saw how to create Part geometry, and how to display it on screen, by attaching it to a "dumb" (non-parametric) document object. This is tedious when we want to change the shape of that object. We would need to create a new shape, then attribute it again to our object.

However, we also saw in all the preceding chapters of this manual how parametric objects are powerful. We only need to change one property, and the shape is recalculated on-the-fly.

Internally, parametric objects don't do anything different than we just did: They recalculate the contents of their Shape property, over and over, each time another property has changed.

FreeCAD provides a very convenient system to build such parametric objects fully in Python. They consist of a simple Python class, which defines all the properties that the object needs, and what will happen when one of these properties changes. The structure of such parametric object is as simple as this:

```
class myParametricObject:

  def __init__(self,obj):
    obj.Proxy = self
    obj.addProperty("App::PropertyFloat","MyLength")
    ...

  def execute(self,obj):
    print ("Recalculating the shape...")
    print ("The value of MyLength is:")
    print (obj.MyLength)
    ...
```

All Python classes usually have an **init** method. What is inside that method is executed when that class is instantiated (which means, in programming slang, that a Python Object is created from that class. Think of a class as a "template" to create live copies of it). In our **init** function here, we do two important things: 1) store our class itself into the "Proxy" attribute of our FreeCAD documetn object, that is, the FreeCAD document object will carry this code, inside itself, and 2) create all the properties our object needs. There are many types of properties available, you can get the full list by typing this code:

```
FreeCAD.ActiveDocument.addObject("Part::FeaturePython","dummy").supportedProperties()
```

Then, the second important part is the execute method. Any code in this method will be executed when the object is marked to be recomputed, which will happen when a property has been changed. That is all there is to it. Inside execute, you need to do all that needs tobe done, that is, calculating a new shape, and attributing to the object itself with something like `obj.Shape = myNewShape`. That is why the execute method takes an "obj" argument, which will be the FreeCAD document object itself, so we can manipulate it inside our python code.

One last thing is important to remember: When you create such parametric objects in a FreeCAD document, when you save the file, the python code above is not stored inside the file. This is for security reasons, if a FreeCAD file contained code, it would be possible for someone to distribute FreeCAD files containing malicious code that could harm other people's computers. So, if you distribute a file that contains objects made with the above code, such code must also be present on the computer that will open the file. The easiest way to achieve that is usually to save the code above in a macro, and distribute the macro together with your FreeCAD file, or share your macro on the FreeCAD macros repository where anybody can download it.

Below, we will do a small exercise, building a parametric object that is a simple parametric rectangular face. More complex examples are available on the parametric object example and in the FreeCAD source code itself.

We will give our object two properties: Length and Width, which we will use to construct a rectangle. Then, since our object will already have a pre-built Placement property (all geometric object have one by default, no need to add it ourselves), we will displace our rectangle to the location/rotation set in the Placement, so the user will be able to move the rectangle anywhere by editing the Placement property.

```
class ParametricRectangle:

  def __init__(self,obj):
    obj.Proxy = self
    obj.addProperty("App::PropertyFloat","Length")
    obj.addProperty("App::PropertyFloat","Width")

  def execute(self,obj):
    # we need to import the FreeCAD module here too, because we might be running out o
f the Console
    # (in a macro, for example) where the FreeCAD module has not been imported automat
ically
    import Part,FreeCAD

    # first we need to make sure the values of Length and Width are not 0
    # otherwise the Part.Line will complain that both points are equal
    if (obj.Length == 0) or (obj.Width == 0):
      # if yes, exit this method without doing anything
      return

    # we create 4 points for the 4 corners
    v1 = FreeCAD.Vector(0,0,0)
    v2 = FreeCAD.Vector(obj.Length,0,0)
    v3 = FreeCAD.Vector(obj.Length,obj.Width,0)
    v4 = FreeCAD.Vector(0,obj.Width,0)

    # we create 4 edges
    e1 = Part.Line(v1,v2).toShape()
    e2 = Part.Line(v2,v3).toShape()
    e3 = Part.Line(v3,v4).toShape()
    e4 = Part.Line(v4,v1).toShape()

    # we create a wire
    w = Part.Wire([e1,e2,e3,e4])

    # we create a face
    f = Part.Face(w)

    # All shapes have a Placement too. We give our shape the value of the placement
    # set by the user. This will move/rotate the face automatically.
    f.Placement = obj.Placement

    # all done, we can attribute our shape to the object!
    obj.Shape = f
```

Instead of pasting the above code in the Python console, we'd better save it somewhere, so we can reuse and modify it later. For example in a new macro (menu Tools -> Macros -> Create). Name it, for example, "ParamRectangle". However, FreeCAD macros are saved with a .FCMacro extension, which Python doesn't recognize when using `import`. So, before

using the above code, we will need to rename the ParamRectangle.FCMacro file to ParamRectangle.py. This can be done simply from your file explorer, by navigating to the Macros folder indicated in menu Tools -> Macros.

Once that is done, we can now do this in the Python Console:

```
import ParamRectangle
```

By exploring the contents of ParamRectangle, we can verify that it contains our ParametricRectangle class.

To create a new parametric object using our ParametricRectangle class, we will use the following code. Observe that we use Part::FeaturePython instead of Part::Feature that we have been using in the previous chapters (The Python version allows to define our own parametric behaviour):

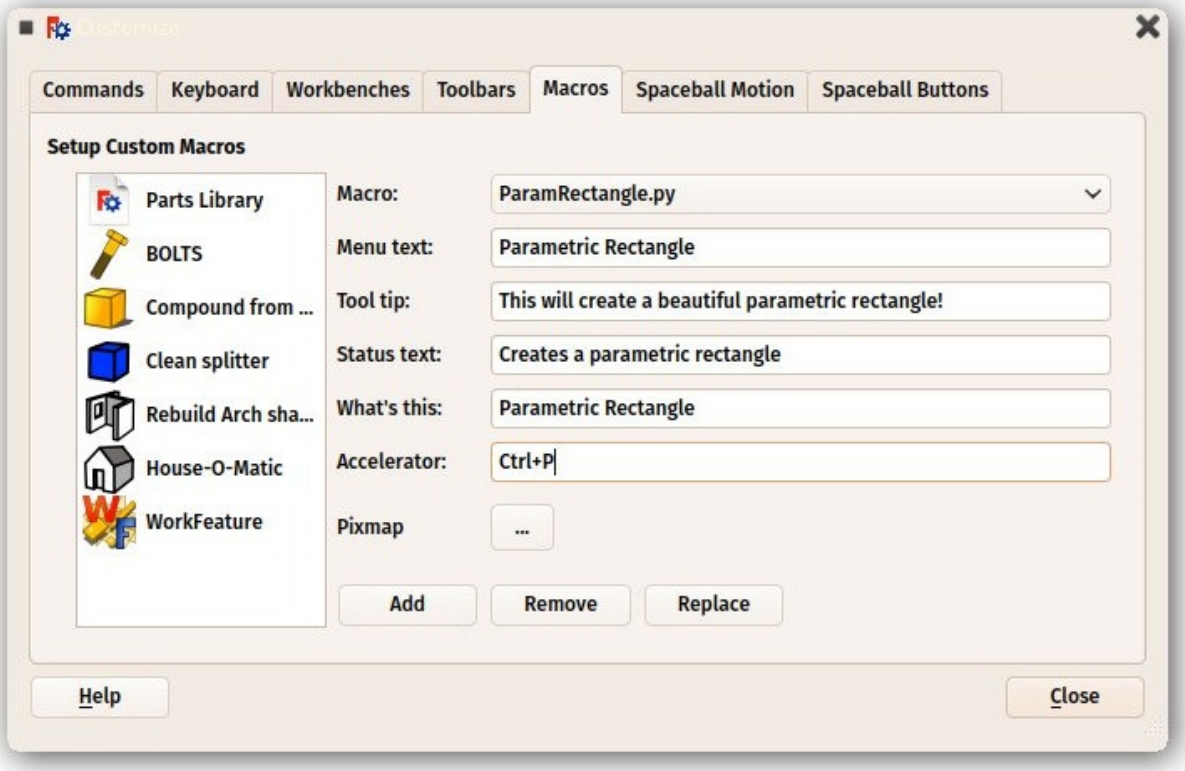
```
myObj = FreeCAD.ActiveDocument.addObject("Part::FeaturePython","Rectangle")
ParamRectangle.ParametricRectangle(myObj)
myObj.ViewObject.Proxy = 0 # this is mandatory unless we code the ViewProvider too
FreeCAD.ActiveDocument.recompute()
```

Nothing will appear on screen just yet, because the Length and Width properties are 0, which will trigger our "do-nothing" condition inside execute. We just need to change the values of Length and Width, and our object will magically appear and be recalculated on-the-fly.

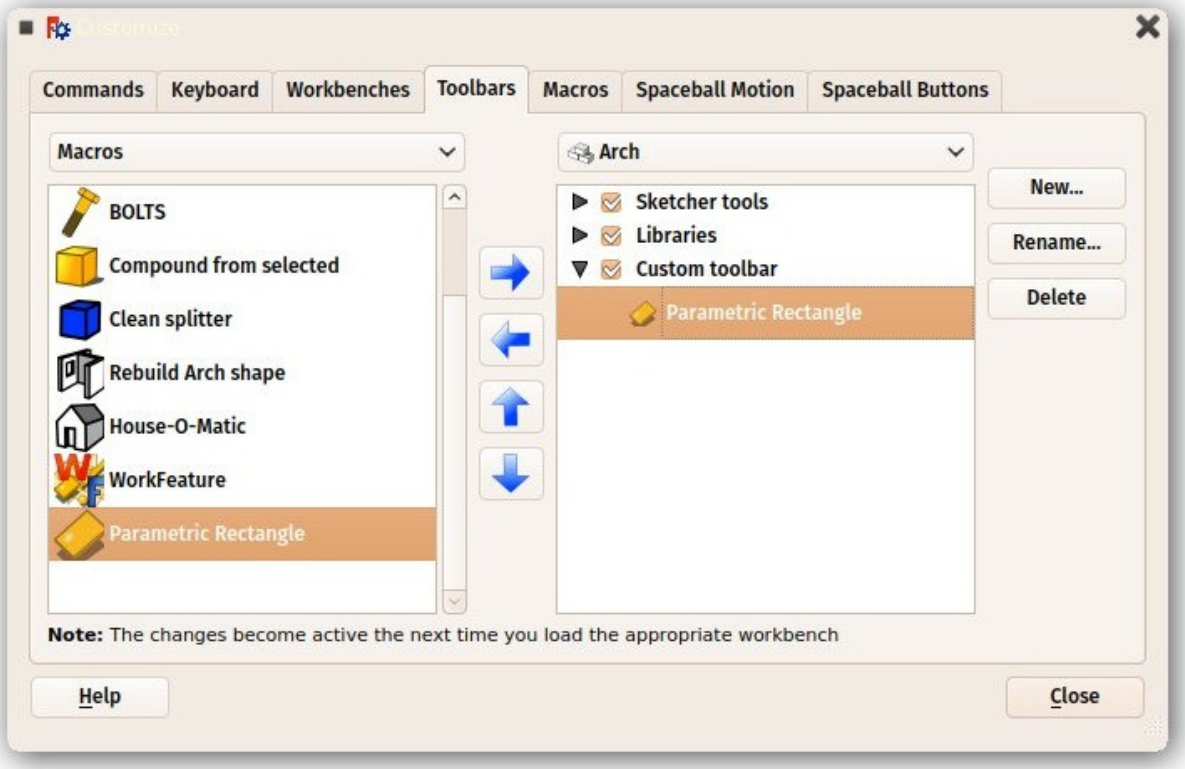
Of course it would be tedious to have to type these 4 lines of Python code each time we want to create a new parametric rectangle. A very simple way to solve this is placing the 4 lines above inside our ParamRectangle.py file, at the end, after the end of the ParametricRectange class (We can do this from the Macro editor).

Now, when we type `import ParamRectangle`, a new parametric rectangle will automatically be created. Even better, we can add a toolbar button that will do just that:

- Open menu **Tools -> Customize**
- Under the "Macros" tab, select our ParamRectangle.py macro, fill in the details as you wish, and press "Add":



- Under the Toolbars tab, create a new custom toolbar in the workbench of your choice (or globally), select your macro and add it to the toolbar:



- That's it, we now have a new toolbar button which, when clicked, will create a parametric rectangle.

Remeber, if you want to distribute files created with this new tool to other people, they must have the ParamRectangle.py macro installed on their computer too.

Read more

- The FreeCAD macros repository: http://www.freecadweb.org/wiki/index.php?title=Macros_recipes
- Parametric object example: http://www.freecadweb.org/wiki/index.php?title=Scripted_objects
- More examples in the FreeCAD code: https://github.com/FreeCAD/FreeCAD/blob/master/src/Mod/TemplatePyMod/FeaturePython.py

Creating interface tools

In the last two chapters, we saw how to create Part geometry and create parametric objects. One last piece is missing to gain full control over FreeCAD: Create tools that will interact with the user.

In many situations, it is not very user-friendly to construct an object with zero-values, like we did with the rectangle in the previous chapter, and then ask the user to fill in the Height and Width values in the Properties panel. This works for a very small number of objects, but will become very tedious if you have a lot of rectangles to make. A better way would be to be able to already give the Height and Width when creating the rectangle.

Python offers a basic tool to have the user enter text on screen:

```
text = raw_input("Height of the rectangle?")
print("The entered height is ",text)
```

However, this requires a running Python console, and when running our code from a macro, we are not always sure that the Python console will be turned on on the user's machine.

The Graphical User Interface, or GUI, that is, all the part of FreeCAD that is displayed on your screen (the menu, toolbars, 3D view, etc), is all there for that purpose: interact with the user. FreeCAD's interface is built with Qt), a very common open-source GUI toolkit that offers a big range of tools such as dialog boxes, buttons, labels, text input boxes or pull-down menus (all these are generically called "widgets").

The Qt tools are very easy to use from Python, thanks to a Python module called Pyside (there are several other Python modules to communicate with Qt from Python too). This module allows you to create and interact with widgets, read what the user did with them (what was filled in text boxes) or do things when, for example, a button was pressed.

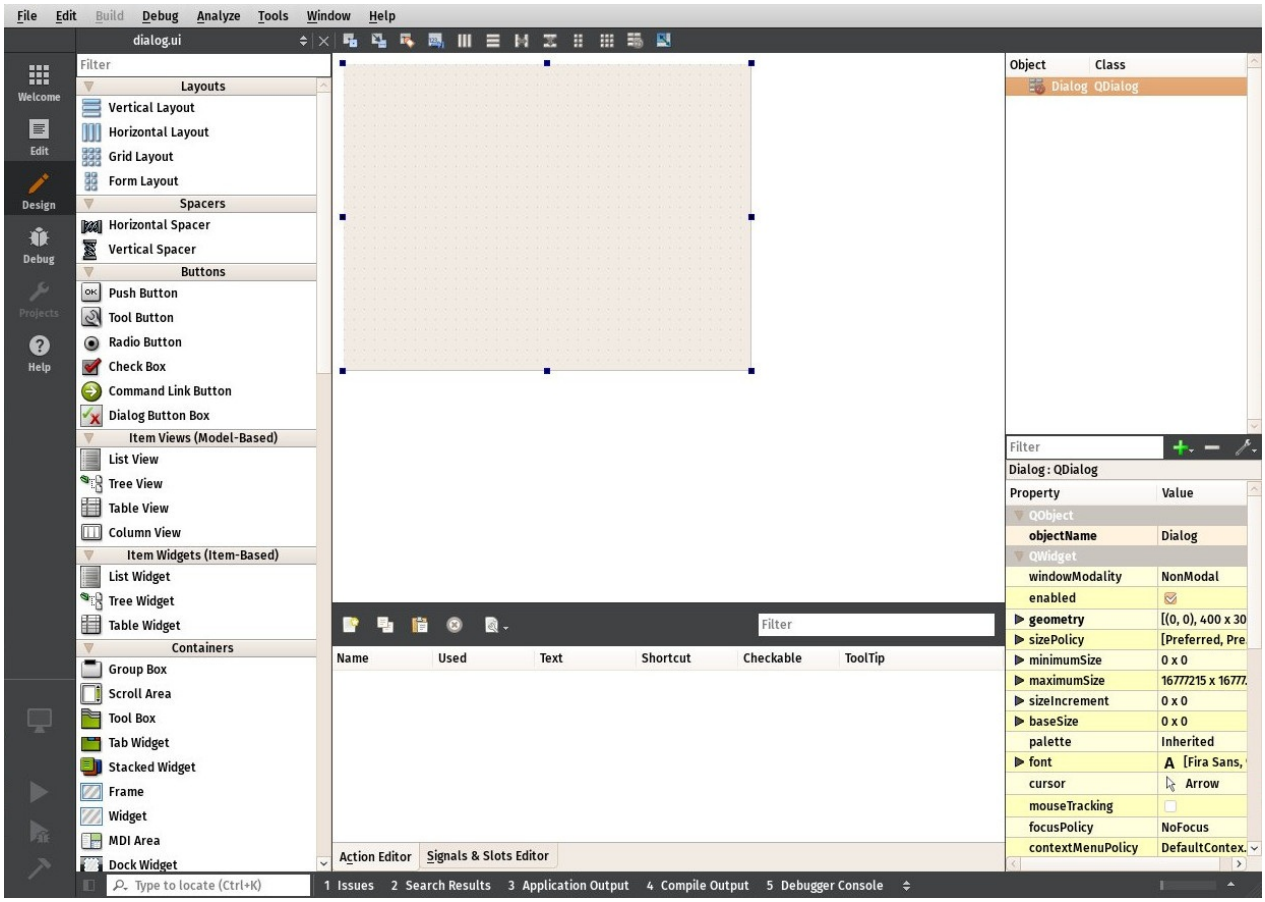
Qt also provides another interesting tool called Qt Designer, which is today embedded inside a bigger application called Qt Creator. It allows to design dialog boxes and interface panels graphically, instead of having to code them manually. In this chapter, we will use Qt Creator to desig a panel widget that we will use in the **Task** panel of FreeCAD. So you will need to download and install Qt Creator from its official page if you are on Windows or Mac (on Linux it will usually be available from your software manager application).

In the following exercise, we will first create a panel with Qt Creator that asks for length, width and height values, then we will create a Python class around it, that will read the values entered by the user from the panel, and create a box with the given dimensions. This

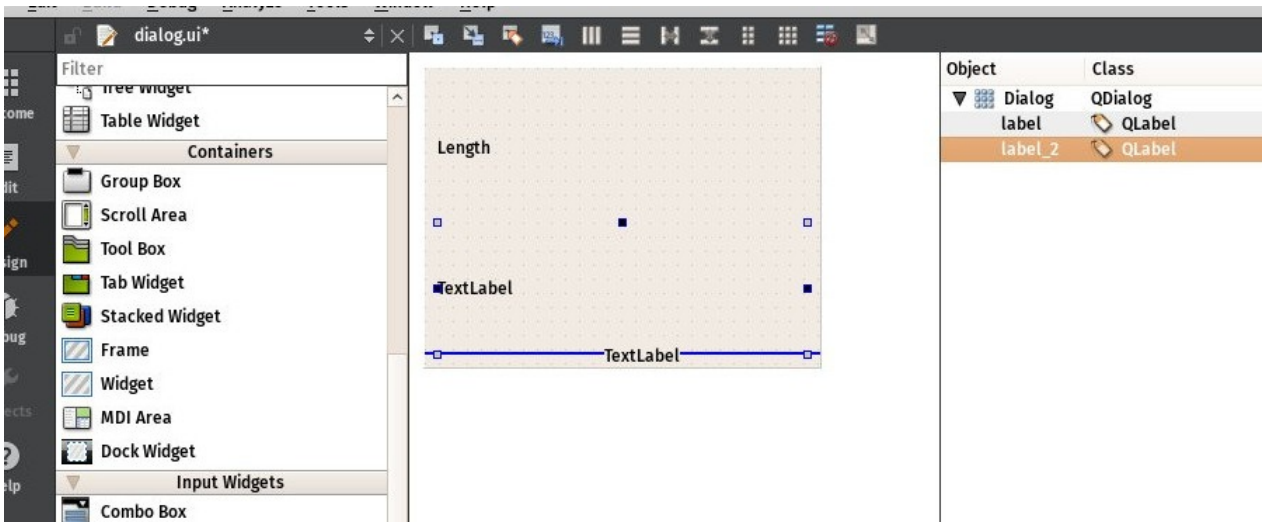
Python class will then be used by FreeCAD to display and control the task panel:



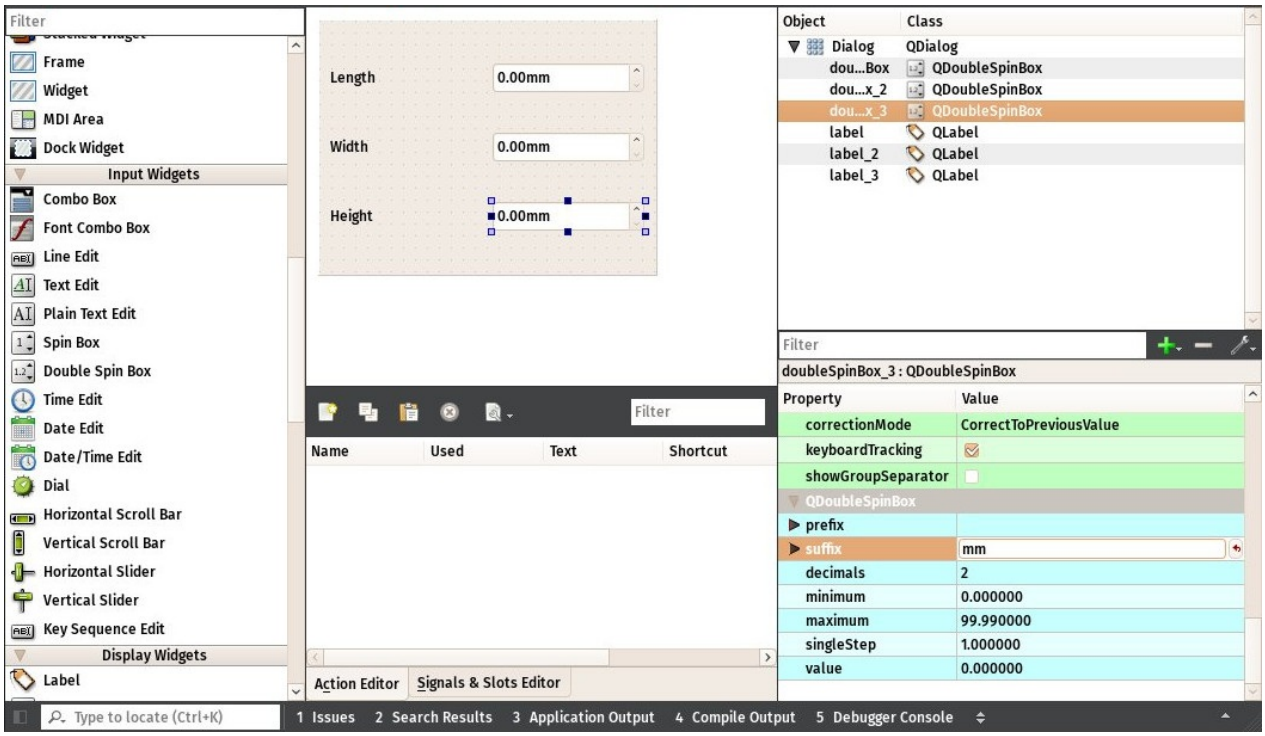
Let's start by creating the widget. Start Qt Creator, then menu **File -> New File or Project -> Files and Classes -> Qt -> Qt Designer Form -> Dialog without buttons**. Click **Next**, give it a filename to save, click **Next**, leave all project fields to their default value (""), and **Create**. FreeCAD's Task system will automatically add OK/Cancel buttons, that's why we chose here a dialog without buttons.



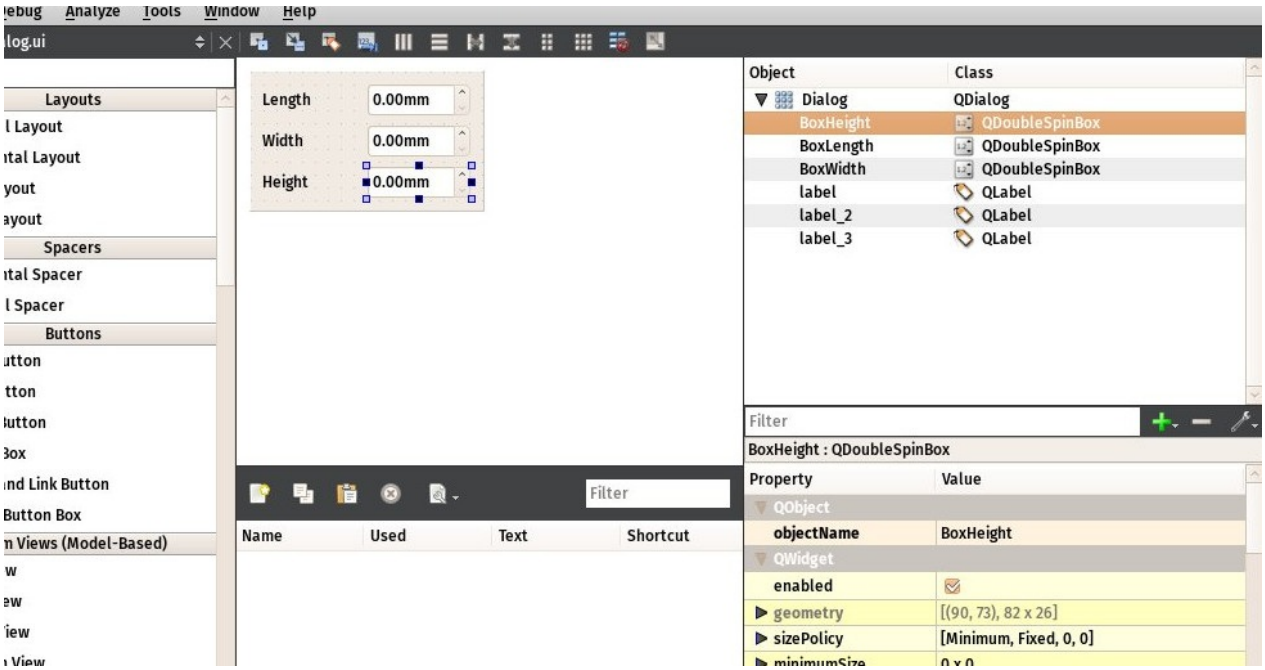
- Find the **Label** in the list in the left panel, and drag it onto the canvas of our widget. Double-click the recent placed Label, and change its text to **Length**.
- Right-click the widget canvas, and choose **Lay out-> Lay out in a Grid**. This will turn our widget into a grid with currently only one cell, occupied by ourfirst label. We can now add the next items at the left, right, top or bottom of our first label, and the grid wil lexpand automatically.
- Add two more labels below the first one, and change their text to Width and Height:



- Now place 3 **Double Spin Box** widgets next to our Length, Width and Height labels. For each of them, in the lower left panel, which shows all the available settings for the selected widget, locate **Suffix** and set their suffix to **mm**. FreeCAD has a more advanced widget, that can handle different units, but that is not available in Qt Creator by default (but can be compiled), so for now we will use a standard Double Spin Box, and we add the "mm" suffix to make sure the user knows in which units they work:



- Now our widget is done, we just need to make sure of one last thing. Since FreeCAD will need to access that widget and read the Length, Width and Height values, we need to give proper names to those widgets, so we can easily retrive them from within FreeCAD. Click each of the Double Spin Boxes, and in the upper right window, double-click their Object Name, and change them to something easy to remember, for example: BoxLength, BoxWidth and BoxHeight:



- Save the file, you can now close Qt Creator, the rest will be done in Python.
- Open FreeCAD and create a new macro from menu **Macro -> Macros -> Create**
- Paste the following code. Make sure you change the file path to match where you saved the .ui file created in QtCreator:

```
import FreeCAD,FreeCADGui,Part

# CHANGE THE LINE BELOW
path_to_ui = "C:\Users\yorik\Documents\dialog.ui"

class BoxTaskPanel:
    def __init__(self):
        # this will create a Qt widget from our ui file
        self.form = FreeCADGui.PySideUic.loadUi(path_to_ui)

    def accept(self):
        length = self.form.BoxLength.value()
        width = self.form.BoxWidth.value()
        height = self.form.BoxHeight.value()
        if (length == 0) or (width == 0) or (height == 0):
            print("Error! None of the values can be 0!")
            # we bail out without doing anything
            return
        box = Part.makeBox(length,width,height)
        Part.show(box)
        FreeCADGui.Control.closeDialog()

panel = BoxTaskPanel()
FreeCADGui.Control.showDialog(panel)
```

In the code above, we used a convenience function (PySideUic.loadUi) from the FreeCADGui module. That function loads a .ui file, creates a Qt Widget from it, and maps names, so we can easily access the subwidget by their names (ex: self.form.BoxLength).

The "accept" function is also a convenience offered by Qt. When there is a "OK" button in a dialog (which is the case by default when using the FreeCAD Tasks panel), any funcion named "accept" will automatically be executed when the "OK" button is pressed. Similarily, you can also add a "reject" function which gets executed when the "Cancel" button is pressed. In our case, we ommitted that function, so pressing "Cancel" will do the default behaviour (do nothing and close the dialog).

If we implement any of the accept or reject functions, their default behaviour (do nothing and close) will not occur anymore. So we need to close the Task panel ourselves. This is done with:

```
FreeCADGui.Control.closeDialog()
```

Once we have our BoxTaskPanel that has 1) a widget called "self.form" and 2) if needed, accept and reject functions, we can open the task panel with it, which is done with these two last lines:

```
panel = BoxTaskPanel()
FreeCADGui.Control.showDialog(panel)
```

Note that the widget created by PySideUic.loadUi is not specific to FreeCAD, it is a standard Qt widget which can be used with other Qt tools. For example, we could have shown a separate dialog box with it. Try this in the Python Console of FreeCAD (using the correct path to your .ui file of course):

```
from PySide import QtGui
w = FreeCADGui.PySideUic.loadUi("C:\Users\yorik\Documents\dialog.ui")
w.show()
```

Of course we didn't add any "OK" or "Cancel" button to our dialog, since it was made to be used from the FreeCAD Task panel, which already provides such buttons. So there is no way to close the dialog (other than pressing its Window Close button). But the function show() creates a non-modal dialog, which means it doesn't block the rest of the interface. So, while our dialog is still open, we can read the values of the fields:

```
w.BoxHeight.value()
```

This is very useful for testing.

Finally, don't forget there is much more documentation about using Qt widgets on the FreeCAD Wiki, in the Python Scripting section, which contains a dialog creation tutorial, a special 3-part PySide tutorial that covers the subject extensively.

Read more

- Qt Creator: https://en.wikipedia.org/wiki/Qt_Creator
- Installing Qt Creator: https://www.qt.io/ide/
- Python scripting documentation: http://www.freecadweb.org/wiki/index.php?title=Power_users_hub
- Dialog creation tutorial: http://www.freecadweb.org/wiki/index.php?title=Dialog_creation
- PySide tutorials: http://www.freecadweb.org/wiki/index.php?title=PySide
- PySide documentation: http://srinikom.github.io/pyside-docs/index.html

The Community

No manual dealing with free and open-source software would be complete without a chapter about the community. As the vast majority of free and open-source software projects, FreeCAD is made by a community, and maintained by that community. Instead of the opaque, unknown, impersonal and inaccessable firm that is more than often found behind commercial software, free and open-source software communities are open spaces, where you as a user are welcome, and where you can get answers very fast, and even have your say in the development of the software itself. You are also more than welcome to help, there are tasks for everybody.

The community is a growing, eclectic group of all kinds of people united by their passion for FreeCAD. All work on FreeCAD voluntarily, during their free time (although sometimes firms or individuals gather to pay a couple of programming hours to a developer to implement a specific function). Some are professional programmers, some are long-time FreeCAD users (some of them are true FreeCAD gurus, who know almost everything, and many of them end up knowing a lot about FreeCAD programming too), and many are new users of FreeCAD. There is nothing specific to do to be part of the community. Just use FreeCAD!

The main place where the community meets and discusses is the FreeCAD forum. All you need to do to participate to the discussions is to register an account on the forum (Your first post will need to be approved by a moderator before you can post more, to prevent spamming). The forum is a great place to ask questions when you are new to FreeCAD. Provided you made a good question (be sure to read the forum rules as they contain useful information to turn your question into a good question), you will usually get several replies within the same hour. If you think someone might have asked your question already, be sure to search, your answer might already be there.

The forum is also a great place to show what you achieved with FreeCAD, to help newcomers when you are more experienced, and to follow and give your opinions in more technical discussions about development. All the FreeCAD development is discussed on the forum, and anybody is free to read or participate.

There are also FreeCAD communities forming outside of the FreeCAD forum, for example on Facebook or Google+.

If you are becoming as enthusiastic about FreeCAD as we are, you might want to help the project. This can be done in many different ways, and there are tasks for everybody, programmers and non-programmers, for example:

- **Help to spread the word**: Many people would get huge benefit from using a free, open-

source 3D modeler like FreeCAD, but simply don't know its existence. Publishing the work you do with FreeCAD, talking about it on social networks, etc... helps these people to discover FreeCAD.

- **Help newcomers**: The vast majority of discussions on the forum are questions asked by new users. You might know good anwers to give them.
- **Help reporting bugs**: The stablility of FreeCAD comes in large part from the fixing of bugs. Since it is not possible for the FreeCAD developers to test all possible use cases, it is important that users report problems when they detect them. Be sure to read the guidelines if you think you found a bug, and then write a report on the bug tracker.
- **Help to write documentation**: The FreeCAD documentation wiki is also written by community members. Some sections of it are still incomplete, or their information incorrect or obsolete. You might be able to help to fix that. To be able to work on the wiki, you will need to familiarize yourself with wiki editing, and ask permission to edit the FreeCAD wiki on the forum.
- **Help to translate FreeCAD**: The translation of FreeCAD is done online by community members, on crowdin. If you don't see your language there, ask one of the administrators to have it added.
- **Help to translate the wiki documentation**: Every page of the wiki is translatable, and requires very little knowledge of the wiki syntax. Helping with translation is also a great way to learn FreeCAD.
- **Write scripts and macros**: FreeCAD has a growing list of Macros. If you wrote some interesting functionality, consider sharing it there.
- **Programming**: For this, you need to know how to program in Python or C++, and have a good knowledge of FreeCAD itself.

The source code of FreeCAD is located on the Github account of the FreeCAD project. Anybody can download, use and modify the code. You can publish your modifications (on Github or any other Git hosting service). If you made interesting modifications, that you wish to see included in the FreeCAD source code, you must ask the community to have them included. This can be done using Github's pull requests mechanism, but the very best way is to discuss what you intend to do first on the forum, and then post an official request in the Pull requests section of the forum when your code is ready. This avoids that you work on something that someone else is already working on too, and ensures that others agree with the way you are doing it, so there is no risk of having your work refused for some reason you didn't foresee.

Hopefully, we managed to give you a good taste of FreeCAD in this manual, and you are already our newest community member. Welcome!

Read more

- The FreeCAD forum: http://forum.freecadweb.org

- The source code of FreeCAD: https://github.com/FreeCAD/FreeCAD
- The Facebook FreeCAD community: https://www.facebook.com/FreeCAD
- The Google+ FreeCAD community: https://plus.google.com/u/0/communities/103183769032333474646
- The FreeCAD documentation wiki: http://www.freecadweb.org/wiki
- Translating FreeCAD on crowdin: https://crowdin.com/project/freecad
- The FreeCAD bug tracker: http://www.freecadweb.org/tracker